

Compiladores

Notas de Aula

Ricardo Vargas Dorneles

November 13, 2023

Compiladores

Notas de Aula

Ricardo Vargas Dorneles

November 13, 2023

- Introdução à Compilação
- Tradução dirigida por sintaxe
 - Esquemas de Tradução
 - Remoção de Recursão
 - Árvores de Sintaxe
- Análise Semântica
 - Verificação de Tipo
 - Análise de Declarações de Variáveis
- Tabela de Precedência de Operadores
- Geração de Código Intermediário
 - Geração de Código para Máquina de Pilha
 - Geração de Código de 3 Endereços
 - Expressões lógicas traduzidas como fluxo de controle
 - Geração de C3E para funções



Otimização de Código



Representação de Blocos Básicos



Construção de GADs para Blocos Básicos



Análise de Vida de Variáveis



Análise de Uso Subsequente



Análise de Laços



Dominadores

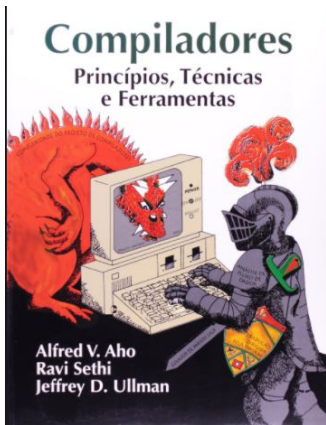


Computações Laço-Invariantes



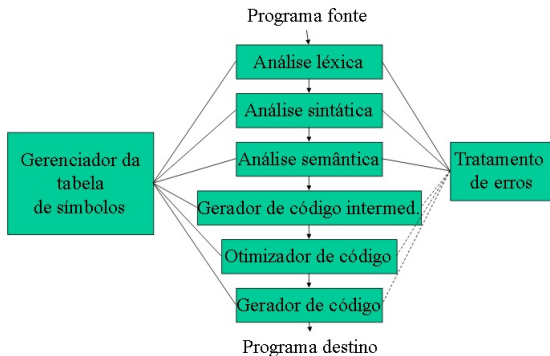
Eliminação de Variáveis de Indução

- **Análise de Fluxo de Dados (Dataflow)**
 - **Análise de definições incidentes (reaching definitions)**
 - **Análise de expressões disponíveis (available expressions)**
 - **Análise de expressões ocupadas (very busy expressions)**
- **Desdobramento de Laços**
- **Expansão Inline**
- **SSA - Static Single Assignment**
- **Geração de Código Objeto**
 - **Seleção de Instruções**
 - **Alocação de Registradores**
 - **Atribuição de Registradores**
 - **Escalonamento de Instruções**
 - **Otimizações Peephole**
 - **Algoritmo de Geração de Código**
- **Lista de Tópicos da Segunda Área**
- **Exercícios de Revisão**
- **Provas Anteriores - Primeira Área**
- **Provas Anteriores - Segunda Área**



Fases da Compilação:

- O processo de compilação pode ser subdividido nas seguintes fases:



- As três primeiras fases constituem, em conjunto, a fase de **análise**, e as três últimas fases constituem a fase de **síntese**.

- A compilação pode ser considerada apenas uma das fases da tradução de um programa fonte para um executável.
- Antes da fase de compilação, o programa fonte passa por um **pré-processamento**, em que são feitas as expansões de macro (defines) e a inclusão dos arquivos definidos nos includes.
 - O arquivo resultante desse passo tem a extensão .i e a opção de compilação -E (ex. gcc -E teste.c) mostra o resultado desse passo na saída padrão.

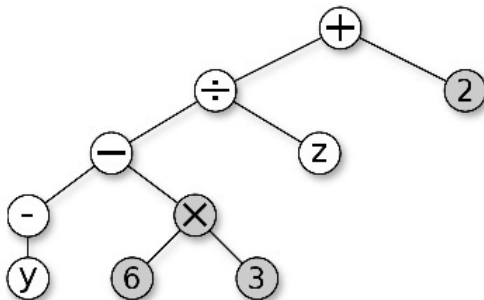
- Após, é feita a **compilação**, que resulta em um arquivo fonte em assembly. Esse arquivo tem a extensão .s e a opção de compilação -S disponibiliza esse arquivo.
- Após, o fonte em assembly é passado por um **montador** que o transforma em um binário (extensão .o, opção de compilação -c) e por fim é **linkado** com as bibliotecas necessárias, gerando o executável .exe.

A análise e a síntese tem tarefas distintas:

- Análise:
 - Reconhece cada uma das partes componentes do programa fonte e sua estrutura
 - Cria uma representação interna
 - A sintaxe a ser analisada é descrita através de gramáticas ou autômatos
- Síntese
 - Gera um programa objeto a partir da representação interna
 - Depende da semântica do programa

[Voltar para o índice](#)

- Após a fase de análise normalmente é gerada uma representação intermediária sobre a qual são feitas otimizações e a partir da qual é gerada a representação alvo. Essa representação intermediária pode ser um pseudo-assembly ou alguma outra representação:
- Exemplo de representação interna : Árvore Sintática



- Análise do Programa Fonte é feita em três fases:
 - Análise **léxica**
 - Análise **sintática**
 - Análise **semântica**
 - Léxico - Descreve os elementos básicos que compõem o programa (Léxico: s.m. É o conjunto de palavras existente em um determinado idioma).
 - Sintaxe - descreve a forma de um comando (forma de escrita) (Sintaxe: s.f. É o estudo das regras que regem a construção de frases nas línguas naturais)
 - Semântica - descreve o significado de um comando (o que o comando faz)

[Voltar para o índice](#)

- Lê seqüências de caracteres que formam o programa fonte
- Agrupa os caracteres em tokens que são as unidades léxicas, os átomos que compõem o programa fonte (ex: números, identificadores, operadores)
- Elimina os caracteres não significativos, como o fim de linha e espaços em branco

[Voltar para o índice](#)

- O analisador léxico normalmente trabalha "por demanda".
- As informações obtidas na análise léxica são armazenadas na tabela de símbolos.

Ex: Identificador
Tipo
Valor
Linha onde foi definido

[Voltar para o índice](#)

- Agrupa os tokens em sentenças da linguagem
- Verifica se os tokens formam sentenças corretas da linguagem de acordo com as regras de sintaxe da linguagem
 - $\text{Atrib} \rightarrow \text{id} = \text{exp}$
 - $\text{Exp} \rightarrow \text{exp} + \text{exp}$
 - $\text{Exp} \rightarrow \text{cte} \mid \text{id}$
- Ex: $x3 = y + 3$

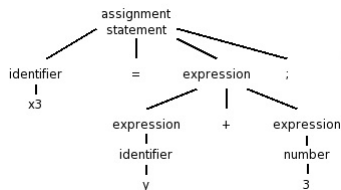


Figure: Árvore de derivação

- Efetua verificações que não podem ser feitas pela análise sintática
 - Verifica se as combinações entre operandos e operadores são válidas (posso aplicar o operador % a float?)
 - Verificação de tipo (posso atribuir um float a um char?)
 - Declarações (duplicadas, variáveis ainda não declaradas...)
 - Alguma coisa sobre estruturas de controle (esse break está dentro de um switch, for ou while?, entradas duplicadas em um switch...)
 - Complementa a tabela de símbolos

[Voltar para o índice](#)

- Na **geração de código intermediário** a seqüência de tokens é transformada em uma representação intermediária como uma árvore sintática ou outra representação.
- Vantagens:
 - Separa a análise do fonte da geração do código objeto, permitindo usar o mesmo front-end para compiladores para diversas plataformas
 - Facilita a implementação de algumas otimizações.

- A **otimização** tenta melhorar a qualidade do código intermediário através de diversos mecanismos como:
 - Identificação de subexpressões comuns
 - Reordenação de expressões

- **Geração de código final:** Geração de código executável a partir de código intermediário otimizado.
- O tratamento de erros e a manutenção da tabela de símbolos não são considerados uma fase por estarem distribuídos ao longo das outras fases.

- A **tabela de símbolos** é uma estrutura de dados que contém um registro para cada identificador. Este registro pode conter:
 - O **identificador** ou um pointer para o identificador
 - **Tipo** : variável inteira, real, procedimento, função, tamanho do vetor (análise sintática)
 - **Pointers para estruturas auxiliares** : árvore sintática ou lista de argumentos de funções e procedimentos

Ferramentas para a geração de compiladores

- Existem muitas ferramentas para, a partir de uma gramática apropriada, gerar analisadores léxicos e sintáticos. Geradores de analisadores léxicos são chamados de parser generators (geradores de parsers) ou compiler compilers (compiladores de compiladores)
- Todos eles utilizam como entrada uma gramática apropriada, que deve ser compatível com o algoritmo utilizado pela ferramenta.
- Em C:
 - Geradores de analisadores léxicos : LEX
 - Geradores de analisadores sintáticos : YACC
- Em Java:
 - Javacc : gerador de analisadores léxicos e sintáticos

Tradução dirigida por sintaxe (TDS)

- É uma técnica de especificação de compiladores e tradutores que permite associar ações semânticas às regras da gramática.
- Estas ações semânticas podem ser, entre outras:
 - Verificação de tipos
 - Geração de código intermediário
 - Preenchimento da tabela de símbolos
 - Emissão de mensagens de erro, etc.

[Voltar para o índice](#)

Complexidade
do Projeto de
Compiladores



Análise de
Fluxo de Dados

Tradução
dirigida por
syntaxe

Gerador de
parsers LALR

- Ex: Conversor de notação infixada para pós-fixada (SPOJ - ONP: Transform the Expression).
 - $3 + 2$ (infixada) $\Rightarrow 32+$ (pós fixada)
 - $3 + 2 - 4 + 7 - 6 \Rightarrow 3 2 + 4 - 7 + 6 -$
- $E \rightarrow T + E \{\text{printf}("+");\}$
- $E \rightarrow T \{\text{nao ha ação semântica}\}$
- $T \rightarrow F * T \{\text{printf}("*");\}$
- $T \rightarrow F \{\text{nao ha ação semântica}\}$
- $F \rightarrow \text{digito} \{\text{printf}("%d", \text{digito.val});\}$
 - Essa gramática dá associatividade à direita para o $+$ e o $*$. Isso é um problema?

```

int E()
{
    if (T())
        if (exp[pos]=='+')
        {
            pos++;
            if (E())
            {
                printf("+");
                return 1;
            }
            else return 0;
        }
        else return 1;
    else return 0;
}

```

$E \rightarrow T + E \{ \text{printf}("+"); \}$

$E \rightarrow T \{ \text{nao ha acao semantica} \}$

```

int T()
{
    if (F())
        if (exp[pos]=='*')
        {
            pos++;
            if (T())
            {
                printf("*");
                return 1;
            }
            else return 0;
        }
        else return 1;
    else return 0;
}

```

$T \rightarrow F * T \{ \text{printf}("*"); \}$

$T \rightarrow F \{ \text{nao ha acao semantica} \}$

```
int F()
{
    if (exp[pos]>='0'&&exp[pos]<='9')
    {
        printf("%c",exp[pos++]);
        return 1;
    }
    return 0;
}
```

- E se mantivéssemos a associatividade dos operadores à esquerda? Lembrando que para um operador ser associativo à esquerda, a produção deve ser recursiva à esquerda; para um operador ser associativo à direita, a produção deve ser recursiva à direita. Para um operador não associativo, a produção remete direto ao nível de precedência seguinte:
 - $E \rightarrow E + T$
 - $E \rightarrow T$
 - $T \rightarrow T * F$
 - $T \rightarrow F$
 - $F \rightarrow \text{digito}$

Algoritmo para obter uma gramática não ambígua para expressões

- 1. Criar um não terminal para cada nível de precedência e um para as unidades básicas, usa-se para os operadores de maior precedência índices maiores
- 2. Para cada operador **op associativo à esquerda** acrescenta-se na gramática uma produção da forma

$$E_i \rightarrow E_i \text{ op } E_{i+1}$$

- **Associativo à direita** (ex: potenciação):

$$E_i \rightarrow E_{i+1} \text{ op } E_i$$

- **Não associativo**

$$E_i \rightarrow E_{i+1} \text{ op } E_{i+1}$$

- **Unários** que podem ser aplicados repetidamente (ex: Not Not A)

$$E_i \rightarrow \text{op } E_i$$

- **Unários** que não podem ser aplicados repetidamente (ex: @Proc)
 $E_i \rightarrow \text{op } E_{i+1}$
 - se não existir E_{i+1} utiliza-se B
- 3. Acrescenta-se na gramática produções na forma $E_i \rightarrow E_{i+1}$ e $E_i \rightarrow B$ para o último nível.
- 4. Acrescenta-se na gramática uma produção que define B como as formas básicas de expressões como constantes, variáveis e expressões entre parênteses.

[Voltar para o índice](#)

Ocorre se algum não terminal A deriva uma forma sentencial $A\alpha$, ou seja $A \rightarrow A\alpha$.

Recursão imediata

$$S \rightarrow \begin{array}{l} S0 \mid \text{(parte recursiva)} \\ 1 \mid \text{(parte não recursiva)} \end{array}$$

Ex: 100

Recursão não imediata

$$\begin{array}{l} S \rightarrow \begin{array}{l} Aa \mid \text{(parte recursiva)} \\ b \mid \text{(parte não recursiva)} \end{array} \\ A \rightarrow Ac \mid Sd \mid \epsilon \end{array}$$

[Voltar para o índice](#)

Algoritmo para remoção de Recursão à esquerda

a) Agrupar as produções recursivas na forma:

$$A \rightarrow A\alpha_1 | A\alpha_2 | A\alpha_3 | \dots | A\alpha_n | \text{ (parte recursiva)} \\ \beta_1 | \beta_2 | \beta_m \text{ (parte não recursiva)}$$

b) Substituir esta produção pelas produções :

$$A \rightarrow \beta_1 B | \beta_2 B | \beta_m B$$

$$B \rightarrow \alpha_1 B | \alpha_2 B | \alpha_3 B | \dots | \alpha_n B | \epsilon$$

[Voltar para o índice](#)

ex: $S \rightarrow Sa|Sb|c|d$

resulta em

$$S \rightarrow cS'|dS'$$
$$S' \rightarrow aS'|bS'|\epsilon$$

[Voltar para o índice](#)

Para remover recursão não imediata substituem-se não terminais pelo lado direito das produções até que a recursão seja imediata

$$S \rightarrow Aa|b$$

$$A \rightarrow Ac|Sd|\epsilon$$

removendo a recursão da segunda produção

$$S \rightarrow Aa|b$$

$$A \rightarrow B|SdB$$

$$B \rightarrow cB|\epsilon$$

substituindo a segunda na terceira

$$S \rightarrow Ba|SdBa|b$$

$$B \rightarrow cB|\epsilon$$

[Voltar para o índice](#)

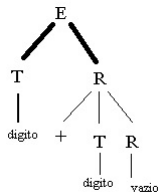
- Removendo a recursão à esquerda...

- $E \rightarrow E + T$
- $E \rightarrow T$
- $T \rightarrow T * F$
- $T \rightarrow F$
- $F \rightarrow \text{digito}$

- $E \rightarrow T E'$
- $E' \rightarrow \epsilon$
- $E' \rightarrow + T E'$
- $T \rightarrow F T'$
- $T' \rightarrow \epsilon$
- $T' \rightarrow * F T'$
- $F \rightarrow \text{digito}$

- Mostre a árvore de derivação para a sentença $3 + 2$.

- $E \rightarrow T R$
- $R \rightarrow \epsilon$
- $R \rightarrow + T R$
- $T \rightarrow \text{digito}$



- Em que momento da derivação deveriam ser executadas as ações semânticas?

- Uma gramática com ações semânticas inseridas no meio das produções se chama um **Esquema de Tradução**.
- Esquemas de tradução:
 - Utilizam uma gramática livre de contexto para especificar a estrutura sintática da gramática
 - Cada símbolo da gramática tem associado um conjunto de atributos. Estes atributos podem ser valores, apontadores para estruturas, entradas em tabelas, etc.
 - Cada produção tem um conjunto de regras semânticas que indicam como calcular o valor dos atributos dos símbolos da produção

[Voltar para o índice](#)

- Ex 1: Avaliação de uma expressão (para simplificação do problema, consideraremos os operadores de soma e multiplicação com associatividade à direita):

$E \rightarrow T + E_1 \{E.val := E_1.val + T.val\}$

$E \rightarrow T \{E.val := T.val\}$

$T \rightarrow F + T_1 \{T.val := T_1.val + F.val\}$

$T \rightarrow F \{T.val := F.val\}$

$F \rightarrow \text{digito} \{F.val := \text{digito.val}\}$

- Os atributos podem ser:
 - Sintetizados : se o seu valor é obtido a partir dos nodos filhos.
 - Herdados : se o seu valor é obtido do nodo pai ou dos irmãos e passado aos nodos filhos

- Na implementação no analisador da função que representa a produção, as ações semânticas da gramática aparecerão nas funções, na mesma ordem em que aparecem na gramática
 - Os atributos do símbolo à esquerda da produção serão representados como argumentos.
 - Os atributos herdados serão argumentos de entrada e os atributos sintetizados serão argumentos de saída.
 - Os atributos de outros símbolos serão representados como variáveis locais.

- Ex : Avaliação de expressão de soma (para simplificação do problema, consideraremos o operador de soma):

$E \rightarrow T + E_1 \{E.val := E_1.val + T.val\}$

$E \rightarrow T \{E.val := T.val\}$

$T \rightarrow \text{digito} \{T.val := \text{digito.val}\}$

- Na função que implementa a produção de E, E.val é o atributo do símbolo à esquerda, e será representado como um argumento. Por ser um atributo sintetizado (calculado a partir dos atributos dos filhos) será um argumento de saída (passado via pointer).
- T.val e E₁.val são representados como variáveis locais.

- Uma implementação para o esquema de tradução acima é mostrada abaixo (código `aval_exp_soma_recurso_direita.cpp`):
- Nesse código a sentença a ser reconhecida é lida do teclado, cada token ocupa uma única posição na expressão, o token corrente é representado pela posição corrente da string (`exp[pos]`) e o avanço do token é representado pelo incremento do índice do string (`pos++`).

```
int main()
{
    int Eval;
    printf("Digite a expressao:");
    scanf("%s",exp);
    if (E(&Eval)) printf("\nExpressão correta vale %d\n",Eval);
    else printf("Erro na posição %d\n",pos);
}
```

```

int E(int *Eval){
    int Tval, E1val;
    if (T(&Tval))
        if (exp[pos]=='+')
        {
            pos++;
            if (E(&E1val))
            {
                *Eval = E1val+Tval;
                return 1;
            }
            else return 0;
        }
        else // próximo token não é "+"
        {
            *Eval=Tval;
            return 1;
        }
    else return 0; // não reconheceu T
}

```

```

#include <stdio.h>
#include <stdlib.h>

int pos=0;
char exp[20];

int T(int *Tval)
{
    if (exp[pos]>='0'&&exp[pos]<='9')
    {
        *Tval=exp[pos]-'0';
        pos++;
        return 1;
    }
    return 0;
}

```

- Expanda o esquema acima para reconhecer números de mais de um dígito, e os operadores de soma, subtração, divisão e multiplicação inteira.

- Ex 2: Atualização de tipo de variáveis na tabela de símbolos para declarações em Pascal.

Decl → id : tipo{PoeTabSimb(id.lex,tipo.val)}

tipo → real{tipo.val := real}

tipo → integerT{tipo.val := integer}

- Obs:
 - Id.lex = atributo que contem o nome do identificador
 - tipo.val = atributo que contem o tipo identificado na declaração
- Como expandir esse esquema de tradução para que suporte listas de identificadores? (Veremos mais adiante)

Ex 3: Idem para declarações em C.

$$\begin{aligned} D &\rightarrow T\{L.tipo := T.tipo\}L \\ T &\rightarrow int\{T.tipo := int\} \\ T &\rightarrow real\{T.tipo := real\} \\ L &\rightarrow id\{PoeTabSimb(id.pos, L.tipo); L_1.tipo := L.tipo\}, L_1 \\ L &\rightarrow id\{PoeTabSimb(id.pos, L.tipo)\} \end{aligned}$$

- Os atributos L.tipo e T.tipo são herdados ou sintetizados?
- Uma declaração em C termina em ponto-e-vírgula. Em que posição do esquema de tradução acima poderia ser colocado o ponto-e-vírgula?
- Expandir o esquema acima para que reconheça listas de declarações ao invés de uma única declaração.

- Implementação em um analisador descendente recursivo preditivo.
 - $D \rightarrow T\{L.tipo := T.tipo\}L$

```
int D()
{
  int Ttipo, Ltipo;
  if (T(&Ttipo))
  {
    Ltipo=Ttipo; //ação semântica
    if (L(Ltipo)) return 1;
    else return 0;
  }
  else return 0;
}
```


- $T \rightarrow \text{int} \{T.\text{tipo} := \text{int}\}$
- $T \rightarrow \text{float} \{T.\text{tipo} := \text{real}\}$

```
int T(int *ttipo)
{
  if (Token==int)
  {
    Letoken();
    *ttipo:=int;
    return 1;
  }
  else if (Token==float)
  {
    Letoken();
    *ttipo:=float;
    return 1;
  }
  else return 0;
}
```

- $L \rightarrow id\{PoeTabSimb(id.pos, L.tipo); L_1.tipo := L.tipo\}, L_1$
- $L \rightarrow id\{PoeTabSimb(id.pos, L.tipo)\}$

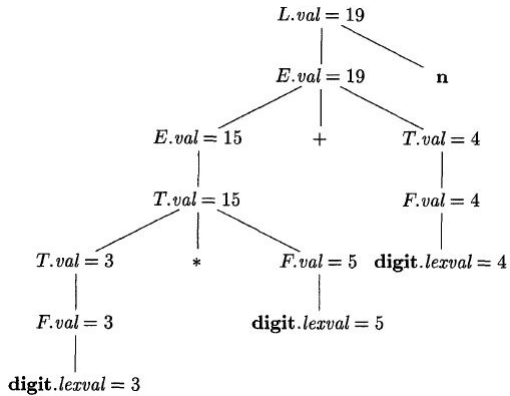
```

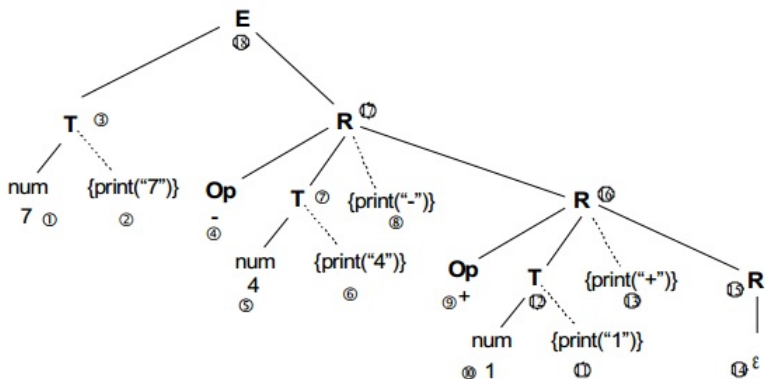
int L(int Ltipo)
{
  int L1tipo;
  if (Token==id)
  {
    Letoken();
    Poetab(id.lex, Ltipo);
    if (Token==' ')
    {
      Letoken();
      L1tipo:=Ltipo;
      if (L(L1tipo)) return 1;
      else return 0;
    }
    else return 1;
  }
  else return 0;
}

```

- Implemente um analisador descendente recursivo que reconheça declarações de char, int, float e vetores unidimensionais em C, verificando a ocorrências de declarações repetidas e emitindo mensagens de erro quando ocorrerem.
- Utilize como tabela de símbolos uma lista de strings.
- Implemente as funções de colocar o identificador na tabela, e de verificar se um identificador já existe na tabela.

- Uma árvore de derivação que mostra os valores dos atributos associados a cada nodo é chamada de árvore de derivação **anotada** (ex da gramática anterior para a expressão $7-4+1$)
- Na árvore abaixo também foram representadas, com linhas pontilhadas, as ações semânticas da produção para auxiliar na identificação do momento em que as mesmas são executadas
- os números em cada nodo representam a ordem em que a visita aos nodos é concluída (o que identifica uma busca ... in-ordem? pós-ordem? pré-ordem?)





- Avaliação de expressões de soma e subtração com recursão à esquerda.

$$E \rightarrow E_1 + T\{E.val := E_1.val + T.val\}$$

$$E \rightarrow E_1 - T\{E.val := E_1.val - T.val\}$$

$$E \rightarrow T\{E.val := T.val\}$$

$$T \rightarrow num\{T.val := num.val\}$$

- Uma **gramática de atributos** é uma DDS na qual as funções nas regras semânticas não produzem efeitos colaterais como imprimir um valor ou atualizar uma variável global.

- $A \rightarrow A_1 Y \{A.a := g(A_1.a, Y.y)\}$
- $A \rightarrow X \{A.a := f(X.x)\}$
- Regra Geral:
 $A \rightarrow X \{R.h := f(X.x)\} R \{A.a := R.s\}$
 $R \rightarrow Y \{R_1.h := g(R.h, Y.y)\} R_1 \{R.s := R_1.s\}$
 $R \rightarrow \epsilon \{R.s := R.h\}$
- Obs: R.h - atributo herdado, R.s - atributo sintetizado

[Voltar para o índice](#)

- Ex:
 - $E \rightarrow E_1 + T \{E.val := E_1.val + T.val\}$
 - $E \rightarrow T \{E.val := T.val\}$
- $A = E$
- $Y = +T$
- $X = T$
- $g(A, Y) = E_1.val + T.val$
- $f(X) = T.val$

$E \rightarrow T \{R.h := T.val\}R \{E.val := R.s\}$

$R \rightarrow +T \{R1.h := R.h + T.val\}R_1 \{R.s := R_1.s\}$

$R \rightarrow \epsilon \{R.s := R.h\}$

Árvore anotada para a expressão 3+3:

```
#include <stdio.h>
#include <conio.h>
```

```
int pos;
char exp[100];
int T()
{
    if (exp[pos]>='0' && exp[pos]<='9')
    {
        pos++;
        return 1;
    }
    else return 0;
}
```

```
int R()
{
    if (exp[pos]=='+')
    {
        pos++;
        if (T())
            if (R())
                return 1;
            else return 0;
        else return 0;
    }
    else return 1;
}
```

```
int E()  
{  
    pos=0;  
    if (T())  
        if (R())  
            return 1;  
        else return 0;  
    else return 0;  
}
```

```
int main()  
{  
    scanf("%s", exp);  
    if (E())  
    {  
        exp[pos]=0;  
        printf("reconheceu %s\n", exp);  
    }  
    else printf("não reconheceu\n");  
    getch();  
}
```

- Expanda o esquema acima para reconhecer operadores de multiplicação e divisão, sub-expressões entre parênteses, operador de troca de sinal e a função $\text{sqrt}(X)$.

- Um grafo de dependência é um grafo que representa as dependências entre os atributos herdados e sintetizados de uma árvore de derivação.
- Escreva o grafo de dependência para a gramática de preenchimento de tipos vista anteriormente para a sentença $int\ a, b$

$D \rightarrow TL \{ L.tipo := T.tipo \}$

$T \rightarrow int \{ T.tipo := int \}$

$T \rightarrow real \{ T.tipo := real \}$

$L \rightarrow L_1, id \{ L_1.tipo := L.tipo; L.x :=$
 $poetipo(id.posição, tipo); \}$

$L \rightarrow id \{ L.x := poetipo(id.posição, L.tipo) \}$

- Para traçar o grafo de dependência deve-se modificar cada regra semântica que consiste da chamada de rotina para a forma $b = f(c_1, c_2, c_3)$ pela introdução de um atributo imaginário b .

- 1) Escreva uma gramática que gere sentenças formadas por sequências de dígitos e retorne, no atributo val do símbolo raiz, o valor do somatório desses dígitos.

- $A \rightarrow A_1 \text{ digit} \{ A.\text{val} = A_1.\text{val} + \text{digit}.\text{lexval} \}$
- $A \rightarrow \text{digit} \{ A.\text{val} = \text{digit}.\text{lexval} \}$

- 2) Escreva uma gramática que gere sentenças formadas por sequências de dígitos e retorne, no atributo val do símbolo raiz, o valor do número formado pelos dígitos.

- $A \rightarrow A_1 \text{ digit } \{ A.\text{val} = A_1.\text{val} * 10 + \text{digit}.\text{lexval} \}$
- $A \rightarrow \text{digit } \{ A.\text{val} = \text{digit}.\text{lexval} \}$

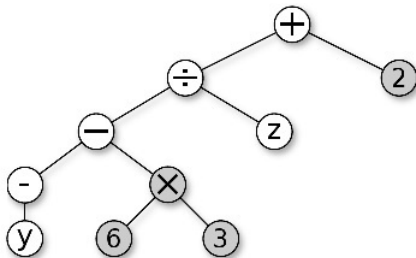
- Um esquema de tradução que utiliza somente atributos sintetizados é chamada S-atribuído.
- Uma árvore de derivação para um esquema s-atribuído pode ser sempre anotada pela avaliação bottom-up das rotinas semânticas para os atributos de cada nodo.

Construção de Árvores de Sintaxe

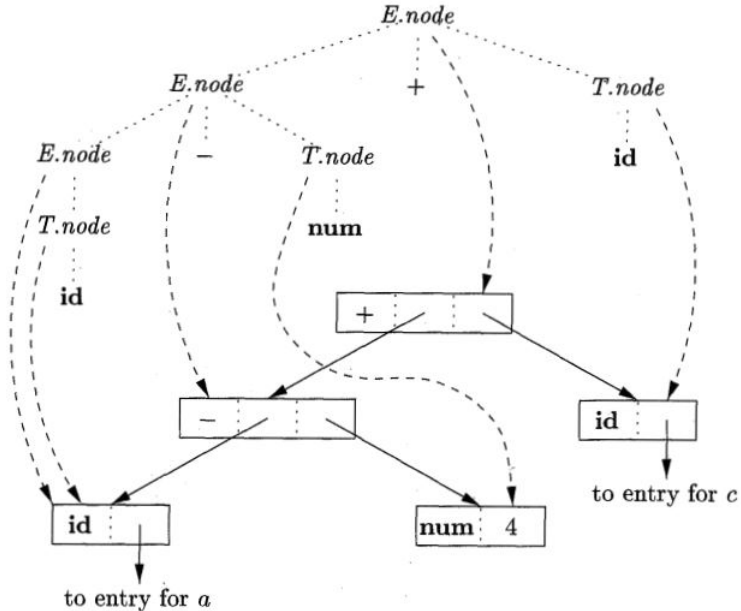
- Uma **árvore de sintaxe** é uma forma condensada de uma árvore de derivação útil para representar construções de linguagens.
- Ela permite separar o processo de análise do processo de tradução. São usadas normalmente em duas situações:
 - ① A gramática é apropriada para análise, mas não reflete a estrutura hierárquica natural das construções da linguagem
 - ② O método de análise restringe a ordem na qual os nodos da árvore de derivação são processados
- Na árvore de sintaxe, operadores e palavras chave não aparecem como folhas, mas como nodos interiores.
- Podem ser utilizadas como uma forma de código intermediário.

[Voltar para o índice](#)

- Ex: Árvore sintática para a expressão $(-y - 6 * 3) / z + 2$



- Ex: Árvore sintática para a expressão $a-4+c$



Funções:

- 1 Geranodo (op, esq, dir) - Cria um nodo de operador com rótulos op e dois campos contendo ponteiros para esq e dir
- 2 Gerafolha(id,índice) - Cria um nodo de identificador com rótulo id e um campo índice contendo um ponteiro para a entrada do identificador na tabela de símbolos
- 3 Gerafolha(num,val) - Cria um nodo de número com rótulo num e um campo val contendo o valor do número

$E \rightarrow E_1 + T \{E.ptr := Geranodo("+", E_1.ptr, T.ptr)\}$

$E \rightarrow E_1 - T \{E.ptr := Geranodo("-", E_1.ptr, T.ptr)\}$

$E \rightarrow T \{E.ptr := T.ptr\}$

$T \rightarrow (E) \{T.ptr := E.ptr\}$

$T \rightarrow id \{T.ptr := Gerafolha(id, posição)\}$

$T \rightarrow num \{T.ptr := Gerafolha(num, num.valor)\}$

- a) Mostre a árvore anotada da derivação da expressão $(3+a) + (3-a)$:
- b) Mostre a árvore sintática da expressão $(3+a) - (3-a)$
- c) Remova a recursão da gramática e ações semânticas

$E \rightarrow T \{R.h := T.ptr\} R \{E.ptr := R.s\}$
 $R \rightarrow + T \{R_1.h := \text{Geranodo}("+", R.h, T.ptr)\} R_1 \{R.s := R_1.s\}$
 $R \rightarrow - T \{R_1.h := \text{Geranodo}("-", R.h, T.ptr)\} R_1 \{R.s := R_1.s\}$
 $R \rightarrow \epsilon \{R.s := R.h\}$
 $T \rightarrow (E) \{T.ptr := E.ptr\}$
 $T \rightarrow id \{T.ptr := \text{Gerafolha}(id, \text{posição})\}$
 $T \rightarrow num \{T.ptr := \text{Gerafolha}(num, num.valor)\}$

- Função E

- Atributos de E: E.ptr - sintetizado - argumento de saída
- Atributos de outros símbolos: Rh, Tptr, Rs - variáveis locais

```
int E(nodoexp **Eptr)
{
    nodoexp *RS,*Rh,*Tptr;
    if (T(&Tptr))
    {
        Rh=Tptr;
        if (R(&Rs, Rh))
        {
            *Eptr=Rs;
            return 1;
        }
    }
    return 0;
}
```

```

int R(nodoexp **Rs, nodoexp *Rh)
{
    nodoexp *Tptr,*R1h,*R1s;
    if (token=='+')
    {
        letoken();
        if (T(&Tptr))
        {
            R1h=geranodo('+',Rh,Tptr);
            if R(&R1s,R1h)
            {
                *Rs=R1s;
                return 1;
            }
            else
            {
                Desaloca R1s;
                return 0;
            }
        }
        else return 0;
    }
    else
    {
        *Rs=Rh;
        return 0;
    }
}

```

```

int T(nodoexp **Tptr)
{
    nodoexp *Eptr;
    if (token=='(')
    {
        letoken();
        if (E(&Eptr))
            if (token==')')
            {
                *Tptr=Eptr;
                return 1;
            }
        else return 0;
    }
    else return 0;
}

else if (token==id)
{
    letoken();
    *Tptr=gerafolha(id);
    return 1;
}

else if (token==num)
{
    letoken();
    *Tptr=gerafolha(id);
    return 1;
}
else return 0;
}

```


- O código gera `_arvore.cpp`, disponível no acervo da disciplina, gera uma árvore sintática para expressões com constantes de 1 dígito e o operador `+`, e avalia o valor da expressão da árvore gerada, escrevendo o resultado.
- Execute o código no PythonTutor, visualizando a estrutura gerada a partir do reconhecimento da expressão `2+3`.

Ferramenta fortemente recomendada

- <http://pythontutor.com/> → Start visualizing your code now
- Selecionar linguagem
- Editar código (p.ex. código abaixo)
- Visualize execution

Write code in C (gcc 4.8, C11) EXPERIMENTAL ▼

```
1 int main() {  
2     int a=0;  
3     printf("%d\n",a);  
4     return 0;  
5 }
```

Keep this tool free for everyone by [making a small d](#)

Support our research by completing a [short user sur](#)

Visualize Execution

- Ao clicar em "visualize execution" é feita uma simulação da execução do código (aparece uma mensagem pedindo para esperar 10 segundos) e é gerada uma sequência de frames sobre a qual é exibida a simulação.
 - Eventualmente o servidor pode estar sobrecarregado, ou a duração da simulação excede o limite de frames permitido, e não será possível fazer a simulação.
- Como a exibição é feita após a simulação, não é possível interação com o usuário (ou seja, não há comandos de entrada (scanf))
- Dados de teste podem ser inseridos na declaração de variáveis. Ex na lâmina seguinte:

C (gcc 4.8, C11) **EXPERIMENTAL!**
see [known bugs](#) and report to philip@pgbovine.net

```
1 #include <stdio.h>
2 typedef struct nodo{int val;struct nodo *prox;} Tnodo;
3 int main() {
4     int m[10]={9,8,7,6,5,4,3,2,1,0},i,j,aux;
5     for (j=0;j<9;j++)
6     for (i=0;i<9;i++)
7         if (m[i]>m[i+1])
8         {
9             aux=m[i];
10            m[i]=m[i+1];
11            m[i+1]=aux;
12        }
13     return 0;
14 }
```

[Edit code](#)

→ line that has just executed

→ next line to execute

Click a line of code to set a breakpoint; use the Back and Forward buttons to jump there.

<< First < Back Forward > Last >>

Stack

main	
	array
m	0 1 2 3 4 5 6 7 8 9
	int int int int int int int int int int
	4 5 3 2 1 0 6 7 8 9
i	int
	1
j	int
	4
aux	int
	5

C (gcc 4.8, C11) **EXPERIMENTAL!**
 see [known bugs](#) and report to philip@pgbovine.net

```

1 #include <string.h>
2 char n[5][10]={"Joao","Jose","Ana","Bea","Carlos"};
3
4 void main(){
5   char aux[10];
6   int i,j;
7   for (j=0;j<4;j++)
8     for (i=0;i<4;i++)
9       if (strcmp(n[i],n[i+1])>0)
10        {
11          strcpy(aux,n[i]);
12          strcpy(n[i],n[i+1]);
13          strcpy(n[i+1],aux);
14        }
15   for (i=0;i<5;i++)
16     printf("%s\n",n[i]);
17 }

```

[Edit code](#)

t has just executed
 e to execute

of code to set a breakpoint; use the Back and Forward buttons to jump there.

Program terminated

·@pgbovine. Support with a [small donation](#).

is improve this tool by clicking below whenever you learn something:

Print output (drag lower right corner to resize)

```

Ana
Bea
Carlos
Joao
Jose

```

Stack

Global variables

array									
0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9
char	char	char	char	char	char	char	char	char	char
'A'	'n'	'a'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'
1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8	1,9
char	char	char	char	char	char	char	char	char	char
'B'	'e'	'a'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'
2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8	2,9
char	char	char	char	char	char	char	char	char	char
'C'	'a'	'r'	'o'	's'	'\0'	'\0'	'\0'	'\0'	'\0'
3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8	3,9
char	char	char	char	char	char	char	char	char	char
'J'	'o'	'a'	'o'	'\0'	's'	'\0'	'\0'	'\0'	'\0'
4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8	4,9
char	char	char	char	char	char	char	char	char	char
'J'	'o'	's'	'e'	'\0'	's'	'\0'	'\0'	'\0'	'\0'

main

array									
0	1	2	3	4	5	6	7	8	9
char	char	char	char	char	char	char	char	char	char
'J'	'o'	'a'	'o'	'\0'	?	?	?	?	?
aux									
i	int								
	5								
j	int								
	4								



- Os botões First e Last permitem ir até o primeiro e o último frame da simulação.
- Os botões Back e Forward permitem avançar ou voltar um frame.
- É possível setar pontos de parada no código para que, ao chegar no ponto selecionado, a execução seja pausada.
 - Isso é feito clicando uma vez sobre a linha a ser inserido o break point. A linha ficará em vermelho.
 - Ao clicar em Forward, ao passar sobre o ponto de parada a simulação será interrompida.
 - Ao clicar em Last os breakpoints serão ignorados
 - Para remover o breakpoint, basta clicar novamente sobre a linha.

- Incremente o código anterior para que ele reconheça e gere a árvore sintática para expressões com os operadores:
 - Precedência 1 : Troca de sinal
 - Precedência 2 : Multiplicação e divisão
 - Precedência 3 : Soma e subtração
 - Itens básicos: constante, identificador, expressão entre parênteses e chamada de função $r(x)$ representando a raiz quadrada

$E \rightarrow E_1 + T \{E.ptr := Geranodo("+", E_1.ptr, T.ptr)\}$

$E \rightarrow E_1 - T \{E.ptr := Geranodo("-", E_1.ptr, T.ptr)\}$

$E \rightarrow T \{E.ptr := T.ptr\}$

$T \rightarrow T_1 * F \{T.ptr := Geranodo("*", T_1.ptr, F.ptr)\}$

$T \rightarrow T_1 / F \{T.ptr := Geranodo("/", T_1.ptr, F.ptr)\}$

$T \rightarrow F \{T.ptr := F.ptr\}$

$F \rightarrow (E) \{T.ptr := E.ptr\}$

$F \rightarrow id \{F.ptr := Gerafolha(id, posição)\}$

$F \rightarrow - F_1 \{F.ptr := Geranodo("-", F_1.ptr)\}$

$F \rightarrow num \{F.ptr := Gerafolha(num, num.valor)\}$

- Assegura que um programa fonte segue as convenções semânticas da linguagem.
- A verificação pode ser estática ou dinâmica, se realizada durante a compilação ou execução.
- Exemplos de verificação estática:
 - Verificação de tipos
 - Verificação de fluxo de controle. Ex: break fora de um laço
 - Verificação de unicidade de identificadores (identificadores redeclarados).

[Voltar para o índice](#)

- Um tipo é representado por uma expressão de tipo que é uma sentença de uma sublinguagem de tipos.
- Uma expressão de tipo é um tipo básico ou é formada aplicando um operador chamado construtor de tipo a outras expressões de tipo.

[Voltar para o índice](#)

1)Tipos básicos : boolean, char, integer, real. Tipos básicos especiais: tipo error (erro de tipo) e void (ausência de valor)

2)Construtores de tipos:

Arrays $\Rightarrow$ array(intervalo, tipo)

ex: Array[1..10] of integer $\Rightarrow$ array([1..10],integer)

Produto $\Rightarrow T_1 * T_2$ onde T_1 e T_2 são expressões de tipo (p.ex. lista de argumentos em funções).

Ex: int X (int, float) $\Rightarrow$ int * float

- $\text{struct} \Rightarrow \text{struct}(E_1 * E_2 * E_3 * \dots * E_n)$ onde
 - $E_i = (\text{nome} * \text{tipo})$
- ex:

```
struct {  
    char nome[20];  
    int fone;  
};
```

```
struct( ( nome * array([0..19],char))*(fone*int))
```

- Apontadores : apont (tipo apontado)
 - ex: $p = \wedge \text{nodo}; \Rightarrow \text{apont}(\text{nodo})$
- Identificadores : podem ser dados nomes a tipos.
- ex:

```
type nodo = record
    id: string [10];
    info: Tinfo;
end
type Tinfo : integer;
```

- Escreva expressões de tipos para os seguintes tipos:
 - a) Um array de ponteiros para reais, onde os índices vão de 1 a 100.
 - b) Um array bidimensional de inteiros (i.e. um array de arrays) com linhas de 0 a 9 e colunas de -10 a 10

a)array([1..100],apont(real))
b)array([0..9],array([-10..10],integer))

- **Funções e Procedimentos**

- Function $X(a:\text{char}; b:\text{real}):\text{real}; \Rightarrow \text{char} * \text{real} \rightarrow \text{real}$
 - Procedure $Y(a:\text{real}; b:\text{char}; c:\text{integer}) \Rightarrow \text{real} * \text{char} * \text{integer}$
- Escreva uma expressão de tipos para uma função cujo domínio sejam funções de inteiros para pointers para inteiros e cujo tipo sejam records consistindo de um inteiro e de um caracter.
- $(\text{integer} \rightarrow \text{apont}(\text{integer})) \rightarrow \text{reg}((\text{campo1} * \text{integer}) * (\text{campo2} * \text{char}))$
- O tipo pode ser representado na forma de uma árvore (como as construções da linguagem).

- Pode variar numa mesma linguagem
- Tudo pode ser dinâmico (causa overhead na execução)
- Estática (mais segurança)
- Algumas coisas só podem ser verificadas dinamicamente (ex: índices de arrays)
- Uma linguagem fortemente tipada (p.ex. pascal) é uma linguagem em que não ocorrem erros de tipo durante a execução.

- Faça uma esquema de tradução (gramática e ações semânticas) para reconhecer declarações de variáveis em C de tipos escalares, bem como arrays de quaisquer dimensões e structures contendo como campos tipos escalares, outras structures e arrays, colocando a informação de tipo na tabela de símbolos.
- Faça uma função que, a partir da tabela de símbolos, escreva as declarações reconhecidas (apenas para testar se as estruturas e a tabela de símbolos estão corretas).
- Exs:

```
int a,b;  
struct data {int dia;int mes;int ano;} d1;  
char a[10],b[10][20];  
struct {int dia;int mes;int ano;} d2;  
struct {int a,b,c;} d3;  
struct {int a;  
        struct {int x; int y;} c1;} s1;
```

```
typedef struct Tnodo {char tipo;  
    union{  
        struct TNCampo *LCampos;  
        struct {int tamanho;  
                struct Tnodo *tipo_ar;};  
        struct Tnodo *Tapont;  
    };  
} Tnodo;
```

```
typedef struct TNCampo{  
    char nome[];  
    Tnodo *tipo;  
    TNCampo *prox;  
} TNCampo;
```

Gramática para declaração de arrays

Dec $\rightarrow$ Tipo $\{ \text{ListaVar.t} = \text{Tipo.t} \}$ ListaVar ;

Tipo $\rightarrow$ int $\{ \text{Tipo.t} = \text{gen_nodot('I')} \}$

Tipo $\rightarrow$ float $\{ \text{Tipo.t} = \text{gen_nodot('F')} \}$

Tipo $\rightarrow$ char $\{ \text{Tipo.t} = \text{gen_nodot('C')} \}$

ListaVar $\rightarrow$ id $\{ \text{ListaDim.t} = \text{ListaVar.t} \}$ ListaDim
 $\{ \text{poetabsimb(id, ListaDim.t)} \} \{ \text{RestoLV.t} = \text{ListaVar.t}; \}$ RestoLV

ListaDim $\rightarrow$ [cte] $\{ \text{ListaDim}_1.t = \text{ListaDim.t} \}$
 $\text{ListaDim}_1 \{ \text{ListaDim.t} = \text{gera_nodo('A', cte, ListaDim}_1.t) \}$

ListaDim $\rightarrow \epsilon$

RestoLV $\rightarrow , \{ \text{ListaVar.t} = \text{RestoLV.t} \}$ ListaVar

RestoLV $\rightarrow \epsilon \{ \text{não faz nada} \}$

- No esquema de tradução acima, na produção tipo é gerado um nodo do tipo escalar (p.ex: na declaração "int a,b,c[10] é gerado um nodo do tipo int) e esse nodo é passado para a lista de identificadores.
- Se a variável declarada é um array, o tipo base do array é passado pelo atributo L.t por todas as dimensões (derivadas por ListaDim). A cada dimensão é criado um nodo construtor de array e a estrutura que descreve as dimensões seguintes é pendurada nele.
- O nodo do tipo base é pendurado no último construtor de array.
- O atributo ListaDim.t é utilizado para descer o pointer para o nodo do tipo base e para subir o pointer para a lista de construtores gerada.

Tipo $\rightarrow$ **struct** Nome {ListaC} {Tipo.t=gera_nodo_s(ListaC.p)}

Nome $\rightarrow \epsilon \mid \mathbf{id}$

ListaC $\rightarrow$ Campo; RLC{Campo.p->prox=RLC.p;
ListaC.p=Campo.p;}

Campo $\rightarrow$ Tipo **id**;{Campo.p=geracampo(id,Tipo,t)}

RLC $\rightarrow \epsilon$ {RLC.p=NULL;}

RLC $\rightarrow$ Campo; RLC₁{Campo.p->prox=RLC₁.p;
RLC.p=Campo.p;}

- A idéia de compatibilidade de tipos está relacionada a questões de verificação de tipos. P. ex.:
 - Geração de código
 - Conversões de tipo
- Compatibilidade de tipos deve ser verificada em diversas situações:
 - Operandos são compatíveis com operadores?
 - Atribuições
 - Passagens de parâmetros em funções
- Métodos para definir compatibilidade de tipos
 - nominal
 - estrutural
- A maior parte das linguagens combina esses métodos

Compatibilidade de Tipos Nominal

- Duas variáveis são compatíveis se:
 - Estão na mesma declaração, ou
 - Estão em declarações distintas e usam o mesmo nome de tipo
- Exemplo:

```
typedef int Int1;  
typedef int Int2;  
struct {int x,y;} v1,v2;  
struct {int z,k;} v3,v4;  
Int1 v5;  
Int1 v6, v7;  
Int2 v;
```
- Quais variáveis são compatíveis entre si?

Compatibilidade de Tipos Nominal

- Compatibilidade de tipos nominal é de fácil implementação porém muito restritiva:
 - Subranges e aliases de tipo integer não são compatíveis com o tipo integer. Exemplo:

```
type indextype= 1..100;  
var  
    count : integer;  
    index : indextype;  
typedef unsigned int uint;  
uint a;  
unsigned int b;  
a=b; ???
```

- Duas variáveis possuem tipos compatíveis se:
 - Seus tipos possuem estruturas idênticas
 - Compatibilidade de tipos estrutural é mais flexível, porém mais difícil de implementar
- Problema
 - Não se consegue diferenciar tipos com a mesma estrutura:
`typedef boolean int;`
`typedef temperature int;`

- Perguntas:
 - Registros: são compatíveis se forem estruturalmente compatíveis porém com nomes de campos distintos?
 - Arrays: são compatíveis se possuírem mesmo tipo base e tamanho, porém com esquema de indexação diferente? (ex.: [0..10] e [1..11])
 - Tipos enumerados: são compatíveis se seus componentes forem escritos de modo diferente?

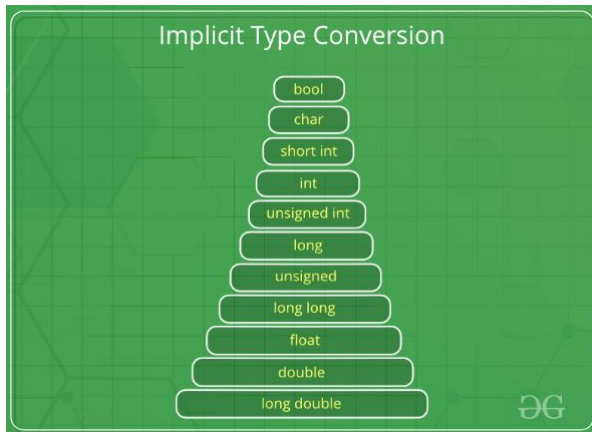
- Exemplos
 - FORTRAN, COBOL - Compatibilidade de tipos estrutural
 - Pascal (ISO Standard Pascal, 1982) - Compatibilidade de tipos estrutural, exceto para parâmetros formais
 - C - Compatibilidade de tipos estrutural, exceto para registros e uniões (equivalência de declaração)

Quais atribuições são válidas?

```
struct teste {int a,b,c};  
typedef struct teste tteste;  
struct teste t1,t3;  
tteste t4;  
struct teste2{int a,b,c};  
struct teste2 t2;  
int main() {  
    t1=t2; //esta é a única inválida  
    t1=t3;  
    t1=t4;  
}
```

- Verificação de tipo é importante não apenas para identificar erros de compatibilidade de tipo, mas para gerar as instruções apropriadas a cada tipo e inserir instruções para conversão implícita de tipo quando necessário (próxima lâmina).
 - $\text{Decl} \rightarrow \text{tipo id} \{ \text{IncTabSimb}(\text{id.val}, \text{tipo.tp}) \}$
 - $\text{Tipo} \rightarrow \text{int} \{ \text{Tipo.tp} := \text{Tint} \}$
 - $\text{Tipo} \rightarrow \text{float} \{ \text{Tipo.tp} := \text{Tfloat} \}$
 - $E \rightarrow \text{num_int} \{ e.tp := \text{tint} \}$
 - $E \rightarrow \text{num_real} \{ e.tp := \text{tfloat} \}$
 - $E \rightarrow \text{id} \{ e.tp := \text{ProcTabSimb}(\text{id}) \}$

- Ao converter um inteiro maior para um menor, copia-se os bytes menos significativos. Ao copiar-se um menor para um maior, se for com sinal propaga-se o bit de sinal para os bytes mais significativos.
- Processadores modernos têm instruções para conversão entre valores inteiros e floats.



- $E \rightarrow E_1 + T$ { if ($E_1.tp = \text{terro}$ || $T.tp = \text{terro}$) $E.tp = \text{terro}$;
else if ($E_1.tp = \text{tint}$ && $T.tp = \text{tint}$) $E.tp = \text{tint}$; else
 $E.tp = \text{tfloat}$; }
- $E \rightarrow T$
- $T \rightarrow T_1 * F$
- $T \rightarrow T_1 \% F$
- $T \rightarrow F$

$S \rightarrow \text{id} := \text{Expr} \{$

```
if id.tipo=e.tipo then  
S.tipo:=tvoid (não ocorreu erro de tipo)  
else S.tipo := terror
```

$\}$

$S \rightarrow \text{if } E \text{ then } S_1 \{ \text{o tipo de } E \text{ deve ser boolean ou relacional} \}$

$S \rightarrow \text{Begin } L \text{ End}$

$L \rightarrow L_1 ; S \mid S$

$S \rightarrow \text{while } E \text{ do } S_1 \{ E \text{ deve ser boolean} \}$

$S \rightarrow \text{for id} := E1 \text{ to } E2 \text{ do } S1 \{$

```
id.tipo := ProcTabSimb(id); (procura id na tabela de símbolos)
if id.tipo <> terror then
  if id.tipo=tint then
    if E1.tipo=tint then
      if E2.tipo=tint then
        if S1.tipo=tvoid then
          S.tipo=tvoid
        else S.tipo:=terror;
      else Begin
        write('limite superior deve ser inteiro');
        S.tipo:=terror;
      end
    else Begin
      write("limite inferior deve ser inteiro");
      S.tipo:=terror;
    end
  else Begin
    write("variável de controle deve ser inteira");
    S.tipo:= terror;
  end
else Begin
  write("variável não declarada");
  S.tipo:=terror;
end

}
```

Um esquema é **L-atribuído** se cada atributo herdado X_j ; $1 < j < n$, no lado direito de $A \rightarrow X_1 X_2 X_3 \dots X_n$, depende somente de

- 1 Dos atributos dos símbolos $X_1, X_2, \dots, X_{j-1}$ à esquerda de X_j na produção e,
- 2 Dos atributos herdados de A

Substituição de atributos herdados por sintetizados

- Às vezes é possível evitar o uso de atributos herdados alterando a gramática.
 - Ex: Declarações em pascal

$$D \rightarrow L : T$$
$$T \rightarrow integer | char$$
$$L \rightarrow L, id | id$$

- Se o não terminal L herda o tipo de T na primeira produção, temos uma **DDS não L-atribuída** de modo que a tradução não pode ser feita durante a análise. A solução para este problema é reestruturar a gramática para incluir o tipo como o último elemento da lista de identificadores.

$D \rightarrow \text{id } L \{ \text{PoeTabSimb}(\text{id}, L.\text{tipo}) \}$

$L \rightarrow , \text{id } L \mid : T$

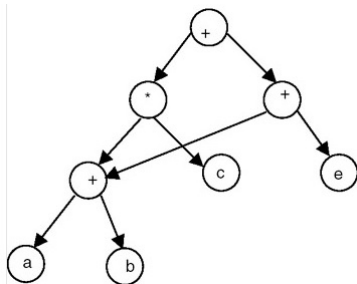
$T \rightarrow \text{integer} \mid \text{char} \{ T.\text{tipo} := \text{tint} \mid T.\text{tipo} := \text{tchar} \}$

$L \rightarrow : T \{ L.\text{tipo} := T.\text{tipo} \}$

$L \rightarrow , \text{id } L \{ L.\text{tipo} := L1.\text{tipo} ; \text{PoeTabSimb}(\text{id}, L1.\text{tipo}) \}$

DAG's - Directed Acyclic Graphs - Grafo Sintático

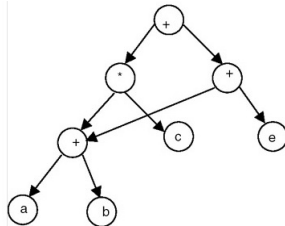
- É um grafo semelhante à árvore sintática mas em que as subexpressões comuns aparecem apenas uma vez.
 - Ex: $(a + b) * c + (a + b + e)$



- Na construção de um grafo sintático as funções de criação de nodo e folha antes devem verificar se já existe um nodo equivalente. Se existe eles retornam um pointer para este nodo.

- Nós podem ser implementados como uma tabela onde cada entrada possui o tipo de nó (operador, operando) e o número das entradas dos nós "filho".

1	a		
2	b		
3	c		
4	e		
5	+	1	2
6	*	5	3
7	+	5	4
8	+	6	7



- Faça um esquema de tradução (gramática e ações semânticas) para reconhecer uma expressão com somas e subtrações de constantes e identificadores, e gerar o grafo sintático equivalente.

$E \rightarrow E_1 + T \{E.p := \text{Poetabela}("+", E_1.p, T.p)\}$

$E \rightarrow E_1 - T \{E.p := \text{Poetabela}("-", E_1.p, T.p)\}$

$E \rightarrow T \{E.p := T.p\}$

$T \rightarrow (E) \{T.p := E.p\}$

$T \rightarrow \text{id} \{T.p := \text{Poetabela}(\text{id})\}$

$T \rightarrow \text{num} \{T.p := \text{Poetabela}(\text{num})\}$

- Obs:

- O atributo .p contém a posição da entrada da tabela que contém o nó gerado
- A função poetabela verifica se o nó criado já existe na tabela. Se existir, retorna sua posição. Se não existir, insere o nó na tabela e retorna a posição.

- (Aho 5.5) Crie um esquema de tradução (gramática e ações semânticas) que gere a derivada de expressões formadas pela aplicação dos operadores aritméticos "+" e "*" a variáveis (assuma que o identificador possui apenas uma letra), constantes de um dígito e expressões entre parênteses. Por exemplo,
 - $x * (3 * y + x * x)$.
- Assuma que nenhuma simplificação ocorra, de tal forma que $3 * x$ avalie em $3 * 1 + 0 * x$.
- Lembrando que:
 - $(fg)' = f'g + fg'$
 - $(f+g)' = f' + g'$
- Implemente o esquema de tradução a partir do código do avaliar de expressões no UCS Virtual

$$\begin{aligned}
 E \rightarrow E_1 + E_2 & \quad \{ E.d := ' (' + E_1.d + ' +' + E_2.d + ') ' \\
 & \quad E.e := ' (' + E_1.e + ' +' + E_2.e + ') ' \} \\
 E \rightarrow E_1 * E_2 & \quad \{ E.d := ' (' + E_1.d + ' *' + E_2.e + \\
 & \quad ' +' + E_2.d + ' *' + E_1.e + ') ' \\
 & \quad E.e := ' (' + E_1.e + ' *' + E_2.e + ') ' \} \\
 E \rightarrow (E_1) & \quad \{ E.d := ' (' + E_1.d + ') ' \\
 & \quad E.e := ' (' + E_1.e + ') ' \} \\
 E \rightarrow \text{const} & \quad \{ E.d := '0'; E.e := \text{const.num} \} \\
 E \rightarrow \text{id} & \quad \{ E.d := '1'; E.e := \text{id.nome} \}
 \end{aligned}$$

• Funciona? Problemas? Exemplos?

- A gramática é ambígua não definindo a precedência entre o + e o *, e não definindo a associatividade do + e do *.

$E \rightarrow E_1 + T$	$\{E.d := '(' + E_1.d + '+' + T.d + ')\'$ $E.e := '(' + E_1.e + '+' + T.e + ')\'$
$E \rightarrow T$	$\{E.d := T.d$ $E.e := T.e\}$
$T \rightarrow T_1 * F$	$\{ T.d := '(' + T_1.d + '*' + F.e +$ $' + '+' + F.d + '*' + T_1.e + ')\'$ $T.e := '(' + T_1.e + '*' + F.e + ')\'$
$T \rightarrow F$	$\{ T.d := F.d$ $T.e := F.e\}$
$F \rightarrow (E)$	$\{ F.d := '(' + E.d + ')\'$ $F.e := '(' + E.e + ')\'$
$F \rightarrow \text{const}$	$\{ F.d := '0'; F.e := \text{const.num} \}$
$F \rightarrow \text{id}$	$\{ F.d := '1'; F.e := \text{id.nome} \}$

- Como fica o modelo de remoção de recursão para calcular mais de um atributo?

- O modelo de remoção de recursão continua valendo mesmo havendo mais de um atributo. Há uma ação semântica para o cálculo de cada atributo, e cada atributo gera um atributo herdado e um atributo sintetizado em R.

$E \rightarrow T$	$\{R.dh = T.d; R.eh=T.e\}$
R	$\{E.d = R.ds; E.e = R.es\}$
$R \rightarrow +T$	$\{R_1.dh = '('+T.d+'+'+R.dh+')'; R_1.eh = '('+T.e+'+'+R.eh+')'\}$
R_1	$\{R.ds = R_1.ds; R.es = R_1.es\}$
$R \rightarrow \epsilon$	$\{R.ds = R.dh; R.es = R.eh\}$
$T \rightarrow T_1 * F$	
$T \rightarrow F$	
$F \rightarrow (E)$	
$F \rightarrow \text{const}$	$\{ F.d := '0'; F.e := \text{const.num} \}$
$F \rightarrow \text{id}$	$\{ F.d := '1'; F.e := \text{id.nome} \}$

- Alterar as ações semânticas do exercício anterior para gerar um string com a expressão pós fixada equivalente
- $2 + A * 3 - B * (3 + 2 + 5 + 7) \Rightarrow 2A3 * +B32 + 5 + 7 + *-$

$$\begin{aligned}
 E \rightarrow & E_1 + T \{ E.p := E_1.p + T.p + ' + ' \} \mid \\
 & E_1 - T \{ E.p := E_1.p + T.p + ' - ' \} \mid \\
 & T \{ E.p := T.p \} \\
 T \rightarrow & T_1 * F \{ T.p := T_1.p + F.p + ' * ' \} \mid \\
 & F \{ T.p := F.p \} \\
 F \rightarrow & \text{id} \{ F.p := \text{id.nome} \} \mid \\
 & \text{cte} \{ F.p := \text{cte.num} \} \mid \\
 & (E) \{ F.p := E.p \}
 \end{aligned}$$

- Removendo a recursão...

- Alterar as ações semânticas da gramática anterior para gerar um string com a expressão equivalente simplificada. As simplificações a serem feitas são:
 - multiplicações por 0 são substituídas pela constante 0
 - multiplicação de uma expressão por 1 resulta na própria expressão
 - somar 0 a uma expressão resulta na própria expressão

- Escreva uma gramática que tendo como terminais dígitos binários e o "." reconheça constantes binárias com parte inteira e fracionária.

$I_n \rightarrow I.F$

$I \rightarrow I0 \mid I1 \mid 0 \mid 1$

$F \rightarrow F0 \mid F1 \mid 0 \mid 1$

- Escreva um esquema de tradução para calcular o valor da constante

$In \rightarrow I.F \{ Im.valor := I.valor + F.valor \}$

$I \rightarrow I0 \mid \{ I.valor := I1.valor * 2 \}$

$I1 \mid \{ I.valor := I1.valor * 2 + 1 \}$

$0 \mid \{ I.valor := 0 \}$

$1 \{ I.valor := 1 \}$

Ex: Árvore de

$F \rightarrow 0F \mid \{ F.valor := F1.valor / 2 \}$

$1F \mid \{ F.valor := F1.valor / 2 + 0.5 \}$

$0 \mid \{ F.valor := 0 \}$

$1 \{ F.valor := 0.5 \}$

derivação anotada para a expressão 101.101 ?

outra solução:

$$\begin{aligned} N &\rightarrow I \mid \{ N.val := I.val \} \\ &\quad I.F \{ N.val := I.val + F.val \} \\ I &\rightarrow ID \mid \{ I.val := I1.val * 2 + D.val \} \\ &\quad D \{ I.val := D.val \} \\ F &\rightarrow DF_1 \mid \{ F.val := F_1.val / 2 + D.val / 2 \} \\ &\quad D \{ F.val := D.val / 2 \} \\ D &\rightarrow 0 \mid \{ D.val := 0 \} \\ &\quad 1 \{ D.val := 1 \} \end{aligned}$$

Removendo a recursão:

$$\begin{aligned} N &\rightarrow I \mid \{ N.val := I.val \} \\ &\quad I.F \{ N.val := I.val + F.val \} \\ I &\rightarrow D \{ R.h := D.val \} \\ &\quad R \{ I.val := R.s \} \\ R &\rightarrow D \{ R1.h := R.h * 2 + D.val \} \\ &\quad R1 \{ R.s := R1.s \} \\ R &\rightarrow \epsilon \{ R.s := R.h \} \\ F &\rightarrow DF_1 \{ F.val := F_1.val / 2 + D.val / 2 \} \\ &\quad D \{ F.val := D.val / 2 \} \\ D &\rightarrow 0 \mid \{ D.val := 0 \} \\ &\quad 1 \{ D.val := 1 \} \end{aligned}$$

Geração de Código Intermediário

- Separa o front-end do compilador (análises léxica, sintática, semântica) da geração de código
- Vantagens:
 - Facilita otimização de código: a maior parte das otimizações é feita sobre o código intermediário
 - Facilita a portabilidade do compilador - O mesmo front-end de um compilador pode ser utilizado para compiladores para várias plataformas.
- Tipos:
 - Árvores e grafos de sintaxe
 - Código de três endereços
 - Código para máquina de pilha

[Voltar para o índice](#)

Geração de Código para uma máquina de pilha abstrata

- Máquinas de pilha se caracterizam por utilizar uma pilha para efetuar operações.
- Atualmente são largamente utilizadas na JVM, a máquina virtual do Java e na máquina virtual do Python.
- A utilização da pilha para armazenar resultados intermediários simplifica bastante o projeto de um gerador de código.

[Voltar para o índice](#)

- O pacote dis, em python, mostra o pcode gerado para uma função:

import dis	3	0	LOAD_CONST	1 (1)
		3	STORE_FAST	1 (f)
def fat(n):	4	6	SETUP_LOOP	36 (to 45)
f=1 (linha 3)		9	LOAD_FAST	0 (n)
		12	LOAD_CONST	1 (1)
		15	COMPARE_OP	4 (>)
while n>1:		18	POP_JUMP_IF_FALSE	44
f*=n	5	21	LOAD_FAST	1 (f)
		24	LOAD_FAST	0 (n)
		27	INPLACE_MULTIPLY	
n-=1		28	STORE_FAST	1 (f)
return f	6	31	LOAD_FAST	0 (n)
dis.dis(fat)		34	LOAD_CONST	1 (1)
		37	INPLACE_SUBTRACT	
fati=fat(4)		38	STORE_FAST	0 (n)
		41	JUMP_ABSOLUTE	9
		44	POP_BLOCK	
print(fati)	7	45	LOAD_FAST	1 (f)
		48	RETURN_VALUE	

Geração de Código para uma máquina de pilha abstrata

- Começaremos nosso estudo de máquina de pilha definindo uma máquina de pilha abstrata com as seguintes características:
- Memória de dados e instruções separadas
- Pilha para auxiliar na execução
- Instruções :
 - Aritmética inteira
 - Manipulação de pilha
 - Fluxo de controle

[Voltar para o índice](#)

- Uma para cada operador da linguagem ($+$, $-$, $*$, $/$).
- O operador é aplicado aos operandos no topo, os operandos são desempilhados e o resultado é empilhado.
- Os operadores relacionais resultam em 0 (falso) e 1 (verdadeiro).
- Obs: O primeiro operando é o subtopo.

- **Push** v - empilha v (constantes)
- **Valor-L** i - empilha o endereço de i
- $:=$ - o valor-R no topo é colocado no valor-L no subtopo e o valor-R é desempilhado (o Valor-L continua)
- **pop** - desempilha e descarta
- Com essas instruções, a atribuição $a=3$ geraria o seguinte código:

- Valor-R i - empilha o conteúdo de i (variáveis)
- copiar - empilha uma cópia do valor no topo da pilha

- Ex: Qual o código para avaliar o comando
 - $\text{dia} := (1461 * y) \text{ div } 4 + (153 * m + 2) \text{ div } 5 + d ?$
- Em notação pós fixada
 - $\text{dia } 1461 y * 4 \text{ div } 153 m * 2 + 5 \text{ div } + d + :=$

valor-L	dia
push	1461
valor-R	y
*	
push	4
div	
push	153
valor-R	m
*	
push	2
+	
push	5
div	
+	
valor-R	d
+	
:=	
pop	

Produções para atribuições e expressões em C

1. $\text{atrib} \rightarrow \text{id} = \text{E}; \quad \{\text{cmd.t} = \text{'valor-L' id} \parallel \text{E.t} \parallel \text{' := ' } \parallel \text{' pop' }\}$
2. $\text{E} \rightarrow \begin{array}{l} \text{E}_1 + \text{T} \mid \\ \text{E}_1 - \text{T} \mid \\ \text{T} \end{array} \quad \begin{array}{l} \{\text{E.t} = \text{E}_1.\text{t} \parallel \text{T.t} \parallel \text{' + ' }\} \\ \{\text{E.t} = \text{E}_1.\text{t} \parallel \text{T.t} \parallel \text{' - ' }\} \\ \{\text{E.t} = \text{T.t} \} \end{array}$
3. $\text{T} \rightarrow \begin{array}{l} \text{T}_1 * \text{F} \mid \\ \text{T}_1 / \text{F} \mid \\ \text{F} \end{array} \quad \begin{array}{l} \{\text{T.t} = \text{T}_1.\text{t} \parallel \text{F.t} \parallel \text{' * ' }\} \\ \{\text{T.t} = \text{T}_1.\text{t} \parallel \text{F.t} \parallel \text{' / ' }\} \\ \{\text{T.t} = \text{F.t} \} \end{array}$
4. $\text{F} \rightarrow \begin{array}{l} \text{id} \mid \\ \text{Cte} \mid \\ (\text{E}) \end{array} \quad \begin{array}{l} \{\text{F.t} = \text{'valor-R' id} \} \\ \{\text{F.t} = \text{'push' cte} \} \\ \{\text{F.t} = \text{E.t} \} \end{array}$

- A partir do código gera_codigo_pilha.cpp que está no acervo da disciplina, que gera código para máquina de pilha para uma atribuição de constante a uma variável, complete o código para reconhecer listas de atribuições de expressões com constantes e variáveis..

Remoção da recursão da produção de E

$$\begin{aligned} E &\rightarrow T \quad \{R.h=T.t\}R\{E.t=R.s\} \\ R &\rightarrow +T \quad \{R_1.h = R.h \parallel T.t \parallel '+'\}R_1\{R.s=R_1.s\} \\ R &\rightarrow \epsilon \quad \{R.s = R.h\} \end{aligned}$$

- A partir do código "gera codigo de pilha para soma.cpp" que está no acervo da disciplina, que gera código para máquina de pilha para uma atribuição de soma a uma variável, complete o código para reconhecer listas de atribuições de expressões com constantes e variáveis e operadores de subtração e multiplicação, com a precedência apropriada.

Produções para listas de atribuições

$$\begin{aligned} L_atrib &\rightarrow \text{atrib } L_atrib_1 && \{ L_atrib.t := \text{atrib.t} \parallel /_atrib_1.t \} \\ L_atrib &\rightarrow \epsilon && \{ L_atrib.t := "" \} \end{aligned}$$

- Código para estruturas de controle é gerado através das instruções para controle de fluxo a seguir:
 - Rótulo l - Define um label para instruções de desvio
 - Goto l - Desvia para label
 - Gofalse l - Desempilha o valor do topo e desvia se é zero
 - Gotrue l - Desempilha topo e desvia se é $\neq 0$
 - Parar - Pára a execução

- Para gerar o código para estruturas de controle, em primeiro lugar deve-se definir exatamente como funciona.
- Qual a semântica do comando if?
- Qual um possível código gerado para o comando abaixo?
 - if ($a > 0$) $a = 1$; else $a = 0$;
- A implementação de estruturas de controle exige labels e desvios condicionais. Para gerar labels usaremos uma função chamada novo_rótulo, que a cada chamada gerará um string com um label distinto.

- Ex: if ($a > 0$) $a = 1$; else $a = 0$;

	valor-r	a	//{a > 0}
	push	0	
	>		
	gofalse	else	
	valor-l	a	
	push	1	
	:=		
	pop		
	goto	saida	
rotulo else	valor-l	a	
	push	0	
	:=		
	pop		
rotulo saida			

com_if $\rightarrow$ if (E) *atrib*₁ else *atrib*₂
 {saida := novo_rotulo;
 else := novo_rotulo;
 cmd.t := E.t ||
 'gofalse' else ||
 atrib1.t ||
 'goto' saida ||
 'rotulo' else ||
 atrib2.t ||
 'rotulo' saida }

- E se ao invés de atrib1 houver um bloco de código? (if (a>0) {a=1;b=2;c=3;}) Comando

- Qual a semântica do comando for?
 - a.A expressão de valor final é avaliada apenas uma vez no início ou a cada iteração?
 - b.A variável de controle é testada por igual ao valor final ou maior que o valor final?
 - a.No turbo pascal, a expressão de valor final é avaliada apenas no início.
 - b.A variável de controle é testada se é igual ao valor final. Se for alterada e passar a ser maior, o for não termina.

- Como seria uma possível implementação para o for?

Avaliar a expressão inicial;

VarCont := Exp inicial

Avaliar a expressão final

Laço: Se VarCont > Expressão Final vá para o fim

Executar Comando

Incrementar VarCont

Vá para Laço

Fim:

- Ex: For i:=1 to 3 do a:=5;

	valor-l	i	//{i:=1}
	push	1	//{Expressão de valor inicial}
	:=		
rotulo laço:	valor-r	i	
	push	3	//{Expressão de valor final}
	<=		
	gofalse	fim	
	valor-l	a	//{Inicio de a:=5}
	push	5	//
	:=		//
	pop		//{Fim de a:=5}
	valor-r	i	//{Início de i=i+1}
	push	1	//
	+		//
	:=		//{Fim de i=i+1}
	goto	laço	
rótulo fim :	pop		
	parar		

- 1 Fazer uma gramática para gerar o for, expressões e atribuição e colocar as ações semânticas
- 2 Retirar a recursão da gramática
- 3 Fazer a árvore de derivação anotada para o comando for do exemplo
- 4 Mostrar um código para um for onde a expressão de valor final seja avaliada somente no início, fora do laço.

Operadores aritméticos, relacionais e lógicos em C

- Foi usada nessa gramática como numeração a precedência nos operadores na tabela abaixo:
- Ao término da execução do código de uma expressão, o resultado da mesma (e somente o resultado) deve estar na pilha. Qual o código gerado para os operadores reduzidos abaixo?

E15 $\rightarrow$ E15 , E14 | E14

E14 $\rightarrow$ id *= E14 | id /= E14 | id %= E14 |
id += E14 | id -= E14 | id = E14 | E13

E13 $\rightarrow$ E13 ? E13 : E13 | E12

E12 $\rightarrow$ E12 || E11 | E11

E11 $\rightarrow$ E11 && E7 | E7

E7 $\rightarrow$ E7 == E6 | E7 != E6 | E6

E6 $\rightarrow$ E6 > E4 | E6 >= E4 | E6 < E4 | E6 <= E4 | E4

E4 $\rightarrow$ E4 + E3 | E4 - E3 | E3

E3 $\rightarrow$ E3 * E2 | E3 / E2 | E3 % E2 | E2

E2 $\rightarrow$ -F | +F | F

F $\rightarrow$ id | cte | (E14)

Tabela de precedência de operadores em C

Prec.	Símbolo	Função	Associatividade
1	++ --	Incremento e decremento pósfixo	esquerda para direita
1	()	Parênteses (chamada de função)	
1	[]	Elemento de array	
1	.	Seleção de campo de structure	
1	->	Seleção de campo de structure a partir de ponteiro	
2	++ --	Incremento e decremento prefixo	direita para a esquerda
2	+ -	Adição e subtração unária	
2	! ~	Não lógico e complemento	
2	(tipo)	Conversão de tipo de dado	
2	*	Desreferência	
2	&	Referência (endereço de elemento)	
3	* / %	Multiplicação, divisão, e módulo (resto)	esquerda para a direita
4	+ -	Adição e subtração	
5	<< >>	Deslocamento de bits à esquerda e à direita	
6	< <=	"menor que" e "menor ou igual que"	
6	> >=	"maior que" e "maior ou igual que"	
7	= = !=	"Igual a" e "diferente de"	
8	&	E bit a bit	
9	^	Ou exclusivo para bits	
10		Ou para bits	
11	&&	E lógico	
12		Ou lógico	
13	c ? t : f	Condição ternária	direita para esquerda
14	=	Atribuição	
14	+= -=	Atribuição por adição ou subtração	
14	*= /= %=	Atribuição por multiplicação, divisão ou módulo (resto)	
14	<<= >>=	Atribuição por deslocamento de bits	
14	&= ^= =	Atribuição por operações lógicas	
15	,	vírgula	esquerda para direita

Produções para geração de código para outras estruturas de controle

$R \rightarrow$ $E_1 > E_2 \mid$ $\{R.t = E_1.t \parallel E_2.t \parallel '>'\}$
 $E_1 < E_2 \mid$ $\{R.t = E_1.t \parallel E_2.t \parallel '<'\}$
 E $\{R.t = E.t\}$

$\text{Com_While} \rightarrow$ while (expr) cmd1
 $\{ \text{saída} := \text{novo_rótulo};$
 $\text{início} := \text{novo_rótulo};$
 $\text{cmd.t} := \text{'rotulo' início} \parallel$
 $\text{expr.t} \parallel$
 $\text{'gofalse' saída} \parallel$
 $\text{cmd1.t} \parallel$
 $\text{'goto' início} \parallel$
 $\text{'rótulo' saída} \}$

- Obs: A função novo_rótulo gera um rótulo diferente a cada vez que é chamada (p.ex. no formato Lxxx: L001,L002...)

Agrupando os comandos...

Com $\rightarrow$ Com_Composto

Com $\rightarrow$ Com_If

Com $\rightarrow$ Com_While

Com $\rightarrow$ Com_Do

Com $\rightarrow$ Com_For

Com $\rightarrow$ Com_Exp

- Que poderia ser implementado como:

```
if (token==TK_Ab_Chaves) return Com_Comp(Com_c)
```

```
if (token==TK_If) return Com_If(Com_c);
```

```
if (token==TK_While) return Com_While(Com_c);...
```

```
return 0;
```

Comando Composto

$\text{Com_Comp} \rightarrow \{ \text{Lista_Comandos} \}$
 $\text{Lista_Comandos} \rightarrow \text{Com Lista_Comandos}$
 $\text{Lista_Comandos} \rightarrow \epsilon$

```
if (tk==TK_Abre_Chaves)
{
    strcpy(Com_Comp_c, "");
    tk=le_token();
    while (tk!=TK_Fecha_Chaves)
        if (Com(Com_c))
            strcat(Com_Comp_c, Com_c);
        else return 0;
    tk=le_token(); // consome o Fecha Chaves
    return 1;
}
```

```
valor-l a // a=1;
push 1
:=
pop
valor-l fat //fat=1
push 1
:=
pop
rotulo LB000 // while (a<10)
valor-r a
push 10
<
gofalse LB001
valor-l fat // fat = fat * a
valor-r fat
valor-r a
*
:=
pop
valor-l a // a = a+1
```

```
valor-r a
push 1
+
:=
pop
goto LB000
rotulo LB001
```

```
a=1;
fat=1;
while (a<10)
{
    fat=fat*a;
    a=a+1;
}
```

6. cmdfor → for(exp1;exp2;exp3) cmd1
{ saída := novo_rótulo;
início := novo_rótulo;
cmd.t := exp1.t ||
'rótulo' início ||
exp2.t ||
'gofalse' saída ||
cmd1.t ||
exp3.t ||
'goto' início ||
'rótulo' saída }

7. cmd $\rightarrow$ for <var> := <exp1> to <exp2> do <cmd>
 {for := novo_rótulo;
 fim := novo_rótulo;
 cmd.t := 'valor-l' var || exp1.t ||
 ':=' || 'rótulo' for ||
 'valor-r' var || exp2.t ||
 '>' || 'go true' fim ||
 cmd1.t || 'valor-r' var || push 1 ||
 '+' || ':=' ||
 'goto' for || 'rotulo' fim || pop }

Obs. Neste for, a expressão de valor final é avaliada a cada iteração. Como seria uma versão para a qual a expressão fosse avaliada somente no início?

- Gere as ações semânticas para gerar código para:
 - a) O repeat do pascal.
 - b) O operador ternário em C.
 - c) O operador vírgula em C.
 - d) O switch em C.
 - e) O operador de coalescência nula (??). $a = E_1 ?? E_2$. Se E_1 é não nula, o operador retorna E_1 , caso contrário retorna E_2 .
 - f) O comando break
 - g) O comando continue
- Obs: Em C e C++ o operador vírgula (representado por ",") é um operador que avalia seu primeiro operando e descarta o resultado, e então avalia o segundo operando e retorna seu valor (e tipo).
- O uso do token vírgula como operador é diferente de seu uso em chamadas e declarações de função, declarações de variáveis e usos similares, em que ela funciona como separador.

Tabela de precedência de operadores em C

Prec.	Símbolo	Função	Associatividade
1	++ --	Incremento e decremento pósfixo	esquerda para direita
1	()	Parênteses (chamada de função)	
1	[]	Elemento de array	
1	.	Seleção de campo de structure	
1	->	Seleção de campo de structure a partir de ponteiro	
2	++ --	Incremento e decremento prefixo	direita para a esquerda
2	+ -	Adição e subtração unária	
2	! ~	Não lógico e complemento	
2	(tipo)	Conversão de tipo de dado	
2	*	Desreferência	
2	&	Referência (endereço de elemento)	
3	* / %	Multiplicação, divisão, e módulo (resto)	esquerda para a direita
4	+ -	Adição e subtração	
5	<< >>	Deslocamento de bits à esquerda e à direita	
6	< <=	"menor que" e "menor ou igual que"	
6	> >=	"maior que" e "maior ou igual que"	
7	= = !=	"Igual a" e "diferente de"	
8	&	E bit a bit	
9	^	Ou exclusivo para bits	
10		Ou para bits	
11	&&	E lógico	
12		Ou lógico	
13	c ? t : f	Condição ternária	direita para esquerda
14	=	Atribuição	
14	+= -=	Atribuição por adição ou subtração	
14	*= /= %=	Atribuição por multiplicação, divisão ou módulo (resto)	
14	<<= >>=	Atribuição por deslocamento de bits	
14	&= ^= =	Atribuição por operações lógicas	
15	,	vírgula	esquerda para direita

Exemplos de código gerado para alguns trechos de código fonte

Trecho 1:

```
i:=1;  
fat:=1;  
while i<n do  
begin  
    fat := fat * i;  
    i := i + 1;  
end;
```

código gerado:

```
valor-l i
push 1
:=          // {i:= 1}
pop
valor-l fat
push 1
:=          // {fat:=1}
pop

rótulo inicio
valor-r i
valor-r n
<
gofalse fim // {i<n}
valor-l fat
valor-r fat
valor-r i
*
:=          {fat := fat * i}
pop
valor-l i
valor-r i
push 1
+
:=
pop        {i := i+1}
goto inicio

rótulo fim
```

Trecho 2

```
for i:=1 to 10 do  
  begin  
    a:=a-1;  
    if a<0 then i:=i*10;  
  end;
```

Código gerado:

```
valor-l i
push 1
:=
for:   valor-r i
      push 10
      >
      gotrue fim
      valor-l a      {a=a-1}
      valor-r a
      push 1
      -
      :=
      valor-r a
      push 0
      >=
      gotrue fim_if
      valor-l i      {i=i*10}
      valor-r i
      push 10
      *
      :=
fim_if: goto for
fim:
```

```
switch (E){  
    case  $L_1$ : comandos1  
    case  $L_2$ : comandos2  
    default: comandos3  
    case  $L_3$ : comandos4  
}
```

- Os labels $L_1, L_2 \dots$ devem ser constantes.
- A entrada default pode estar em qualquer ponto da lista e é executada se a expressão não for igual a nenhum dos labels
- Ao entrar em um dos cases, a execução segue através das listas de comandos até o final de todos os cases ou até executar um break.
- A lista de comandos de cada entrada pode ser vazia
- A lista de cases pode ser vazia

- Uma forma de implementar o **switch** é agrupando os blocos de código das diversas entradas do case e os blocos de teste em dois blocos disjuntos, gerando para o exemplo abaixo, para o código a seguir, o código de máquina de pilha mostrado na lâmina seguinte:

```
switch (a) {  
    case 1: comando1;  
    case 2: comando2;  
    case 3: comando3;  
    default: comando4;  
}
```


	valor-R a	{codigo de avaliação de A}
	copiar	{duplica o valor da expressão no topo da pilha para comparar}
	push 1	{label da primeira entrada}
	=	{compara o valor da expressão com primeiro label}
	gotrue LB1	
	copiar	{duplica o valor da expressão no topo da pilha para comparar}
	push 2	{label da segunda }
	=	{compara o valor da expressão com segundo label}
	gotrue LB2	
	copiar	{duplica o valor da expressão no topo da pilha para comparar}
	push 3	{label da terceira entrada}
	=	{compara o valor da expressão com terceiro label}
	gotrue LB3	
	goto LBdefault	{se não existir default, gera goto LBFim}
LB1:	comando1	
LB2:	comando2	
LB3:	comando3	
LBdefault:	Comando 4	
	pop	{desempilha o valor da expressão}

- E uma gramática para gerar o código acima é:

cmd $\rightarrow$ **switch** (E) { LCases }
LCases $\rightarrow$ **Case** LCases
LCases $\rightarrow$ ϵ
Case $\rightarrow$ **case** cte : LCom
Case $\rightarrow$ **default** : LCom

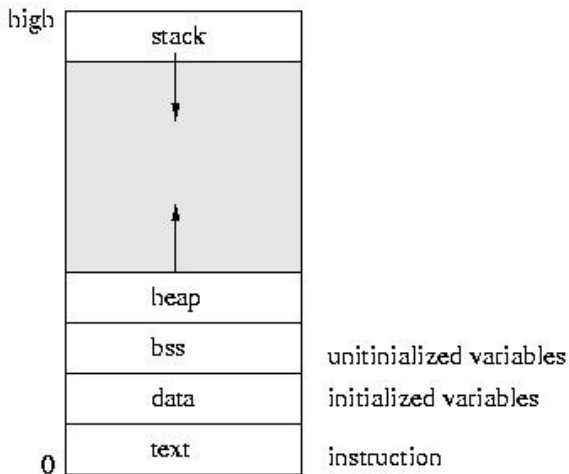
- E um início de ações semânticas é:
 - **cmd** $\rightarrow$ **switch** (E) { LCases } { **cmd.cod = E.cod || LCases.tst || LBCases.cod** }
 - LCases $\rightarrow$ Case LCases₁
 - { **Lcases.cod=Case.cod || Lcases₁.cod** }
 - LCases $\rightarrow \epsilon$
 - Case $\rightarrow$ **case** cte : LCom
 - Case $\rightarrow$ default : LCom

- Qual o código gerado para o comando (considerando a numeração correta dos labels e variáveis temporárias)?

```
switch (a) {  
    case 1: a=1;  
    case 2: b=b+1;  
    case 3: a=a+1;  
           b=b+2;  
    default: a=3;  
}
```

- Na análise de uma declaração são atualizados na tabela de símbolos o tipo e o endereço da variável.
- Este endereço pode ser o deslocamento em um bloco de memória alocado para variáveis globais ou o deslocamento em relação ao início do stack frame em uma função.
- Lembrando que (figurinha na próxima lâmina):
 - Variáveis locais são alocadas na pilha (stack) do sistema.
 - Variáveis globais e variáveis locais estáticas são alocadas em um segmento de dados na carga do programa pelo sistema operacional.

- Variáveis globais e variáveis estáticas inicializadas com valores diferentes de zero são alocadas no segmento de dados e variáveis globais e estáticas não inicializadas ou inicializadas com zero são alocadas no segmento BSS.
- Todas as variáveis globais e estáticas não inicializadas são inicializadas com 0.
- A pilha cresce "para baixo". A cada empilhamento o stack pointer é decrementado
- Variáveis alocadas dinamicamente são alocadas no heap (toda a memória física e virtual disponível)



```

long long int l;
short x;
int a;
char b;
float c;

int main()
{
    long long int ll;
    short int xl;
    int al;
    char bl;
    float cl;
    printf("&l=%08x\n",&l);
    printf("&x=%08x\n",&x);
    printf("&a=%08x\n",&a);
    printf("&b=%08x\n",&b);
    printf("&c=%08x\n",&c);
    printf("&ll=%08x\n",&ll);
    printf("&xl=%08x\n",&xl);
    printf("&al=%08x\n",&al);
    printf("&bl=%08x\n",&bl);
    printf("&cl=%08x\n",&cl);
}

```

```

&l=00430030
&x=00430038
&a=0043003c
&b=00430040
&c=00430044

&ll=0022fe48
&xl=0022fe46
&al=0022fe40
&bl=0022fe3f
&cl=0022fe38

```



```
typedef struct nodo {int valor; struct nodo *prox;} Tnodo;
Tnodo *inicg=NULL;
Tnodo *inic2;
```

```
void ins_inic(Tnodo **einic, int valor)
```

```
{
    Tnodo *ptaux=(Tnodo *)malloc(sizeof(Tnodo));
    ptaux->valor=valor;
    ptaux->prox=*einic;
    *einic=ptaux;
    printf("_ptaux=%08x\n", ptaux);
    printf("&ptaux=%08x\n", &ptaux);
    printf("&ptaux->valor=%08x\n", &ptaux->valor);
    printf("&ptaux->prox=%08x\n", &ptaux->prox);
    printf("einic=%08x\n", *einic);
    printf("&einic=%08x\n", &*einic);
    printf("**einic=%08x\n", **einic);
}
```

```
int main()
```

```
{
    Tnodo *inicio=NULL;
    printf("_inicg=%08x\n", inicg);
    printf("&inicg=%08x\n", &inicg);
    printf("_inic2=%08x\n", inic2);
    printf("&inic2=%08x\n", &inic2);
    printf("_inicio=%08x\n", inicio);
    printf("&inicio=%08x\n", &inicio);
    ins_inic(&inicio, 10);
}
```

```
inicg = 00000000
&inicg = 00430030
inic2 = 00000000
&inic2 = 00430038
inicio = 00000000
&inicio = 0022fe48

ptaux = 003476c0
&ptaux = 0022fe08
&ptaux->valor = 003476c0
&ptaux->prox = 003476c8
einic=0022fe48
&einic=0022fe20
*einic=003476c0
**einic=0022fdf0
```

- Faça um esquema de tradução de blocos de declaração de variáveis em pascal (integer, char ou real) que calcule o endereço de cada variável e a área total de armazenamento do bloco.

```

Bloco → Var{end:=0}D ;
D → id L {poe_tab_simb(id,L.tipo,end);end:=end+L.tam}
L → , id L1 | { L.tipo:=L1.tipo;
poe_tab_simb(id,L.tipo,end);
end:=end+L1.tam;
L.tam:=L1.tam}
: T { L.tipo := T.tipo; L.tam := T.tam }
T → integer | { T.tipo:=integer; T.tam:=2 }
Char | { T.tipo:=char; T.tam:=1 }
Real { T.tipo:=real; T.tam:=4 }

```

- Idem para a linguagem C:

tipo	Tamanho
short int	2
int	4
long int	4
long long int	8
double	8
float	4

Figure: Número de bytes dos tipos em C

Gramática para declaração de arrays unidimensionais em pascal:

Ex: Var a : array[1..10] of integer;

Bloco $\rightarrow$ Var D; {end:=0}

T $\rightarrow$ integer | {T.tipo := integer; T.tam:=2}

Real | {T.tipo := real; T.tam:=4}

Char {T.tipo := char; T.tam:=1}

D $\rightarrow$ id : array [dig .. dig₂] of T

{D.tipo:=array(T.tipo,dig₂.valor,dig.valor);

Poetabsimb(id,D.tipo,end);

End:=End + T.tam * (dig₂.valor - dig.valor + 1);}

Ex: `int a,b,c[10];`

$$\begin{aligned}
 D &\rightarrow T \{V.tb=T.tb; V.tam=T.tam;\} V \{R.tb=T.tb; R.tam=T.tam;\} R; \\
 T &\rightarrow \text{int} \mid \{T.tb := \text{tint}; T.tam := 4\} \\
 &\quad \text{double} \mid \{T.tb := \text{tdouble}; T.tam := 8\} \\
 R &\rightarrow , \{V.tb=R.tb; V.tam=R.tam;\} V \{R_1.tb=R.tb; R_1.tam=R.tam;\} R_1 \\
 R &\rightarrow \epsilon \\
 V &\rightarrow \text{id} \{ \text{Poetabsimb}(\text{id}, V.tb, \text{end}); \\
 &\quad \text{end} := \text{end} + V.tam; \} \\
 V &\rightarrow \text{id}[\text{cte}] \{ \text{Poetabsimb}(\text{id}, \text{array}(V.tb, \text{cte}), \text{end}); \\
 &\quad \text{end} := \text{end} + V.tam * \text{cte.val}; \}
 \end{aligned}$$

- T.tb: tipo base.

Gramática para declaração de arrays multidimensionais em pascal:

```
D → id : array [ dig .. dig2 R
    { D.tam:= R.tam*(dig2-dig+1);
      D.tipo:=array(R.tipo,dig.valor,dig2.valor);
      Poetabsimb(id,D.tipo,end);
      End=end+D.tam;}

R → , dig .. dig2 R1 { R.tam:= R1.tam*(dig2-dig+1);
    R.tipo:=array(R1.tipo,dig.valor,dig2.valor); }

R → ] of T { R.tam:= T.tam; R.tipo:=T.tipo; }
```

Ex: `int a,b,c[10];`

$V \rightarrow \text{id}\{R.tb=V.tb\}R\{\text{Poetabsimb}(\text{id},R.tipo,\text{end});$
 $\text{end}:=\text{end} + V.tam*R.tdm;\}$

$R \rightarrow [\text{cte}] \{R_1.tb=R.tb\}R_1$
 $\{R.tdm=R_1.tdm*cte; R.tipo=\text{array}(R_1.tipo,cte)\}$

$R \rightarrow \epsilon\{R.tipo=R.tb; R.tdm=1;\}$

- $V.tb$: tipo base.
- $R.tdm$: tamanho das dimensões
- $V.tam$: tamanho do tipo base

Endereçamento de Elementos de Vetores

- O endereço do i -ésimo elemento de um vetor A é obtido por
 - $\text{Base} + (i - \text{low}) * w$
- onde
 - w = comprimento de cada elemento
 - low = limite inferior
 - base = endereço relativo da memória reservada para o vetor, ou seja, é o endereço de $A(\text{low})$

- A expressão acima pode ser avaliada em tempo de compilação se reescrita como
 - $i * w + (\text{base} - \text{low} * w)$
- A subexpressão $c := \text{base} - \text{low} * w$ pode ser avaliada quando a declaração do vetor é analisada.
- O valor de c pode ser armazenado na tabela de símbolos, na entrada para o vetor A , tal que o endereço relativo de $A[i]$ é obtido somando-se $(i*w)$ a C .
- Na linguagem C é mais simples, uma vez que o limite inferior (low) de cada dimensão é 0.
 - Nesse caso basta multiplicar o índice pelo tamanho do elemento e somar ao endereço base

- Ex: código gerado para a atribuição $A[3]:=5$;

```
valor-l  A
push     3
push     4 ; o tamanho de um int
*
+
push     5
:=
pop
```

- Faça um esquema de tradução (gramática e ações semânticas) para reconhecimento de atribuições a vetores em C.

$\text{atrib} \rightarrow \text{id}[E] = E_1$

Opções de compilação para geração de fonte assembly no gcc

- `gcc -c -Wa,-ahl=test.s test.c`
 - `-c` processa só o arquivo fonte, gerando um objeto, sem efetuar a linkagem.
 - `-Wa` passa parâmetros direto pro assembler (que no caso do gcc é o `gnu as`).
 - `-ahl` são parâmetros do `as`.
 - `-a` gera um arquivo com a listagem do código, conforme as opções que tu passa pra ele:
 - `h` inclui o código em alto nível (de "high level", do tempo que C era considerado alto nível =).
 - `l` inclui o código em assembly.

- O Clang usa um código intermediário chamado llvm IR (intermediate representation)
 - O flag de compilação -emit-llvm gera um fonte com o código intermediário
- O gcc usa um código intermediário chamado gimple (ou gcc IR)
 - O flag de compilação -fdump-tree-original gera um fonte com o primeiro nível do código intermediário (mostrando a implementação em baixo nível do for)
 - O flag -fdump-tree-all mostra os intermediários de todos os passos (aparentemente o .11.t mostra a análise dos laços)

- E matrizes?
- Como é feita a referência à posição $M[i][j][k]$ da matriz $M[5][7][9]$?
- Faça um esquema de tradução (gramática e ações semânticas) para reconhecimento de referência de vetores em C.

$E \rightarrow \text{id}[E_1]$ (Não dá. A máquina de pilha não tem instruções para empilhar o conteúdo de uma variável cujo endereço está no topo da pilha)

- Faça um esquema de tradução (gramática e ações semânticas) para reconhecimento de atribuições a elementos de matrizes bidimensionais em C.

$\text{atrib} \rightarrow \text{id}[E][E_1] = E_2$

Endereçamento de campos de structures

- Deve ser guardado para cada campo o seu deslocamento em relação ao endereço base da struct.
- Na referência a um campo da structure, pode-se usar o deslocamento do campo em relação ao endereço base da struct usando instruções para manipulação de vetores ou pointers.
 - $A[\text{deslocamento do campo}] = X$ ou
 - $\text{*}(&A + \text{deslocamento do campo}) = X$

- Ao encontrar uma declaração de função é criada uma tabela de símbolos para a função e apontada para a tabela de símbolos da função mãe, no caso de funções aninhadas (quando a linguagem o permite), blocos de escopos diferentes ou para a tabela de símbolos de globais.
- A tabela de símbolos de cada função tem a informação do tamanho da área de variáveis que necessita.


```

. . .
int value=10;

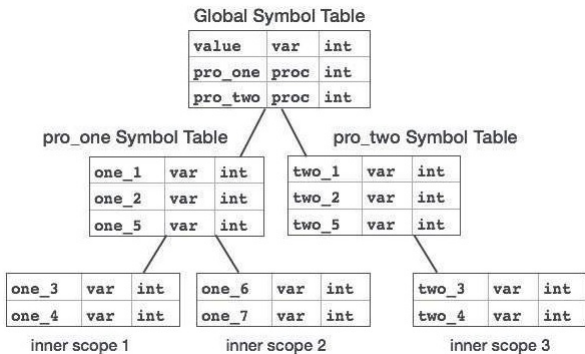
void pro_one()
{
    int one_1;
    int one_2;
    {
        int one_3;
        int one_4;
    }
    int one_5;
    {
        int one_6;
        int one_7;
    }
}
void pro_two()
{
    int two_1;
    int two_2;
    {
        int two_3;
        int two_4;
    }
    int two_5;
}
. . .

```

\
| - inner scope 1
/

\
| - inner scope 2
/

\
| - inner scope 3
/



- Um registro (ou quadro) de ativação é um bloco de memória alocado na chamada de uma procedure.
- Normalmente é formado pelos seguintes campos:
 - 1 Valor de retorno
 - 2 Parâmetros atuais
 - 3 Link de controle (endereço de retorno)
 - 4 Link de acesso (aponta para o registro de ativação do procedimento externo, é utilizado em procedimentos aninhados)
 - 5 Estado da máquina (estado dos flags)
 - 6 Dados locais (variáveis locais e parâmetros)
 - 7 Temporários (variáveis criadas com geratemp)

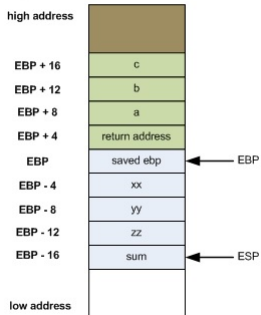
```

int foobar(int a, int b, int c)
{
    int xx = a + 2;
    int yy = b + 3;
    int zz = c + 4;
    int sum = xx + yy + zz;

    return xx * yy * zz + sum;
}

int main()
{
    return foobar(77, 88, 99);
}

```



Código de três endereços (C3E)

- É uma forma de código intermediário usada para gerar código para máquinas de registradores (em oposição a máquinas de pilha)
- Tem esse nome porque as instruções tem no máximo 3 operandos
- Forma Geral:
 - $A := B \text{ op } C$
- Onde A, B e C representam endereços de variáveis

Ex: $A := X + Y * Z$

- $T1 := Y * Z$
- $T2 := X + T1$
- $A := T2$

[Voltar para o índice](#)

- C3E é uma representação linear de uma árvore ou DAG (grafo dirigido acíclico) onde os nomes correspondem aos nodos do interior do grafo
- Ex: $A := B * - C + B * - C$

T1:=-C

T2:=B*T1

T3:=-C

T4:=B*T3

T5:=T2+T4

A:=T5

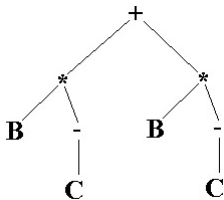


Figure: Árvore sintática equivalente

- Código otimizado e DAG equivalente:

T1:=-C

T2:=B*T1

T3:=T2+T2

A:=T3



Figure: Grafo sintático equivalente

Tipos de códigos de três endereços mais utilizados

- ① Comando de atribuição na forma $x := y \text{ op } z$ onde op é um operador binário lógico ou aritmético
- ② Atribuição com operador unário $x := \text{op } y$ onde op pode ser, por exemplo, negação, troca de sinal, conversão de tipo
- ③ Comando de cópia $x := y$
- ④ Desvio incondicional goto L
- ⑤ Desvio condicional como if $x \text{ op } y$ goto L
- ⑥ Endereçamento de elemento de vetor $X := Y[i]$ e $X[i] := Y$
- ⑦ Chamada de função
 - Param x - empilha parâmetro
 - Call p, n - chama função com n parâmetros
- ⑧ Atribuições com apontadores
 - $X := \& y$
 - $X := * Y$
 - $*X := Y$

- A principal representação interna do GCC, chamada GIMPLE, é uma versão de C3E.
- O flag de compilação `fdump-tree-all` gera arquivos de saída com o resultado de cada uma das 200 etapas da compilação (ou de grande parte delas):
- O código abaixo (`teste.c`):

```
int main()  
{  
    int a=1,b=1,i;  
    a=a*i+2*(b+i);  
}
```

- gera a seguinte representação intermediária no primeiro passo do gimple (`teste.c.004t.gimple`)

```
1 main ()
2 {
3   int D.1397;
4   int D.1398;
5   int D.1399;
6   int a;
7   int b;
8   int i;
9   a = 1;
10  b = 1;
11  D.1397 = a * i;
12  D.1398 = b + i;
13  D.1399 = D.1398 * 2;
14  a = D.1397 + D.1399;
15 }
```

- Nesse passo é gerada a primeira versão do código em Gimple. Nos passos anteriores é utilizada uma representação em árvore chamada GENERIC, sem representação textual.
- Nesse passo as expressões são "quebradas" em expressões de 2 operandos e o resultado colocado em temporários.
- Os temporários tem uma numeração sequencial no formato "D.YYYYY".
- Também é gerado código para inicialização das variáveis locais.
- Para variáveis globais inicializadas na declaração não é necessário código de inicialização, porque quando o programa é carregado na memória, as variáveis globais inicializadas são carregadas já inicializadas em um segmento separado.

- Funções :
 - Cria_temp : cria uma nova variável temporária
 - Gen : Gera uma instrução de três operandos
- Atributos :
 - E.place : variável que recebe o valor de E
 - E.cod : seqüência de C3E avaliando E

$A \rightarrow id = A_1 \{A.p=id;$
 $\quad A.cod=A_1.cod \parallel gen(=,id,A_1.p); \}$
 $A \rightarrow E \{A.cod:=E.cod$
 $\quad A.place:=E.place \}$
 $E \rightarrow E_1 + T \{E.place:=cria_temp$
 $\quad E.cod := E_1.cod \parallel T.cod \parallel gen(+, E.place, E_1.place, T.place) \}$
 $E \rightarrow T \{E.cod:=T.cod$
 $\quad E.place:=T.place \}$
 $T \rightarrow T_1 * F \{T.place:=cria_temp$
 $\quad T.cod:=T_1.cod \parallel F.cod \parallel gen(*, T.place, T_1.place, F.place) \}$
 $T \rightarrow F \{T.cod:=F.cod$
 $\quad T.place:=F.place \}$

- Para implementar temos que tirar a recursão à esquerda...

$$\begin{aligned}
 E &\rightarrow E_1 + T \{E.place:=cria_temp \\
 &\quad E.cod := E_1.cod \parallel T.cod \parallel gen(+, E.place, E_1.place, T.place)\} \\
 E &\rightarrow T \{E.cod:=T.cod \\
 &\quad E.place:=T.place\}
 \end{aligned}$$

- Removida a recursão...

$$\begin{aligned}
 E &\rightarrow T \{R.hplace=T.place; R.hcod=T.cod; \} R\{E.place=R.splace; E.cod=R.scod; \} \\
 R &\rightarrow + T\{R_1.hplace=geratemp; \\
 &\quad R_1.hcod = R.hcod + T.cod+gen(+, R_1.hplace, R.hplace, T.place)\} \\
 &\quad R_1\{R.splace=R_1.splace; R.scod=R_1.scod \} \\
 R &\rightarrow \epsilon \{R.splace=R.hplace; R.scod=R.hcod \}
 \end{aligned}$$

- Como ficaria a árvore de derivação anotada de "a+3"? Use para T as produções:

$$\begin{aligned}
 T &\rightarrow id \{T.place:=id; T.cod:="" \} \\
 T &\rightarrow cte \{T.place:=cria_temp; T.cod:=gen(:=, T.place, cte.val)\}
 \end{aligned}$$

$F \rightarrow \text{id} \{F.\text{place} := \text{id}; F.\text{cod} := " " \}$

$F \rightarrow \text{cte} \{F.\text{place} := \text{cria_temp};$
 $F.\text{cod} := \text{gen}(:=, F.\text{place}, \text{cte.val}) \}$

$F \rightarrow (E) \{F.\text{cod} := E.\text{cod}; F.\text{place} := E.\text{place} \}$

$F \rightarrow \text{id} [E] \{F.\text{place} = \text{cria_temp}; \text{Taux} = \text{cria_temp};$
 $F.\text{cod} = E.\text{cod} || \text{gen}(*, \text{Taux}, E.\text{place}, 4) ||$
 $\text{gen}(=[], F.\text{place}, \text{id}, \text{Taux}) \}$

$F \rightarrow \text{id} [E][E_1]$

$\text{atrib} \rightarrow \text{id} := E \{ \text{atrib.cod} := E.\text{cod} || \text{gen}(:=, \text{id}, E.\text{place}) \}$

$\text{atrib} \rightarrow \text{id}[E] := E_1$

$\text{atrib} \rightarrow \text{id}[E][E_1] := E_2$

- Na verdade a atribuição em C não é um comando, e sim um operador, como os outros da gramática abaixo:
- Foi usada nessa gramática como numeração a precedência nos operadores na tabela a seguir:
- Ao término da execução do código de uma expressão, o resultado da mesma (e somente o resultado) deve estar no EPlace. Qual o código gerado para os operadores reduzidos abaixo?
- No caso de um "comando expressão", o valor final no Eplace é simplesmente ignorado.

Gramática (quase) completa de operadores em C

$E15 \rightarrow E15, E14 \mid E14$

$E14 \rightarrow E13 * = E14 \mid E13 / = E14 \mid E13 \% = E14 \mid$
 $E13 + = E14 \mid E13 - = E14 \mid E13 = E14 \mid E13$

$E13 \rightarrow E13 ? E13 : E13 \mid E12$

$E12 \rightarrow E12 \parallel E11 \mid E11$

$E11 \rightarrow E11 \&\& E7 \mid E7$

- Obs: O operador vírgula, do nível E15, é um operador que avalia o primeiro operando e descarta o resultado, e avalia o segundo operando e retorna seu resultado.
- O uso da vírgula como operador é distinto de seu uso como separador de listas de elementos como parâmetros de funções e variáveis em uma declaração.
- Ex: Quanto vale 'a' após ser executado o comando "a=2,3;" ?
- E "a = (2,3);" ?

$E7 \rightarrow E7 == E6 \mid E7 != E6 \mid E6$

$E6 \rightarrow E6 > E4 \mid E6 >= E4 \mid E6 < E4 \mid E6 <= E4 \mid E4$

$E4 \rightarrow E4 + E3 \mid E4 - E3 \mid E3$

$E3 \rightarrow E3 * E2 \mid E3 / E2 \mid E3 \% E2 \mid E2$

$E2 \rightarrow -F \mid +F \mid F$

$F \rightarrow id \mid cte \mid (E14)$

Tabela de precedência de operadores em C

Prec.	Símbolo	Função	Associatividade
1	++ --	Incremento e decremento pósfixo	esquerda para direita
1	()	Parênteses (chamada de função)	
1	[]	Elemento de array	
1	.	Seleção de campo de structure	
1	->	Seleção de campo de structure a partir de ponteiro	
2	++ --	Incremento e decremento prefixo	direita para a esquerda
2	+ -	Adição e subtração unária	
2	! ~	Não lógico e complemento	
2	(tipo)	Conversão de tipo de dado	
2	*	Desreferência	
2	&	Referência (endereço de elemento)	
3	* / %	Multiplicação, divisão, e módulo (resto)	esquerda para a direita
4	+ -	Adição e subtração	
5	<< >>	Deslocamento de bits à esquerda e à direita	
6	< <=	"menor que" e "menor ou igual que"	
6	> >=	"maior que" e "maior ou igual que"	
7	= = !=	"Igual a" e "diferente de"	
8	&	E bit a bit	
9	^	Ou exclusivo para bits	
10		Ou para bits	
11	&&	E lógico	
12		Ou lógico	
13	c ? t : f	Condição ternária	direita para esquerda
14	=	Atribuição	
14	+= -=	Atribuição por adição ou subtração	
14	*= /= %=	Atribuição por multiplicação, divisão ou módulo (resto)	
14	<<= >>=	Atribuição por deslocamento de bits	
14	&= ^= =	Atribuição por operações lógicas	
15	,	vírgula	esquerda para direita

- O C3E para estruturas de controle é gerado a partir da combinação dos elementos que o compõem e mais algumas instruções de desvios condicionais, incondicionais e labels.
- O comando "if (E) Com1 else Com2" poderia ser implementado com a seguinte estrutura:

```
E.cod // avalia a condição
if E.place==0 goto lbelse
Com1.cod // comandos do "then"
goto lbsaida
lbelse:
Com2.cod // comandos do "else"
lbsaida:
```

- Implementada pelo esquema de tradução a seguir:

```
com_cond → if E then Com else Com2 { Final := Geralabel  
Else := Geralabel  
ComCond.cod := E.cod ||  
gen("if" , E.place, "=", "0", "goto", Else) ||  
com.cod ||  
gen("goto", final) ||  
gen("else", ":") ||  
com2.cod ||  
gen("final", ":") }
```

- E o comando "if (a>b) a=c; else b=c;" seria traduzido para:

T1=a>b

if T1==0 goto LB01

a=c

goto LB02

LB01:

b=c

LB02:

- O GIMPLE utiliza a forma abaixo de desvio condicional:
if (a oprel b) goto <LBtrue>; else goto <LBFalse>;
- E o código gerado para o mesmo comando "if (a>b) a=c; else b=c;" é

```

    if (a > b) goto <D.1397>; else goto <D.1398>;
<D.1397>: // início do bloco do then
    a = c;
    goto <D.1399>;
<D.1398>: // início do bloco do else
    b = c;
<D.1399>: // saída do if

```

- E os labels seguem a mesma notação <D.YYYY> das variáveis temporárias.

- E o gimple gerado para $a=b>c?1:2$ é:
if (b > c) goto <D.1398>; else goto <D.1399>;
<D.1398>:
iftmp.0 = 1;
goto <D.1400>;
<D.1399>:
iftmp.0 = 2;
<D.1400>:
a = iftmp.0;
- Havendo um contador de temporários de ternários no padrão iftmp.nnn.

- O comando "while (E) Comando" poderia ser implementado com a seguinte estrutura:

lblaço:

```
E.cod // avalia a condição  
if E.place==0 goto lbsaida  
Comando.cod  
goto lblaço
```

lbsaida:

- Implementado pelo esquema de tradução a seguir:

```
com_while → while E do Com { Final := Geralabel  
Início := Geralabel  
ComWhile.cod := gen(início,":") ||  
E.cod ||  
gen("if" , E.place, "=", "0", "goto", Final) ||  
com.cod ||  
gen("goto", início) ||  
gen(final,":") }
```

- No Gimple, o código "while (a<10) a++" é traduzido para:
goto <D.1397>;
<D.1396>:
a = a + 1;
<D.1397>:
if (a <= 9) goto <D.1396>; else goto <D.1398>;
<D.1398>:

```
com_repeat → repeat Lcom until E { Inicio := Geralabel  
Fim := Geralabel  
Com_repeat.cod := gen(inicio,":") ||  
Lcom.cod ||  
E.cod ||  
gen("if" , E.place, "<>", "0", "goto", Fim) ||  
gen("goto", inicio) ||  
gen(fim, ":") }
```

```

com_for → for id := E to  $E_2$  do com { Inicio := Geralabel
Fim := Geralabel
Com_for.cod := E.cod ||
gen(´:=´, id.place, E.place) ||
gen(inicio, ´:´) ||
 $E_2$ .cod ||
gen(´if´, id.place, ´>´,  $E_2$ .place, ´goto´, fim) ||
com.cod ||
gen(´+´, id.place, id.place, 1) ||
gen(´goto´, inicio) ||
gen(fim, ´:´) }

```

```

com_for_C → for (  $E_1$  ;  $E_2$  ;  $E_3$  ) {Fim := Geralabel
Comando.break=Fim; LbContinue:=Geralabel;
Comando.continue=LbContinue} Comando
{LbLaco := Geralabel
Com_for.cod :=  $E_1$ .cod || gen(LbLaco:) ||  $E_2$ .cod ||
gen('if' ,  $E_2$ .place, '=0', 'goto',Fim) ||
com.cod ||
gen(LbContinue:) ||  $E_3$ .cod ||
gen('goto',inicio) ||
gen(fim,':')}

```

- O gimple gerado para o comando
for (i=0;i<10;i++) printf("%d",i)

- é

```
i = 0;
```

```
goto <D.1838>;
```

```
<D.1837>:
```

```
printf ("%d", i);
```

```
i = i + 1;
```

```
<D.1838>:
```

```
if (i <= 9) goto <D.1837>; else goto <D.1839>;
```

```
<D.1839>:
```

- O for aceita uma condição vazia que é interpretada como true, e usada em "laço infinito". O código gerado para
for (i=0;;i++) printf("%d",i)

- é

i = 0; // bloco de inicialização

<D.1837>:

printf ("%d", i); // corpo do for

i = i + 1; // bloco de incrementos

goto <D.1837>;

- O comando **continue** é traduzido para um desvio para o início do bloco de incrementos (se houver). O trecho:

```
for (i=0;;i++)  
    {if (i<5) continue;  
     printf("%d",i);}
```

- é traduzido para

```
i = 0;  
<D.1838>:  
if (i <= 4) goto <D.1840>; else goto <D.1841>;  
<D.1840>: // label do "then no if"  
goto <D.1837>; // tradução do continue  
<D.1841>: // label da saída do if (ou else)  
printf ("%d", i);  
<D.1837>: // label do bloco de incrementos  
i = i + 1;  
goto <D.1838>;
```

Com → Com_Composto

Com → Com_If

Com → Com_While

Com → Com_Do

Com → Com_For

Com → Exp;

- Que poderia ser implementado como:

```
if (token==TK_Ab_Chaves) return Com_Comp(Com_c)
```

```
if (token==TK_If) return Com_If(Com_c);
```

```
if (token==TK_While) return Com_While(Com_c);...
```

```
return 0;
```

- Por que a atribuição e chamadas de função não estão na lista de comandos acima?

Comando Composto

$\text{Com_Comp} \rightarrow \{ \text{Lista_Comandos} \}$
 $\text{Lista_Comandos} \rightarrow \text{Com Lista_Comandos}$
 $\text{Lista_Comandos} \rightarrow \epsilon$

```
if (tk==TK_Abre_Chaves)
{
    strcpy(Com_Comp_c, "");
    tk=le_token();
    while (tk!=TK_Fecha_Chaves)
        if (Com(Com_c))
            strcat(Com_Comp_c, Com_c);
        else return 0;
    tk=le_token(); // consome o Fecha Chaves
    return 1;
}
```

$\text{com_for} \rightarrow \text{for } (E_1 ; E_2 ; E_3) \{ \text{comandos} \}$

$\text{com} \rightarrow \text{continue}$

$\text{com} \rightarrow \text{break}$

$E \rightarrow E_1 ? E_2 : E_3$

$E \rightarrow E_1 ?? E_2$

Com $\rightarrow$ com_atrib | com_repeat | com_while | com_if

Case expressão of

Var 1: comando;

Var2, var3 : comando;

...

else comando;

end;

```
switch (E){  
    case  $L_1$ : comandos1  
    case  $L_2$ : comandos2  
    default: comandos3  
    case  $L_3$ : comandos4  
}
```

- Os labels $L_1, L_2 \dots$ devem ser constantes.
- A entrada default pode estar em qualquer ponto da lista e é executada se a expressão não for igual a nenhum dos labels
- Ao entrar em um dos cases, a execução segue através das listas de comandos até o final de todos os cases ou até executar um break.
- A lista de comandos de cada entrada pode ser vazia
- A lista de cases pode ser vazia

- Uma forma de implementar o **switch** é agrupando os blocos de código das diversas entradas do case e os blocos de teste em dois blocos disjuntos, gerando para o exemplo abaixo, para o código a seguir, o código de máquina de pilha mostrado na lâmina seguinte:

```
switch (a) {  
    case 1: comando1;  
    case 2: comando2;  
    case 3: comando3;  
}
```

```
if a==1 goto L1  
if a==2 goto L2  
if a==3 goto L3  
goto fim
```

L1: comando 1

L2: comando 2

L3: comando 3

fim:

- E uma gramática para gerar o código acima é:

cmd $\rightarrow$ **switch** (E) { LCases }

LCases $\rightarrow$ Case LCases

LCases $\rightarrow$ ϵ

Case $\rightarrow$ **case** cte : LCom

- E um início de ações semânticas é:
 - **cmd** $\rightarrow$ **switch** (E) { {LCases.p=E.place} LCases} {
 LBFim=geralabel(); cmd.cod = E.cod || LCases.tst ||
 gen(goto, LBFim) || LCases.cod}
 - LCases $\rightarrow$ **case** cte : LCom {***} LCases₁
 {Lcases.cod=Case.cod || Lcases₁.cod}
 - LCases $\rightarrow \epsilon$
 *** LB=geralabel();LCases1.p=LCases.p;
 LCases1.cod=LCases.cod||gen(label,LB)||LCom.cod;
 LCases1.tst=LCases.tst||gen(if==,LCases.p,cte,LB)

- Qual o código gerado para o comando (considerando a numeração correta dos labels e variáveis temporárias)?

```
switch (a) {  
    case 1: a=1;  
    case 2: b=b+1;  
    case 3: a=a+1;  
           b=b+2;  
    default: a=3;  
}
```

Lista de tópicos para a primeira prova

- Definição de esquemas de tradução em geral (avaliação de expressões, conversões)
 - Definição de atributos herdados e sintetizados
 - Anotação de árvore de derivação
 - Árvore de sintaxe
 - Verificação de tipo
- Implementação de esquemas de tradução em analisadores descendentes recursivos
 - Remoção de recursão de produções com ações semânticas
 - Representação de atributos como parâmetros de entrada ou saída e variáveis locais
- Geração de código de pilha para expressões e estruturas de controle
- Geração de C3E para :
 - Expressões
 - Estruturas de Controle
 - Referência a vetores e matrizes

- Uma forma de implementar um comando condicional é através da avaliação completa da expressão de controle e utilização de instruções de desvio do tipo `If...goto`.

```
If (a>3) or (b<2) or (c>3)
then a:=3
else a:= 4
```

E
if E.place=false goto Lelse
a:=3 goto fim
Lelse: a:=4
fim:

- Em algumas situações não é necessário avaliar completamente a expressão para obter o resultado da mesma.
- Em uma expressão contendo o "OR" de duas subexpressões, se a primeira subexpressão é verdadeira, a expressão é verdadeira independentemente do resultado da segunda subexpressão, não sendo necessário portanto, avaliá-la.
- Essa avaliação parcial é chamada avaliação short-circuit e é a avaliação usada por definição da linguagem C, o que torna seguro o uso de expressões como:
if (ptaux==NULL || ptaux->valor<10)
- Em algumas linguagens é feita a avaliação completa da expressão, em outras isso é configurado no compilador.

- O código (teste.c) abaixo:

```
if (!(i>0 && i>1 || i==3))
```

```
    a=1;
```

```
else
```

```
    a=2;
```

- gerado pelo gcc no passo 4 (teste.c.004t.gimple) é:

```
if (i <= 0) goto <D.1839>; else goto <D.1842>;
```

```
<D.1842>:
```

```
if (i <= 1) goto <D.1839>; else goto <D.1840>;
```

```
<D.1839>:
```

```
if (i != 3) goto <D.1843>; else goto <D.1840>;
```

```
<D.1843>:
```

```
a = 1;
```

```
goto <D.1841>;
```

```
<D.1840>:
```

```
a = 2;
```

```
<D.1841>:
```


Eor $\rightarrow$ { *Eor*₁.true := Eor.true ;
 Lb := Geralabel
 *Eor*₁.false := Lb } *Eor*₁ ||
 { *Eand*.true := Eor.true
 Eand.false := Eor.false } *Eand*
 { Eor.cod := *Eor*₁.cod || gen (label LB) || *Eand*.cod }
 Eor $\rightarrow$ { *Eand*.true := Eor.true ; *Eand*.false := Eor.false } *Eand*
 { Eor.cod := *Eand*.cod }

$E_{and} \rightarrow \{ E_{and_1}.false := E_{and}.false;$
 $LB := \text{geralabel};$
 $E_{and_1}.true := LB \} E_{and_1} \ \&\&$
 $\{ E_{rel}.true := E.true;$
 $E_{rel}.false := E.false \} E_{not}$
 $\{ E_{and}.cod := E_{and_1}.cod \parallel \text{gen (label LB) } \parallel E_{rel}.cod \}$

$E_{and} \rightarrow \{ E_{rel}.true := E_{and}.true ; E_{rel}.false := E_{and}.false \} E_{rel}$
 $\{ E_{and}.cod := E_{rel}.cod \}$

- Operadores relacionais também podem ser implementados como short-circuit. Uma possível produção para isso, quando os operadores relacionais são não associativos (o que obriga que os operandos sejam expressões numéricas) seria:

$E_{\text{Rel}} \rightarrow E_1 > E_2$ $\{ E_{\text{Rel}}.\text{cod} := E_1.\text{cod} \parallel E_2.\text{cod} \parallel$
 $\text{gen}(>, E_1.\text{place}, E_2.\text{place}, E_{\text{Rel}}.\text{true}) \parallel$
 $\text{gen}(\text{goto}, E_{\text{Rel}}.\text{false}) \}$

$E_{\text{Rel}} \rightarrow E$ $\{ E_{\text{Rel}}.\text{cod} = E.\text{cod} \}$

- Se os operadores relacionais são associativos, os primeiros são avaliados para valores numéricos, e o último é avaliado de forma short-circuit. Isso é implementado criando dois níveis de operadores relacionais. Os primeiros avaliados são os mais à esquerda, de forma numérica, e o último é avaliado de forma short-circuit.
- No esquema de tradução abaixo, ERelS deriva expressões relacionais por short-circuit, e na chamada deve receber .true e .false, e ERelA deriva expressões relacionais que retornam em .place um valor numérico.

$$\begin{aligned} \text{ERelS} \rightarrow \text{ERelA} > E & \quad \{ \text{ERel.cod} := E_1.\text{cod} \parallel E_2.\text{cod} \parallel \\ & \quad \text{gen}(>, E_1.\text{place}, E_2.\text{place}, \text{ERel.true}) \parallel \\ & \quad \text{gen}(\text{goto}, \text{ERel.false}) \} \\ \text{ERelS} \rightarrow \text{ERelA} & \quad \{ \text{ERel.cod} = E.\text{cod} \} \end{aligned}$$

$E\text{Rel}A \rightarrow E\text{Rel}A_1 > E$	<pre> { ERelA.place=criatemp; Lbtrue=geralabel; Lbfim=geralabel;ERelA.cod := ERelA₁.cod E.cod gen(>,ERelA₁.place,E.place,Lbtrue) gen(=,ERelA.place,0) gen(goto, Lbfim)} gen(:,Lbtrue) gen(=,ERelA.place,1) gen(:,Lbfim)} </pre>
$E\text{Rel}A \rightarrow E$	<pre> {ERelA.cod=E.cod; ERelA.place=E.place} </pre>

- Em algumas situações operadores lógicos devem retornar valores numéricos como, por exemplo, no comando:
 - `a = b && c;`
- Para isso pode-se diferenciar a expressão lógica short-circuit e a expressão lógica que resulta valor numérico, da mesma forma que foi feito para os operadores relacionais.
- Qual tipo de expressão seria chamada depende do contexto em que a expressão aparece.
- Expressões lógicas short-circuit seriam chamadas quando ocorrerem em condições de comandos condicionais ou de repetição.

Um esquema reduzido de short-circuit

ComIf $\rightarrow$ If {EA.true=geralabel(); EA.false=geralabel();}
(EA) Com₁ else Com₂{ LbFim=geralabel();
ComIf.cod=EA.cod+gen(rotulo EA.true)+
Com₁.cod+gen("goto" Lbfim)+gen(rotulo EA.false)+
Com₂.cod+gen(rotulo Lbfim);}
EA $\rightarrow$ {LBaux=geralabel(); EA₁.true=LBaux; EA₁.false=EA.false}
EA₁ && {ER.true=EA.true; ER.false=EA.false;}ER
{EA.cod=EA₁.cod+gen(rótulo LBaux)+ER.cod}
EA $\rightarrow$ {ER.true=EA.true; ER.false=EA.false}ER{EA.cod=ER.cod}
ER $\rightarrow$ EN₁ > EN₂ {ER.cod=EN₁.cod+EN₂.cod+
gen("if"+EN₁.place+">" +EN₂.place+
" goto " ER.true)+gen("goto " ER.false)}
EN $\rightarrow$ id {EN.place=id}

- obs:

- EA - Expressão AND
- ER - Expressão relacional
- EN - Expressão numérica

- Removendo a recursão da produção de A:

EA $\rightarrow$ {LBaux=geralabel(); EA₁.true=LBaux; EA₁.false=E.false}
 EA₁ && {ER.true=EA.true; ER.false=EA.false;} ER
 {EA.cod=EA₁.cod+gen(rótulo LBaux)+ER.cod}
 EA $\rightarrow$ {ER.true=EA.true; ER.false=EA.false} ER {EA.cod=ER.cod}

- Resulta em:

EA $\rightarrow$ {ER.true=geralabel(); ER.false=EA.false;}
 ER {EAL.true=EA.true; EAL.false=EA.false;} EAL
 {EA.cod=ER.cod+gen(rotulo ER.true)+EAL.cod}
 EAL $\rightarrow$ && {ER.true=geralabel(); ER.false=EAL.false;} ER
 {EAL₁.true=EAL.true; EAL₁.false=EAL.false;} EAL₁
 {EAL.cod=ER.cod+gen(rotulo ER.true)+EAL₁.cod}
 EAL $\rightarrow$ € {EAL.cod=gen(goto EAL.true)}

- Na linguagem C, uma expressão é um comando, de modo que:
 - a
 - $a+1$
 - $\text{fat}(n)$
- são comandos válidos, e o símbolo de atribuição (" $=$ ") é um operador, de modo que $a=b=c=1$ é uma expressão válida. Observe que o operador $=$ exige como operador à esquerda um identificador, de modo que $3=4=5$ não é uma expressão válida. Observe também que a prioridade da atribuição é menor que os operadores aritméticos, de modo que a expressão $a+b=c$ não é válida.
- Faça um esquema de tradução para reconhecer e gerar C3E para atribuição e expressões contendo o operador de atribuição, soma, identificadores, constantes e parênteses, como, por exemplo:
 - $a=3+(b=2)$

- $A \rightarrow \text{id} = A_1$
- $A \rightarrow E$
- $E \rightarrow E_1 + T$
- $E \rightarrow T$
- $T \rightarrow \text{id}$
- $T \rightarrow \text{cte}$
- $T \rightarrow \text{id}++$

- Em uma declaração são atualizados na tabela de símbolos o tipo e o endereço da variável.
- Este endereço pode ser o deslocamento em um bloco de memória alocada para variáveis globais ou locais de uma procedure.

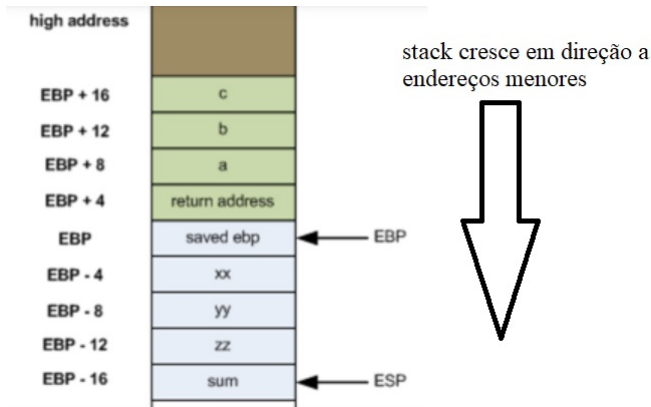
- No esquema a seguir a função Enter (nome, tipo, offset) cria uma entrada na tabela de símbolos com nome, tipo e endereço relativo da variável.

```
P → {offset:=0} D
D → id : T { enter (id.nome,T.type,offset)
           Offset := offset + T.width }
T → integer { T.type := integer
             T.width := 4 }
T → array [ num ] of T1
    { T.type := array(num_val, T1.type)
      T.width := num_val * T1.width }
```

- A chamada, execução e retorno de uma função envolve diversos passos na geração do código.
- A troca de valores entre o ponto de chamada e a função pode ser feita através da pilha ou de registradores, definidos em padrões específicos para cada arquitetura.
- Ao ser chamada uma função uma série de ações deve ser executada:
 - Deve ser reservado espaço para o valor de retorno da função (se for retornado pela pilha)
 - Os parâmetros devem ser avaliados e empilhados (ou colocados em registrador)
 - O endereço de retorno deve ser salvo
 - O fluxo de controle deve desviar para o início da função (a função deve ser chamada)

- O espaço na pilha utilizado para a comunicação com a função se chama **Registro de Ativação (stack frame)**
- Ele é um bloco de memória alocado na chamada de uma função e normalmente é formado pelos seguintes campos:
 - 1 Valor de retorno
 - 2 Parâmetros atuais
 - 3 Link de controle (endereço de retorno)
 - 4 Link de acesso (aponta para o registro de ativação do procedimento externo, é utilizado em procedimentos aninhados)
 - 5 Estado da máquina (estado dos flags)
 - 6 Dados locais (variáveis locais e parâmetros)
 - 7 Temporários (variáveis criadas com geratemp)

- a figura abaixo mostra a pilha para uma chamada da função foobar a seguir:



```
int foobar(int a, int b, int c)
{
    int xx = a + 2;
    int yy = b + 3;
    int zz = c + 4;
    int sum = xx + yy + zz;
    return xx * yy * zz + sum;
}
```

- A pilha mostra os 3 parâmetros empilhados, o endereço de retorno, o BP salvo e as variáveis locais

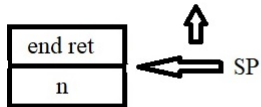
- A figura abaixo mostra o código gerado para a função à esquerda
- O Stack Pointer (SP) aponta para a última posição **ocupada** na pilha
- O Base Pointer (BP) será utilizado como referência para os argumentos e variáveis locais na pilha. Por ser a referência para o *stack frame* corrente, ele também é chamado de *frame pointer*.

```
x86-64 gcc 11.2  -O0 -m32

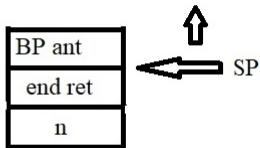
// type your code here
int square(int n) {
    int x;
    return x+n+31;
}

square:
    push    ebp
    mov     ebp, esp
    sub     esp, 16
    mov     edx, DWORD PTR [ebp-4]
    mov     eax, DWORD PTR [ebp+8]
    add     eax, edx
    add     eax, 31
    leave
    ret
```

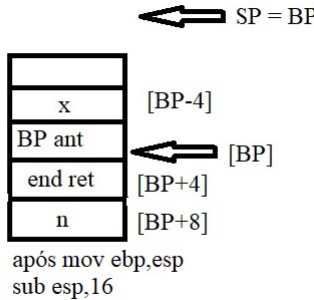
- A primeira linha do código gerado (`push ebp`) salva o EBP na pilha. O EBP até este momento aponta para o stack frame da função chamadora, e seu valor deve ser salvo para que ao encerrar a execução da função corrente, ele volte a apontar para o stack frame da função chamadora.
- Após, (`mov ebp, esp`) o BP passa a apontar para o stack frame corrente, em cujo topo está o BP salva.
- Acima (`[BP-4]`, `[BP-8]`) ficarão as variáveis locais.
- Em `[BP+4]` está o endereço de retorno da função
- Abaixo (`[BP+8]`, `[BP-12]`) estão os parâmetros passados pela pilha



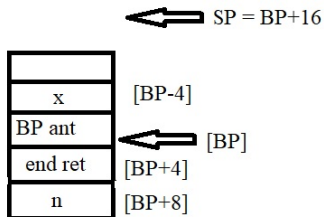
Ao entrar
na função



Após push ebp



- A figura abaixo mostra o estado da pilha após executar o código de entrada da função.



- A linha seguinte (`sub esp,16`) reserva na pilha espaço para as variáveis locais (no caso, `x`). Essa alocação é feita em blocos de 16 bytes, de modo que se forem necessários 20 bytes para variáveis locais (ou temporárias), serão alocados 32 bytes.
- As três linhas a seguir calculam a expressão $x+n+31$ colocando o resultado em `ax`, por onde será retornado.
- A instrução `leave` faz o inverso do código de entrada. Desempilha o espaço para as variáveis locais e restaura o BP anterior.

- Ao analisar uma chamada de função, os parâmetros devem ser avaliados e empilhados (se os parâmetros são passados pela pilha) ou movidos para os registradores correspondentes.
- Um padrão que define como é feita a passagem de parâmetros é chamado *calling convention* (▶ *calling convention*)
- Em compiladores C para plataformas x86-64, definido pela AMD e seguida pela Intel, os parâmetros são passados por registradores, bem como o valor de retorno da função.

- O **Compiler Explorer** é uma plataforma que permite gerar o código objeto de programas em mais de 30 linguagens, para diferentes plataformas, compiladores, versões e opções de compilação, o que permite analisar diversas características dos compiladores e das linguagens.

- A figura abaixo mostra o código gerado para a chamada de função à esquerda, pelo gcc x86-64, versão 10.3, mas com o flag de compilação -m32, para gerar código de 32 bits.
- Observa-se que o parâmetro 5 é passado pela pilha (push 5) e o valor de retorno é retornado pelo registrador eax.

x86-64 gcc 10.3 -O0 -m32

<pre>int main() { int a=square(5); }</pre>	<pre>14 push 5 15 call square 16 add esp, 4 17 mov DWORD PTR [ebp-4], eax 18 mov eax, 0 19</pre>
--	---

- A figura abaixo mostra o mesmo código, mas sem o flag -m32. No código para 64 bits o compilador segue uma convenção diferente e o parâmetro é passado pelo registrador DI, e não pela pilha.
- Ao compilar com a opção -O3 setada (máxima otimização), o compilador identifica que o resultado da função não é usado, e não gera código para a chamada.

x86-64 gcc 10.3 -O0

```
int main()
{
    int a=square(5);
}
```

```
mov     edi, 5
call    square
mov     DWORD PTR [rbp-4], eax
mov     eax, 0
```

- Ao acrescentar-se mais parâmetros à função, o código gerado para 64 bits para a chamada `square(1,2,3,4,5,6,7,8,9,10)` é o código abaixo. Os primeiros seis parâmetros são passados por registradores, e os restantes pela pilha. Para 32 bits, os 10 parâmetros são passados pela pilha.

```
push    10
push    9
push    8
push    7
mov     r9d, 6
mov     r8d, 5
mov     ecx, 4
mov     edx, 3
mov     esi, 2
mov     edi, 1
call    square
add     rsp, 32
mov     DWORD PTR [rbp-4], eax
mov     eax, 0
```

- A convenção acima se chama *Application Binary Interface* e é utilizada para arquiteturas de 64 bits.
- Parâmetros float são passados pelos registradores SSE (xmm0, xmm1...)
- O retorno de valores da função é feito pelos registradores RAX e RDX, para valores inteiros, e XMM0 e XMM1 para valores float.
- Se o valor de retorno não cabe em registradores (p.ex. uma struct) é passado pelo DI o endereço da struct a receber o valor de retorno.
- A seleção final das instruções que serão utilizadas para a passagem dos parâmetros é feita na fase de seleção de instruções na geração de código objeto, uma vez que depende da arquitetura.

Call rotulo

$a := pilha[i]$

$pilha[i] := a$

registradores : SP - Aponta para a última posição utilizada na pilha
 BP - Aponta para o registro de ativação corrente

- Ao reconhecer uma chamada de função o código gerado deve executar as seguintes funções:
 - Se a função é uma função com tipo, que retorna um valor, e o padrão de chamada especifica que o retorno da função é pela pilha, deve ser reservado espaço na pilha para a função colocar o valor de retorno. Isso pode ser feito decrementando o SP. Se a função vai retornar um inteiro de 4 bytes, $SP = SP - 4$ reserva 4 bytes na pilha para o valor de retorno.
 - Se a função tem parâmetros, o valor de cada parâmetro deve ser avaliado e empilhado (push n) ou colocado nos registradores apropriados.
 - Deve ser efetuada a chamada da função, que automaticamente empilha o valor de retorno (call funcao)

- Após efetuar a chamada da função o código gerado deve executar as seguintes funções:
 - Se foram empilhados parâmetros antes de efetuar a chamada, eles devem ser desempilhados. Isso pode ser feito por $SP = SP + X$ onde X é o tamanho total dos parâmetros que foram empilhados.
 - Se a função retornou algum valor, o valor de retorno deve ser desempilhado para um temporário (p.ex: `pop T0`). Ou, se o padrão especificar, deve ser gerada uma instrução para mover o valor de retorno do registrador apropriado para o seu ponto de uso.

- Ao reconhecer a declaração de uma função, o código gerado deve efetuar as seguintes operações:
 - Salvar na pilha o valor atual do registrador BP (push BP)
 - $BP=SP$, a partir daí BP passa a ser a referência para acessar na pilha os parâmetros, valor de retorno e variáveis locais
 - Reservar na pilha espaço para as variáveis locais ($SP=SP-tam$), onde tam é o tamanho total das variáveis locais

- O código de saída da função deve efetuar as seguintes operações:
 - Atualizar na pilha o valor de retorno da função (se for uma função com tipo e o padrão especificar retorno pela pilha) $[BP+...]=\text{valor de retorno}$. Ou mover o valor de retorno para o registrador apropriado.
 - Restaurar BP (pop BP)
 - Retornar (return)

Exemplo:

```
int soma(int a,int b)
{
    int s;
    s=a+b;
    return s;
}
...
x=soma(2,5)
```

	$\leftarrow$ SP
S	
BP	$\leftarrow$ BP
end. ret.	
b (=5)	
a (=2)	
val. ret.	

- Assumindo que as posições da pilha tenham o mesmo tamanho das variáveis e parâmetros inteiros e floats (4 bytes):
 - o valor de retorno será referenciado como [BP+16]
 - o primeiro parâmetro (a) como [BP+12]
 - o segundo parâmetro (b) como [BP+8]
 - a variável local s como [BP-4]
 - a segunda variável local (se existisse) como [BP-8]

	←SP
S	←BP
BP	
end. ret.	
b (=5)	
a (=2)	
val. ret.	

- E o código gerado para a função e na chamada seria:

```
int soma(int a,int b)
{
    int s;
    s=a+b;
    return s;
}
...
x=soma(2,5)
```

	←SP
S	←BP
BP ant	
end. ret.	
b (=5)	
a (=2)	
val. ret.	

Soma:

```
push _BP
_BP = _SP
_SP = _SP-4
T0=[_BP+16]
T1=[_BP+12]
T2 = T0+T1
[_BP] = T2
T3 = [_BP]
[_BP+20] = T3
_SP = _SP+4
pop _BP
ret
...
...
_SP = _SP-4
push 2
push 5
call soma
_SP = _SP+8
pop x
```

C3E comentado:

Soma:

```
push _BP      : Salva o valor anterior do BP na pilha
_BP = _SP     : BP passa a apontar para o stack frame atual
_SP = _SP-4   : Reserva espaço para a variável local S
T0=[_BP+16]   : Busca o valor de a
T1=[_BP+12]   : Busca o valor de b
T2 = T0+T1    : S = a+b;
[_BP] = T2    : coloca o resultado da soma em S na pilha
T3 = [_BP]    : busca S para colocar no valor de retorno
[_BP+20] = T3 : coloca S no valor de retorno
_SP = _SP+4   : "desaloca" espaço das variáveis locais
pop _BP       : restaura BP
ret           : retorna, isto é, desempilha o endereço de retorno e desvia para ele
...
...
_SP = _SP-4   : reserva espaço para o valor de retorno
push 2        : empilha o primeiro parâmetro
push 5        : empilha o segundo parâmetro
call soma     : chama a função
_SP = _SP+8   : "desaloca" parâmetros
pop x         : desempilha o valor de retorno
```

- Gramática para chamada de função:

$E \rightarrow \text{id (LP)}$

$\{E.\text{place} = \text{criatemp}; E.\text{cod} = "SP = SP + 4" + LP.\text{cod} + "call"$
 $\text{id} + "SP = SP + " LP.\text{tam} + "pop" E.\text{place}\}$

$LP \rightarrow E \text{ RLP}$

$LP \rightarrow \epsilon$

$RLP \rightarrow , LP$

$RLP \rightarrow \epsilon$

- Mostre a árvore de derivação para a chamada de função a seguir:
 - `a=fat(a+1,b);`

Passos a serem executados na análise:

- Ao iniciar uma procedure:
- Criar uma nova tabela de símbolos apontando para a anterior.
O cálculo de endereço para variáveis locais é diferente do cálculo para argumentos. Criar um bloco de variáveis locais e um bloco de argumentos.
- $\text{End_VL} \leftarrow 0$
- $\text{End_Args} \leftarrow 0$

Ao analisar argumentos e variáveis locais:

Para cada um colocar na tabela de símbolos:

- Tipo
- Endereço
- Nome

Ao sair da análise da procedure, a tabela de símbolos correspondente é eliminada e a tabela anterior passa a ser a tabela corrente.

- Ao analisar chamada:
 - Gerar código para avaliar argumentos
 - Empilhar argumentos
 - Gerar código para chamada
 - Gerar código para desempilhar argumentos.

- As técnicas de geração de código frequentemente produzem código objeto ou intermediário com instruções redundantes e construções sub-ótimas.
- Isso se deve à forma como é gerado o código intermediário, em que pequenos trechos de código são gerados independentemente do contexto
 - p.ex. a expressão $a=b+1$ poderia ser gerada como
$$\begin{aligned}T1 &= 1 \\ T2 &= b + T1 \\ a &= T2\end{aligned}$$

[Voltar para o índice](#)

- A qualidade pode ser melhorada aplicando transformações que se não tornam o código ótimo, podem melhorá-lo bastante.
- Uma razão para não gerar um código ótimo, é que a definição de ótimo é relativa, podendo ser o código que execute o mais rapidamente possível, ou o menor código possível.
- Outra razão é que algumas análises tem um custo computacional alto (envolvendo problemas np-completos).
- Por essa razão os compiladores usam flags de compilação para definir as prioridades na compilação.

- No gcc, o flag -O0 a -O3 define 4 níveis de otimização. O nível -O0 não implementa otimizações, os outros 3 implementam mais e mais otimizações, com um custo maior de compilação.
- A lista completa de flags de compilação pode ser vista com:
 - gcc - -help=optimizers
- A otimização feita pelo compilador não tornará eficiente um algoritmo ou estrutura de dados ineficiente.
- Para obter um código de boa qualidade o primeiro passo é a escolha correta do algoritmo e estrutura de dados apropriados.

[Voltar para o índice](#)

- Algumas análises tem um custo computacional alto e de modo geral não são disponibilizadas em compiladores, mas programas chamados analisadores estáticos de código (static code analysis) fazem algumas dessas análises.
- O Cppcheck é um ferramenta (que pode ser adicionado como plugin ao vscode ou codeblocks para fazer essas análises).

- Aplicado ao código abaixo:

```
int a[5],i;  
for (i=0;i<=5;i++) a[i]=0;
```

- Emitiu a mensagem:
error : Array 'a[5]' accessed at index 5, which is out of bounds.

- As otimizações feitas pelo compilador podem ser independentes da máquina, feitas sobre o código intermediário ou dependentes da máquina, feitas sobre o código final pelo back-end do compilador.
- A geração de código objeto de qualidade inclui tópicos como
 - Alocação de registradores
 - Ordem de execução de instruções para melhor utilizar os recursos do hardware (pipelines, unidades funcionais múltiplas, múltiplos cores)
 - Uso de instruções especiais (processadores vetoriais, instruções de movimentação de blocos no x86...)

- A maior parte das otimizações é feita sobre o código intermediário.
- Algumas transformações que podem levar a um código melhor são:
 - Eliminação de instruções redundantes
 - Otimizações no fluxo de controle
 - Simplificações algébricas
 - Uso de instruções específicas de máquina

- a) **Movimentações redundantes**

- Na seqüência de instruções

Mov R0,A

Mov A,R0

- Se não há nenhum label entre as duas instruções é desnecessário a segunda.

b) Código Inalcançável : Código após um jump incondicional como o gerado por um break ou return.

c) Otimizações no fluxo de controle : Se há desvios sobre desvios como a sequência a seguir :

```
If debug=1 goto L1
```

```
Goto L2
```

```
L1: —
```

```
—
```

```
L2: —
```

```
—
```

Equivale a

```
If debug<>1 goto L2
```

```
L1: —
```

```
L2: —
```

Desvios para desvios podem ser substituídos:

goto L1 goto L2

Ex: ... ⇒ ...
L1: goto L2 L1: goto L2
 (a) (b)

Se no caso (b) antes de L1 houver um desvio incondicional e não houver desvios para L1 a instrução L1:goto L2 pode ser eliminada. Isto pode ser obtido mantendo-se um contador de desvios para cada label.

d)Simplificação algébrica

- Partes de expressões algébricas podem ser resolvidas durante a compilação.
- De modo geral operações entre constantes ou operações entre variáveis e constantes que possam ser simplificadas. Algumas são:
- $x := x + 0 \Rightarrow 0$ 0 é o elemento neutro da soma
- $x := x * 1 \Rightarrow 1$ 1 é o elemento neutro da multiplicação
- $x := x * 0 \Rightarrow 0$ 0 é o que?
- E expressões entre constantes que podem aparecer no código porque estavam no código fonte (às vezes por legibilidade) ou por terem aparecido como resultado de outras transformações
- Ex : DDS para gerar derivada da expressão: $3 * x + 4 * x$ gera
- $(3 * 1 + x * 0) + (4 * 1 + x * 0) \Rightarrow 3 + 4 \Rightarrow 7$

- **e) Redução de esforço**

- É a substituição de operações complexas por operações mais simples. Ex: x^2 é mais fácil de implementar como $x*x$ do que como uma chamada a uma rotina de potenciação e um produto por uma potência de 2 é mais fácil de implementar por um deslocamento do que por produto.

- **f) Uso de instruções específicas da máquina**

- A detecção de situações que permitem utilizar instruções específicas pode melhorar o código gerado. Ex: Uso de instruções de string com auto incremento no 8086.

- g) Remoção do código independente do laço para fora do laço
- A análise de laços tem um papel importante na otimização. Programas passam a maior parte do seu tempo de execução dentro de laços, e pequenas reduções no número de instruções pode ter impacto significativo na redução do tempo total.

Ex: Otimização do código gerado para um quicksort.

```
int a[tam]
void qs (m,n) {
    int i,j,v,x;
    if (n<=m) return;
    i = m-1;
    j = n;
    v = a[n];
    while (1){
        do i=i+1; while (a[i]<v);
        do j=j-1; while (a[j]>v);
        if (i>=j) break;
        x = a[i];
        a[i] = a[j];
        a[j] = x;
    }
    x = a[i];
    a[i] = a[n];
    a[n] = x;
    qs(m,j);
    qs(i+1,n);
}
```

Código Gerado (considerando que cada inteiro ocupa 32 bits):

```
if n<m goto fim
i:=m-1
j:=n
T1:=n*4
V:=a[T1]
While:
Do:
i:=i+1
T2:=i*4
T4:=a[T2]
If T4 < v goto Do
Do1:
j := j - 1
T3 := j * 4
T5 := a[T3]
If T5 > v goto Do1
If i>=y goto fimwhile
T6 := i * 4
```

```
x := a[T6]
T7 := j * 4
T8 := a[T7]
T9 := i * 4
a[T9] := T8
T10 := j * 4
a[T10] := x
goto While
fimwhile :
T11:= i * 4
X:= a[T11]
T12 := n * 4
T13 := a[T12]
T14 := i * 4
A[T14] := i * 4
T15 := n * 4
A[T15] := x
```

Código sem os labels:

```
1. if n<m goto 27
2. i:=m-1 Início do Bloco 1
3. j:=n
4. T1:=n*4
5. V:=a[T1]
6. i:=i+1 Início do Bloco 2
7. T2:=i*4
8. T3:=a[T2]
9. If T3 < v goto 6
10. j := j - 1 Início do Bloco 3
11 T2 := j * 4
12 T3 := a[T2]
13 If T3 > v goto 10
```

```
14 If i>=y goto 22 Bloco 4
15. T2 := i * 4 Início do Bloco 5
16. x := a[T2]
17. T3 := j * 4
18. T4 := a[T3]
19. a[T2] := T4
20. a[T3] := x
21. goto 6
22. X:= a[T2] Início do Bloco 6
23. T4 := n * 4
24. T5 := a[T4]
25.A[T2] := T5
26. A[T4] := x
```


Blocos básicos e grafo de fluxo de controle

- Para algumas otimizações é necessário levar em conta as dependências entre as variáveis e instruções.
- Para isso o código deve ser convertido em um grafo de fluxo que reflete os ciclos e dependências entre os diversos blocos.
- Um bloco onde não há nenhuma instrução de desvio e para cujo interior não ocorre nenhuma instrução de desvio é chamado de bloco básico.

Como definir os blocos básicos

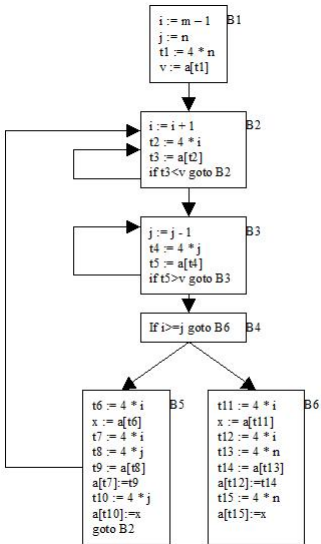
- 1 Determinar os líderes
 - Primeiro comando
 - Qualquer comando que seja objeto de desvio
 - Qualquer comando que segue goto condicional
- 2 Blocos são formados pelo líder + comandos seguintes até o próximo líder

Líderes em azul

```
1. if n<m goto 27
2. i:=m-1/// Bloco 1
3. j:=n
4. T1:=n*4
5. V:=a[T1]
6. i:=i+1// Bloco 2
7. T2:=i*4
8. T3:=a[T2]
9. If T3 < v goto 6
10. j := j - 1// Bloco 3
11 T2 := j * 4
12 T3 := a[T2]
13 If T3 > v goto 10
```

```
14 If i>=y goto 22//Bloco 4
15. T2 := i * 4// Bloco 5
16. x := a[T2]
17. T3 := j * 4
18. T4 := a[T3]
19. a[T2] := T4
20. a[T3] := x
21. goto 6
22. X:= a[T2]// Bloco 6
23. T4 := n * 4
24. T5 := a[T4]
25.A[T2] := T5
26. A[T4] := x
```

Grafo de Fluxo Resultante (o teste da primeira instrução não foi representado por não sofrer nenhuma otimização):



- A obtenção do grafo de fluxo de controle (CFG) é feita logo no início da otimização.
- No gcc, a representação em blocos básicos aparece a partir do passo 11 (teste.c.011t.cfg)
- Inicialmente ele lista os sucessores de cada nodo:

```
:: 2 succs { 4 3 }
```

```
:: 3 succs { 4 6 }
```

- seguido do código:

```
<bb 2>:
```

```
if (i <= 0) goto <bb 4>; else goto <bb 3>;
```

```
<bb 3>:
```

```
if (i <= 1)
```

```
goto <bb 4>;
```

```
else
```

```
goto <bb 6>;
```

- Algumas otimizações que podem ser feitas no grafo acima são:
 - Eliminação de expressões comuns locais (no sentido de estarem no mesmo bloco básico)
- Podem ser eliminadas, pois já existem com o mesmo valor:
 - $t_{12} := 4 * i$ (B6)
 - $t_{15} := 4 * n$ (B6)
 - $t_7 := 4 * i$ (B5)
 - $t_{10} := 4 * j$ (B5)

- E o bloco B5 seria reduzido de

```
T6 := 4 * i  
x := a[T6]  
T7 := 4 * i  
T8 := 4 * j  
T9 := a[T8]  
a[T7] := T9  
T10 := 4 * j  
a[T10] := x  
goto B2
```

```
T6 := i * 4  
x := a[T6]  
T8 := 4 * j  
T9 := a[T8]  
a[T6] := T9  
a[T8] := x  
goto B2
```

2. Eliminação de Subexpressões Globais

- Em B5 $t6 := 4 * i$ e $x := a[t6]$ podem ser substituídos por $x := t3$, pois $a[4 * i]$ já está em $t3$ desde o bloco B2.
- $t8 := 4 * j$ e $t9 := a[t8]$ podem ser substituídos por $a[t2] := t5$, pois $a[4 * j]$ já está em $t5$ desde o bloco B3.
- $a[t6] := t9$ é substituído por $a[t2] := t5$. $t8$ é substituído por $t4$.
- O bloco B5 após essas transformações fica:

```
T6 := i * 4  
x := a[T6]  
T8 := 4 * j  
T9 := a[T8]  
a[T6] := T9  
a[T8] := x  
goto B2
```

```
x := t3  
a[T2] := T5  
a[T8] := x  
goto B2
```


- Praticamente as mesmas transformações podem ser feitas em B6.
- **3. Propagação de Cópia** - A idéia da transformação de propagação de cópias é usar g em lugar de f sempre que possível após comandos de cópia $f:=g$ de modo a, eventualmente, tornar desnecessário o comando de cópia.
- Nos blocos B5 e B6, as atribuições de x foram substituídas por atribuições de t3 permitindo eventualmente a remoção da atribuição de t3 a x.

```
x := t3  
a[T2] := T5  
a[T8] := x  
goto B2
```

```
x := t3  
a[T2] := T5  
a[T8] := t3  
goto B2
```

- 4. **Eliminação de Código Morto** - Código morto é código que não afeta o resultado final da computação. Isso inclui tanto código não alcançável (unreachable codes) como atribuições a variáveis que não são utilizadas posteriormente (dead variables).
- Ao se analisar a utilização da variável x ao sair dos blocos B5 e B6, constata-se que a mesma não é utilizada e as atribuições $x:=t3$ podem ser removidas gerando os blocos B5 e B6 a seguir.

```
x := t3  
a[T2] := T5  
a[T8] := t3  
goto B2
```

```
a[T2] := T5  
a[T8] := t3  
goto B2
```

- 5. Redução de força (Substituição de uma operação por outra mais barata)
 - A instrução $t4 := 4*j$ no bloco B3 pode ser substituída por $t4 := t4 - 4$, pois como a cada ciclo a variável j é decrementada de 1, a variável $t4$ é decrementada de 4. Para manter a relação de $t4 = 4*j$ ao entrar pela primeira vez no bloco B3, esta inicialização pode ser feita no bloco B1. O mesmo pode ser feito para o bloco B2. Os blocos resultantes destas operações são:

6. Remoção do código independente do laço para fora do laço

É a remoção para fora do laço de expressões que mantenham o mesmo valor em todas as execuções. No exemplo do quicksort não ocorre esta situação.

7. Eliminação das variáveis de indução

- Variáveis dentro de um laço que tem sempre a mesma relação entre seus valores são chamadas de variáveis de indução e normalmente uma é suficiente para controlar o laço podendo as outras serem eliminadas.
- As variáveis i e $t2$ assim como j e $t4$ tem esta propriedade.
- i e j são utilizadas apenas para o teste em B4 e neste teste podem ser substituídas por $t2$ e $t4$ permitindo sua eliminação do código do laço.
- Após esta substituição o código resultante é o seguinte:

A variável j em B1 é utilizada apenas para o cálculo de $t4$. n pode ser utilizado tornando desnecessária a variável j .
Código resultante

```
1.  $i := m - 1$  Início do Bloco 1
2.  $T1 := n * 4$ 
3.  $v := a[T1]$ 
4.  $T4 := T1$ 
5.  $t2 := 4 * i$ 
6.  $T2 := T2 + 4$  Início do Bloco 2
7.  $T3 := a[T2]$ 
8. If  $T3 < v$  goto 6 (B2)
9.  $T4 := T4 - 4$  Início do Bloco 3
```

```
10.  $T5 := a[T4]$ 
11. If  $T5 > v$  goto 9 (B3)
12. If  $T2 > T4$  goto 16 (B6) Bloco 4
13.  $a[T2] := T5$  Início do Bloco 5
14.  $a[T4] := T3$ 
15. goto 6 (B2)
16.  $T14 := a[T1]$  Início do Bloco 6
17.  $A[T2] := T14$ 
18.  $A[T1] := T3$ 
```

Representação de Blocos básicos através de grafos

A representação de blocos básicos através de grafos acíclicos dirigidos (GADs) permite:

- A identificação de subexpressões comuns no bloco
- Identificação de variáveis usadas dentro do bloco e avaliadas fora

Um GAD para um bloco básico é um grafo acíclico orientado tal que

- As folhas representam variáveis ou constantes
- Nodos internos são identificados por operadores e representam valores computados

[Voltar para o índice](#)

Ex: Bloco básico e GAD correspondente

$$T1 = a + b$$

$$T2 = a - b$$

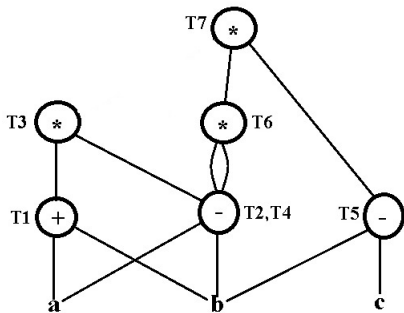
$$T3 = T1 * T2$$

$$T4 = a - b$$

$$T5 = b - c$$

$$T6 = T2 * T4$$

$$T7 = T6 * T5$$



Algoritmo para construção de GAD's para blocos básicos

Para cada comando do bloco básico faça os passos (1) a (3). Suponha que cada instrução (de três endereços) segue um dos formatos:

- i $x = y \text{ op } z$
- ii $x = \text{op } y$
- iii $x = y$

- ❶ Se ainda não existe um nodo com o nome y , então crie uma folha para y . Faça o mesmo para z no caso (i)
- ❷ No caso (i), verifique se existe um nodo op com filhos y (à esquerda) e z (à direita). Caso exista chame-o também de x ; senão crie um nodo op com nome x e arcos de x para y e z (identificados no passo (1))
 - No caso (ii), verifique se existe um nodo op com um único filho y , se não existir, crie tal nodo e estabeleça um arco deste nodo para y , chamando de x o nodo criado ou encontrado.
 - No caso (iii), chame também de x o nodo y .
- ❸ Remover x da lista de nós aos quais está associado. Associar x ao nodo criado (ou selecionado) no passo anterior.

Ex : $T1 = 4 * i$

$T2 = a[T1]$

$T3 = 4 * i$

$T4 = a[T3]$

$T5 = T2 * T4$

$T6 = \text{prod} + T5$

$\text{Prod} = T6$

$T7 = i + 1$

$I = T7$

- O código do bloco pode ser reconstruído gerando uma instrução para cada nodo, na mesma ordem em que os nodos foram acrescentados ao GAD.
- Quando o GAD é uma árvore, o algoritmo a seguir garante que o código gerado é o mais eficiente possível, em utilização de registradores (visto mais adiante, em geração de código)

- O algoritmo abaixo produz uma ordenação de nodos que corresponde ao inverso do código otimizado gerado a partir do bloco.
 - 1 Crie a lista L, inicialmente vazia
 - 2 Escolha um nodo n, que não esteja em L, tal que, se existirem arestas incidentes a n, estas se originam em nodos que já estão em L. Adicione n a L. Se não existe tal n, encerre.
 - 3 Se
 - i n1 é o último nodo adicionado a L e
 - ii a aresta mais à esquerda que se origina em n1 incide em um nodo interno n que não está em L, e
 - iii todos os predecessores diretos de n estão em L, então adicione n a L e repita 3
 - 4 Senão execute (2)

Passo	L	
1	T6	/ \
2	T5	
3	T2	
4	T1	
5	T7	

Geração de código otimizado
removendo expressões comuns

$T7 = i + 1$

$T1 = 4 * i$

$T2 = a[T1]$

$T5 = T2 * T2$

$Prod = prod + T5$

$I = T7$

- Os nós podem ser implementados como um array. P.ex. o código abaixo à esquerda, que implementa o comando $x=(a+b)+a*(a+b)$ poderia ser implementado com o array à direita:

$T1=a+b$

$T2=a+b$

$T3=a*b$

$T4=T1+T3$

$x=T4$

1	a			
2	b			
3	+	1	2	T1,T2
4	*	1	3	T3
5	+	3	4	T4,x

- Se uma análise de fluxo entre os blocos identifica que algumas variáveis atualizadas no bloco estão mortas na saída do mesmo (i.e. não serão utilizados posteriormente), a subárvore da qual a variável é raiz pode ser eliminada. Ex: T3 no exemplo anterior.

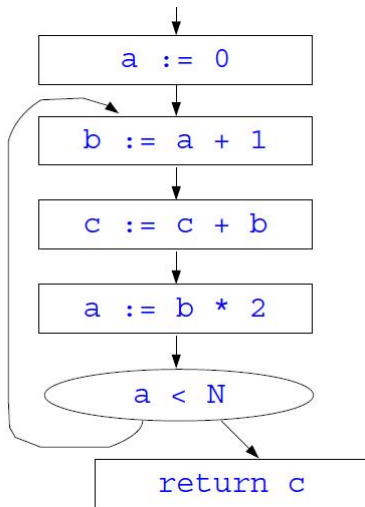
Por que Análise de Vida?

- O compilador traduz programas em uma linguagem intermediária com um número infinito de temporários.
- Mas, o programa deve ser executado em uma máquina com número limitado de registradores.
- Dois temporários podem usar o mesmo registrador se não estão em uso ao mesmo tempo.
- Uma variável é dita **viva** se ela contém um valor que será utilizado no futuro.

[Voltar para o índice](#)

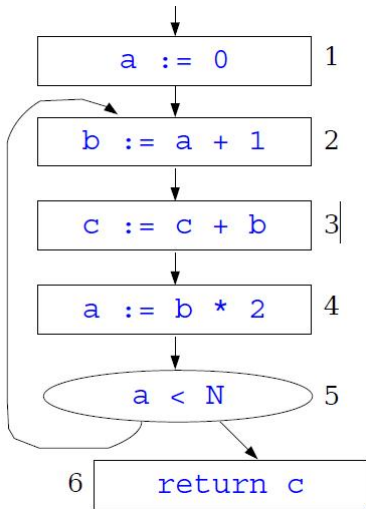
Grafo de Fluxo de Controle

$a \leftarrow 0$
L1: $b \leftarrow a + 1$
 $c \leftarrow c + b$
 $a \leftarrow b * 2$
if $a < N$ goto L1
return c



Grafo de Fluxo de Controle

- intervalo de vida da variável a é $\{1 \rightarrow 2, 4 \rightarrow 5 \rightarrow 2\}$
- intervalo de vida da variável b é $\{2 \rightarrow 3 \rightarrow 4\}$
- variável c está viva na entrada e na saída



- Determinar o intervalo de vida de cada variável é um problema de *fluxo de dados* (a vida das variáveis muda ao longo das arestas do grafo de fluxo).
- $pred[n]$ é o conjunto de todos os predecessores do nodo N (i.e., todos os nodos do qual saem arestas terminando no nodo N).
- $succ[n]$ é o conjunto de todos os sucessores do nodo N (i.e., todos os nodos no qual chegam arestas que saem do nodo N).
- Predecessores e sucessores no grafo de exemplo?

- Uma atribuição a uma variável (mesmo temporária) **define** aquela variável.
- Uma ocorrência de uma variável do lado direito de uma atribuição sem que haja uma definição da mesma em comando anterior do mesmo bloco **usa** a variável.
- **def**(var a) = conjunto de nodos do grafo que definem a.
- **def**(nodo N) = conjunto de variáveis que são definidas no nodo N.
- **uso**(var a) = conjunto de nodos do grafo que usa a variável a.
- **uso**(nodo N) = conjunto de variáveis usadas por N

- Uma variável está **viva** em uma aresta se há um caminho da aresta até um uso da variável que não passa por nenhuma outra definição da variável.
- Uma variável a está **viva** na entrada de um nodo N (live-in) se ela está viva em qualquer uma das arestas incidentes a N (arestas que chegam a N).
- $\text{in}[n]$ = conjunto de variáveis vivas na entrada de N .
- Uma variável está **viva** na saída (live-out) de um nodo N se ela está viva em qualquer uma das arestas que saem de N ($\text{out}[n]$).

- 1 Se a variável a está em $uso[N]$, então a está viva na entrada do nodo N .
- 2 Se a variável a está viva na entrada do nodo N , então a está viva na saída de todos os nodos em $pred[N]$.
- 3 Se a variável a está viva na saída do nodo N e não está em $def[n]$, então a está também viva na entrada do nodo N .

$$in[n] = use[n] \cup (out[n] - def[n])$$

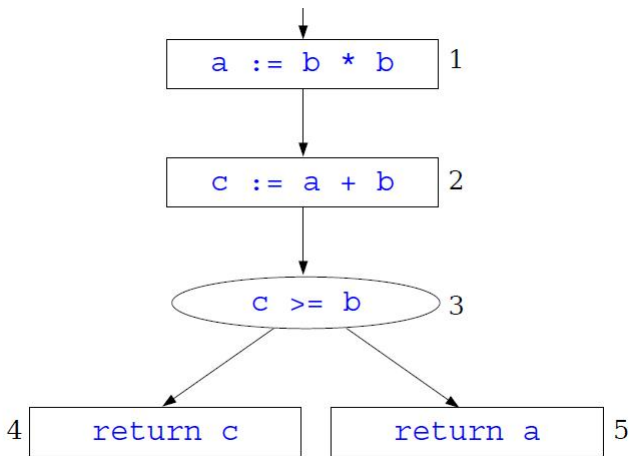
$$out[n] = \bigcup_{s \text{ in } succ[n]} in[s]$$

- Para cada nodo N
 - $in[N] \leftarrow \emptyset$; $out[n] \leftarrow \emptyset$;
- repita
 - Para cada nodo N
 - $in'[n] \leftarrow in[n]$; $out'[n] \leftarrow out[n]$
 - $out[n] \leftarrow \bigcup_{s \text{ em } succ[n]} in[s]$
 - $in[n] \leftarrow use[n] \cup (out[n] - def[n])$
- until $in'[n] = in[n]$ e $out'[n] = out[n]$ para todo n

Exemplo do Algoritmo

n	uso	def	iter. out	1 in	iter. out	2 in	iter. out	3 in
6	c			c		c		c
5	a		c	ac	ac	ac	ac	ac
4	b	a	ac	bc	ac	bc	ac	bc
3	bc	c	bc	bc	bc	bc	bc	bc
2	a	b	bc	ac	bc	ac	bc	ac
1	c	a	ac	c	ac	c	ac	c

- Qualquer solução para as equações de fluxo de dados é uma *aproximação conservadora*.
- Se o valor de a é necessário quando a execução alcança o nodo N , deve-se garantir que a está vivo na entrada de N .
- Mas d pode também estar vivo na entrada de N e não ser utilizado.
- Pode-se erroneamente pensar que uma variável está viva (i.e., em um registrador, mas não é realmente necessária), mas nunca erroneamente pensar que a variável está morta.



- Objetivo: manter uma variável em registrador se esta vai ser usada em seguida: analisa-se o código de um bloco básico do fim para o início.
- **Variável Viva:**
 - Uma variável é dita viva em uma linha i de um bloco se ela vai ser subsequentemente usada no bloco (em uma linha maior do que i).
 - Caso contrário, a variável está morta na linha i .
 - Pela análise de fluxo entre blocos é possível identificar quais variáveis estarão vivas na saída do bloco.
 - Na falta dessa análise, todas as variáveis não temporárias serão consideradas vivas na saída do bloco.

[Voltar para o índice](#)

- Essa análise identifica, para cada definição de variável, onde será o próximo uso da variável definida
- A análise é feita dentro do bloco básico, do fim para o início.
- É mantida na tabela de símbolos para cada variável a informação de próximo uso e se a variável está viva naquele ponto ou não.
- Inicialmente as variáveis que estão vivas na saída do bloco são marcadas na tabela de símbolos como vivas e com próximo uso fora do bloco. As outras são marcadas como mortas e sem próximo uso.

- Encontrando-se o comando $i : x := y \text{ op } z$ fazer:
 - 1) Registrar no comando i a informação encontrada na tabela de símbolos com relação ao próximo uso e vida de x , y e z . Se x não está vivo então o comando pode ser desprezado.
 - 2) Na tabela de símbolos, registrar x como não vivo e não tendo próximo uso
 - 3) Na tabela de símbolos, registrar y e z como vivos e $i =$ próximo-uso de y e z .

- Considere o código a seguir que lê tres coeficientes de uma equação de segundo grau e escreve a soma de suas raízes reais:

```
leia(a)
leia(b)
leia(c)
if ( $b^2 - 4*a*c < 0$ )
    printf("não tem");
else
{
     $r1 = (-b + (b^2 - 4*a*c)^{1/2}) / (2*a)$ ;
     $r2 = (-b - (b^2 - 4*a*c)^{1/2}) / (2*a)$ ;
    soma = r1 + r2;
    printf("%d", soma);
}
```

1: $t0 = -b$	$t0:v$ e $p.u.=7$; $b:n.v.$
2: $t1 = b^2$	$t1$ e $b:n.v.$ (código morto)
3: $t2 = 4 * a$	$t2:v$ e $p.u.=4$; $a:v$ e $p.u.=8$
4: $t2 = t2 * c$	$t2:v$ e $p.u.=6$; $c:n.v$
5: $t3 = 1/2$	$t3:v$ e $p.u.=6$
6: $t2 = t2^t3$	$t2:v$ e $p.u.=7$; $t3:n.v.$
7: $t3 = t0 + t2$	$t3:v$ e $p.u.=9$; $t0$ e $t2: v$ e $p.u.=10$
8: $t4 = 2 * a$	$t4:v$ e $p.u.=9$; $a:n.v.$
9: $t5 = t3 / t4$	$t5:v$ e $p.u.=12$; $t4:v$ e $p.u.=11$; $t3 n.v.$
10: $t6 = t0 - t2$	$t6:v$ e $p.u.=11$; $t0$ e $t2 n.v.$
11: $t4 = t6 / t4$	$t4:v$ e $p.u.=12$; $t6 n.v.$
12: $soma = t5 + t4$	$soma: v$ e $p.u.$ fora; $t5$ e $t4 n.v.$
	$soma$ é a única variável viva e próximo uso é fora do bloco

- ① Gere o C3E correspondente
- ② Mostre o grafo de fluxo de controle
- ③ Calcule a informação de uso subsequente para cada definição de variável.

Ex: (Aho 95 - pg 311 - Ex. 10.1.a) 10.1 Considere a multiplicação matricial a seguir:

```
begin
  for i:=1 to n do
    for j:=1 to n do
      c[i,j]:=0;
    for i:=1 to n do
      for j:=1 to n do
        for k:=1 to n do
          c[i,j] := c[i,j] + a[i,k] * b[k,j]
        end.
      end.
    end.
  end.
```


a) Assumindo que a , b e c tenham a memória alocada estaticamente e que haja quatro bytes por palavra numa memória endereçada em bytes, produza os enunciados de três endereços para o programa de multiplicação matricial. (observação: a solução abaixo considera como primeiro elemento a posição 0 e a última a posição n além de estar errada, pois considera cada linha com 10 elementos.. O exercício deve ser feito considerando a primeira posição como sendo a posição 1).

```

T1 :=0
FOR1: if T1>n goto PROXCOM
T2:=0
FOR2: if T2>n goto INCFOR1
T3 = T1 * 10
T4 = T3 + T2
T4 = T4 * 4
C[T4] = 0
T2 = T2 + 1
Goto FOR2
INCFOR1: T1 = T1 + 1
Goto FOR1
PROXCOM: T1 = 0
FOR3: if T1>n goto FIM
T2 = 0
FOR4: if T2 >n goto INCFOR3
T3=0
FOR5: if T3>n goto INCFOR4
T4 = T3 * 10
T5 = T4 + T2

```

```

T5 = T5 * 4
T6 = T1 * 10
T7 = T3 + T6
T7 = T7 * 4
T 8 = T1 * 10
T9 = T8 + T2
T8 = T9 * 4
T20 = b[T5]
T10 = a[T7]
T11 = T20 + T10
T21 = c[T9]
T12 = T21 + T11
C[T9] = T12
T3 = T3 +1
Goto FOR5
INCFOR4: T2 = T2 + 1
Goto FOR4
INCFOR3: T1 = T1 + 1
Goto FOR3
FIM:

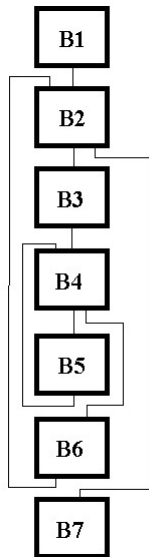
```

Código para a inicialização da matriz

```
        i = 1
laco1:  if i>n goto fim1
        j = 1
laco2:  if j>n goto fim2
        t1 = i-1
        t2 = t1*40
        t3 = j-1
        t4 = t3*4
        t5 = t2+t4
        c[t5] = 0
        j= j + 1
        goto laco2
fim2:  i = i+1
        goto laco1
fim1:
```

Líderes em vermelho

	$i = 1$	B1
laco1:	if $i > n$ goto fim1	B2
	$j = 1$	B3
laco2:	if $j > n$ goto fim2	B4
	$t1 = i - 1$	B5
	$t2 = t1 * 40$	
	$t3 = j - 1$	
	$t4 = t3 * 4$	
	$t5 = t2 + t4$	
	$c[t5] = 0$	
	$j = j + 1$	
	goto laco2	
fim2:	$i = i + 1$	B6
	goto laco1	
fim1:	continua...	B7



Instrução Condicional

- Desvios condicionais são normalmente implementados em processadores através de flags de estado.
- Assim, uma instrução de desvio condicional
if $a > b$ goto L1
pode ser implementada através de uma instrução de comparação entre os dois operandos, seguido de uma instrução de desvio condicional.
A instrução `cmp op1,op2` efetua a subtração de um operando pelo outro, descartando o resultado mas atualizando os flags de estado
- O operador relacional do C3E definirá qual flag de estado deve ser testado:

Cmp R1,R2

JZ, JE

JL

$R1 < R2$

JG

$R1 > R2$

JLE

$R1 \leq R2$

JGE

$R1 \geq R2$

JNZ

$R1 \neq R2$

- Exemplo do Algoritmo

$$d := (a - b) + (a - c) + (a - c)$$

- Mostre o código gerado (intermediário e final) e o estado dos descritores de registrador e endereços após cada instrução.

$$T1 = a - b$$

$$T2 = a - c$$

$$T3 = T1 + T2$$

$$d = T3 + T2$$

C.I.	Código Gerado	Descritor dos Regs	Descritor de End
$T1 = a - b$	mov a, R0 sub b,R0	R0 contém T1	T1 está em R0
$T2 = a - c$	mov a,R1 sub c,R1	R0 contém T1 R1 contém T2	T1 está em R0 T2 está em R1
$T3 = T1 + T2$	add R1,R0	R1 contém T2 R0 contém T3	T2 está em R1 T3 está em R0
$D = T3 + T2$	add R1,R0 mov R0,d	R1 contém T2 R0 contém d R0 contém d	T2 está em R1 d está em R0 d está em R0 e memória

- Um **laço** em um grafo de fluxo de controle é um conjunto de nós S incluindo um nó cabeçalho h , de tal forma que
 - a partir de qualquer nó em S existe um caminho para h ,
 - existe um caminho de h a qualquer nó em S ,
 - não há nenhuma aresta de algum nodo fora de S para qualquer nó em S exceto para h .

- Laços como o anterior, chamados **irredutíveis**, não são considerados porque a existência de mais de um ponto de entrada no laço impede de implementar algumas otimizações.
- Linguagens estruturadas não geram laços irredutíveis. Eles podem ser causados pelo uso de goto.
- Identificação de laços pode ser efetuada através da análise de dominadores.

- No gcc, a identificação dos laços é feita no mesmo passo da construção do cfg, no passo 11, gerando para o código abaixo à esquerda, o código à direita (teste.c.011t.cfg):

```
int a=10;
while (a>0)
{
    if (a==5)
        break;

    a- -;
}
```

```
<bb 2>:
a = 10;
goto <bb 6>;

<bb 3>:
if (a == 5)
    goto <bb 4>;
else
    goto <bb 5>;

<bb 4>:
goto <bb 7>;

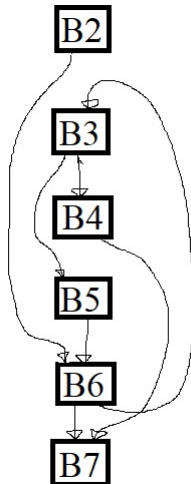
<bb 5>:
a = a + -1;

<bb 6>:
if (a > 0)
    goto <bb 3>;
else
    goto <bb 7>;

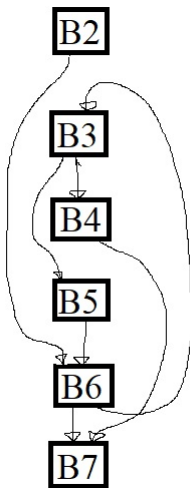
<bb 7>:
return;
```

- Que pode ser visualizado como o cfg à direita:

```
<bb 2>:  
a = 10;  
goto <bb 6>;  
  
<bb 3>:  
if (a == 5)  
    goto <bb 4>;  
else  
    goto <bb 5>;  
  
<bb 4>:  
goto <bb 7>;  
  
<bb 5>:  
a = a + -1;  
  
<bb 6>:  
if (a > 0)  
    goto <bb 3>;  
else  
    goto <bb 7>;  
  
<bb 7>:  
return;
```



- Além do código gerado, o arquivo também contém no início a estrutura do cfg (listando os sucessores de cada nodo) e os laços identificados:



```
:: Loop 1
:: header 6, latch 5
:: depth 1, outer 0
:: nodes: 6 5 3
:: 2 succs { 6 }
:: 3 succs { 4 5 }
:: 4 succs { 7 }
:: 5 succs { 6 }
:: 6 succs { 3 7 }
:: 7 succs { 1 }
```

- Cada grafo de fluxo de controle tem um nó de início s_0 com nenhum antecessor (onde começa a execução).
- Diz-se que um nó d **domina** um nó n se todos os caminhos de s_0 a n deve passar por d . (Cada nó domina a si mesmo.)
- Considere nó n com antecessores $p_1, \dots, p_k$ e nó $d \neq n$.
 - Se d domina cada um dos p_i em seguida, d deve dominar n .
 - Se d domina n então d deve dominar todos os nodos p_i .

[Voltar para o índice](#)

- Seja $D[n]$ o conjunto de nós que dominam n . Esse conjunto pode ser encontrado resolvendo a seguinte equação de fluxo de dados:

$$D[s_0] = \{s_0\}$$
$$D[n] = \{n\} \cup \left(\bigcap_{p \text{ em } \text{pred}[n]} D[p] \right) \text{ para } n \neq s_0$$

- Como a maioria das equações de fluxo de dados , pode-se resolver estas equações por iteração.
- Inicializar cada conjunto $D[n](n \neq s_0)$ com todos os nós no grafo. Cada atribuição fará $D[n]$ menor, devido à intersecção.

- Sendo N o conjunto de todos os nodos do grafo:
 $D(n_0) := \{n_0\};$
para cada n em $N - \{n_0\}$ faça $D(n) := N;$
enquanto ocorrer alteração em qualquer $D(n)$ faça
 para cada n em $N - \{n_0\}$ faça
 $D[n] = \{n\} \cup (\cap_{p \text{ em } \text{pred}[n]} D[p])$

Exemplo

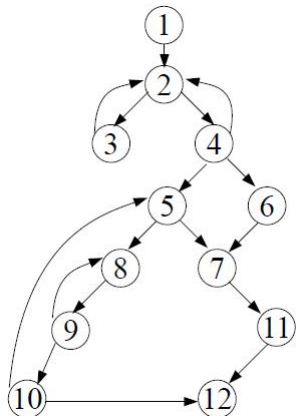


Figure: Grafo de Fluxo de Controle

n	pred[n]	D[n]
1		1
2	1 3 4	1 2
3	2	1 2 3
4	2	1 2 4
5	4 1 0	1 2 4 5
6	4	1 2 4 6
7	5 6	1 2 4 7
8	5 9	1 2 4 5 8
9	8	1 2 4 5 8 9
10	9	1 2 4 5 8 9 10
11	7	1 2 4 7 11
12	10 11	1 2 4 12

- Uma representação eficiente de conjuntos em equações de fluxo de dados, usada na implementação de compiladores, é através de bitsets. Cada bit representa um dos nodo possíveis do conjunto, e as operações de conjuntos são implementadas com uma operação bitwise:
 - União: implementada pela operação or
 - Intersecção: implementada pelo and
 - Complemento: implementada pela negação bitwise
 - Diferença simétrica: operador xor
- Há algoritmos quadráticos e até lineares para a solução das equações de fluxo de dados (mas são complicados).

Dominadores imediatos

- Em um grafo conexo, suponha que **d** domina **n** e **e** domina **n**. Então, ou **d** domina **e** ou **e** domina **d**.
- Ou, em outras palavras, se dois nodos **a** e **b** dominam um terceiro nodo **n**, ou **a** domina **b**, ou **b** domina **a**.
- Como consequência, cada nó n tem não mais do que 1 **dominador imediato**, $iDom(n)$, de tal modo que
 - $iDom(n) \neq n$
 - $iDom(n)$ domina n
 - $iDom(n)$ não domina qualquer outro dominador de n
- Dominadores imediatos tem um papel importante na identificação de laços.

Exemplo: Encontrar dominador imediato

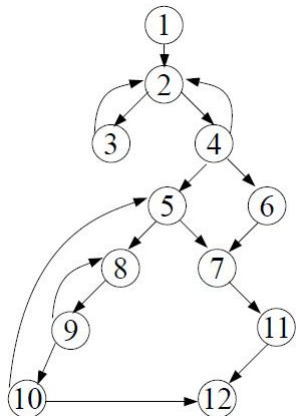


Figure: Grafo de Fluxo de Controle

n	dom[n]	idom[n]
1	1	
2	1 2	1
3	1 2 3	2
4	1 2 4	2
5	1 2 4 5	4
6	1 2 4 6	4
7	1 2 4 7	4
8	1 2 4 5 8	5
9	1 2 4 5 8 9	8
10	1 2 4 5 8 9 10	9
11	1 2 4 7 11	7
12	1 2 4 12	4

Exemplo: Árvore de dominadores

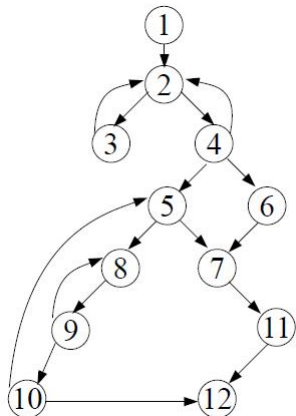


Figure: Grafo de Fluxo de Controle

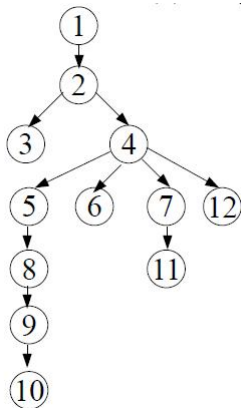
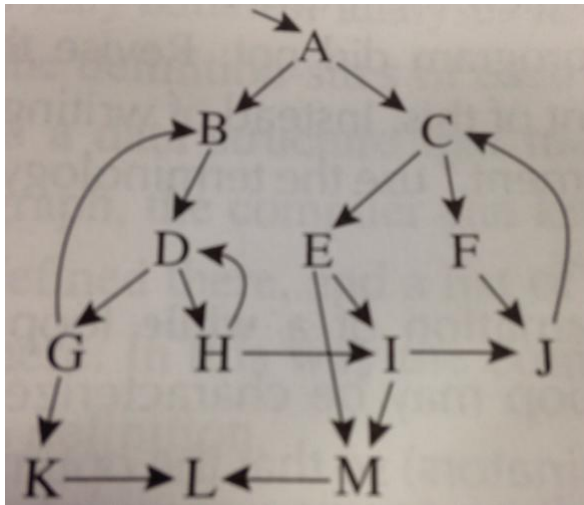
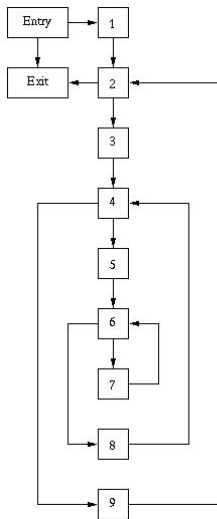


Figure: Árvore de dominadores (arestas de $\text{idom}(n)$ para n)

1) (2.0 pontos) Mostre a árvore de dominadores para o Grafo de Fluxo de Controle a seguir:



2) (2.0 pontos) Mostre a árvore de dominadores para o Grafo de Fluxo de Controle a seguir:



Exemplo: Arestas de Retorno

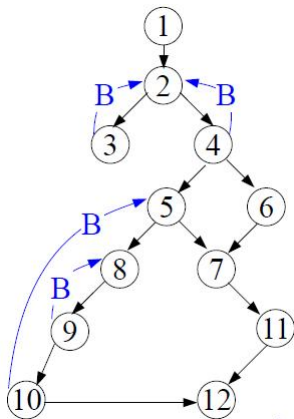


Figure: Grafo de Fluxo de Controle

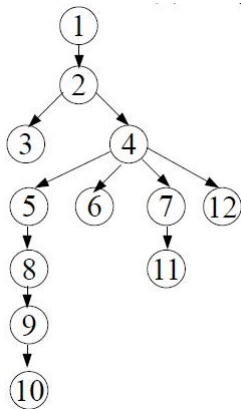


Figure: Uma aresta de n para h em $D[h]$ é uma **aresta de retorno**

- Uma **aresta de retorno** (**back edge**) é uma aresta em que a cabeça (vértice destino) da aresta domina a cauda (vértice de origem).
- O **laço natural** de uma aresta de retorno $n \rightarrow h$, onde h domina n , é o conjunto de nós x tal que
 - h domina x e
 - existe um caminho de x para n não contendo h .
- Em outras palavras, é o conjunto de nodos x que há um caminho para n sem passar por h . Os nodos que alcançam n passando por h estão antes do laço, os que não alcançam n estão depois do laço.
- O cabeçalho deste laço é h (pode ser para mais de um laço).

Algoritmo para identificar laços

- Entrada: aresta de retorno $a \rightarrow b$, onde **b** domina **a**.
- Saída: conjunto *loop* de nodos do laço.

```
stack := empty;  
loop := {b};  
insert(a);  
while stack is not empty do begin  
    pop m of the stack;  
    for each predecessor p of m do insert(p)  
end;
```

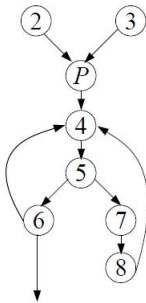
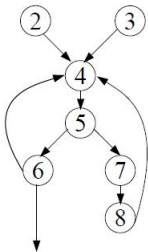
- Função insert:
 procedure insert(m);
 if m is not in loop then begin
 loop := loop $\cup$ {m};
 push m onto stack
 end;

- A identificação de laços aninhados define uma ordem em que os laços devem ser analisados, uma vez que quanto mais interno é um laço, mais tempo um programa fica nele.
- Além disso, alterações em um laço interno, como remoção de expressões laço-invariantes ou eliminação de variáveis de indução acarretam alterações no laço imediatamente externo a ele, e dessas alterações podem ocorrer novas oportunidades de otimização.

- Se A e B são laços com cabeçalhos a e b , de tal modo que $a \neq b$ e b está em A , então os nodos de B formam um subconjunto próprio de nodos em A (isto é, o laço B está aninhado no laço A).
- Para construir uma árvore de aninhamento de laços do grafo de fluxo G :
 - 1 Calcule os dominadores de G .
 - 2 Construa a árvore de dominação.
 - 3 Encontre todos os laços naturais (e nodos cabeçalhos desses laços).
 - 4 Para cada cabeçalho h , fundir todos os laços naturais de h em um único laço $[h]$.
 - 5 Construa a árvore de cabeçalhos de laço, de tal forma que h_1 está acima de h_2 na árvore se h_2 está no laço $[h_1]$.

Pré-cabeçalho do laço

- Muitas otimizações de laço inserem comandos imediatamente antes do início do laço.
- Pode-se usar um nó de pré-cabeçalho inicialmente vazio P.
- O código ao lado geraria um grafo com várias arestas chegando na cabeça do laço (ele não corresponde às figuras mostradas).



```
i=0;
for ( i=0; i < 10; i++)
{
    scanf ("%d",&x);
    if (x>0) break;
    if (x %2==0) break;
    if (x %3==0) break;
}
do
{
    c[i]=0;
    i++;
} while (i < 10);
```

- Se um laço contem $t \leftarrow a \text{ opbin } b$ tal que a, b tem mesmo valor em cada iteração, o mesmo ocorrerá com t .
- $d : t \leftarrow a_1 \text{ opbin } a_2$ é laço-invariante se para cada a_i ,
 - a_i é uma constante, ou
 - todas as definições de a_i que alcançam d estão fora do laço, ou
 - apenas uma definição de a_i chega em d , e esta definição é laço-invariante

[Voltar para o índice](#)

- Se uma atribuição $d : t \leftarrow a$ *opbin* b é laço-invariante, sob algumas condições ela pode ser removida para fora do laço, no final do pré-cabeçalho:
 - 1 d domina todas as saídas do laço nas quais t está viva, e
 - 2 existe somente uma definição de t no laço, e
 - 3 t não está viva na saída do pré-cabeçalho do laço:

Remoção de computações laço-invariantes

- Deve-se remover a computação laço-invariante $t \leftarrow a \text{ binop } b$ para fora desses laços?

laço 1	laço 2	laço 3
L_0 $t \leftarrow 0$ L_1 $i \leftarrow i+1$ $t \leftarrow a \text{ binop } b$ $M[i] \leftarrow t$ $\text{if } i < N \text{ goto } L_1$ L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $\text{if } i \geq N \text{ goto } L_2$ $i \leftarrow i+1$ $t \leftarrow a \text{ binop } b$ $M[i] \leftarrow t$ $\text{goto } L_1$ L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $i \leftarrow i+1$ $t \leftarrow a \text{ binop } b$ $M[i] \leftarrow t$ $t \leftarrow 0$ $M[j] \leftarrow t$ $\text{if } i < N \text{ goto } L_1$ L_2

Remoção de computações laço-invariantes

- Critérios para remover a computação laço-invariante $t \leftarrow a$ binop b para o fim do pré-cabeçalho:

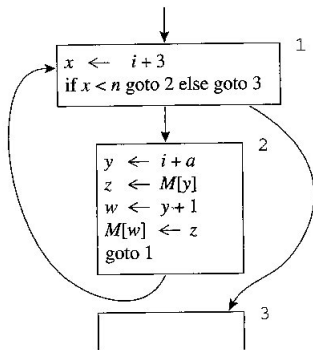
laço 4	
L_0	$t \leftarrow 0$
L_1	$M[j] \leftarrow t$ $i \leftarrow i+1$ $t \leftarrow a \text{ binop } b$ $M[i] \leftarrow t$ if $i < N$ goto L_1
L_2	$x \leftarrow t$

- 1 d domina todas as saídas do laço nas quais t está vivo, e
- 2 há apenas uma definição de t no laço, e
- 3 t não está vivo na saída do pré-cabeçalho do laço.

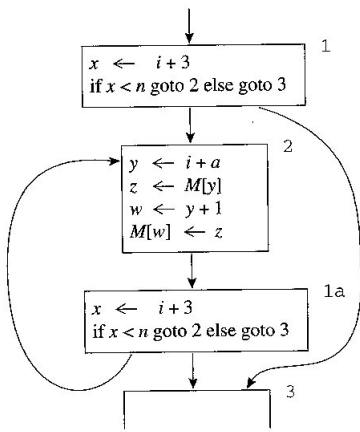
Transformando *while*'s em *repeat*'s

- A condição 1 pode impedir remoções de computações laço-invariantes por colocar uma saída no início do laço.
- Uma solução para isso é transformar o *while* em um *repeat* com um comando *if* antes dele.
- para isso os comandos do nodo cabeçalho são duplicados ao final

Transformando while's em repeat's



(a)



(b)

Eliminação de Variáveis de Indução

- Suponha que um laço tem uma variável i que é incrementada ou decrementada a cada iteração e uma variável j variável que está definida no laço como $i * c + d$, onde c e d são laço-invariantes.
- Pode-se calcular j sem utilizar i .
- Sempre que i é incrementado de a , incrementa-se j de $c * a$.
- As variáveis i e j são chamados de *variáveis de indução*.
- Uma variável i é dita uma **variável de indução básica** se ela é incrementada ou decrementada de um valor fixo em cada iteração.
- Uma variável j é uma variável de indução **derivada** na família de i se o valor de j depende somente de i e expressões laço-invariantes.

Otimizações de Variáveis de Indução

Antes:

```
s ← 0
i ← 0
L1 if  $i \geq n$  goto L2
    j ← i*4
    k ← j+a
    x ← M[k]
    s ← s+x
    i ← i+1
    goto L1
```

L₂

Após

```
s ← 0
k' ← a
b ← n*4
c' ← a+b
L1 if  $k' \geq c$  goto L2
    x ← M[k]
    s ← s+x
    k' ← k'+4
    goto L1
```

L₂

:

- Análise de variáveis de indução descobre que i , j e k são variáveis de indução relacionadas.
- Redução de Força substitui a multiplicação por 4 por uma adição.
- Eliminação de variáveis de indução substitui $i \geq n$, com $k \geq 4n + a$.
- Para limpar: propagação de cópias e eliminação de código morto.

- A única definição da variável de indução básica i em L é da forma $i \leftarrow i + c$ ou $i \leftarrow i - c$, em que c é laço-invariante.
- A variável k é uma variável de indução derivada em L se:
 - 1 Existe apenas uma definição de k em L , da forma $k \leftarrow j * c$ ou $k \leftarrow j + d$, onde j é uma variável de indução e c e d são laço-invariantes; e
 - 2 Se j é uma variável de indução derivada da família de i , então
 - 1 a única definição de j que atinge k é aquela no laço,
 - 2 e não há nenhuma definição de i em qualquer caminho entre a definição de j e a definição de k .

- Dada a definição $g \leftarrow i * b_g + a_g$, podemos descrever a variável de indução g como uma tripla (i, a_g, b_g) , onde i é a variável básica de indução.
- A partir do exemplo:
 - variável de indução derivada j com $j \leftarrow i * 4$ é descrita como $(i, 0, 4)$, e
 - variável de indução derivada k com $k \leftarrow j + a$ é descrita como $(i, a, 4)$.
- A variável de indução básica i é descrita como $(i, 0, 1)$.

Algoritmo para identificar variáveis de indução

- Entrada: Um laço L consistindo de instruções C3E, informações sobre uso-definição e informação sobre laço-invariantes.
- Saída: Um conjunto de variáveis de indução, cada uma definida por uma tripla.
- Algoritmo:
 1. Para cada instrução em L que está no formato $i = i+c$ or $i=i-c$ crie uma tripla $(i, 0, 1)$.
 2. Variáveis de indução derivadas: Para cada comando de L ,
 - Se o comando está na forma $k=j+c1$ ou $k=j*c2$
 - e j é uma variável de indução com a tripla (x, a, b)
 - e $c1$ e $c2$ são laço-invariantes
 - e k é definido apenas uma vez no laço
 - e se j é uma variável de indução derivada pertencente à família de i então
 - a única definição de j que alcança k deve estar em L
 - e nenhuma definição de i deve ocorrer em nenhum caminho entre as definições de j e k
 - então crie a tripla $(x, a+c1, b)$ para $k=j+c1$ ou $(x, a*c2, b*c2)$ para $k=j*c2$

- Identifique e represente como triplas as variáveis de indução do bloco abaixo:

$i=0$

laço: if $i = 5$ goto fim

$T1 = i * 4$

$T2 = 9 - i$

$T3 = 4 * T2$

$v[T1] = V[T3]$

$i = i + 1$

goto laço

- Gostaríamos de ter uma variável de indução derivada cuja definição é da forma $j \leftarrow i * b + a$ e substituí-la por uma soma. Por quê?
- Depois de $i \leftarrow i + c$ (em cada iteração), faça $j' \leftarrow j' + c * b$.
- Expressão $c * b$ é laço-invariante, então pode ser computada no pré-cabeçalho.
- Substitua a atribuição original a j por $j \leftarrow j'$.
- Coloque $j' \leftarrow a + i * b$ no final do pré-cabeçalho do laço.

Algoritmo para redução de força

- Entrada: Um laço consistindo de instruções de 3 endereços e triplas de variáveis de indução.
- Saída: Um laço modificado com um novo pré-cabeçalho
- Algoritmo:
- 1. Para cada variável de indução derivada j com tripla (i, a, b)
 - - crie uma nova j'
 - - ponha a computação $t=b*c$ no pré-cabeçalho
 - - após cada definição de i em L , onde $i = i+c$ insira $j' = j' + t$
 - - substitua a definição de j por $j=j'$
 - - inicialize j' ao final do pré-cabeçalho com $j' = a+b*i$
- Obs:
 - - j' também tem a tripla (i, a, b)
 - - a multiplicação foi movida para fora do laço

Exemplo: redução de força

Antes:

```
s ← 0
i ← 0
L1 if  $i \geq n$  goto L2
    j ← i * 4
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    goto L1
L2
```

Após a redução
de força em j e k :

```
s ← 0
i ← 0
j' ← 0
k' ← a
L1 if  $i \geq n$  goto L2
    j ← j'
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    k' ← k' + 4
    goto L1
L2
```

- Para a variável de indução derivada g com tripla (i, x, y) e variável de indução $i \leftarrow i + z$, faça $g' \leftarrow g' + z * y$ e inicialize com $g' \leftarrow x + i * y$.
- Porque o programa transformado é equivalente ao original?
- É mais rápido?

- Após a redução de força, algumas variáveis de indução não são mais usadas em todo o laço e outras são usadas somente em comparações com laço-invariantes.
- Uma variável é **inútil** em L se está morta nas saídas do laço e seu único uso é em uma definição de si mesma.
- As definições de tais variáveis podem ser removidas do laço.

Algoritmo para eliminação de variáveis de indução

- Entrada: Um laço consistindo de instruções C3E, informações sobre uso-definição, informação de laço-invariantes e informação de variáveis vivas.
- Saída: Um laço revisado.
- Algoritmo:
 1. Para cada variável de indução básica i
 - Se ela é usada somente para computar outras variáveis de indução de sua família e em desvios condicionais
 - Use uma tripla $j=(i, c, d)$ da família, preferencialmente com $c = 0$
 - Modifique cada condicional envolvendo i de modo a que passe a usar j
 - "if i relop x goto L " se torna "if j relop y goto L " onde $y = c + d*x$
 - Substitua todos os usos de i por $(j-c)/d$, fazendo os ajustes necessários
 - Substitua a atualização de i ($i \leftarrow i + a$) por $j \leftarrow j + a*d$
 - Coloque no pré-cabeçalho do laço a inicialização $j \leftarrow d*i + c$ Defina j como variável de indução básica e recalcule as triplas das variáveis de indução da família de i transformando-as em variáveis da família de j

Exemplo: Eliminação

Antes:

```
s ← 0
i ← 0
j' ← 0
k' ← a
L1 if i ≥ n goto L2
    j ← j'
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    k' ← k' + 4
    goto L1
L2
```

Após a elimi-
nação de j e j' :

```
s ← 0
i ← 0
k' ← a
L1 if i ≥ n goto L2
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    k' ← k' + 4
    goto L1
L2
```

- A variável j é inútil no laço.
- Após a remoção de j , j' é inútil no laço.
- Usando eliminação de código morto, podemos também remover a definição de j' do pré-cabeçalho.

- A variável i é dita **quase inútil**
 - se for utilizado apenas em comparações com laço-invariantes e nas definições de si mesmo, e
 - se há alguma outra variável de indução na mesma família que não é inútil.
- Pode-se tornar i inútil, modificando a comparação para usar a outra variável de indução.

- Suponhamos que temos $k < n$, onde j e k são variáveis de indução coordenadas da família de i e n é laço-invariante.
- Isto é, para j com (i, a_j, b_j) e com k (i, a_k, b_k) sabemos $(j - a_j) / b_j = (k - a_k) / b_k$.
- Podemos reescrever $k < n$, como $j < (b_j / b_k) * (n - a_k) + a_j$, se b_j / b_k é positivo. Todo o lado direito da expressão é laço invariante.

Exemplo: Reescrita de Comparações

Antes:		Após	reescr-
		ever $i \geq n$	e eliminar i :
	$s \leftarrow 0$		$s \leftarrow 0$
	$i \leftarrow 0$		$k' \leftarrow a$
	$k' \leftarrow a$		$b \leftarrow n * 4$
L_1	if $i \geq n$ goto L_2		$c \leftarrow a + b$
	$k \leftarrow k'$	L_1	if $k' \geq c$ goto L_2
	$x \leftarrow M[k]$		$k \leftarrow k'$
	$s \leftarrow s + x$		$x \leftarrow M[k]$
	$i \leftarrow i + 1$		$s \leftarrow s + x$
	$k' \leftarrow k' + 4$		$k' \leftarrow k' + 4$
	goto L_1		goto L_1
L_2			
		L_2	

- k' e i são variáveis de indução coordenadas porque $(i-0)/1 = (k'-a)/4$.
- Assim, $i \geq n$ pode ser substituído por $k' \geq a + 4 * n$.
- Então, i é inútil no laço.

Exemplo: Versão Final

	Antes:	Após reescrever
	$s \leftarrow 0$	$i \geq n$
	$k' \leftarrow a$	e eliminar i :
	$b \leftarrow n * 4$	$s \leftarrow 0$
	$c \leftarrow a + b$	$k' \leftarrow a$
L_1	if $k_1 \geq c$ goto L_2	$b \leftarrow n * 4$
	$k \leftarrow k'$	$c \leftarrow a + b$
	$x \leftarrow M[k]$	L_1 if $k' \geq c$ goto L_2
	$s \leftarrow s + x$	$x \leftarrow M[k]$
	$k' \leftarrow k' + 4$	$s \leftarrow s + x$
	goto L_1	$k' \leftarrow k' + 4$
L_2		goto L_1
		L_2

Finalmente, a propagação de cópia pode eliminar $k \leftarrow k'$.

- 2) (2.5 pontos) Considere o C3E gerado para o código a seguir, que copia um vetor $v[1..100]$ para uma matriz $M[1..10,1..10]$.

```

i=1;
for l:=1 to 10 do
  for c:=1 to 10 do
    begin
      M[l , c]=v [ i ];
      i:=i+1;
    end
  end
end

```

```

i=1
l=1
L1:  if l>10 goto F1
      c=1
L2:  if c>10 goto F2
      t1=l-1
      t2=t1*10
      t3=t2+c
      t4=t3-1
      t5=i-1
      t6=v[t5]
      m[t4]=t6
      i=i+1
      c=c+1
      goto L2
F2:  l=l+1
      goto L1
F1:

```

- Identifique as variáveis de indução representando-as por triplas. Aplique a seguir as transformações de eliminação de variáveis de indução, remoção de computações laço-invariantes e propagação de cópia mostrando, a cada passo, quais transformações foram feitas.

Análise de Fluxo de Dados (Dataflow Analysis)

- Análise de fluxo de dados é o processo de coletar informações sobre como o programa manipula os dados.
- É usada em diversas etapas da compilação para obter várias informações para a otimização
- Normalmente usa-se para obter informações na entrada e saída dos blocos básicos.

- Ela pode ser forward, quando a informação é gerada "para a frente" no grafo de fluxo (i.e, a saída de um bloco define a informação na entrada dos seus sucessores) ou "backward" (a entrada de um bloco define a saída de seus predecessores).
- A análise de dominadores para a identificação de laços é um exemplo de análise "forward", onde os dominadores de um nodo eram encontrados a partir dos dominadores de seus predecessores.

- E a análise de variáveis vivas é um exemplo de uma análise "backward", onde a entrada de um bloco era calculada a partir da saída do mesmo, e a saída de um bloco a partir da entrada de seus sucessores.
- Normalmente o resultado pode ser obtido por uma equação de fluxo de dados, que resolvida gera os conjuntos buscados.
- Uma abordagem simples para resolver as equações é aplicar repetidamente as equações a cada bloco, até que os conjuntos na entrada e saída de cada bloco convirjam para um ponto fixo, onde não há mais alterações nos conjuntos.

- Esse método é conhecido por Método de Kildall, porque foi apresentado por Gary Kildall em sua tese de doutorado em 1972.
- Depois ele inventou o CP/M, primeiro sistema operacional para computadores portáteis, abriu sua empresa e ficou rico.



- Uma abordagem mais eficiente é manter uma lista dos blocos a serem processados. Em uma análise forward, como a entrada de cada bloco depende de seus predecessores, coloca-se na lista todos os blocos e, a partir daí:
 - Tira-se um bloco B da lista e calcula-se sua saída.
 - Se a saída foi alterada, coloca-se na lista os sucessores do bloco
 - Repete-se isso até que a lista esteja vazia.

- Entre os usos de análise dataflow em compiladores encontra-se:
 - Análise de laços (cálculo de dominadores)
 - Análise de vida de variáveis na entrada e saída de blocos
 - Análise de definições incidentes (Reaching definition)
 - Análise de expressões disponíveis: : Uma expressão $X+Y$ está disponível em um ponto p se em cada caminho até p a expressão $X+Y$ é avaliada e depois da última avaliação não há atribuição a X ou Y
 - Busy expression: Uma expressão $B \text{ op } C$ é dita "ocupada" em um ponto p se em todo caminho a partir de p há o cálculo da expressão antes da primeira redefinição de B ou C .

- A análise entre blocos normalmente utiliza informações internas ao bloco, obtidas por uma análise local (dentro do bloco)
 - Analisa o efeito de cada instrução
 - Compõe o efeito de cada instrução para derivar informação a partir do início do bloco básico para cada instrução
 - Pode-se descrever o comportamento do bloco como uma "função de transferência" em que a saída do bloco é descrita em função da entrada e dos comandos no bloco ou, na análise backward, a entrada do bloco é descrita em função da saída do bloco e seus comandos.
- Análise de fluxo de dados (entre blocos)
 - Analisa o efeito de cada bloco básico
 - Compõe o efeito dos blocos básicos para derivar informações nas fronteiras dos blocos básicos.

- A análise de definições incidentes identifica para cada ponto do código quais as definições de variáveis que alcançam aquele ponto.
- Pontos de um bloco básico:
 - Imediatamente antes do primeiro comando
 - Entre comandos
 - Após o último comando

- A informação gerada por essa análise simplifica bastante a implementação de algumas otimizações.
 - Identificação e cálculo de expressões constantes.
 - Propagação de Cópias
 - Eliminação de subexpressões comuns, etc.
 - Remoção de expressões laço-invariantes
- A identificação das definições incidentes permite também realizar algumas análises estáticas de código, como identificar variáveis usadas antes de receberem algum valor, ou variáveis que recebem valores que não são utilizados (dead code).

- Definições incidentes : uma definição **d** **alcança** um ponto **p** se há um caminho de **d** a **p** e **d** não é morta naquele caminho.
- Uma **definição d** de uma variável **x** é um enunciado que atribui ou pode atribuir um valor a **x**.
 - Definição inambígua: uma atribuição a **x**
 - Definição ambígua: Um enunciado que pode ou não alterar o valor de **x** (uma chamada de função com **x** passado por referência ou em que **x** está no escopo)
- Normalmente associa-se um identificador **d** a uma definição, representando-a como "**d**: $x = a \text{ op } b$ " onde **d** é o identificador da definição.

- No exemplo abaixo, a definição **d1** **incide** na definição **d2** porque o valor de **y** usado em **d2** é o que foi atribuído em **d1**

d1 : y := 3

d2 : x := y

- Já no exemplo abaixo, a definição **d1** não incide na definição **d3** porque a definição **d1** é morta na definição **d2**, uma vez que **y** é redefinido.

d1 : y := 3

d2 : y := 4

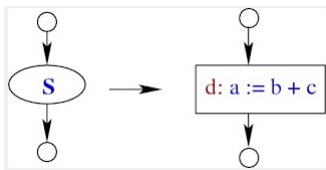
d3 : x := y

Efeito de um bloco básico:

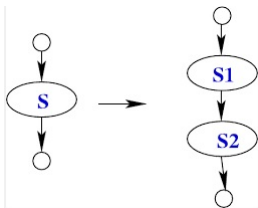
- Efeito da atribuição $a = b + c$
 - **Usa** as variáveis (b,c)
 - **Mata** todas as **outras** definições de a
 - **Gera** uma nova definição de a
- Atribuições com outros tipos de expressão (p.ex. $a = b[i]$ ou $a = \text{fat}(i)$) tem o mesmo efeito.
 - Outras instruções como desvios (condicionais ou incondicionais) não geram e nem matam nada.
 - Assim como comandos-expressão que não atualiza variáveis (incluindo chamadas de funções void).

- A composição dos efeitos das atribuições é o efeito do bloco básico
- $GEN[B]$ = conjunto de todas as definições dentro de B que são visíveis imediatamente após o bloco.
- $KILL[B]$ = união de todas as definições em todos os blocos básicos do grafo de fluxo que são mortas por comandos em B

- Constroem-se comando a comando
 - O comando s , " $d: v \leftarrow x \text{ op } y$ ", **gera** d
 - O comando s , " $d: v \leftarrow x \text{ op } y$ ", **mata** todas as outras definições de v
- Ou,
 - $\text{Gen}[s] = \{ d \}$
 - $\text{Kill}[s] = \text{defs}(v) - \{ d \}$
- onde $\text{defs}(v)$ são todas as outras definições de v no programa.



- $\text{Gen}[s] = \{ d \}$
- $\text{Kill}[s] = \text{defs}(a) - \{ d \}$



- $\text{Gen}[s] = \text{Gen}[S2] \cup (\text{Gen}[S1] - \text{Kill}[S2])$
- $\text{Kill}[s] = \text{Kill}[S2] \cup (\text{Kill}[S1] - \text{Gen}[S2])$

Definições incidentes (Reaching Definitions)

- Uma definição de uma variável **v** no ponto **d** de um programa **alcança** um ponto **u** do programa se existe um caminho de arestas de **d** até **u** que não contem uma nova definição de **v**

```
1: n <- 10
2: older <- 0
3: old <- 1
4: result <- 0
5: if n <= 1 goto 14
6: i <- 2
7: if i > n goto 13
8: result <- old + older
9: older <- old
10: old <- result
11: i <- i + 1
12: JUMP 7
13: return result
14: return n
```

- Para cada bloco b básico, determinar se cada definição no programa alcança b
- $In[n]$: Conjunto de definições que alcança o início do nodo n
- $Out[n]$: Conjunto de definições que alcança o fim do nodo n
- Representação: $IN [b]$, $OUT [b]$: um vetor de bits, um bit para cada definição

Problema das definições incidentes

- Conjuntos de definições constituem o domínio desse problema de fluxo de dados
- Computa-se superconjuntos de definições como valores seguros
- É seguro assumir que uma definição alcança um ponto, mesmo que ele não atinja.
- No exemplo a seguir assume-se que ambas definições $a=2$ e $a=4$ alcançam o ponto após o if, mesmo que a atribuição $a=4$ nunca seja executada.

```
if (a==b)
```

```
    a=2;
```

```
else if (a==b)
```

```
    a=4;
```

- Análises mais complexas, como análise de intervalo de valores podem identificar a situação acima.

Problema das Definições Incidentes (3)

- Equações de Fluxo de Dados (restrições)
 - $IN[B] = \cup_{P \text{ em } pred(B)} OUT[P]$
 - $OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$
 - $IN[B] = \emptyset$, para todo B (inicialização)
- Se algumas definições alcançam B1 (entrada) então $IN[B1]$ é inicializado com essas definições
- É um problema de análise dataflow Forward (já que $OUT[B]$ é expresso em termos de $IN[B]$), e o operador de confluência é $\cup$

- $\text{out}(S) = \text{gen}(S) \cup [\text{in}(S) - \text{kill}(S)]$
 - Gen : definições geradas
 - Kill : definições mortas
 - In : definições na entrada
 - Out : definições na saída

- **Gen e kill** para cada comando:

	gen	kill
1: $n \leftarrow 10$	1	
2: $older \leftarrow 0$	2	9
3: $old \leftarrow 1$	3	10
4: $result \leftarrow 0$	4	8
5: if $n \leq 1$ goto 14		
6: $i \leftarrow 2$	6	11
7: if $i > n$ goto 13		
8: $result \leftarrow old + older$	8	4
9: $older \leftarrow old$	9	2
10: $old \leftarrow result$	10	3
11: $i \leftarrow i + 1$	11	6
12: JUMP 7		
13: return result		
14: return n		

Um algoritmo iterativo para computar alcance de definições

para cada bloco básico B faça

$\{ \text{IN}[B] = \emptyset; \text{OUT}[B] = \text{GEN}[B]; \}$

trocou = true;

while trocou do $\{ \text{trocou} = \text{false};$

 for each block B do $\{$

$\text{IN}[B] = \bigcup_{P \text{ em } \text{pred}(B)} \text{OUT}[P]$

 oldout = $\text{OUT}[B]$;

$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] - \text{KILL}[B]);$

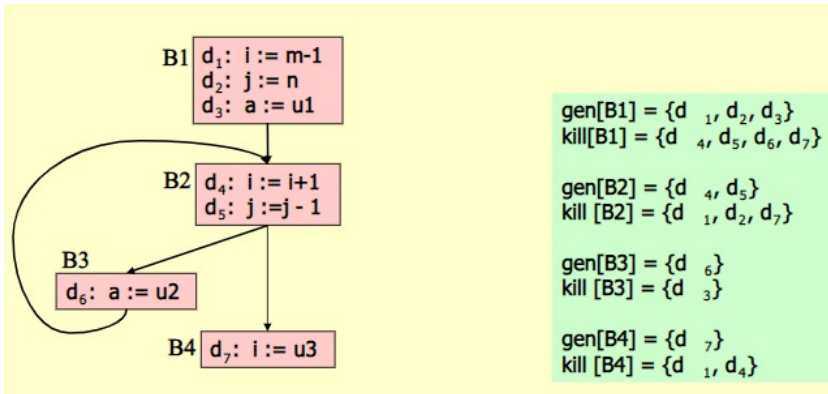
 if $(\text{OUT}[B] \neq \text{oldout})$ trocou = true;

$\}$

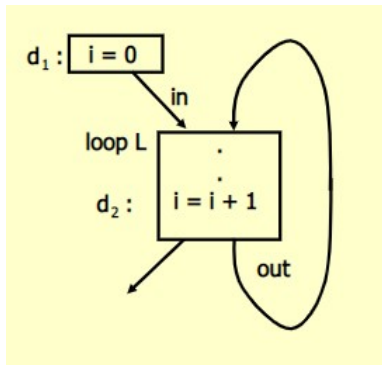
$\}$

- GEN, KILL, IN, e OUT são todos representados como vetores de bits com um bit para cada definição no grafo de fluxo

Exemplo clássico (do livro do Dragão)



- Ex: Quais definições alcançam a saída do bloco L?



Uso de definições incidentes na identificação de variáveis não inicializadas

- Um uso da informação de definições incidentes é na identificação de uso de variáveis que ainda não receberam algum valor, na análise estática de código.
- Isso pode ser feito criando no início do grafo um bloco com definições "dummy" de todas as variáveis do código e calculando as definições incidentes.
- Se a definição falsa alcança algum ponto em que a variável é usada, então há um uso da variável antes de receber algum valor.
- Nesse caso o compilador pode gerar um warning, um erro ou até utilizar algum valor *default*.

- A figura abaixo, à direita, mostra o código gerado para a função à esquerda, sem nenhuma otimização.

x86-64 gcc 10.3

-O0

square:

```
int square(int n) {  
    short int x;  
    return x*31;  
}
```

```
push    rbp  
mov     rbp, rsp  
mov     DWORD PTR [rbp-20], edi  
movsx   edx, WORD PTR [rbp-2]  
mov     eax, edx  
sal     eax, 5  
sub     eax, edx  
pop     rbp  
ret
```

- A figura abaixo, à direita, mostra o código gerado para a mesma função à esquerda, com nível -O1 de otimização.
- Que otimizações foram feitas?

x86-64 gcc 10.3





-O1

```
int square(int n) {  
    short int x;  
    return x*31;  
}
```

```
square:  
    mov     eax, 0  
    ret
```


- Propagação de Cópias é o processo de substituir as ocorrências de variáveis que são alvos de atribuição direta ($x \leftarrow y$) pelos seus valores.
- No código a seguir:

$y = x$

$z = 3 + y$

- Propagação de cópia resultaria em

$z = 3 + x$

- Atribuições diretas como essa podem ocorrer no código-fonte, como resultado do processo de geração de código ou como resultado de otimizações.

- Para que a propagação de cópia seja aplicada algumas condições devem ser atendidas.

- Considere a atribuição direta

$s: x = y$

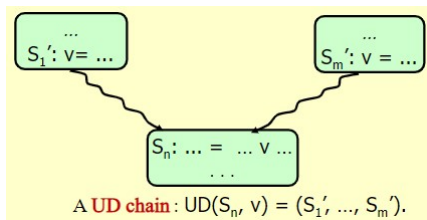
- Para que x possa ser substituído por y em um comando u onde x é usado:
 - A definição s deve ser a única definição de x que alcança o comando u
 - Não pode haver definições de y em qualquer caminho de s a u
- A primeira condição pode ser verificada pela análise de definições incidentes e cadeias use-def.

Problema da Propagação de cópias

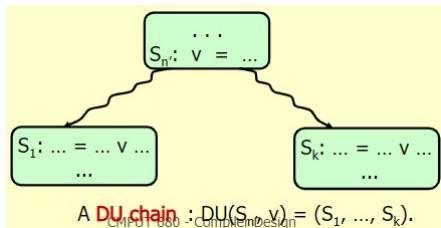
- Para cada bloco b básico, determinar quais comandos de cópia alcançam o início do bloco b
- $In[n]$: Conjunto de comandos de cópia que alcança o início do nodo n
- $Out[n]$: Conjunto de comandos de cópia que alcança o fim do nodo n
- $Kill[n]$: Conjunto de comandos de cópia $s: x = y$ que são mortos (por uma redefinição de x ou y) no bloco n
- $Gen[n]$: Conjunto de comandos de cópia $s: x = y$ gerados no bloco n

- Conjuntos de comandos de cópia constituem o domínio desse problema de fluxo de dados
- Equações de Fluxo de Dados (restrições)
 - $IN[B] = \bigcap_{P \text{ em } \text{pred}(B)} OUT[P]$
 - $OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$
 - $IN[B] = \emptyset$, para todo B (inicialização)
- É um problema de análise dataflow Forward (já que $OUT[B]$ é expresso em termos de $IN[B]$, e $IN[B]$ é expresso em termos das saídas dos predecessores), e o operador de confluência é $\cap$

- Uma cadeia uso-definição (use-def chain) é uma estrutura de dados que consiste de um uso, U , de uma variável, e todas as definições D da variável que podem alcançar o uso sem outra definição entre D e U .



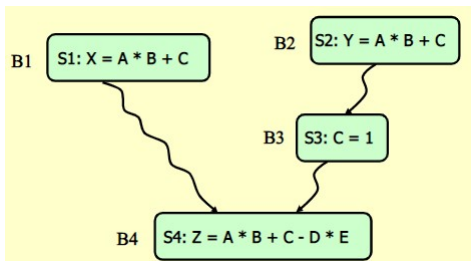
- De forma análoga, uma cadeia definição-uso (def-use chain) é uma estrutura que consiste de uma definição D e todos os usos U que podem ser alcançados sem que haja uma nova definição da variável entre D e U .



- Propagação de cópia faz uso de definições incidentes, cadeias use-def e cadeias def-use para computar quais ocorrências da variável alvo podem ser substituídas
- Se todos os usos da variável alvo podem ser substituídas, a atribuição pode ser eliminada
- Propagação de cópias é uma otimização frequentemente usada após outras otimizações.
- Algumas otimizações, como eliminação de subexpressões comuns exigem que a propagação de cópias seja feita após para melhorar a eficiência da eliminação de subexpressões comuns.

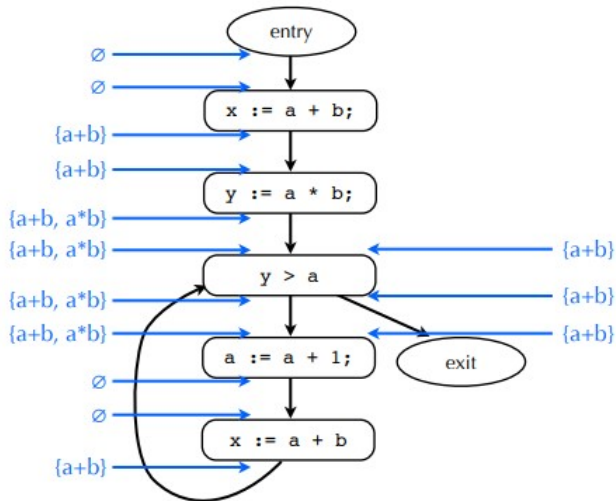
Análise de Expressões Disponíveis (available expressions)

- A análise de expressões disponíveis permite identificar em cada ponto (possivelmente em outro bloco) expressões que foram calculadas anteriormente e que ainda estão disponíveis (eliminação de subexpressões comuns), eliminando a necessidade de recalculá-las.
- No exemplo abaixo, a expressão $a*b$ está disponível na entrada do bloco B4?



- Uma expressão está **disponível** em um ponto **p** se
 - a expressão é calculada em todos os caminhos que levam a **p** e
 - O valor da expressão (e de nenhuma das variáveis envolvidas em seu cálculo) foi alterado em qualquer dos caminhos, desde a definição.

- As equações de fluxo de dados para análise de expressões disponíveis são semelhantes às equações de definições incidentes.
- O domínio das equações são expressões.
- Cada comando de atribuição pode matar e gerar expressões.
- A atribuição
$$A = B + C$$
 - **gera** a expressão **B+C**
 - **mata** todas as expressões em que **A** é usado
 - No caso particular em que o operando destino é um dos operandos à direita da instrução (ex: $A = A + B$), a expressão não se torna disponível, uma vez que a atribuição a **A** é feita **após** a avaliação da expressão.



- A análise de expressões disponíveis é uma análise forward. As expressões disponíveis na entrada de um bloco são definidas pela saída dos predecessores, e as expressões disponíveis na saída de um bloco são definidas pela função de transferência do bloco (Gen, Kill) e pelas expressões disponíveis na entrada do mesmo.
- $Gen[B]$ é o conjunto de definições no bloco, após as quais nenhum dos seus operandos é alterado até o final do bloco.
- $Kill[B]$ é o conjunto de expressões que utilizam qualquer variável que é alvo de uma atribuição dentro do bloco.

- Como, para uma expressão estar disponível em um ponto, ela tem que ter sido definida em todos os caminhos até o ponto, o operador de confluência é a intersecção.
- A função de transferência de cada bloco B e a equação de fluxo de dados podem ser descritas por:
 - $out[B] = Gen[B] \cup (In[B] - Kill[B])$
 - $in[B] = \bigcap_{p \text{ em } pred[n]} Out[p]$
- E inicialmente, $In(S) = \emptyset$ e $Out(S) = Gen(S)$ sendo S o bloco inicial e
- $In(B) =$ Todas as expressões no programa e $Out(B) = Gen(B) \cup (In(B) - Kill(B))$ para todos os outros blocos

$\text{In}(\text{Start}) = \emptyset$

$\text{Out}(\text{Start}) = \text{Gen}(\text{Start})$

FOR B $\neq$ Start DO

$\text{In}(B) = \text{All Expressions in program}$

$\text{Out}(B) = (\text{In}(B) - \text{Kill}(B)) \cup \text{Gen}(B)$

ENDFOR

WHILE there are changes DO

 FOR B $\neq$ Start DO

$\text{In}(B) = \bigcap \text{Out}(P) \text{ para } P \text{ in Pred}(B)$

$\text{Out}(B) = (\text{In}(B) - \text{Kill}(B)) \cup \text{Gen}(B)$

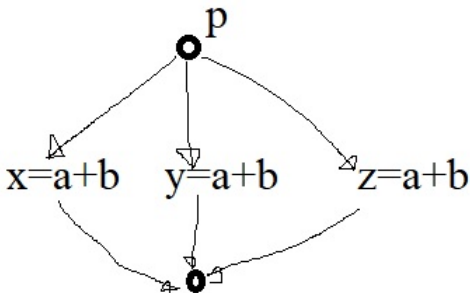
 ENDFOR

ENDWHILE

Análise de Expressões "Ocupadas" (Very Busy Expressions)

- Uma expressão **A op B** é dita "ocupada" em um ponto **p** se, ao longo de todo caminho após **p** a expressão é avaliada sem que haja uma redefinição de **A** ou **B**.
- Se uma expressão é ocupada em um ponto **p**, pode ser vantajoso calculá-la, para reduzir o tamanho do código evitando que a mesma expressão seja calculada em vários pontos.
- Pode ser implementada por análise dataflow backward.

- Na figura abaixo, a expressão $a+b$ é calculada em todos os ramos a partir do ponto p.
- Nesse caso ela pode ser calculada no ponto p, e o resultado utilizados nos 3 ramos.



- O efeito de um bloco **B** em uma expressão **Z = X op Y** é o mesmo que para a análise de expressões disponíveis, mas na direção contrária.
- Assim, a expressão **d: a = b + c** dentro de um bloco "gera" um uso da expressão "**b+c**" do ponto **d** para cima, e "mata" qualquer expressão que utilize **a** após o ponto **d**.
 - $Kill(B) = \{ X \text{ op } Y \mid X \text{ ou } Y \text{ são definidos antes do uso de } X \text{ op } Y \text{ em } B \}$
 - $Gen(B) = \{ x \text{ op } Y \mid X \text{ op } Y \text{ é usada em } B \text{ antes de qualquer definição de } X \text{ ou } Y \}$

- Quais os conjuntos **Kill(B)** e **Gen(B)** para o bloco abaixo?

```
a=a+b  
b=c+d  
c=3+b  
b=d+3  
e=e*2
```

- Equações para "very busy expressions"
 - $\text{Kill}(B) = \{X \text{ op } Y\}$

- Expansão inline é uma técnica de otimização que substitui a chamada de uma função pelo corpo da função.
- Expansão inline é a substituição de uma chamada de função por uma cópia do corpo da função, com alguma "cola" para substituir passagem de parâmetros e retorno de resultados
- Essa otimização pode melhorar o desempenho de um programa (evitando o overhead de chamadas de função), à custa de um aumento no tamanho do código gerado.

[Voltar para o índice](#)

- Em algumas situações a expansão pode apresentar novas oportunidades de otimização
- Expansão inline pode ser feita automaticamente pelo compilador ou pode ser configurada por diretivas de compilação específicas ou usando a palavra inline no código fonte.

[Voltar para o índice](#)

- Compiladores utilizam diversos mecanismos para decidir quantas e quais chamadas de função devem ser expandidas.
- Critérios utilizados para decidir isso pode ser o tamanho do corpo da função bem como o tipo de parâmetros passados (constantes ou expressões, passagem por valor ou referência..)
- Passagem de constantes como parâmetros cria mais oportunidades de otimização.
- No gcc, a expansão é feita nos primeiros passos sobre o Gimple, no passo 18 (.018t.inline), nos níveis de otimização -O2 e -O3.
- A lâmina seguinte mostra o gimple no passo 17 e no passo 18, imediatamente após fazer a expansão online

```

int fat(int N)
{
    int i=1,fat=1;
    while (i<=N)
    {
        fat=fat*i;
        i=i+1;
    }
    return fat;
}

int main()
{
    int a,b;
    scanf("%d",&a);
    b=fat(a);
    printf("%d",b);
}

```

```

main ()
{
    int b;
    int a;
    int a.0_3;

    <bb 2>:
    scanf ("%d", &a);
    a.0_3 = a;
    b_4 = fat (a.0_3);
    printf ("%d", b_4);
    return;
}

```

```

<bb 3>:
fat_9 = fat_7 * i_8;
i_10 = i_8 + 1;

<bb 4>:
# i_8 = PHI <1(2), i_10(3)>
# fat_7 = PHI <1(2), fat_9(3)>
if (a.0_3 >= i_8)
    goto <bb 3>;
else
    goto <bb 5>;

<bb 5>:
fat_11 = fat_7;
_12 = fat_11;
b_4 = _12;
printf ("%d", b_4);

```

Vantagens da Expansão Inline

- Elimina o custo da chamada de função e instrução de retorno, bem como código inicial e final acrescentados a cada chamada de função pelo compilador
- A eliminação de desvios mantendo próximo na memória todo o código a ser executado pode melhorar a localidade temporal melhorando o desempenho da cache
- A expansão pode criar oportunidades para outras otimizações, como uma constante passada como parâmetro ser propagada ou parte da função ser retirada de um laço

- A figura abaixo mostra o código gerado para uma chamada da função `fat`, no nível de otimização `-O1`, sem ser aplicada a expansão `inline`:

```
int fat(int n) {  
    int f=1;  
    for (int i=1;i<=n;i++) f*=i;  
    return f;  
}
```

```
int a,b;  
scanf("%d",&a);  
b=fat(a);  
printf("%d",b);
```

x86-64 gcc 11.2



-O1

sem expansão inline

```
mov     edi, DWORD PTR [rsp+12]  
call    fat  
mov     esi, eax
```

- E a figura abaixo mostra o código gerado para a mesma chamada da função fat, no nível de otimização -O2, ao ser aplicada a expansão inline:

```
int fat(int n) {  
    int f=1;  
    for (int i=1;i<=n;i++) f*=i;  
    return f;  
}  
  
int a,b;  
scanf("%d",&a);  
b=fat(a);  
printf("%d",b);
```

x86-64 gcc 11.2  -O2

com expansão inline

```
mov     edx, DWORD PTR [rsp+12]  
test    edx, edx  
jle     .L10  
add     edx, 1  
mov     eax, 1  
mov     esi, 1  
.L9:  
imul    esi, eax  
add     eax, 1  
cmp     eax, edx  
jne     .L9
```

- A figura abaixo mostra o código gerado para a chamada da função `fat`, nos níveis de otimização `-O1` (sem expansão) `-O2` (com expansão), passando um valor constante como parâmetro:

```
int fat(int n) {  
    int f=1;  
    for (int i=1;i<=n;i++) f*=i;  
    return f;  
}
```

```
int main()  
{
```

```
    int b;  
    b=fat(5);  
    printf("%d",b);  
}
```

x86-64 gcc 11.2



-O1

sem expansão inline

```
mov     edi, 5  
call    fat
```

x86-64 gcc 11.2



-O2

com expansão inline

```
mov     esi, 120
```

- Ao fazer a expansão da função, juntamente com o desdobramento de laço, o compilador conseguiu calcular durante a compilação o fatorial de 5 e substituiu a chamada da função pelo valor correspondente.

- O corpo da função expandida substitui a chamada
- Na expansão, cada parâmetro formal recebe a expressão do parâmetro atualé substituída pelo parâmetro atual

- Se temos uma chamada de função:

```
x = f(exp1 , ... , expn );
```

- e a função f é definida como:

```
type0 f(type1 x1 , ... , typen xn).  
{  
  corpo da função  
  return(exp);  
}
```

- onde o return mostrado é o único ponto de saída da função, podemos substituir a chamada pelo bloco

```
{  
  type1 x1 = exp1;  
  typen xn = expn;  
  corpo da função  
  x = exp;  
}
```

- Note que se, digamos, `expn` se refere a uma variável com o mesmo nome que `x1`, a referência será a uma instância diferente de `x1` depois da transformação.
- Portanto evita-se variáveis na função expandida mascarando variáveis usadas no comando de chamada da função.
- Pode-se conseguir isso renomeando as variáveis na função expandida para nomes novos, não utilizados antes da expansão da função.
- Como, a não ser que o corpo da função seja muito pequeno, o tamanho do código pode aumentar muito.
- Por essa razão, pode-se usar um limite no tamanho das funções a serem expandidas

- Qual limite utilizar depende do equilíbrio desejado entre velocidade e tamanho.
- Mas mesmo ao otimizar para velocidade, cuidado deve ser tomado para não expandir muito, já que programas maiores podem executar mais devagar que programas menores devido ao pior uso da cache.
- Cuidado deve ser tomado ao expandir funções recursivas: Se o corpo da função expandida contem uma chamada que vai ser expandida, isso pode levar a uma expansão infinita.
- Por isso é comum limitar a expansão a apenas um ou dois níveis ou tratar funções indiretamente recursivas como um caso especial.

- Um caso particular é a recursão de cauda, em que o último comando da função é uma chamada recursiva, como a função abaixo:

```
void esc(int n){  
    if (n==0) return;  
    printf("%d\n",n);  
    esc(n-1);}
```

- Nesse caso a chamada recursiva pode ser substituída por um desvio para o início da função com poucas alterações.
- A figura abaixo mostra o código gerado para a função acima:

```
#include <stdio.h>
```

```
void esc(int n)
```

```
{
```

```
if (n==0) return;
```

```
printf("%d\n",n);
```

```
esc(n-1);
```

```
}
```

```
int main()
```

```
{
```

```
esc(5);
```

```
}
```

```
main:
```

```
push    rbx
```

```
mov     ebx, 5
```

```
.L14:
```

```
mov     esi, ebx
```

```
xor     eax, eax
```

```
mov     edi, OFFSET FLAT:LC0
```

```
call    printf
```

```
sub     ebx, 1
```

```
jne     .L14
```

```
xor     eax, eax
```

```
pop     rbx
```

```
ret
```

Remoção de recursão de cauda (tail recursion)

- Uma otimização importante em chamadas de funções é a eliminação de chamadas de cauda.
- Nela, o compilador substitui a sequência de código
 - Call routine
 - Return
- por
 - Jump routine
- Isto não apenas reduz código e melhora velocidade, mas também reduz o uso da pilha.

- Uma **chamada de cauda** (tail call), é uma chamada de função em que a chamada da função é o último comando a ser executado no corpo da função.
- Corpos de funções podem conter mais uma chamada de cauda; por exemplo, na routine abaixo ambas as chamadas $\text{gcd}(b, a)$ e $\text{gcd}(a-b, b)$ são chamadas de cauda:

```
int gcd(int a, int b) {  
    if (b == 0) return a;  
    if (b == 1) return 1;  
    if (b > a)  
        return gcd(b, a);  
    return gcd(a-b, b);  
}
```

Figure: Implementação Recursiva do Maior Divisor Comum

- chamadas de cauda são importantes por várias razões:
 - Ocorrem com frequência;
 - São fáceis de identificar
 - Podem ser facilmente eliminadas, levando a ganhos significativos

- É comum em funções terminarem em uma chamada de cauda.
- O código do quicksort, a seguir, mostra uma chamada de cauda e uma chamada que não é de cauda.
- A frequência de chamadas de cauda é ainda maior em código gerado para linguagens funcionais e lógicas.
- Chamadas de cauda são fáceis de detectar no grafo de fluxo de controle: correspondem a nós de chamada de rotina cujo sucessor é um nodo de retorno.
- Muito do custo de gerenciar uma chamada de cauda é supérfluo e pode ser eliminado.

```
PROCEDURE QuickSort(A: ArrayType);  
VAR Low, High: ArrayType;  
BEGIN  
  Split (A, Low, High);  
  QuickSort(Low);  
  QuickSort(High)  
END
```

- Há diversas razões para isso. Em primeiro lugar, não há razão para retornar à rotina R que fez a chamada (a chamadora), quando a rotina C que foi chamada terminou, quando não há nenhum código no chamador para ser executado em seguida.
- Portanto não há necessidade de um endereço de retorno para quando C termina, nem há necessidade de armazenar um endereço de retorno quando C é chamado. A transferência de C pode ser feita por uma simples instrução de desvio.

- Segundo, os parâmetros formais e variáveis locais da função chamadora R não são mais necessários após os parâmetros atuais serem computados, contanto que esses parâmetros não contenham pointers para os parâmetros formais ou variáveis locais de R.
- E terceiro, como os parâmetros formais e variáveis locais não são mais necessários, e não há necessidade de um endereço de retorno, o registro de ativação de R está livre antes da chamada de C, e pode ser utilizado, com alguma modificação, como registro de ativação de C.

- Quando R e C são a mesma rotina, há uma recursão de cauda direta.
- No exemplo anterior:
- O corpo da rotina contém duas chamadas recursivas.
- Para eliminar as chamadas e reutilizar o mesmo registro de ativação, deve-se :
 - 1 atribuir os parâmetros atuais (os enviados na chamada) aos parâmetros formais e,
 - 2 substituir a chamada por um desvio (jump ou goto).
- Para realizar (1) pode-se utilizar variáveis temporárias.
- Os parâmetros atuais são atribuídos a variáveis temporárias, e os temporários são atribuídos aos parâmetros formais, o que previne problemas com alterações dos parâmetros formais durante a avaliação das expressões (overlap)
- O jump utiliza o label no início do corpo da rotina.
- Isso gera o código a seguir:

```

int gcd(int a, int b) {
L_gcd:
if (b == 0) return a;
if (b == 1) return 1;
if (b > a) { /* return gcd(b, a); */
int tmp_1 = b, tmp_2 = a;
a = tmp_1, b = tmp_2;
goto L_gcd;
}
else
{ /* return gcd(a-b, b); */
int tmp_1 = a-b, tmp_2 = b;
a = tmp_1, b = tmp_2;
goto L_gcd;
}
}
}

```

Figure: Implementação iterativa do GCD por eliminação de chamadas de cauda

- Otimizações de blocos básicos podem ser aplicadas ao código iterativo, levando ao código a seguir:

```
int gcd(int a, int b) {  
L_gcd:  
    if (b == 0) return a;  
    if (b == 1) return 1;  
    if (b > a) { /* return gcd(b, a); */  
        int tmp_2 = a;  
        a = b, b = tmp_2;  
        goto L_gcd;  
    }  
    else { /* return gcd(a-b, b); */  
        a = a-b;  
        goto L_gcd;  
    }  
}
```

Figure: Implementação iterativa do GCD após otimizações de blocos básicos

- Quando R e C não são a mesma função, ainda há uma forma simples de eliminar a chamada a C: expandi-la. Se isso resulta em uma ou mais chamadas recursivas diretas, R e C são mutuamente recursivas e as chamadas podem ser eliminadas como visto. Se não, repete-se o processo tomando cuidado com os pontos críticos da expansão de funções.
- Essa abordagem falha se nem todo o código fonte está disponível, por exemplo, ao chamar uma função de uma biblioteca compilada. Mas na maioria dos casos, funções mutuamente recursivas ocorrem no mesmo módulo.

- Ao gerar código assembly, a eliminação da chamada é ainda mais fácil, já que o início do código de C está disponível.
- Os parâmetros de C são escritos no registro de ativação de R, usando variáveis temporárias se necessário; o registro de ativação parece com o registro de ativação de C, e um desvio para o início de C basta para transferir o controle.

Desdobramento de laços (loop unrolling)

- Alguns laços tem um corpo tão pequeno que a maior parte do tempo é gasto atualizando a variável contadora do laço e testando a condição de saída do laço.
- É mais eficiente desenrolar tais laços, colocando duas ou mais cópias do corpo do laço em fileira.
- Isso evita atualizar e testar o contador do laço em cada corpo do laço.

[Voltar para o índice](#)

Como Desenrolar um laço

- Dado um laço L com cabeçalho h e arestas de retorno $s_i \leftarrow h$, pode-se desenrolar L como se segue:
 - 1 Copie nós para criar um laço L' com cabeçalho h' e arestas de retorno $s'_i \leftarrow h'$.
 - 2 Mude todas arestas de retorno em L de $s_i \leftarrow h$ para $s_i \leftarrow h'$.
 - 3 Altere todas as arestas de retorno em L' de $s'_i \leftarrow h'$ para $s'_i \leftarrow h$.

Exemplo: Desdobramento Inútil de Laço

Antes:

```
L1  x ← M[i]
      s ← s + x
      i ← i + 4
      if i < n goto L1
L2
```

Depois:

```
L1  x ← M[i]
      s ← s + x
      i ← i + 4
      if i < n goto L'1 else L2
L'1 x ← M[i]
      s ← s + x
      i ← i + 4
      if i < n goto L1 else L2
L2
```

Como Desenrolar um laço

- Precisamos de uma variável de indução i tal que $i \leftarrow i + \Delta$ domina cada aresta de retorno do laço.
- Então sabemos que cada iteração incrementa i por exatamente a soma de todos os Δs .
- Pode-se combinar todos os incrementos e testes de saída e colocar apenas um na última iteração.

Exemplo: Desdobramento de Laço

Antes:

```
L1  x ← M[i]
      s ← s + x
      i ← i + 4
      if i < n goto L1
L2
```

Depois:

```
L1  x ← M[i]
      s ← s + x
      x ← M[i+4]
      s ← s + x
      i ← i + 8
      if i < n goto L1
L2
```

- Será que isto vai funcionar se o laço original tiver um número ímpar de iterações?
- Como podemos tratar um número ímpar de iterações?
- Como podemos modificar a nossa estratégia e desenrolar o laço por um fator de K ?

Quantidade ímpar de iterações

```

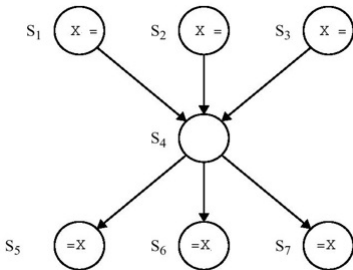
      if  $i < n - 8$  goto  $L_1$  else  $L_2$ 
 $L_1$    $x \leftarrow M[i]$ 
       $s \leftarrow s + x$ 
       $x \leftarrow M[i+4]$ 
       $s \leftarrow s + x$ 
       $i \leftarrow i + 8$ 
      if  $i < n - 8$  goto  $L_1$  else  $L_2$ 
 $L_2$    $x \leftarrow M[i]$ 
       $s \leftarrow s + x$ 
       $i \leftarrow i + 4$ 
      if  $i < n$  goto  $L_2$  else  $L_3$ 
 $L_3$ 
```

- Como podemos tratar um número ímpar de iterações?
 - Há um "epílogo" com o código do laço ao final para executar a iteração faltante (no caso de um desdobramento de 2)
- Como podemos modificar a nossa estratégia e desenrolar o laço por um fator de K ?
 - O "epílogo" fica dentro de um laço para completar as iterações restantes.

SSA - Static single assignment

- Em projeto de compiladores, a forma SSA (static single assignment) é uma representação de código intermediário em que cada variável tem exatamente **uma definição**, e toda variável deve ser definida antes de ser usada.
- Variáveis existentes no código intermediário original são divididas em versões, normalmente associando-se índices a cada nova versão de uma variável definida anteriormente.

- Na figura abaixo, 3 definições possivelmente diferentes de x chegam em S_4 , o que dificulta algumas otimizações como propagação de cópias ou qualquer outra que use cadeias uso-definição.



- Na forma SSA, cadeias use-def são explícitas e cada uma contem exatamente um elemento, uma vez que cada variável só é definida em um ponto.
- A definição de uma variável pode estar em um laço e ser executada várias vezes.

[Voltar para o índice](#)

- A forma SSA é utilizada na grande maioria dos compiladores atuais.
- O gcc converte a representação intermediária para a forma SSA nos primeiros passos do GIMPLE, e o mantém até o final da representação em GIMPLE, antes de converter para RTL.
- O LLVM mantém a representação SSA durante todo o processo, até a tradução para código específico da arquitetura alvo.

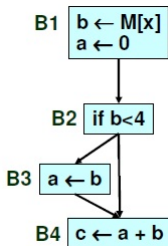
Vantagens da forma SSA

- Análise de fluxo de dados mais simples
- Não é necessário calcular In, Out, Gen, Kill
- Simplifica/melhora diversas otimizações:
 - Propagação de constantes
 - Identificação de condições constantes
 - Eliminação de Código Morto
 - Redução de força
 - Alocação de registradores
- Forma SSA é relacionada com estruturas de dominância

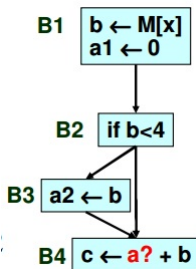
- Converter um código para a forma SSA consiste em substituir a variável destino de uma atribuição por uma nova "versão" da variável (normalmente atribuindo um índice a ela) e substituindo todos os usos da variável (informação obtida na cadeia definição-uso) pela versão que alcança o uso.

Forma SSA no fluxo de execução

- Este código está na forma SSA?
- Não, duas definições de `a` aparecem no código (em B1 e B3)
- Como transformar este código para a forma SSA?
- Criamos duas versões de `a`, uma para B1 e uma para B3.



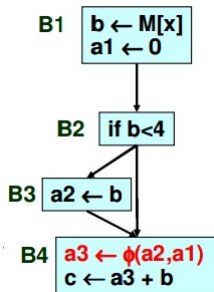
- Mas qual versão deve ser usada em B4?
- Define-se uma função que "sabe" qual caminho foi utilizado para alcançar o bloco básico B4:



$$\phi(a_1, a_2) = \begin{cases} a_1 & \text{se chegarmos a B4 vindo de B2} \\ a_2 & \text{se chegarmos a B4 vindo de B3} \end{cases}$$

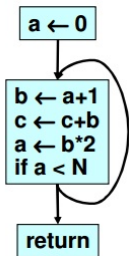
Forma SSA no fluxo de execução

- Mas qual versão deve ser usada em B4?
- Define-se uma função que "sabe" qual caminho foi utilizado para alcançar o bloco básico B4:

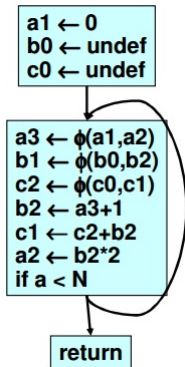


$$\phi(a_1, a_2) = \begin{cases} a_1 & \text{se chegarmos a B4 vindo de B2} \\ a_2 & \text{se chegarmos a B4 vindo de B3} \end{cases}$$

Um exemplo de laço

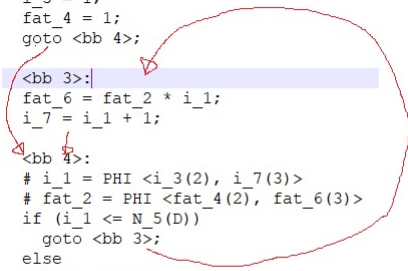


- $\phi(b_0, b_2)$ não é necessário porque b_1 nunca é usado. Mas a fase do compilador que gera funções ϕ não sabe disso.
- Funções ϕ desnecessárias são posteriormente eliminadas por "eliminação de código morto".



- No gcc a forma SSA é utilizada a partir do passo 15 do gimple (teste.c.015t.ssa) e as funções ϕ são inseridas.
- A figura à direita mostra a representação SSA do trecho de código à esquerda, de uma função de fatorial.

```
int i=1,fat=1;
while (i<=N)
{
    fat=fat*i;
    i=i+1;
}
return fat;
```



```
<bb 2>:
i_3 = 1;
fat_4 = 1;
goto <bb 4>;

<bb 3>:
fat_6 = fat_2 * i_1;
i_7 = i_1 + 1;

<bb 4>:
# i_1 = PHI <i_3(2), i_7(3)>
# fat_2 = PHI <fat_4(2), fat_6(3)>
if (i_1 <= N_5(D))
    goto <bb 3>;
else
    goto <bb 5>;

<bb 5>:
_8 = fat_2;
return _8;
```

- Há duas definições de **i** que atingem a condição do while, bem como duas definições de **fat**.
- São inseridas na entrada do bloco bb_4 funções ϕ para ambas.

```
i3=1  
int i=1,fat=1;  
  
while (i<=N)  
    {  
        fat=fat*i;  
i7 i=i+1;  
    }  
return fat;
```

A red arrow points from the variable **i3=1** to the variable **i** in the condition **i<=N** of the while loop. Another red arrow points from the variable **i7** in the statement **i=i+1;** back to the variable **i** in the condition **i<=N**, indicating a loop iteration.

- No passo 21 (teste.c.021t.ccp) é aplicada a transformação de "constant propagation" e o identificador `i_3` na função ϕ é substituído pelo valor da inicialização

```
<bb 2>:  
i_3 = 1;  
fat_4 = 1;  
goto <bb 4>;
```

```
<bb 3>:  
fat_6 = fat_2 * i_1;  
i_7 = i_1 + 1;
```

```
<bb 4>:  
# i_1 = PHI <i_3(2), i_7(3)>  
# fat_2 = PHI <fat_4(2), fat_6(3)>  
if (i_1 <= N_5(D))  
    goto <bb 3>;  
else  
    goto <bb 5>;
```

```
<bb 2>:  
i_3 = 1;  
fat_4 = 1;  
goto <bb 4>;
```

```
<bb 3>:  
fat_6 = fat_2 * i_1;  
i_7 = i_1 + 1;
```

```
<bb 4>:  
# i_1 = PHI <1(2), i_7(3)>  
# fat_2 = PHI <1(2), fat_6(3)>  
if (i_1 <= N_5(D))  
    goto <bb 3>;  
else  
    goto <bb 5>;
```

- E no passo 26 (teste.c.026t.copyprop) é aplicada a transformação de propagação de cópia e as inicializações de `i` e `fat` são removidas.

```
<bb 2>:  
i_3 = 1;  
fat_4 = 1;  
goto <bb 4>;
```

```
<bb 3>:  
fat_6 = fat_2 * i_1;  
i_7 = i_1 + 1;
```

```
<bb 4>:  
# i_1 = PHI <1(2), i_7(3)>  
# fat_2 = PHI <1(2), fat_6(3)>  
if (i_1 <= N_5(D))  
    goto <bb 3>;  
else  
    goto <bb 5>;
```

```
<bb 2>:  
goto <bb 4>;
```

```
<bb 3>:  
fat_6 = fat_2 * i_1;  
i_7 = i_1 + 1;
```

```
<bb 4>:  
# i_1 = PHI <1(2), i_7(3)>  
# fat_2 = PHI <1(2), fat_6(3)>  
if (i_1 <= N_5(D))  
    goto <bb 3>;  
else  
    goto <bb 5>;
```

```
<bb 5>:  
fat_8 = fat_2;  
return fat_8;
```

- Como implementar uma função que "sabe" qual caminho foi utilizado?
- Não é necessário. A função ϕ é utilizada apenas para conectar usos e definições durante a otimização, nunca é executada.

- Pode-se inserir funções ϕ para cada variável em cada join (nodos com mais de um antecessor), mas é desnecessário.
- Qual critério deve ser usado para inserir uma função ϕ para uma variável a no nodo z do grafo?
- Intuitivamente, deve-se adicionar funções ϕ se duas ou mais definições de a podem chegar no nodo z por caminhos distintos
- Uma forma eficiente de identificar os pontos de inserção de funções ϕ é através do conceito de **fronteiras de dominância**.

```
int i=1, fat=1;  
while (i<=5)  
{  
    fat=fat*i;  
    i=i+1;  
}
```

```
<bb 2>:  
i_3 = 1;  
fat_4 = 1;  
goto <bb 4>;
```

```
<bb 3>:  
fat_5 = fat_2 * i_1;  
i_6 = i_1 + 1;
```

```
<bb 4>:  
# i_1 = PHI <i_3(2), i_6(3)>  
# fat_2 = PHI <fat_4(2), fat_5(3)>  
if (i_1 <= 5)  
    goto <bb 3>;  
else  
    goto <bb 5>;
```

```
<bb 5>:  
printf ("%d", fat_2);  
return;
```

```
}
```

- Diz-se que um nodo A **domina estritamente** um nodo diferente B se é impossível alcançar o nodo B sem passar pelo nodo A (diz-se que A domina B se A estritamente domina B ou $A=B$).
- Um nodo B está na **fronteira de dominância** do nodo A se A não domina estritamente B mas domina algum predecessor imediato de B (A pode ser o predecessor imediato de B).
- Do ponto de vista de A, os nodos da fronteira de dominância são os nodos nos quais outros fluxos de execução que não passam por A são "vistos" por A pela primeira vez.

Exemplo

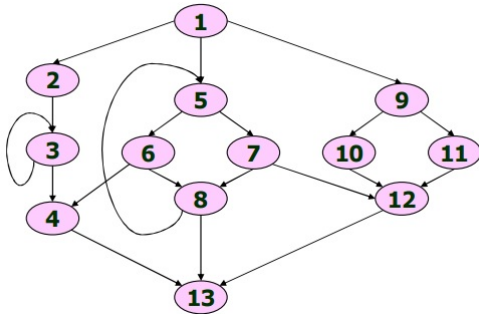


Figure: Qual a fronteira de dominância de 5?

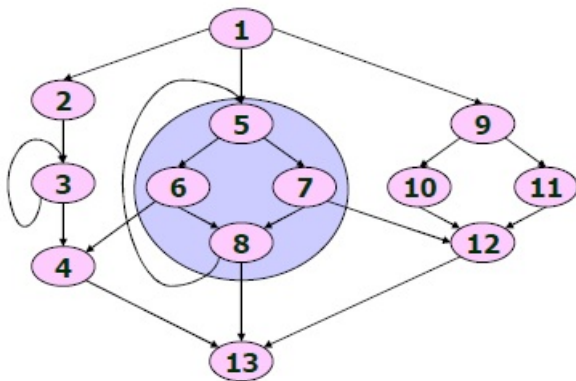
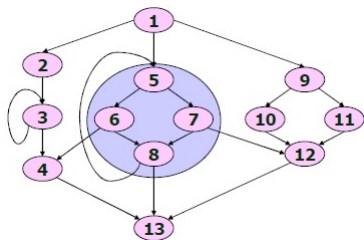
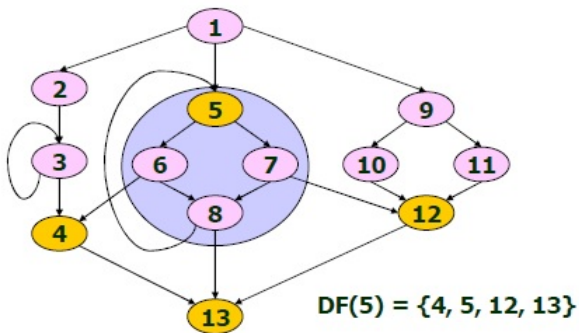


Figure: Nodos dominados pelo nodo 5 (nodos 6, 7 e 8)



- Um nodo w está na fronteira de dominância do nodo 5 se 5 domina um antecessor de w , mas 5 não domina w estritamente. Qual a fronteira de dominância de 5? São os sucessores de 6, 7 e 8 que não são dominados estritamente pelo nodo 5.



- Fronteira de dominância de 5: $DF(5)=\{4, 5, 12, 13\}$

- Fronteiras de dominância capturam os pontos precisos em que são necessárias funções ϕ : Se um nodo A define uma certa variável então aquela definição alcançará todo nodo que A domina. Apenas ao alcançar a fronteira de dominância pode haver outras definições incidentes da mesma variável.
- Assim, não é necessária nenhuma outra função ϕ para as definições existentes em A.

Algoritmo para calcular a fronteira de dominância

- O algoritmo abaixo calcula, para cada nodo **b**, a quais fronteiras de dominância ele pertence.
- Se ele só tem um predecessor, não faz parte de nenhuma fronteira de dominância (não é necessária nenhuma função ϕ)

for each node b

 dominance_frontier(b)={}

for each node b

 if the number of immediate predecessors of b ≥ 2

 for each p in immediate predecessors of b

 runner := p

 while runner $\neq$ idom(b)

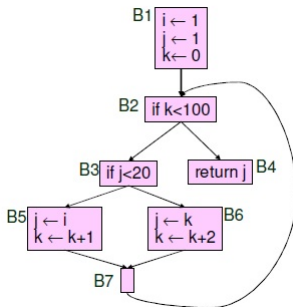
 add b to runner's dominance frontier set

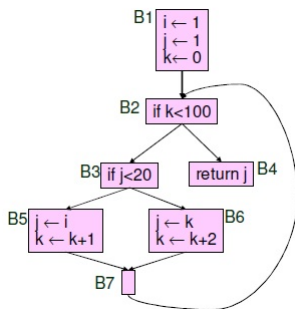
 runner := idom(runner)

Exemplo

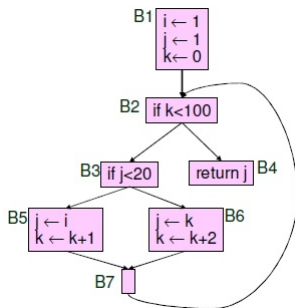
Monte o grafo de fluxo de controle do código a seguir e calcule a fronteira de dominância de cada nodo:

```
i=1;
j=1;
k=0;
while (k<100)
{
  if (j<20)
  {
    j=i;
    k=k+1;
  }
  else
  {
    j=k;
    k=k+2;
  }
}
return j;
}
```





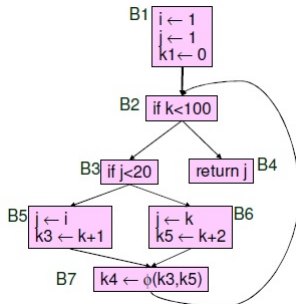
	B	pred	p	run
FD(1)={ }	1	não tem		
FD(2)={2}	2	1,7	1	1
FD(3)={2}			7	7
FD(4)={ }				3
FD(5)={ }				2
FD(6)={ }				1
FD(7)={2}				



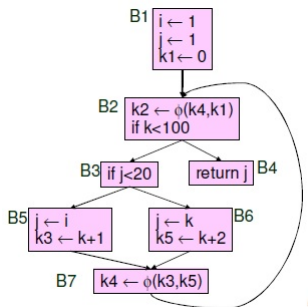
	B	pred	p	run
FD(1)={ }	1	não tem		
FD(2)={2}	2	1,7	1	1
FD(3)={2}			7	7
FD(4)={ }				3
FD(5)={7}				2
FD(6)={7}				1
FD(7)={2}				
	3	2		
	4	2		
	5	3		
	6	3		
	7	5,6	5	5
			6	3
				6
				3

Critério da Fronteira de Dominância

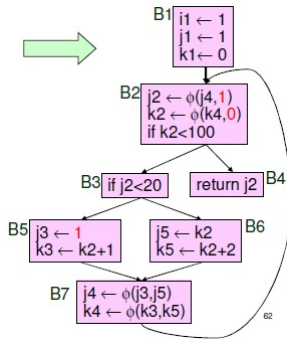
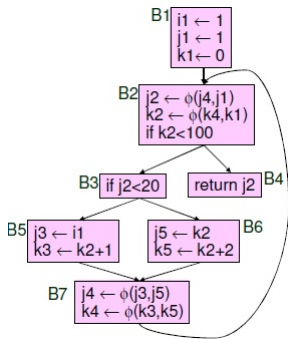
- Se um nodo x , diferente do nodo inicial, contém uma definição para a variável a , então cada nodo z na fronteira de dominância de x precisa de uma função ϕ para a .
- Esse critério determina a colocação das funções ϕ .
- No exemplo anterior:
 - Os nodos B5 e B6 tem definições de k . É necessária uma função ϕ para k em B7.



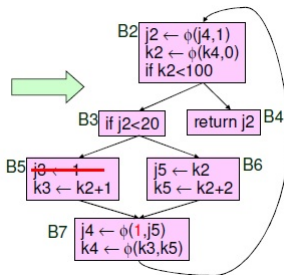
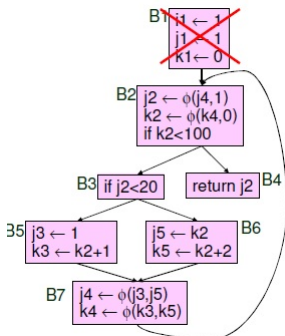
- Como agora B7 tem uma definição de k , é necessária uma função ϕ para k em B2.



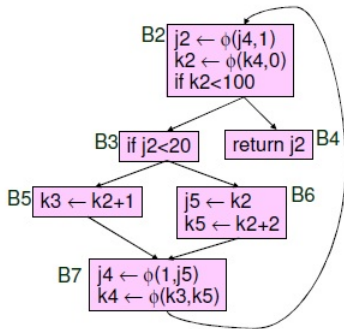
- Inserindo as funções ϕ para j e completando a numeração e propagação de constantes



- Eliminação de código morto e propagação de constantes



- Fim? Propagação condicional de constantes encontra em algumas situações ramos que nunca serão executados.



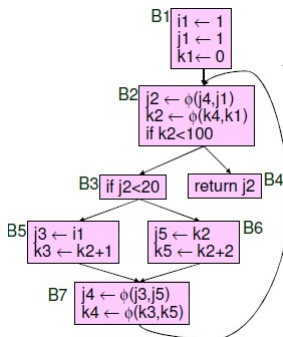
Algoritmo para propagação condicional de constantes

- O algoritmo opera executando uma interpretação abstrata do código na forma SSA.
- Ele identifica em algumas situações variáveis que podem ter somente um valor em um ponto de código, bem como blocos que nunca são executados.
- Ele utiliza a seguinte notação para controlar os possíveis valores de uma variável v :
 - $\nu(v)=\perp$ - Não há evidência de que alguma atribuição a v foi executada.
 - $\nu(v)=4$ - Há evidência que $v \leftarrow 4$ foi executado, mas não há evidência de que v recebeu qualquer outro valor.
 - $\nu(v)=\top$ - Há evidências de que v teve, em diferentes momentos, ao menos dois valores diferentes, ou algum valor que não pode ser avaliado em tempo de compilação.
- e a notação a seguir para a executabilidade de blocos:
 - $\epsilon(B)=\text{false}$ - Não há evidência de que o bloco B alguma vez será executado
 - $\epsilon(B)=\text{true}$ - Há evidência de que o bloco B pode ser executado

- Inicialmente o algoritmo inicia com $\nu(v) = \perp$ para todas as variáveis e $\epsilon(B) = \text{false}$ para todos os blocos. A partir daí observa-se o seguinte:
 - ① Toda variável v que não tenha definição (variável não inicializada, variável global ou parâmetro formal de uma função) é inicializada com $\nu(v) = \top$.
 - ② O bloco inicial B_1 é marcado como executável: $\epsilon(B_1) = \text{true}$
 - ③ Para cada bloco executável B com apenas um sucessor C , faça $\epsilon(C) = \text{true}$
 - ④ Para cada comando executável (comando que está dentro de um bloco marcado como executável) $v \leftarrow x \text{ op } y$ onde $\nu[x] = c_1$ e $\nu[y] = c_2$, faça $\nu(v) = c_1 \text{ op } c_2$.
 - ⑤ Para cada comando executável $v \leftarrow x \text{ op } y$ onde $\nu[x] = \top$ ou $\nu[y] = \top$, faça $\nu(v) = \top$.
 - ⑥ Para cada comando executável $v \leftarrow \phi(x_1, \dots, x_n)$ onde $\nu[x_i] = c_1$, $\nu[x_j] = c_2$, $c_1 \neq c_2$, o i -ésimo predecessor é executável e o j -ésimo predecessor é executável, faça $\nu(v) = \top$.

- 7 Para cada comando executável $v \leftarrow \text{MEM}()$ ou $v \leftarrow \text{CALL}()$, faça $\nu(v) = \top$.
- 8 Para cada comando executável $v \leftarrow \phi(x_1, \dots, x_n)$ onde $\nu[x_i] = \top$ e o i -ésimo predecessor é executável, faça $\nu(v) = \top$.
- 9 Para cada comando executável $v \leftarrow \phi(x_1, \dots, x_n)$ cujo i -ésimo predecessor é executável e $\nu[x_i] = c_1$; e para todo $j \neq i$, ou o predecessor j não é executável, ou $\nu[x_j] = \perp$ ou $\nu[x_j] = c_1$, faça $\nu(v) = c_1$
- 10 Para cada desvio condicional executável **if** $x < y$ **goto** L_1 **else** L_2 , onde $\nu[x] = \top$ ou $\nu[y] = \top$, faça $\epsilon(L_1) \leftarrow \text{true}$ e $\epsilon(L_2) \leftarrow \text{true}$
- 11 Para cada desvio condicional executável **if** $x < y$ **goto** L_1 **else** L_2 , onde $\nu[x] = c_1$ e $\nu[y] = c_2$, faça $\epsilon(L_1) \leftarrow \text{true}$ ou $\epsilon(L_2) \leftarrow \text{true}$ dependendo de $c_1 < c_2$.

- Inicialização

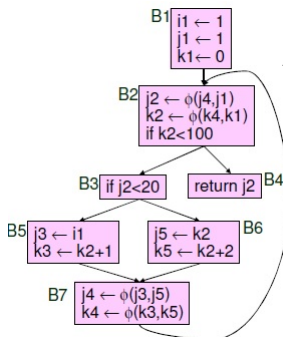


B	$\epsilon[B]$
B_1	false
B_2	false
B_3	false
B_4	false
B_5	false
B_6	false
B_7	false

x	$\nu[x]$
i_1	$\perp$
j_1	$\perp$
j_2	$\perp$
j_3	$\perp$
j_4	$\perp$
j_5	$\perp$
k_1	$\perp$
k_2	$\perp$
k_3	$\perp$
k_4	$\perp$
k_5	$\perp$

- Pela regra 2, o bloco B1, por ser o ponto de entrada, é executável.
- Pela regra 3, o bloco B2, por ser o único sucessor de B1, também é executável.
- Pela regra 4, as variáveis **i1**, **j1** e **k1** são marcadas como a constante 1.
- Pela regra 9, as variáveis **j2** e **k2** recebem os valores de **j1** e **k1**
- Pela regra 11, os blocos B3 e B5 são executáveis.

- Primeira iteração

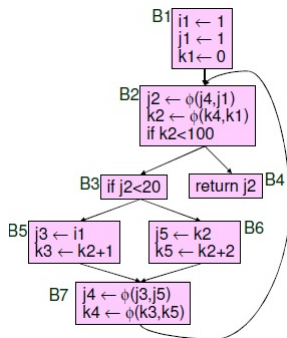


B	$\epsilon[B]$
B_1	true (2)
B_2	true (3)
B_3	true (11)
B_4	false
B_5	true (11)
B_6	false
B_7	false

x	$\nu[x]$
i_1	1(4)
j_1	1(4)
j_2	1(9)
j_3	$\perp$
j_4	$\perp$
j_5	$\perp$
k_1	0(4)
k_2	0(9)
k_3	$\perp$
k_4	$\perp$
k_5	$\perp$

- Pela regra 4, j_3 recebe o valor 1.
- Pela regra 4, k_3 recebe o valor 1.
- Pela regra 9 j_4 recebe o valor 1
- Pela regra 9 k_4 recebe o valor 1

- Continuando...

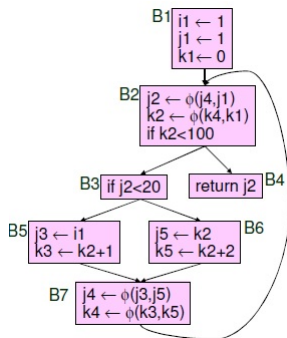


B	$\epsilon[B]$
B_1	true
B_2	true
B_3	true
B_4	true
B_5	true
B_6	false
B_7	true

x	$\nu[x]$
i_1	1
j_1	1
j_2	1
j_3	1
j_4	1
j_5	$\perp$
k_1	0
k_2	0
k_3	1
k_4	1
k_5	$\perp$

- Pela regra 6, k_2 recebe $\top$
- Pela regra 6, k_3 recebe $\top$
- Pela regra 6, k_4 recebe $\top$
- Após o que mais nenhum valor é alterado

- Final

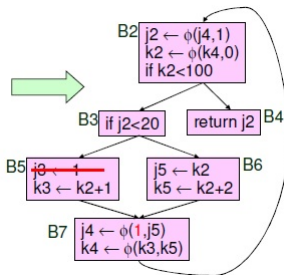
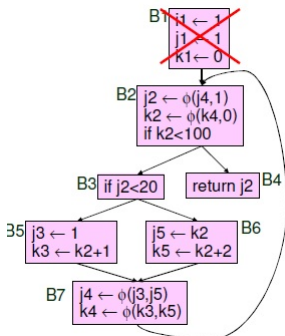


B	$\epsilon[B]$
B_1	true
B_2	true
B_3	true
B_4	true
B_5	true
B_6	false
B_7	true

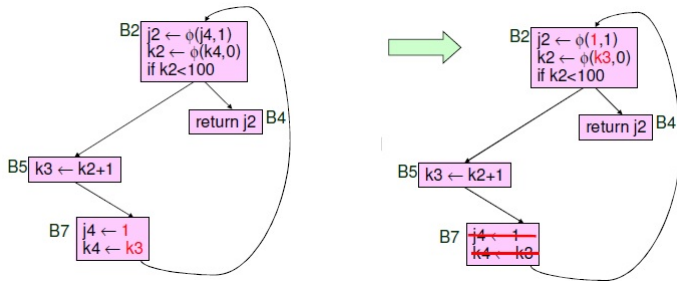
x	$\nu[x]$
i_1	1
j_1	1
j_2	1
j_3	1
j_4	1
j_5	$\perp$
k_1	0
k_2	$\top$
k_3	$\top$
k_4	$\top$
k_5	$\perp$

- Esses passos são aplicados repetidamente até que não haja mais alterações no estado dos blocos e dos comandos.
- Após completada a análise as seguintes transformações podem ser feitas:
 - Os blocos que estão marcados como não-executáveis podem ser removidos
 - As variáveis v que estão marcadas como constantes ($\nu[v] = c$) podem ter todos os seus usos substituídos pelo valor da constante e o comando de definição da variável pode ser removido.
 - As funções ϕ nos blocos sucessores imediatos dos blocos removidos devem ser corrigidas, retirando os argumentos correspondentes ao bloco eliminado.

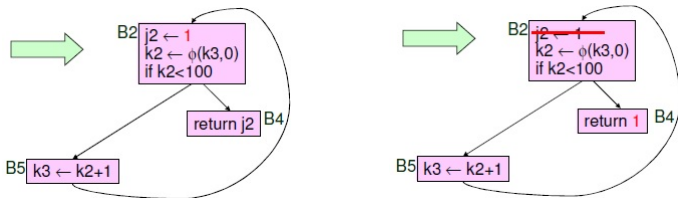
- Eliminação de código morto e propagação de constantes



- Eliminação de funções ϕ com um argumento e propagação de constantes e cópias



- Eliminação de funções ϕ com argumentos iguais e propagação de constantes



- Fim? Não, ainda pode-se aplicar *aggressive dead-code elimination*

Grafo de Dependência de Controle

- É utilizado para implementação de *aggressive dead-code elimination*
- Todo grafo de fluxo de controle tem um nó de saída.
 - Se o grafo representa uma função, o nó de saída é o nó do **return**
 - Se a função possui mais de um return, cria-se um nó com um return e todos os outros return são transformados em arestas para o nó com o return
- Diz-se que um nó **y** tem dependência de controle de um nó **x**, se há arestas de **x** para nodos **u** e **v**, sendo que há algum caminho de **u** ao nodo de saída que evita **y**, e todos os caminhos de **v** ao nodo de saída passam por **y**.
 - De modo que a "decisão" de passar ou não por **y** a partir de **x** está em **x**.

Construção do Grafo de Dependência de Controle

- Para construir o grafo de dependência de controle:
 - 1 Adicione um novo nodo de entrada e , com uma aresta $r \rightarrow s$ ao nodo inicial s do grafo e uma aresta do nodo r ao nodo de saída (indicando que o programa principal pode ou não chamar a função representada pelo grafo)
 - 2 Faça G' o grafo de fluxo de controle reverso, invertendo todas as arestas do grafo original. O nodo de saída do grafo original corresponderá ao nodo de entrada de G' .
 - 3 Calcule a fronteira de dominância do grafo G' .
 - 4 O grafo de dependência de controle possui aresta $x \rightarrow y$ para todo " x pertence à fronteira de dominância de y "

Eliminação agressiva de código morto

- É baseada em eliminar comandos que não tem efeito na "saída" do programa.
- O algoritmo é basicamente o seguinte:
 - Marque como vivos os comandos que:
 - 1 Executam entrada/saída, armazenam valores em vetores/matrizes/variáveis dinâmicas ou chamam alguma outra função que possa ter efeitos colaterais;
 - 2 Definem alguma variável v que é usada em algum outro comando vivo;
 - 3 É um desvio condicional, que tem dependência de controle sobre algum outro comando vivo.
 - E elimine todos os outros comandos.

- Para converter um código em SSA de volta para o código original (C3E ou outra representação), cada função $y \leftarrow \phi(x_1, x_2, x_3)$ em um nodo N é convertida para $y \leftarrow x_i$ para cada nodo i predecessor do nodo N, e cada atribuição $y \leftarrow x_i$ é colocada ao final do i-ésimo predecessor N.

- Eliminação de Código Morto:
 - Se a lista de usos da variável está vazia, a definição é "morta"
 - Quando o comando $v \leftarrow x \text{ op } y$ é eliminado devido a v estar morto, esse comando deve ser removido da lista de usos de x e y . O que pode acarretar na "morte" de suas definições.
 - Portanto, deve-se iterar o algoritmo de eliminação de código morto.
- Propagação de Constantes
 - Se existe $v \leftarrow c$, onde c é uma constante, então todos os usos de v podem ser alterados para c .
 - Uma função ϕ no formato $v \leftarrow \phi(c_1, c_2, \dots, c_n)$ onde todos c_i são idênticos pode ser alterada para $v \leftarrow c$

- Para variáveis x , y e z e constantes a , b , c
- **Propagação de cópias:** $x \leftarrow \phi(y)$ ou $x \leftarrow y$ podem ser removidos e y pode substituir cada uso de x .
- **Constant folding** (resolução de expressões constantes): Se existir $x \leftarrow a \text{ op } b$ pode-se calcular $a \text{ op } b$ e substituir a soma por c , utilizando o comando $x \leftarrow c$
- **Condições constantes:** O comando `if  $a < b$  goto L1 else L2` pode ser substituído por `goto L1` ou `goto L2`, de acordo com o cálculo, em tempo de compilação para $a < b$ ajustando o grafo de fluxo.
- Código inacessível: Eliminar blocos

- A fase de geração de código ocorre após as otimizações do código intermediário. Essa fase começa com a seleção de instruções. Nesta fase, o compilador irá substituir instruções da representação intermediária por instruções da arquitetura alvo.
- Essa tradução nem sempre é uma tradução um para um, arquiteturas CISC, com as suas instruções complexas e diferentes modos de endereçamento, possuem instruções equivalentes a mais de uma instrução de representação intermediária

- O seletor de instruções deve encontrar esse padrões compostos de uma ou mais instruções de representação intermediária e traduzir para um código objeto equivalente com o menor custo (de execução, espaço, ou qualquer outro critério que o compilador esteja usando) possível.
- Certas arquiteturas utilizam instruções que operam apenas sobre dois registradores(alterando um dos operandos com o resultado da operação) enquanto o código de três endereços por definição opera sobre três temporários.
- Pode-se resolver isso adicionando uma instrução de movimentação antes da operação que garanta que o temporário de um dos operandos seja o mesmo temporário do resultado da operação

- O problema da seleção de instruções pode ser abordado como um problema de reescrita de árvores, onde o código intermediário é primeiramente traduzido para uma representação baseada em árvores. No caso do GCC, do gimple para o RTL (documentação do RTL [aqui](#)).
- A conversão do gimple para rtl é feita no passo 169 gerando a saída 170r.expand.
- Arquivos com a representação em rtl podem ser gerados com a opção de compilação -fdump-rtl-all (ou -fdump-rtl... para passos específicos)
- A representação rtl (register transfer logic) é uma representação em baixo nível, próximo ao nível de registrador. Internamente usa uma representação em árvore, mas sua representação textual lembra um lisp.

- A representação em rtl para o comando "a=1;", sendo **a** a única variável local da função (portanto referenciado como [BP-4]) é a seguinte:

```
(insn 6 3 9 2 (set (mem/c:SI (plus:SI (reg/f:SI 78  
virtual-stack-vars)
```

```
(const_int -4 [0xffffffffffffffc])) [0 a+0 S4 A32])
```

```
(const_int 1 [0x1])) teste.c:6 -1
```

```
(nil))
```

- Esse exemplo (da [Wikipedia](#)) significa "Some o conteúdo do registrado 139 ao registrador 138 e atribua ao registrador 140.
(set (reg:SI 140)
 (plus:SI (reg:SI 138)
 (reg:SI 139)))

- Essa árvore é então reescrita de forma a substituir grupos de nodos representando instruções de CI por grupos de nodos representando instruções da arquitetura alvo
- Para assegurar a portabilidade do compilador, a descrição da arquitetura alvo é feita através de uma linguagem de descrição de arquitetura. No gcc é um arquivo com extensão .md (machine description) e a descrição de cada instrução da arquitetura é feita em rtl.
- O que explica como o gcc tem porte para tantas arquiteturas (e o Compiler Explorer também)

- Os arquivos de descrição da arquitetura contem modelos em RTL para cada instrução.
- Arquiteturas RISC tem menos modelos, e mais simples. Arquiteturas CISC tem mais modelos e mais complexos, devido à maior quantidade de modos de endereçamento.
- A figura abaixo mostra a representação de uma instrução para a arquitetura 68000 ([aqui](#))

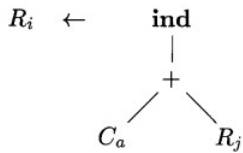
```
(define_insn "tstsi"
  [(set (cc0)
        (match_operand:SI 0 "general_operand" "rm"))]
  ""
  "*"
  {
    if (TARGET_68020 || ! ADDRESS_REG_P (operands[0]))
      return \"tstl %0\";
    return \"cmpl #0,%0\";
  })
```

- O processo de identificar a melhor combinação de instruções da arquitetura alvo equivalente ao código é denominado *tree tiling*.
- Instruções são escolhidas através de heurísticas que podem incluir: instruções que eliminam o número máximo de nodos de uma só vez, instruções com o menor custo de execução, instruções que ocupam o menor espaço em memória, etc.

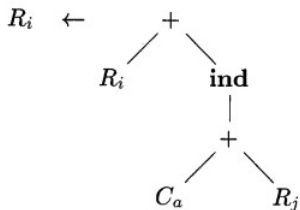
- O Livro do Dragão apresenta um conjunto de modelos de instruções para uma arquitetura-alvo hipotética:

1)	$R_i \leftarrow C_a$	{ LD R_i , # a }
2)	$R_i \leftarrow M_x$	{ LD R_i , x }
3)	$ \begin{array}{c} M \leftarrow \quad = \\ \quad \swarrow \quad \searrow \\ M_x \quad \quad R_i \end{array} $	{ ST x , R_i }
4)	$ \begin{array}{c} M \leftarrow \quad = \\ \quad \swarrow \quad \searrow \\ \text{ind} \quad \quad R_j \\ \\ R_i \end{array} $	{ ST $*R_i$, R_j }

5)

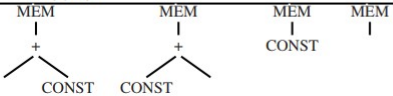
{ LD R_i , $a(R_j)$ }

6)

{ ADD R_i , R_i , $a(R_j)$ }

7)	$R_i \leftarrow \begin{array}{c} + \\ / \quad \backslash \\ R_i \quad R_j \end{array}$	{ ADD R_i , R_i , R_j }
8)	$R_i \leftarrow \begin{array}{c} + \\ / \quad \backslash \\ R_i \quad C_1 \end{array}$	{ INC R_i }

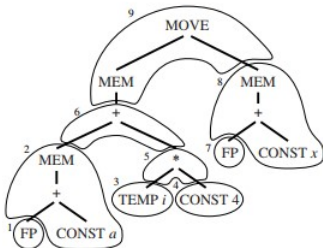
- O Livro do Tigre usa uma representação ligeiramente diferente:

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	
MUL	$r_i \leftarrow r_j \times r_k$	
SUB	$r_i \leftarrow r_j - r_k$	
DIV	$r_i \leftarrow r_j / r_k$	
ADDI	$r_i \leftarrow r_j + c$	
SUBI	$r_i \leftarrow r_j - c$	
LOAD	$r_i \leftarrow M[r_j + c]$	

- As instruções da lâmina anterior geram resultados em registradores. As instruções abaixo geram o resultado em posições de memória.

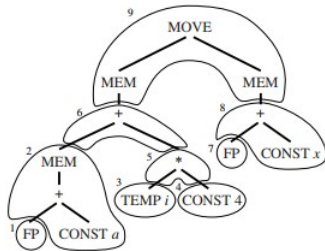
STORE	$M[r_j + c] \leftarrow r_i$	<pre> graph TD MOVE1[MOVE] --- MEM1[MEM] MOVE1 --- R1[] MEM1 --- P1[+] P1 --- C1[CONST] P1 --- C2[CONST] MOVE2[MOVE] --- MEM2[MEM] MOVE2 --- R2[] MEM2 --- P2[+] P2 --- C3[CONST] P2 --- C4[CONST] MOVE3[MOVE] --- MEM3[MEM] MOVE3 --- R3[] MEM3 --- C5[CONST] MOVE4[MOVE] --- MEM4[MEM] MOVE4 --- R4[] </pre>
MOVEM	$M[r_j] \leftarrow M[r_i]$	<pre> graph TD MOVE[MOVE] --- MEM1[MEM] MOVE --- MEM2[MEM] MEM1 --- R1[] MEM2 --- R2[] </pre>

- A figura abaixo mostra dois tilings diferentes (não muito) para o mesmo CI, e o código gerado correspondente



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

(a)



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	ADDI	$r_2 \leftarrow \mathbf{fp} + x$
9	MOVEM	$M[r_1] \leftarrow M[r_2]$

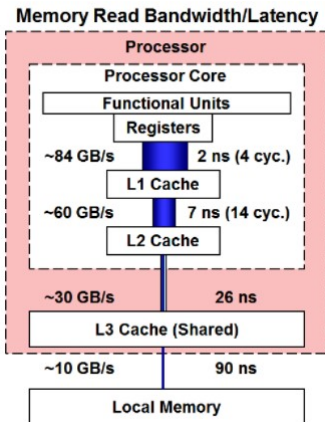
(b)

- Há diversos algoritmos para efetuar o *tiling*. No maximal munch, o algoritmo inicia do nodo raiz procurando o modelo de instrução que consegue incluir a maior quantidade de nodos.
- No exemplo anterior, seria escolhido o modelo do segundo exemplo, que cobre 3 nodos da árvore (a subárvore 9: MOVE-MEM-MEM).
- Após, é gerada a instrução correspondente ao modelo usado e o processo se repete para as subárvores restantes.
- Isso se repete até que toda a árvore tenha sido "coberta" pelas instruções.
- Nessa abordagem as instruções são geradas na ordem reversa. A instrução que "cobre" o nodo raiz será a última a ser executada.

- Uma segunda abordagem alternativa é usar uma abordagem bottom-up, baseada em programação dinâmica.
- A árvore do CI é varrida das folhas em direção à raiz encontrando, dentre os possíveis conjuntos de instruções equivalentes a cada subárvore, o de menor custo.

Alocação de Registradores

- Ao gerar código objeto, o compilador deve decidir a que registrador cada variável será atribuída, seja porque operações em registradores são mais rápidas, ou porque algumas arquiteturas exigem que os operandos de algumas instruções estejam em registradores



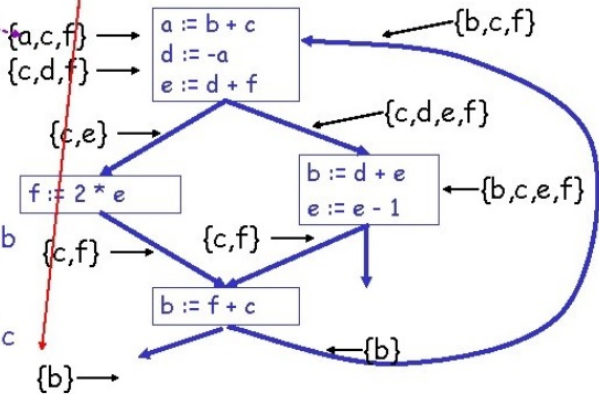
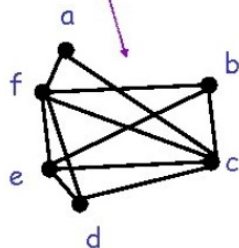
- Na geração de código intermediário considera-se que há uma quantidade ilimitada de registradores (as variáveis temporárias), mas arquiteturas reais possuem um número finito (e bem pequeno).
- Por essa razão busca-se atribuir cada registrador físico a mais de uma variável (mas não simultaneamente).

Architecture	32 bits	64 bits
ARM	15	31
Intel x86	8	16
MIPS	32	32
POWER/PowerPC	32	32
RISC-V	16/32	32
SPARC	31	31

- Considere o programa
 - d1: $a=c+d$
 - d2: $e=a+b$
 - d3: $f=e-1$
 - assumindo que **a** e **e** não estejam vivos após o uso...
- Como **a** não está vivo após o uso em **d2**, ele pode ser reutilizado mais adiante.
- O mesmo ocorre com **e** após o uso em **d3**.
- **a**, **e** e **f** podem ser atribuídos ao mesmo registrador:
 - d1: $r1 = r2 + r3$
 - d2: $r1 = r1 + r4$
 - d3: $r1 = r1 - 1$

- A ideia básica na alocação de registradores é que duas variáveis podem utilizar o mesmo registrador se em nenhum momento as duas estarão **vivas** simultaneamente.
- Essa informação pode ser obtida a partir da análise de vida de variáveis, que mostra para cada ponto, quais variáveis estão vivas (i.e. contem um valor que será utilizado mais adiante)
- A partir dessa informação, monta-se um grafo onde cada vértice representa uma das variáveis e são colocadas arestas entre variáveis que estão vivas simultaneamente.
- Esse grafo é chamado de **grafo de interferência entre registradores** ou só **grafo de interferência**.

$\{a,b\}$ não aparecem
juntos em nenhum
dos conjuntos



- O grafo de interferência fornece informação **global** (ou seja, entre blocos) sobre quais variáveis serão atribuídas a cada registrador.
- Normalmente o número de registradores disponíveis é menor que o número de registradores necessário, e nesse caso algumas variáveis são salvas em memória durante a execução do código (normalmente na pilha de execução).
 - Esse processo de salvar o registrador em memória é chamado de **spilling**
- Sobre o **grafo de interferência** é aplicado um **algoritmo de coloração**, buscando utilizar a menor quantidade possível de registradores para armazenar as variáveis.
- Coloração ótima de um grafo (com o menor número de cores) é um problema NP-Completo, e por isso são utilizados algoritmos aproximativos.

- É trabalho do alocador de registradores identificar os casos onde será necessário armazenar um temporário na pilha de execução, identificar o temporário que resultará no menor custo de acesso após ser passado para a pilha de execução e adicionar as instruções para o acesso e armazenamento desse temporário na pilha de execução, processo este chamado de *spilling*.
- Uma vez que o alocador de registradores tiver feito o *spilling* de uma quantidade suficiente de registradores, cabe ao atribuidor de registradores atribuir para cada temporário o seu respectivo registrador

- A última fase é a fase (opcional) de *scheduling*. Nesta fase o compilador reordena as instruções geradas a fim de maximizar o uso dos recursos do processador, explorando o **paralelismo a nível de instrução** em arquiteturas **pipeline**.
- O gcc suporta scheduling para quase todas suas arquiteturas-alvo, e é ativado pelo flag **-march**.
- Scheduling normalmente é feito por blocos uma vez que ao sair de um bloco normalmente há mais de um caminho possível, impossibilitando prever qual a próxima instrução.

- Para garantir que o comportamento do programa continuará o mesmo ao mudar a ordem das instruções deve ser levada em conta a **dependência** entre elas.
- Há 3 dependências entre instruções que devem ser levadas em consideração:
 - RAW (Read After Write) - ocorre quando a instrução $i+1$ tenta ler um dado que a instrução i ainda não atualizou;
 - WAW (Write After Write) ? ocorre quando a instrução $i+1$ tenta escrever um dado antes que a instrução i atualize o mesmo;
 - WAR (Write After Read) ? ocorre quando a instrução $i+1$ tenta escrever um dado antes que a instrução i possa lê-lo;

- Para garantir a correção do código gerado, é montado um grafo representando as dependências entre as instruções.
- Nesse grafo os vértices representam as instruções, e há um arco de I_1 para I_2 se I_1 deve ser executada antes de I_2 .
- Sobre esse grafo é executada uma ordenação topológica, e a cada passo é retirada uma instrução de grau zero (que não dependa de ninguém).
- Se houver mais de uma instrução, algumas heurísticas são utilizadas, entre elas:
 - Os recursos (unidades funcionais) utilizados pelas instruções mais recentes.
 - Se uma instrução está no caminho crítico do grafo, sua prioridade sobe.
 - Se a escolha de uma instrução liberar várias outras, sua prioridade sobe.

- Ao longo da etapa de geração de código são aplicadas diversas otimizações, chamadas de otimizações *peephole*, que ocorrem sobre um pequeno conjunto de instruções a cada vez (o *peephole*).
- Entre essas otimizações inclui-se a redução de força, em que uma operação mais cara é substituída por uma de menor custo

- Requisitos impostos a geradores de código:
 - Código correto e de alta qualidade
 - Uso efetivo dos recursos de máquina
- O problema de gerar código ótimo é insolúvel (indecidível).
 - Na prática utilizam-se técnicas heurísticas que geram bom, mas não necessariamente ótimo código.

[Voltar para o índice](#)

1.Considerações no projeto de um gerador de código:

a)Formas de código objeto

- Linguagem absoluta (geração de código mais fácil)
- Linguagem relocável
 - Maior flexibilidade
 - Permite ligação entre diversos módulos
- Linguagem assembly - facilita o processo de geração de código. Pode-se usar um assembler em separado. Há um passo adicional na montagem

- b) Seleção de instruções

A escolha da seqüência apropriada pode resultar num código mais curto e mais rápido.

- c) Alocação de registradores

Instruções com registradores são mais curtas e rápidas que instruções envolvendo memória, portanto o uso efetivo de registradores é muito importante.

- d) Escolha da ordem de avaliação

A melhor ordem é um problema insolúvel. Algumas computações requerem menos registradores para resultados intermediários.

2.A máquina objeto

- Familiaridade com a máquina e o conjunto de instruções é necessária para projetar um bom gerador de código. Será usada aqui uma máquina com n registradores de propósito geral $R_0, R_1, \dots, R_n$ e instruções da forma
Op fonte, destino

Modos de endereçamento	forma	endereço	Custo Adicional
1. Absoluto	M	M	1
2. Registrador	R	R	0
3. Indexado	C(R)	C + conteúdo de R	1
4. Registrador Indireto	*R	Conteúdo (R)	0
5. Indexado Indireto	*C(R)	Conteúdo(C + Conteúdo(R))	1
6. Imediato	#	Constante C	1

Mov R0,M →	Armazena conteúdo de R0 em M
Mov 4(R0),M →	Armazena conteúdo de (4 + conteúdo de R0) em M
Mov *4(R0),M →	Armazena conteúdo de (conteúdo(4+conteúdo(R0))) em M
Mov #1,R0 →	Armazena 1 em R0

Custo de uma instrução é 1 + custo dos modos de endereçamento utilizados na instrução.

Instrução	Custo
mov R0,R1	1
mov R5,M	2
add #1,R3	2

- Sub 4(R0),*12(R1) (busca o conteúdo da posição R1+12)
Custo total = 3 (byte com o código da operação + 1 byte (4)
+ 1 byte (12))

- Possibilidades de código gerado para $a := b + c$

mov b,R0

add c,R0

mov R0,a

- Custo:6

mov b,a

add c,a

- Custo:6

Supondo que R0,R1 e R2 contêm os endereços de a,b e c

```
mov *R1, *R2
```

```
add *R2, *R0
```

Custo:2

R1 e R2 contêm os valores de b e c, e o valor de b não é necessário após a atribuição

```
add R2,R1
```

```
mov R1,a
```

Custo:3

Gerador de código simplificado

Suposições:

- Para cada operador do código intermediário, existe um operador no código objeto
- Resultados computados podem ser deixados em registradores tanto tempo quanto desejado, salvando-se somente
 - 1 Se o registrador é necessário
 - 2 Imediatamente antes de uma chamada de procedure, desvio ou comando com rótulo

Código ótimo para $a := b + c$ supondo b e c armazenados em R_j e R_i , e deixando o resultado em R_i .

Add R_j, R_i Custo=1 ($R_i = c \Rightarrow R_i = a$)

Se c está em memória e R_i contem b

Add c, R_i Custo=2

Caso c for usado a seguir

Mov c, R_j

Add R_j, R_i

Descritores de Registradores e endereços

- O algoritmo de geração de código manipula descritores que informam o que contém cada registrador.
- Inicialmente todos os registradores estão vazios.
- O descritor de registradores é consultado a cada vez que um registrador é necessário.
- O descritor de endereços informa onde o valor da variável pode ser encontrado.
- A localização pode ser um registrador ou um endereço de memória.
- Esta informação pode ser armazenada na tabela de símbolos.

Algoritmo de Geração de Código

Entrada: bloco básico de código de três endereços

Para cada instrução $X:=Y \text{ op } Z$ faça

- 1 Chame getreg para determinar a posição L onde o resultado deve ser armazenado. Normalmente L será um registrador, mas poderá ser memória.
- 2 Consulte o descritor de endereços para Y: se Y estiver em memória e registrador, prefira o registrador. Se o valor de Y não está ainda em L, gere a instrução `mov Y,L` que armazena uma cópia de Y em L.
- 3 Gere a instrução `op Z',L`, onde Z' é a posição corrente de Z (prefira o registrador se Z está em memória e registrador). Atualize o descritor de endereço de X para L. Se L for um registrador, atualizar o descritor para indicar que contém o valor de X.
- 4 Se os valores de Y e/ou Z não tem próximo-uso, se não estão vivos na saída do bloco e estão em registradores então altere o descritor de registradores para indicar que estes registradores não contêm mais Y e Z.

Função Getreg

Retorna a localização L para armazenar o valor de X na atribuição $X = Y \text{ op } Z$.

- 1 Se Y está em um registrador que não contém outras variáveis e Y não está vivo, nem tem próximo uso após a execução de $X := Y \text{ op } Z$, então retorna o registrador de Y como L. Atualize o descritor de endereço de Y indicando que ele não está mais em L.
- 2 Falhando (1) retorne um registrador vazio p/ L, se existir algum.
- 3 Falhando (2), se X tem um próximo-uso no bloco, ou op é um operador que requer um registrador (p.ex.: indexação), encontre um registrador ocupado R. Armazene o valor de R em memória (mov R,M) se ainda não estiver, atualize o descritor de M e retorne R. Se R contem o valor de várias variáveis, uma instrução MOV deve ser gerada p/ cada variável que necessite ser salva.
- 4 Se X não é usada no bloco, ou nenhum registrador pode ser liberado, escolha o endereço de memória X para L.

- Definição dos Blocos Básicos e do Grafo de Fluxo Ex
- Otimizações em blocos básicos:
 - Eliminação de expressões comuns
 - Propagação de cópias
 - Eliminação de variáveis de indução
 - Eliminação de código morto
- Representação do bloco básico como um Grafo Dirigido Acíclico e Montagem do código a partir do GAD Ex

[Voltar para o índice](#)

- Análise de vida de variáveis Ex
- Informação de próximo uso Ex
- Geração de código objeto (alocação de registradores)
- Identificação de Laços:
 - Cálculo de dominadores Ex
 - Dominador imediato
 - Árvore de dominação
 - Identificação de Laços Naturais (algoritmo)
 - Laços aninhados

- Remoção de Computações laço-invariantes
 - Condições
- Definições incidentes (algoritmo)
 - Cálculo
 - Propagação de Cópias
 - Variáveis de Indução
 - Conceituação: Básicas, derivadas
 - Identificação
 - Representação por triplas
 - Remoção
 - Desdobramento de laços

- Forma SSA:
 - Conversão para a forma SSA
 - Colocação de funções ? : Árvore de dominância
 - Otimizações na forma SSA
 - Propagação de cópias
 - Eliminação de código morto
 - Propagação de condições constantes
- Conversão de SSA para C3E

- Considere o código a seguir que escreve todos os números primos entre 2 e 100:

```
for (n=2;n<=100;n++)  
{  
    d=2;  
    while (n%d!=0) d++;  
    if (n==d) printf("%d",n);  
}
```

[Voltar para o índice](#)

- 1 Gere o C3E correspondente
- 2 Mostre o grafo de fluxo de controle
- 3 Calcule os dominadores de cada nodo
- 4 Mostre a árvore de dominadores e o dominador imediato de cada nodo
- 5 Identifique os laços naturais
- 6 Mostre a árvore de aninhamento de laços

- Considere o código a seguir que lê três coeficientes de uma equação de segundo grau e escreve a soma de suas raízes reais:

```
leia(a)
```

```
leia(b)
```

```
leia(c)
```

```
if ( $b^2 - 4 * a * c < 0$ )
```

```
    printf("não tem");
```

```
else
```

```
{
```

```
     $r1 = (-b + (b^2 - 4 * a * c)^{(1/2)}) / (2 * a);$ 
```

```
     $r2 = (-b - (b^2 - 4 * a * c)^{(1/2)}) / (2 * a);$ 
```

```
    soma = r1 + r2;
```

```
    printf("%d", soma);
```

```
}
```

- 1 Gere o C3E correspondente
- 2 Mostre o grafo de fluxo de controle
- 3 Gere o grafo dirigido acíclico que representa o maior bloco básico
- 4 Gere o código otimizado a partir do grafo acíclico dirigido

- Considere o código a seguir:

```
for (i=1;i<=N;i++) {  
    x[i]=Y+I;  
    Z=Y; }
```

- 1 Gere o C3E e o grafo de fluxo de controle correspondente
- 2 A atribuição $Z=Y$ pode ser removida para fora do laço?
- 3 Em caso contrário, qual (ou quais) das condições para remoção de código laço-invariante não é respeitada?
- 4 E que alterações podem ser feitas no código para que ela possa ser removida?
- 5 Gere o C3E e o GFC (grafo de fluxo de controle) correspondente ao código alterado.

- Considere o C3E a seguir que inicializa um vetor de 100 posições com valores decrescentes a partir de 202:
 - Considere que a instrução "t3 $\leftarrow$ addr a" coloca em t3 o endereço base de a (uma alternativa às instruções de referência a array)

t1 $\leftarrow$ 202

i $\leftarrow$ 1

L1: if i > 100 goto L2

t1 $\leftarrow$ t1-2

t3 $\leftarrow$ addr a

t4 $\leftarrow$ t3-4

t5 $\leftarrow$ 4 * i

t6 $\leftarrow$ t4 + t5

*t6 $\leftarrow$ t1

i $\leftarrow$ i+1

goto L1

L2:

- 1 Mostre o grafo de fluxo de controle correspondente
- 2 Identifique as variáveis de indução, básicas e derivadas, mostrando para cada uma a tripla correspondente
- 3 Efetue a remoção das variáveis de indução
- 4 Efetue a remoção de código laço-invariante quando possível

- Considere o C3E a seguir:

$a \leftarrow 0$

$c \leftarrow 0$

L1: $b \leftarrow a + 1$

$c \leftarrow c + b$

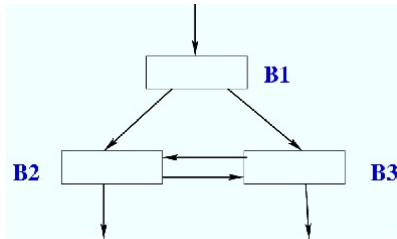
$a \leftarrow b * 2$

if $a < N$ goto L1

return c

- 1 Mostre o gfc correspondente
- 2 Identifique na entrada e na saída de cada bloco básico as variáveis vivas. Assuma que ao executar o return, apenas a variável c deve estar viva
- 3 Calcule a informação de uso subsequente das variáveis em cada comando
- 4 Gere o código objeto correspondente para uma máquina de 2 registradores

- O gfc a seguir contem laços naturais? Em caso afirmativo, identifique-os. Em caso negativo, justifique.



- Considere o código abaixo, para o cálculo da potência b^e :

```
int pow(int b, int e){  
    p=1;  
    for (i=1;i<=e;i++)  
        p=p*b;  
    return p;  
}
```

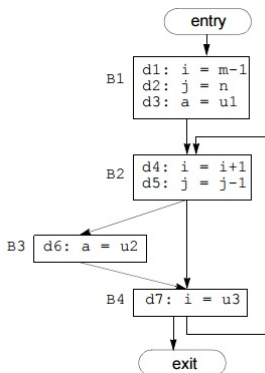
- Gere o C3E e o grafo de fluxo de controle, e aplique a transformação de desdobramento de laços por um fator de 2, mostrando o código correspondente.

- Considere a função recursiva abaixo, para escrever um valor em binário:

```
void escbn(int V, int pos)
{
    printf("%d", V»pos&1);
    if (pos>0) escbn(V,pos-1);
}
```

- Aplique a transformação vista em aula para eliminar a recursão de cauda, gerando uma função equivalente não-recursiva.

- Aplique o algoritmo visto em aula para calcular as definições que alcançam o início de cada bloco básico do grafo de fluxo de controle a seguir:



- A fim de controlar em tempo de compilação e uso de memória do compilador, e os trade-offs entre velocidade e espaço para o executável resultante, GCC fornece uma gama de níveis gerais de otimização, numerados 0-3, bem como opções individuais para tipos específicos de otimização.
- Um nível de otimização é escolhido com a opção de linha de comando `-O LEVEL`, onde LEVEL é um número de 0 a 3. Os efeitos dos diferentes níveis de otimização são os seguintes:

- -O0 ou nenhuma opção -O (default)
 - Neste nível de otimização o GCC não realiza nenhuma otimização e compila o código fonte da forma mais simples possível. Cada comando no código fonte é convertido diretamente para as instruções correspondentes no arquivo executável, sem rearranjo. Esta é a melhor opção para usar ao depurar um programa um programa e é o padrão se nenhuma opção nível de otimização é especificado.

- -O1 ou -O
 - Este nível liga as formas mais comuns de otimização que não requerem quaisquer compensações espaço-velocidade.
 - Com esta opção, os executáveis resultantes devem ser menores e mais rápido do que com -O0 .
 - As otimizações mais caras, tais como a programação de instruções, não são utilizados a este nível.
 - Compilando com a opção -O1 muitas vezes pode levar menos tempo do que a compilação com -O0 , devido às quantidades reduzidas de dados que precisam ser processados após otimizações simples.

- -O2
 - Esta opção ativa otimizações adicionais, além daquelas utilizadas por -O1 .
 - Essas otimizações adicionais incluem programação instrução.
 - Somente otimizações que não requerem quaisquer compensações espaço-velocidade são utilizados, de modo que o executável não deve aumentar de tamanho.
 - O compilador irá demorar mais tempo para compilar programas e exigem mais memória do que com -O1 .
 - Esta opção é geralmente a melhor escolha para a implantação de um programa, porque proporciona otimização máxima, sem aumentar o tamanho do executável.
 - É o nível de otimização padrão para lançamentos de pacotes GNU.

- -O3
 - Esta opção ativa otimizações mais caras, tais como expansão de funções inline, além de todas as otimizações dos níveis anteriores -O2 e -O1 . O nível de otimização -O3 pode aumentar a velocidade da executável resultante, mas também pode aumentar o seu tamanho. Em algumas circunstâncias em que essas otimizações não são favoráveis, essa opção pode gerar um executável mais lento.
- -funroll-loops
 - Esta opção ativa desdobramento de laços (loop-unrolling) , e é independente das outras opções de otimização. Isso irá aumentar o tamanho do executável. Se esta opção trará ou não benefícios, deve ser analisado caso-a-caso.

- -Os
 - Esta opção seleciona otimizações que reduzem o tamanho do executável.
 - O objetivo desta opção é produzir o menor executável possível, para sistemas limitados pela memória ou espaço em disco.
 - Em alguns casos, um executável menor também irá executar mais rápido, devido ao melhor aproveitamento da cache.
- É importante lembrar que os benefícios da otimização nos níveis mais altos devem ser pesados em relação ao custo.
- O custo de otimização inclui uma maior complexidade na depuração, e aumento do tempo e os requisitos de memória durante a compilação.
- Na maioria dos casos, é satisfatório usar -O0 para depuração, e -O2 para o desenvolvimento e implantação.

Opções de compilação para geração de fonte assembly no gcc

- `gcc -c -Wa,-ahl=test.s test.c`
 - `-c` processa só o arquivo fonte, gerando um objeto, sem efetuar a linkagem.
 - `-Wa` passa parâmetros direto pro assembler (que no caso do gcc é o `gnu as`).
 - `-ahl` são parâmetros do `as`.
 - `-a` gera um arquivo com a listagem do código, conforme as opções que tu passa pra ele:
 - `h` inclui o código em alto nível (de "high level", do tempo que C era considerado alto nível =).
 - `l` inclui o código em assembly.

- Aho : Praticamente tudo
- Appel : A parte de análise de fluxo de dados

Questões do POSCOMP

(45 Fundamentos 2005) No processo de geração de um código executável (em linguagem de máquina) a partir de um programa fonte, escrito em linguagem de alto nível (por exemplo, C) o programa original passa por transformações e análises que são realizadas em diversas fases. De forma simplificada, pode-se dividi-las nas oito (8) fases apresentadas, em ordem alfabética, a seguir:

- | | |
|--------------------------------|-------------------------------------|
| (A) Alocação de Registradores | (E) Link Edição |
| (B) Análise Léxica | (F) Montagem |
| (C) Análise Sintática | (G) Seleção de Instruções |
| (D) Emissão de Código Assembly | (H) Verificação de Tipos e Símbolos |

Durante o processo de geração do código executável a partir do código fonte em qual ordem essas fases são possíveis de serem executadas?

- | | |
|---------------------|---------------------|
| (a) B C H G A D F E | (d) B H C G A D F E |
| (b) C B H G A D F E | |
| (c) B C H G A D E F | (e) B C H A G D E F |

(2010 - Técnicas - 64) Considere o bloco básico a seguir.

```
e:=i+1  
c:=i+j  
b:=j*4  
e:=M[b]  
if e<b goto B12
```

Assinale a alternativa que apresenta a transformação de código local que pode ser aplicada no bloco básico.

- a) Eliminação de código morto.
- b) Propagação de cópias.
- c) Eliminação de subexpressões comuns.
- d) Propagação de constantes.
- e) Eliminação de variáveis de indução.

(2014 - Técnicas - 70) Considere o trecho de código a seguir.

$a := 0$

$b := a + 1$

$c := c + b$

$a := b * 2$

Assinale a alternativa que apresenta, corretamente, as variáveis que estarão vivas no início do trecho desse código.

a) a

b) b

c) c

d) a, b

e) b, c

(2017 - Técnicas - 63) Sobre a técnica de tradução dirigida por sintaxe, é correto afirmar que:

- A) Uma definição dirigida por sintaxe é uma gramática livre de contexto acrescida de atributos e regras. Os atributos são associados às produções, e as regras aos símbolos terminais e não terminais da gramática.
- B) Uma definição dirigida por sintaxe é denominada definição S-atribuída quando nessa estão envolvidos apenas atributos herdados.
- C) As regras semânticas só são aplicadas após a construção total da árvore sintática pelo parser do compilador.
- D) Grafos de dependência são utilizados para determinar uma ordem de avaliação para as instâncias dos atributos de uma árvore de derivação.
- E) Sendo "S" um símbolo da gramática presente em uma árvore de derivação, um atributo sintetizado é computado através dos valores dos atributos dos nós irmãos ou do nó pai de "S".

1)(5 pontos) Faça uma DDS para reconhecimento de structures em C, com campos de inteiros e floats. As ações semânticas devem atualizar a tabela de símbolos com o tipo, posição relativa da variável no bloco e tamanho da structure. A lista de campos da structure deve ser armazenada em uma lista encadeada com as mesmas informações. A posição de um campo da structure deve ser relativa ao início da structure. Ex:

```
struct data {int dia, int mes, int ano;} d1,d2;
```

2)(5 pontos) Faça uma DDS para gerar código de 3 endereços para atribuição a um elemento de array considerando uma linguagem que aceite arrays unidimensionais com 0 como primeira posição e aceitando somas na expressão de índice.

Ex: $X[3+I+Y[3]]:=3$; $Y:=X[3]$;

Para endereçamento indexado utilize o código de 3 endereços nos formatos $X[i]:=Y$ e $Y:=X[i]$.

1) Considere a seguinte gramática do Pascal.

$\langle \text{comando} \rangle \rightarrow \langle \text{for} \rangle \mid \langle \text{while} \rangle \mid \langle \text{if} \rangle \mid \langle \text{atribuição} \rangle$
 $\langle \text{for} \rangle \rightarrow \text{TFOR TID TATRIB} \langle \text{expressão} \rangle \text{TTO} \langle \text{expressão} \rangle \text{TDO}$
 $\langle \text{comando} \rangle \text{TPTVG} \langle \text{while} \rangle \rightarrow \text{TWHILE} \langle \text{expressão} \rangle \text{TDO}$
 $\langle \text{comando} \rangle \text{TPTVG} \langle \text{if} \rangle \rightarrow \text{TIF} \langle \text{expressão} \rangle \text{TTHEN}$
 $\langle \text{comando} \rangle \text{TPTVG} \langle \text{atribuição} \rangle \rightarrow \text{TID TATRIB} \langle \text{expressão} \rangle$
 $\langle \text{expressão} \rangle \rightarrow \langle \text{exp1} \rangle \text{TMAIOR} \langle \text{exp1} \rangle \mid \langle \text{exp1} \rangle \langle \text{exp1} \rangle \rightarrow$
 $\langle \text{exp1} \rangle + \langle \text{termo} \rangle \mid \langle \text{termo} \rangle \langle \text{termo} \rangle \rightarrow \langle \text{termo} \rangle *$
 $\langle \text{fator} \rangle \mid \langle \text{fator} \rangle \langle \text{fator} \rangle \rightarrow \text{TID} \mid (\langle \text{expressão} \rangle)$

terminais:

TID - identificador. TID.pos - atributo de posição na tabela de símbolos
TATRIB- := TPTVG - ponto e vírgula

a) (3 pontos) Faça um esquema de tradução que verifique os seguintes erros de tipo em um programa emitindo mensagens de erro apropriadas.

- **BOOLEAN EXPRESSION EXPECTED** - se houver uma expressão de tipo numérico onde for esperada uma expressão booleana (comandos if e while)
- **UNKNOWN IDENTIFIER** - se aparecer um identificador não declarado
- **INVALID FOR CONTROL VARIABLE** - se a variável de controle do FOR é real (pode ser boolean, mas os valores inicial e final também devem ser)
- **TYPE MISMATCH** - tipo incompatível. Foi tentado atribuir um valor a uma variável incompatível com aquele valor (real a integer ou integer a boolean)
- **TO OR DOWNTON EXPECTED** - não há o TO no comando FOR após a variável de controle
Suponha que você tem uma função que procura um identificador na tabela de símbolos pela posição e retorna seu tipo (Gettipo - retorna integer, real, boolean ou não_existe)

b) (3 pontos) Implemente o esquema em um parser descendente recursivo

2) (4 pontos) Considere a seguinte gramática para geração de expressões com constantes, x , $+$ e $*$.

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow \text{cte}$$

$$F \rightarrow (E)$$

$$F \rightarrow x$$

Faça uma DDS para gerar a expressão da derivada da expressão de entrada. Sugestão : Associe a cada símbolo não terminal um atributo com a expressão da derivada e outro com a expressão.

Exemplos:

Entrada	Saída
X	1
3	0
$3 * X$	$(3 * 1 + 0 * 3)$
$3 * 3$	$(3 * 0 + 0 * 3)$
$3 * X + 4 * X$	$((3 * 1 + X * 0) + (4 * 1 + X * 0))$

Relações:

1)(5 pontos) Faça uma DDS para declarações de variáveis do tipo integer, real, arrays com início em 0 e records do pascal. As ações semânticas devem atualizar a tabela de símbolos com o tipo, posição relativa da variável no bloco e tamanho do elemento (para arrays) ou tamanho do record. A lista de campos do record deve ser armazenada em uma lista encadeada com as mesmas informações. A posição de um campo do record deve ser relativa ao início do record.

2)(5 pontos) Faça uma DDS para gerar código de 3 endereços para atribuição a um elemento de array considerando uma linguagem que aceite arrays unidimensionais com 0 como primeira posição e aceitando somas na expressão de índice. Ex: $X[3+I+Y[3]]:=3$; $Y:=X[3]$; Para endereçamento indexado utilize o código de 3 endereços nos formatos $X[i]:=Y$ e $Y:=X[i]$.

1)(3 pontos) Considerando a seguinte declaração de array:
M:array[1..10,1..10,1..10] of integer; Mostre o código de três endereços gerado para o seguinte trecho de programa em pascal:

```
for i := 1 to 10 do
  for j := 1 to 10 do
    for k := 1 to 10 do
      M [ i , j , k ] := M [ i , j , k+1 ] + 1;
```

• Solução:

```
T0=1
i=T0
L1:  T1=10
     if i>T1 goto Fim1
     T2=1
     j=T2
L2:  T3=10
     if j>T3 goto Fim2
     T4=1
     k=T4
L3:  T5=10
     if k>T5 goto Fim3
     t6=i-1          subtrai índice inicial da primeira dimensão
     T7=T6*100
     T8=j-1          subtrai índice inicial da segunda dimensão
     T9=T8*10
     T10=1
     T11=k+T10
     T12=T11-1       subtrai índice inicial da terceira dimensão
     T13=T7+T9
     T14=T13+T12
     T15=T14*4
     T16=M[T15]
     T17=1
     T18=T16+T17
```

t19=i-1	subtrai índice inicial da primeira dimensão
T20=T19*100	
T21=j-1	subtrai índice inicial da segunda dimensão
T22=T21*10	
T23=1	
T24=k+T23	
T25=T24-1	subtrai índice inicial da terceira dimensão
T26=T20+T22	
T27=T26+T25	
T28=T27*4	
M[T28]=T18	
T29=1	
T30=k+T29	
k=T30	
goto L3	
Fim3: T31=1	j=j+1 (implícito no for)
T32=j+T31	
j=T32	
goto L2	
Fim2: T33=1	i=i+1 (implícito no for)
T34=i+T33	
i=T34	
goto L1	
Fim1:	

• Propagação de cópias

```
T0=1
i=T0
L1:  T1=10
    if i>T1 goto Fim1
    T2=1
    j=T2
L2:  T3=10
    if j>T3 goto Fim2
    T4=1
    k=T4
L3:  T5=10
    if k>T5 goto Fim3
    t6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T10=1
    T11=k+T10
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T17=1
    T18=T16+T17
```

```
i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    t6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T19=i-1
    T20=T19*100
    T21=j-1
    T22=T21*10
    T24=k+1
    T25=T24-1
```


- Subexpressões Comuns: T19 é substituído por T6 e T21 por T8

```
L1:  i=1
      if i>10 goto Fim1
      j=1
L2:  if j>10 goto Fim2
      k=1
L3:  if k>10 goto Fim3
      t6=i-1
      T7=T6*100
      T8=j-1
      T9=T8*10
      T11=k+1
      T12=T11-1
      T13=T7+T9
      T14=T13+T12
      T15=T14*4
      T16=M[T15]
      T18=T16+1
      T19=i-1
      T20=T19*100
      T21=j-1
      T22=T21*10
      T24=k+1
      T25=T24-1
```

```
      i=1
L1:  if i>10 goto Fim1
      j=1
L2:  if j>10 goto Fim2
      k=1
L3:  if k>10 goto Fim3
      T6=i-1
      T7=T6*100
      T8=j-1
      T9=T8*10
      T11=k+1
      T12=T11-1
      T13=T7+T9
      T14=T13+T12
      T15=T14*4
      T16=M[T15]
      T18=T16+1
      T20=T6*100
      T22=T8*10
      T24=k+1
      T25=T24-1
      T26=T20+T22
      T27=T26+T25
      T28=T27*4
```

- Subexpressões Comuns: T20 é substituído por T7 e T22 por T9

```
i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T20=T6*100
    T22=T8*10
    T24=k+1
    T25=T24-1
    T26=T20+T22
    T27=T26+T25
    T28=T27*4
```

```
i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T24=k+1
    T25=T24-1
    T26=T7+T9
    T27=T26+T25
    T28=T27*4
```

- Subexpressões Comuns: T20 é substituído por T7, T22 por T9 e T24 por T11

```
L1:  i=1
      if i>10 goto Fim1
      j=1
L2:  if j>10 goto Fim2
      k=1
L3:  if k>10 goto Fim3
      T6=i-1
      T7=T6*100
      T8=j-1
      T9=T8*10
      T11=k+1
      T12=T11-1
      T13=T7+T9
      T14=T13+T12
      T15=T14*4
      T16=M[T15]
      T18=T16+1
      T20=T6*100
      T22=T8*10
      T24=k+1
      T25=T24-1
      T26=T20+T22
      T27=T26+T25
      T28=T27*4
```

```
      i=1
L1:  if i>10 goto Fim1
      j=1
L2:  if j>10 goto Fim2
      k=1
L3:  if k>10 goto Fim3
      T6=i-1
      T7=T6*100
      T8=j-1
      T9=T8*10
      T11=k+1
      T12=T11-1
      T13=T7+T9
      T14=T13+T12
      T15=T14*4
      T16=M[T15]
      T18=T16+1
      T25=T11-1
      T26=T7+T9
      T27=T26+T25
      T28=T27*4
      M[T28]=T18
      k=k+1
      goto L3
Fim3: j=j+1
      goto L2
Fim2: i=i+1
      goto L1
Fim1:
```

- Subexpressões Comuns: T25 é substituído por T12 e T26 é substituído por T13

```
i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T25=T11-1
    T26=T7+T9
    T27=T26+T25
    T28=T27*4
    M[T28]=T18
    k=k+1
    goto L3
Fim3: j=j+1
      goto L2
Fim2: i=i+1
      goto L1
Fim1:
```

```
i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T27=T13+T12
    T28=T27*4
    M[T28]=T18
    k=k+1
    goto L3
Fim3: j=j+1
      goto L2
Fim2: i=i+1
      goto L1
Fim1:
```

- Subexpressões Comuns: T27 é substituído por T14 e T28 é substituído por T15

```

i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T27=T13+T12
    T28=T27*4
    M[T28]=T18
    k=k+1
    goto L3
Fim3: j=j+1
Fim2: i=i+1
      goto L1
Fim1:

```

```

i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    M[T15]=T18
    k=k+1
    goto L3
Fim3: j=j+1
      goto L2
Fim2: i=i+1
      goto L1
Fim1:

```

- Expressões laço-invariantes: i e j não variam no laço interno

```

i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
L3:  if k>10 goto Fim3
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T11=k+1
    T12=T11-1
    T13=T7+T9
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    T27=T13+T12
    T28=T27*4
    M[T28]=T18
    k=k+1
    goto L3
Fim3: j=j+1
    goto L2
Fim2: i=i+1
    goto L1
Fim1:

```

```

i=1
L1:  if i>10 goto Fim1
    j=1
L2:  if j>10 goto Fim2
    k=1
    T6=i-1
    T7=T6*100
    T8=j-1
    T9=T8*10
    T13=T7+T9
L3:  if k>10 goto Fim3
    T11=k+1
    T12=T11-1
    T14=T13+T12
    T15=T14*4
    T16=M[T15]
    T18=T16+1
    M[T15]=T18
    k=k+1
    goto L3
Fim3: j=j+1
    goto L2
Fim2: i=i+1
    goto L1
Fim1:

```

- Variáveis de indução no laço interno (L3). T11 é função de k.
 $k = T11 - 1$, $T11 = k + 1$

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	k=1
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
L3:	if k>10 goto Fim3
	T11=k+1
	T12=T11-1
	T14=T13+T12
	T15=T14*4
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	k=k+1
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	T11=2
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
L3:	if T11>11 goto Fim3
	T12=T11-1
	T14=T13+T12
	T15=T14*4
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T11=T11+1
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Variáveis de indução no laço interno (L3). T12 é função de T11.

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	T11=2
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
L3:	if T11>11 goto Fim3
	T12=T11-1
	T14=T13+T12
	T15=T14*4
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T11=T11+1
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	T12=1
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
L3:	if T12>10 goto Fim3
	T14=T13+T12
	T15=T14*4
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T12=T12+1
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Variáveis de indução no laço interno (L3). T14 é função de T12 (e de T13, mas T13 é laço-invariante em relação ao laço interno). Como $T12 = T14 - T13$, a condição se torna $T14 - T13 > 10$, que se torna $T14 > T13 + 10$. O cálculo de $T13 + 10$ é passado para antes da condição.

i=1	
L1:	if i>10 goto Fim1
j=1	
L2:	if j>10 goto Fim2
T12=i	
T6=i-1	
T7=T6*100	
T8=j-1	
T9=T8*10	
T13=T7+T9	
L3:	if T12>10 goto Fim3
T14=T13+T12	
T15=T14*4	
T16=M[T15]	
T18=T16+1	
M[T15]=T18	
T12=T12+1	
goto L3	
Fim3:	j=j+1
goto L2	
Fim2:	i=i+1
goto L1	
Fim1:	

i=1	
L1:	if i>10 goto Fim1
j=1	
L2:	if j>10 goto Fim2
T6=i-1	
T7=T6*100	
T8=j-1	
T9=T8*10	
T13=T7+T9	
T14=T13+1	
T30=T13+10	
L3:	if T14>T30 goto Fim3
T15=T14*4	
T16=M[T15]	
T18=T16+1	
M[T15]=T18	
T14=T14+1	
goto L3	
Fim3:	j=j+1
goto L2	
Fim2:	i=i+1
goto L1	
Fim1:	

- Variáveis de indução no laço interno (L3). T15 é função de T14. E isso encerra as otimizações no laço L3.

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
	T14=T13+1
	T30=T13+10
L3:	if T14>T30 goto Fim3
	T15=T14*4
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T14=T14+1
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	j=1
L2:	if j>10 goto Fim2
	T6=i-1
	T7=T6*100
	T8=j-1
	T9=T8*10
	T13=T7+T9
	T14=T13+1
	T30=T13+10
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Expressões laço-invariantes no laço L2. i , $T6$ e $T7$ são laço invariantes no laço L2 e podem ser jogados para a inicialização.

	$i=1$
L1:	if $i > 10$ goto Fim1
	$j=1$
L2:	if $j > 10$ goto Fim2
	$T6=i-1$
	$T7=T6*100$
	$T8=j-1$
	$T9=T8*10$
	$T13=T7+T9$
	$T14=T13+1$
	$T30=T13+10$
	$T31=4*T30$
	$T15=T14*4$
L3:	if $T15 > T31$ goto Fim3
	$T16=M[T15]$
	$T18=T16+1$
	$M[T15]=T18$
	$T15=T15+4$
	goto L3
Fim3:	$j=j+1$
	goto L2
Fim2:	$i=i+1$
	goto L1
Fim1:	

	$i=1$
L1:	if $i > 10$ goto Fim1
	$T6=i-1$
	$T7=T6*100$
	$j=1$
L2:	if $j > 10$ goto Fim2
	$T8=j-1$
	$T9=T8*10$
	$T13=T7+T9$
	$T14=T13+1$
	$T30=T13+10$
	$T31=4*T30$
	$T15=T14*4$
L3:	if $T15 > T31$ goto Fim3
	$T16=M[T15]$
	$T18=T16+1$
	$M[T15]=T18$
	$T15=T15+4$
	goto L3
Fim3:	$j=j+1$
	goto L2
Fim2:	$i=i+1$
	goto L1
Fim1:	

- Variáveis de indução em L2. T8 é função de j.

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	j=1
L2:	if j>10 goto Fim2
	T8=j-1
	T9=T8*10
	T13=T7+T9
	T14=T13+1
	T30=T13+10
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	j=j+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	T8=0
L2:	if T8>9 goto Fim2
	T9=T8*10
	T13=T7+T9
	T14=T13+1
	T30=T13+10
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	T8=T8+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Variáveis de indução em L2. T9 é função de T8.

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	T8=0
L2:	if T8>9 goto Fim2
	T9=T8*10
	T13=T7+T9
	T14=T13+1
	T30=T13+10
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	T8=T8+1
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	T9=0
L2:	if T9>90 goto Fim2
	T13=T7+T9
	T14=T13+1
	T30=T13+10
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	T9=T9+10
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Variáveis de indução em L2. T13 é função de T9, uma vez que T7 é laço-invariante dentro de L2. $T9 = T13 - T7$, a condição se torna $T13 - T7 > 90$, $T13 > 90 + T7$ e $90 + T7$ pode ser calculado antes da condição.

	i=1
L1:	if i>10 goto Fim1 T6=i-1 T7=T6*100 T9=0
L2:	if T9>90 goto Fim2 T13=T7+T9 T14=T13+1 T30=T13+10 T31=4*T30 T15=T14*4
L3:	if T15>T31 goto Fim3 T16=M[T15] T18=T16+1 M[T15]=T18 T15=T15+4 goto L3
Fim3:	T9=T9+10 goto L2
Fim2:	i=i+1 goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1 T6=i-1 T7=T6*100 T13=T7 T20=90+T7
L2:	if T13>T20 goto Fim2 T14=T13+1 T30=T13+10 T31=4*T30 T15=T14*4
L3:	if T15>T31 goto Fim3 T16=M[T15] T18=T16+1 M[T15]=T18 T15=T15+4 goto L3
Fim3:	T13=T13+10 goto L2
Fim2:	i=i+1 goto L1
Fim1:	

- Variáveis de indução em L2. T14 e T30 são função de T13. T13=T14-1, e a condição fica T14-1>T20, T14>T20+1, e T20+1 pode ser calculado antes da condição.

```

i=1
L1:  if i>10 goto Fim1
      T6=i-1
      T7=T6*100
      T13=T7
      T20=90+T7
L2:  if T13>T20 goto Fim2
      T14=T13+1
      T30=T13+10
      T31=4*T30
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T13=T13+10
      goto L2
Fim2: i=i+1
      goto L1
Fim1:

```

```

i=1
L1:  if i>10 goto Fim1
      T6=i-1
      T7=T6*100
      T13=T7
      T20=90+T7
      T21=T20+1
      T14=T13+1
      T30=T13+10
L2:  if T14>T21 goto Fim2
      T31=4*T30
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T30=T30+10
      goto L2
Fim2: i=i+1
      goto L1
Fim1:

```

- Variáveis de indução em L2. T31 é função de T30.

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	T13=T7
	T20=90+T7
	T21=T20+1
	T14=T13+1
	T30=T13+10
L2:	if T14>T21 goto Fim2
	T31=4*T30
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	T14=T14+10
	T30=T30+10
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

	i=1
L1:	if i>10 goto Fim1
	T6=i-1
	T7=T6*100
	T13=T7
	T20=90+T7
	T21=T20+1
	T14=T13+1
	T30=T13+10
	T31=4*T30
L2:	if T14>T21 goto Fim2
	T15=T14*4
L3:	if T15>T31 goto Fim3
	T16=M[T15]
	T18=T16+1
	M[T15]=T18
	T15=T15+4
	goto L3
Fim3:	T14=T14+10
	T31=T31+40
	goto L2
Fim2:	i=i+1
	goto L1
Fim1:	

- Variáveis de indução em L1: $T6$ é função de i . $i = T6 + 1$. A condição fica $T6 + 1 > 10$, $T6 > 9$.

	$i = 1$
L1:	if $i > 10$ goto Fim1
	$T6 = i - 1$
	$T7 = T6 * 100$
	$T13 = T7$
	$T20 = 90 + T7$
	$T21 = T20 + 1$
	$T14 = T13 + 1$
	$T30 = T13 + 10$
	$T31 = 4 * T30$
L2:	if $T14 > T21$ goto Fim2
	$T15 = T14 * 4$
L3:	if $T15 > T31$ goto Fim3
	$T16 = M[T15]$
	$T18 = T16 + 1$
	$M[T15] = T18$
	$T15 = T15 + 4$
	goto L3
Fim3:	$T14 = T14 + 10$
	$T31 = T31 + 40$
	goto L2
Fim2:	$i = i + 1$
	goto L1
Fim1:	

	$T6 = 0$
L1:	if $T6 > 9$ goto Fim1
	$T7 = T6 * 100$
	$T13 = T7$
	$T20 = 90 + T7$
	$T21 = T20 + 1$
	$T14 = T13 + 1$
	$T30 = T13 + 10$
	$T31 = 4 * T30$
L2:	if $T14 > T21$ goto Fim2
	$T15 = T14 * 4$
L3:	if $T15 > T31$ goto Fim3
	$T16 = M[T15]$
	$T18 = T16 + 1$
	$M[T15] = T18$
	$T15 = T15 + 4$
	goto L3
Fim3:	$T14 = T14 + 10$
	$T31 = T31 + 40$
	goto L2
Fim2:	$T6 = T6 + 1$
	goto L1
Fim1:	

- Variáveis de indução em L1: T7 é função de T6. $T6 = T7/100$.
A condição fica $T7 > 900$.

	T6=0
L1:	if T6>9 goto Fim1 T7=T6*100 T13=T7 T20=90+T7 T21=T20+1 T14=T13+1 T30=T13+10 T31=4*T30
L2:	if T14>T21 goto Fim2 T15=T14*4
L3:	if T15>T31 goto Fim3 T16=M[T15] T18=T16+1 M[T15]=T18 T15=T15+4 goto L3
Fim3:	T14=T14+10 T31=T31+40 goto L2
Fim2:	T6=T6+1 goto L1
Fim1:	

	T7=0
L1:	if T7>900 goto Fim1 T13=T7 T20=90+T7 T21=T20+1 T14=T13+1 T30=T13+10 T31=4*T30
L2:	if T14>T21 goto Fim2 T15=T14*4
L3:	if T15>T31 goto Fim3 T16=M[T15] T18=T16+1 M[T15]=T18 T15=T15+4 goto L3
Fim3:	T14=T14+10 T31=T31+40 goto L2
Fim2:	T7=T7+100 goto L1
Fim1:	

- Propagação de cópias em L1: T13 pode ser substituído por T7.

```

T7=0
L1:  if T7>900 goto Fim1
    T13=T7
    T20=90+T7
    T21=T20+1
    T14=T13+1
    T30=T13+10
    T31=4*T30
L2:  if T14>T21 goto Fim2
    T15=T14*4
L3:  if T15>T31 goto Fim3
    T16=M[T15]
    T18=T16+1
    M[T15]=T18
    T15=T15+4
    goto L3
Fim3: T14=T14+10
    T31=T31+40
    goto L2
Fim2: T7=T7+100
    goto L1
Fim1:

```

```

T7=0
L1:  if T7>900 goto Fim1
    T20=90+T7
    T21=T20+1
    T14=T7+1
    T30=T7+10
    T31=4*T30
L2:  if T14>T21 goto Fim2
    T15=T14*4
L3:  if T15>T31 goto Fim3
    T16=M[T15]
    T18=T16+1
    M[T15]=T18
    T15=T15+4
    goto L3
Fim3: T14=T14+10
    T31=T31+40
    goto L2
Fim2: T7=T7+100
    goto L1
Fim1:

```

- Variáveis de indução em L1: T20 e T30 são função de T7.
 $T7 = T20 - 90$, e a condição fica $T20 > 990$. T14 não é variável de indução de T7 porque é incrementada no laço L2.

```

      T7=0
L1:   if T7>900 goto Fim1
      T20=90+T7
      T21=T20+1
      T14=T7+1
      T30=T7+10
      T31=4*T30
L2:   if T14>T21 goto Fim2
      T15=T14*4
L3:   if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      goto L1
Fim1:

```

```

      T7=0
      T20=90+T7
      T30=T7+10
L1:   if T20>990 goto Fim1
      T21=T20+1
      T14=T7+1
      T31=4*T30
L2:   if T14>T21 goto Fim2
      T15=T14*4
L3:   if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      T20=T20+100
      T30=T30+100
      goto L1
Fim1:

```

- Variáveis de indução em L1: T21 é função de T20.

```

T7=0
T20=90+T7
T30=T7+10
L1:  if T20>990 goto Fim1
      T21=T20+1
      T14=T7+1
      T31=4*T30
L2:  if T14>T21 goto Fim2
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      T20=T20+100
      T30=T30+100
      goto L1
Fim1:

```

```

T7=0
T20=90+T7
T30=T7+10
T21=T20+1
L1:  if T21>991 goto Fim1
      T14=T7+1
      T31=4*T30
L2:  if T14>T21 goto Fim2
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      T21=T21+100
      T30=T30+100
      goto L1
Fim1:

```

- Propagação de Cópias no cabeçalho de L1. T7 pode ser substituída em T20 e T30 e T20 pode ser substituída em T21.

```

T7=0
T20=90+T7
T30=T7+10
T21=T20+1
L1:  if T21>991 goto Fim1
      T14=T7+1
      T31=4*T30
L2:  if T14>T21 goto Fim2
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      T21=T21+100
      T30=T30+100
      goto L1
Fim1:

```

```

T7=0
T30=10
T21=91
L1:  if T21>991 goto Fim1
      T14=T7+1
      T31=4*T30
L2:  if T14>T21 goto Fim2
      T15=T14*4
L3:  if T15>T31 goto Fim3
      T16=M[T15]
      T18=T16+1
      M[T15]=T18
      T15=T15+4
      goto L3
Fim3: T14=T14+10
      T31=T31+40
      goto L2
Fim2: T7=T7+100
      T21=T21+100
      T30=T30+100
      goto L1
Fim1:

```

2)(4 pontos)Escreva uma DDS para gerar código de três endereços para referências a elementos de arrays [1..10,1..10] of integer em pascal na seguinte produção:

$E \rightarrow \text{id} [E , E]$

3)(3 pontos)Monte o grafo de fluxo do código do exercício 1 e otimize-o removendo expressões laço invariantes e variáveis de indução.

1) (2.5 pontos) Faça uma DDS (gramática e ações semânticas) que reconheça declarações de arrays bidimensionais de inteiros e chars em pascal (ex: `var a:array[1..100,1..100] of integer`). Na tabela de símbolos devem ser colocados o limite inferior e superior de cada dimensão e o tipo de cada elemento.

2) (2.5 pontos) Considere a gramática a seguir para expressões simples envolvendo somas de variáveis e constantes. Acrescente produções na gramática com as ações semânticas para gerar código de três endereços para permitir que uma referência a array bidimensional apareça em uma expressão (ex: $a := b[45 + c, 10]$) ou do lado esquerdo de uma atribuição ($a[10, 12] := 20$). Para endereçamento indexado utilize o código de 3 endereços nos formatos $X[i] := Y$ e $Y := X[i]$, onde i contém o deslocamento do elemento dentro do array. Considere que as matrizes são armazenadas por linhas, ou seja, para o cálculo do endereço de um elemento, na memória estão armazenados em primeiro lugar todos os elementos da primeira linha, depois os da segunda linha e assim por diante..

3) (2.5 pontos) Considere a gramática a seguir:

$S \rightarrow \text{while expr do begin } S \text{ end} \mid$

$S \rightarrow S ; S \mid$

$S \rightarrow \text{break} \mid$

$S \rightarrow \text{outro}$

Escreva uma DDS que gere C3E para as 4 produções. Considere que o comando Break é traduzido para um desvio para o comando após o While envolvente mais próximo.

4) (2.5 pontos) Na linguagem C a atribuição é considerado um operador que pode aparecer em expressões e o resultado da operação é o valor que foi atribuído (em pascal isto não é permitido porque a atribuição é um comando e não um operador). Considere a gramática a seguir que gera expressões com soma e atribuição

$$E \rightarrow E + E$$

$$E \rightarrow \text{id} = E$$

$$E \rightarrow \text{id}$$

$$E \rightarrow \text{cte}$$

$$E \rightarrow (E)$$

Insira ações semânticas para gerar C3E para a gramática.

- 1) (4.0 pontos) Uma forma de tratar declarações de Records em pascal é criar uma tabela de símbolos em separado para a lista de campos. Escreva uma gramática para reconhecimento de REFERÊNCIAS a Records em atribuições em pascal em que os campos podem ser do tipo integer ou record. Coloque as ações semânticas para gerar o código da atribuição e explique como buscar na tabela de símbolos o endereço de cada campo.

Ex: Para o record

A:record

 A1:integer;

 A2:integer;

 A3:record

 A31:integer;

 End;

 A4:integer;

End;

A gramática deve reconhecer a referencia A.A3.A31 := 10;

5) (2.5 pontos) O comando continue dentro de uma estrutura de repetição na linguagem C faz com que a iteração corrente seja interrompida passando a estrutura a executar a próxima iteração. Considere a gramática a seguir para o comando for:

comando $\rightarrow$ for ($e_1; e_2; e_3$) comando

comando $\rightarrow$ continue

comando $\rightarrow$ outro

A semântica do comando for na linguagem C é a seguinte:

```
e1;  
while e2 do  
    {  
        comando;  
        e3;  
    }
```

Faça um esquema de tradução para gerar código de 3 endereços para o comando for.

6) (2.5 pontos) Escreva uma DDS para converter uma expressão que está em notação pós fixada com os operadores $+$, $-$, $*$ e $/$, e operandos de um único dígito em notação infixada. A expressão resultante (na forma de um string, como no exercício da derivada das expressões) deve ser retornada no atributo `.exp` do símbolo inicial. A expressão resultante pode ter parênteses redundantes. A gramática deve ser apropriada para análise descendente recursiva, ou seja, não deve conter recursão à esquerda.

Resposta :

$E \rightarrow E_1 E_2 * \{E.exp := '(' + E_1.exp + '*' + E_2.exp + ')\}$
 $E \rightarrow E_1 E_2 + \{E.exp := '(' + E_1.exp + '+' + E_2.exp + ')\}$
 $E \rightarrow E_1 E_2 - \{E.exp := '(' + E_1.exp + '-' + E_2.exp + ')\}$
 $E \rightarrow E_1 E_2 / \{E.exp := '(' + E_1.exp + '/' + E_2.exp + ')\}$
 $E \rightarrow \text{dígito } E.exp := \text{dígito}$

Removendo a Recursão:

$$E \rightarrow \text{dig} \{R.h := \text{dig}\} R \{E.exp := R.s \}$$

$$R \rightarrow E + \{R_1.h := '(' + R.h + '+' + E.exp + ')' \} R_1 \{R.s := R_1.s\}$$

$$R \rightarrow E * \{R_1.h := '(' + R.h + '*' + E.exp + ')' \} R_1 \{R.s := R_1.s\}$$

$$R \rightarrow E - \{R_1.h := '(' + R.h + '-' + E.exp + ')' \} R_1 \{R.s := R_1.s\}$$

$$R \rightarrow E / \{R_1.h := '(' + R.h + '/' + E.exp + ')' \} R_1 \{R.s := R_1.s\}$$

$$R \rightarrow \epsilon \{R.s := R.h\}$$

7) (2.5 pontos) Mostre o C3E gerado para o trecho a seguir, considerando as gramáticas e ações semânticas vistas em aula e considerando as operações booleanas implementadas como desvios no fluxo de controle. Considere na numeração das variáveis temporárias e labels a ordem em que os mesmos são criados.

```
if (a<b+c) and (b<a+c) and ((c<a+b) or (e=a+b)) then
  begin
    a:=(b+c)*d;
    d:=(c*d)+(e*f);
  end
else if (d+e>3) then
  c:=3
  else
  c:=4;
\end{tiny}
```

8) (2.5 pontos) Considere a gramática a seguir para geração de expressões pós-fixadas. Faça um esquema de tradução para avaliação do valor das expressões geradas e implemente o mesmo em um analisador descendente recursivo. Tome o cuidado de remover a recursão da gramática para implementar o tradutor. O valor da expressão deve ser retornado no atributo .val do símbolo inicial.

- $E \rightarrow E E + \mid E E * \mid \text{digito}$

- Resposta:

- $E \rightarrow E_1 E_2 * \{E.val := E_1.val * E_2.val\}$
- $E \rightarrow E_1 E_2 + \{E.val := E_1.val + E_2.val\}$
- $E \rightarrow \text{dígito} \{E.val := \text{dig.val}\}$

- Removendo a Recursão:

- $E \rightarrow \text{dig} \{R.h := \text{dig.val}\}$
- $R \{E.val := R.s\}$
- $R \rightarrow E + \{R1.h := R.h + E.val\}$
- $R_1 \{R.s := R_1.s\}$
- $R \rightarrow \epsilon \{R.s := R.h\}$

9) Para o trecho de código a seguir:

```
Var A , B, C : array [5..10,5..10] of integer;  
    lin , col , ind : integer;  
begin  
    for lin:=5 to 10 do  
        for col:=5 to 10 do  
            begin  
                C[lin , col]:=0;  
                for ind:=5 to 10 do  
                    C[lin , col]:=C[lin , col]+A[lin , ind]*B[ind , col];  
                end;  
            end;  
        end;  
    end;
```

a. Mostre o C3E gerado utilizando o padrão Tnn para variáveis temporárias e Lnn para labels. A numeração dos labels e variáveis temporárias deve estar de acordo com as ações semânticas da gramática (2 pontos)

9) Mostre o C3E gerado para o comando a seguir considerando, na numeração das variáveis temporárias, a ordem em que o código foi gerado.

$a := (a + b) * (a - b + d) / (c + b + d);$

1. (2.5 pontos) Considere a gramática a seguir que gera C3E para o While e atribuição:

$$S \rightarrow \text{while } E \text{ do } S_1 \{ \text{lbfim geralabel; lblaco=geralabel; } \\ S.\text{cod}=\text{gen}(\text{rotulo, lblaco}) \parallel E.\text{cod} \parallel \text{gen}(\text{if } E.\text{place}=0 \text{ goto } \\ \text{lbfim}) \parallel S_1.\text{cod} \parallel \text{gen}(\text{goto, lblaco}) \parallel \text{gen}(\text{rotulo,lbfim}) \}$$

$$S \rightarrow \text{Begin } L \text{ End } \{ S.\text{cod} := L.\text{cod} \}$$

$$S \rightarrow \text{break}$$

$$S \rightarrow \text{id} := E \{ S.\text{cod} := E.\text{cod} \parallel \text{gen}(:=,\text{id},E.\text{place}) \}$$

$$L \rightarrow S \ R \{ L.\text{cod} := S.\text{cod} \parallel R.\text{cod} \}$$

$$R \rightarrow ; \ S \ R_1 \mid \{ R.\text{cod} := S.\text{cod} \parallel R_1.\text{cod} \}$$

$$R \rightarrow \epsilon \{ R.\text{cod} := "" \}$$

Coloque as ações semânticas necessárias para a geração de C3E do break. Considere que o comando Break é traduzido para um desvio para o comando após o While envolvente mais próximo. Considere a possibilidade de Whiles aninhados.

3. (2.5 pontos) Considere a produção a seguir para geração de código de avaliação de expressões.

$$E \rightarrow E_1 + T \{ E.place := cria_temp \quad E.cod := E_1.cod \parallel T.cod \\ \parallel gen(+, E.place, E_1.place, T.place) \}$$
$$E \rightarrow T \{ E.cod := T.cod; E.place := T.place \}$$

Escreva uma produção equivalente não recursiva, com as devidas ações semânticas.

4. (2.5 pontos) A seguinte gramática gera expressões formadas pela aplicação do operador aritmético $+$ a constantes inteiras e reais. Quando dois inteiros são adicionados, o tipo resultante é inteiro, em caso contrário é real.

$$E \rightarrow E_1 + T \mid T$$

$$T \rightarrow \text{num.num} \mid \text{num}$$

- Coloque as ações semânticas para traduzir a expressão analisada para uma expressão pós-fixada (atributo exp) e retornar o tipo da expressão no atributo tipo.
- Use o operador unário inttoreal, que converte o valor no topo da pilha para real para converter um valor inteiro num valor real equivalente de forma que ambos os operadores de + na forma posfixa tenham o mesmo tipo.
- P.ex. a expressão $1+2+3.4+5$ resultaria em $1\ 2\ +\ \text{inttoreal}\ 3.4\ +\ 5\ \text{inttoreal}\ +$

- 1) (2.5 pontos) Considere a seguinte produção para o operador ternário em C:

$E \rightarrow E_1 ? E_2 : E_3 \{ \text{if } (E_1.\text{val} == \text{TRUE}) E.\text{val} = E_2.\text{val}; \text{ else } E.\text{val} = E_3.\text{val} \}$

$E \rightarrow T \{ E.\text{val} = T.\text{val} \}$

- Utilizando a técnica vista em aula, remova a recursão à esquerda da produção atualizando as ações semânticas.

2) (2.5 pontos) Considerando a mesma produção para o operador ternário em C, escreva as ações semânticas para gerar código de 3 endereços para o operador ternário.

$$E \rightarrow E_1 ? E_2 : E_3$$

- 3) (2.5 pontos) Escreva as ações semânticas para as produções a seguir para gerar código de 3 endereços para atribuição a um elemento de array considerando uma linguagem que aceite arrays unidimensionais com 0 como primeira posição e aceitando somas na expressão de índice.
- Ex:

$$X[3+I+Y[3]]:=3;$$

$$Y:=X[3];$$
- Para endereçamento indexado utilize o código de 3 endereços nos formatos $X[i]:=Y$ e $Y:=X[i]$.
- Produções:

$$\text{atrib} \rightarrow \text{id} \rightarrow E \mid \text{id} [E] = E$$

$$E \rightarrow E+T \mid T$$

$$T \rightarrow \text{cte} \mid \text{id} \mid \text{id}[E]$$

1) (2.5 pontos) Escreva uma gramática e ações semânticas para gerar um string com valores reais, e ações semânticas para calcular, em um atributo "val", o valor real gerado no string. Ex: Se o valor gerado é "3.14" o atributo val do símbolo inicial deve retornar 3.14. Considere que os terminais da gramática são os dígitos de 0 a 9, e que o número real obrigatoriamente contem casas decimais antes e após o ponto.

- 2) (2.5 pontos) Considere a gramática a seguir que gera expressões envolvendo produtos de variáveis e constantes:

$E \rightarrow E * T \mid T$

$F \rightarrow \text{cte} \mid \text{id}$

- Especifique ações semânticas na gramática para gerar o string equivalente à expressão reconhecida, mas efetuando as seguintes simplificações na expressão:
 - a) Multiplicações pela constante 1 são eliminadas
 - b) Multiplicações pela constante 0 resultam em 0
 - c) Multiplicações entre constantes são avaliadas na análise e a expressão é substituída pelo resultado da avaliação

- Ex:
 - A expressão $3 * 2 * a$ gera o string $6 * a$
 - A expressão $3 * 0 * x$ gera o string 0
 - A expressão $3 * 1 * y$ gera o string $3 * y$
- Assuma a existência de funções para converter de strings para valores inteiros e vice-versa.
- Não é necessário remover a recursão da gramática.

- 3)(2.5 pontos) Considere a gramática a seguir que gera expressões, da linguagem C, contendo somente atribuições e identificadores:

$$E \rightarrow \text{id} = E1 \mid \text{cte} \mid \text{id}$$

- Coloque ações semânticas para gerar, a partir da análise de uma expressão em C, um string com uma sequência de atribuições equivalente em pascal. Por exemplo, a expressão "a=b=c=d" resulta na sequência de atribuições "c:=d;b=c;a=b;"

- 4) (2.5 pontos) Considere a produção para o comando PARA a seguir:

Com _Para $\rightarrow$ Para id de E ate E faca Comando

- Escreva uma função para o reconhecimento do comando Para, retornando TRUE se reconheceu corretamente ou FALSE se ocorreu algum erro no reconhecimento. A função deve também emitir as seguintes mensagens de Erro:
 - (a) Tipo da variável de controle deve ser inteiro
 - (b) Variável de controle não declarada
 - (c) Expressão de valor inicial deve ser do tipo inteiro
 - (d) Expressão de valor final deve ser do tipo inteiro
 - (e) Faltou a palavra de
 - (f) Faltou a palavra ate
 - (g) Faltou a palavra faca

Descreva os atributos utilizados na função e os parâmetros da função para buscar informações na tabela de símbolos.

- 1) (2.5 pontos) Escreva um esquema de tradução (gramática e ações semânticas) que reconheça números romanos e calcule o valor do número romano reconhecido, retornando-o como atributo do símbolo inicial da gramática. Veja alguns exemplos de números romanos e seus respectivos valores em números arábicos.

I=1	VI=6	XX=20	LXX = 70	CCC = 300	DCCC = 800
II=2	VII=7	XXX=30	LXXX = 80	CD = 400	CM = 900
III=3	VIII=8	XL=40	XC = 90	D = 500	M = 1000
IV=4	IX=9	L=50	C = 100	DC = 600	MM = 2000
V=5	X=10	LX = 60	CC = 200	DCC = 700	MMM = 3000

- Obs.:
 - O menor número romano é I (=1)
 - O maior número romano possível é MMMCMXCIX (=3999)

- 3) (2.5 pontos) O código de 3 endereços de uma chamada de uma função void com parâmetros passados por valor se resume a empilhar os parâmetros e executar um Call. Assim, a chamada de função

X(a,b,c,d)

- seria traduzida como

push a
push b
push c
push d
call X

- Escreva um esquema de tradução (produções e ações semânticas) para o reconhecimento e geração de C3E de uma chamada de função void com uma lista de expressões passadas como parâmetros por valor. Toda a troca de informação entre as produções deve ocorrer via atributos (nada de variáveis globais).

- Resposta:

- O simbolo L representa uma lista de expressões passadas como parâmetros, podendo ser vazia

$\text{Chamada_F} \rightarrow \text{id} (L)$	$\{ \text{Chamada_F.cod} = \text{L.cod} + \text{gen}(\text{Call}, \text{id}) \}$
$L \rightarrow E R$	$\{ \text{L.cod} = \text{E.cod} + \text{gen}(\text{"push"}, \text{E.place}) + \text{R.cod} \}$
$L \rightarrow \epsilon$	$\{ \text{L.cod} = "" \}$
$R \rightarrow , E R_1$	$\{ \text{R.cod} = \text{E.cod} + \text{gen}(\text{"push"}, \text{E.place}) + \text{R}_1.\text{cod} \}$
$R \rightarrow \epsilon$	$\{ \text{R.cod} = "" \}$

4) (2.5) Algumas linguagens como PHP possuem o comando break N, onde N designa o número de for's aninhados para fora dos quais o break desvia. Escreva um esquema de tradução (produções e ações semânticas) para gerar o C3E do for (considere para o for a mesma semântica utilizada na linguagem C), da lista de comandos e do break N pelas produções abaixo:

$\text{ComFor} \rightarrow \text{for} (\text{Exp1}; \text{Exp2}; \text{Ex3}) \text{ Lcom}$

$\text{Lcom} \rightarrow \text{Com RLCom}$

$\text{RLCom} \rightarrow ; \text{Com RLCom} \mid \epsilon$

$\text{Com} \rightarrow \text{Atrib} \mid //$ não é necessário colocar ações semânticas para esse comando

$\text{ComIf} \mid //$ nem esse

$\text{ComFor} \mid$

ComBreakN

$\text{ComBreakN} \rightarrow \text{break cte}$

- 2. (2.5 pontos) Considere a produção a seguir para geração de código de avaliação de expressões.

$E \rightarrow E_1 + T \{E.place := cria_temp; E.cod := E_1.cod \parallel T.cod \parallel gen(+, E.place, E_1.place, T.place)\}$

$E \rightarrow T \{E.cod := T.cod; E.place := T.place\}$

- Escreva uma produção equivalente não recursiva, com as devidas ações semânticas.

3) (2.5 pontos) O comando Loop, na linguagem Oberon, causa a execução repetida da lista dos comandos entre o LOOP e o END, até que seja executado o comando EXIT, como mostrado no exemplo abaixo:

```
LOOP
```

```
    Read(i);
```

```
    IF  $i < 0$  then EXIT END;
```

```
END;
```

- As produções a seguir reconhecem o comando LOOP e o EXIT:

ComLOOP $\rightarrow$ LOOP Lista_comandos END

Lista_comandos $\rightarrow$ Comando

- Lista_comandos $\rightarrow$ Comando ; Lista_comandos
- Comando $\rightarrow$ ComEXIT |
- ComLOOP |
- Outros
- ComEXIT $\rightarrow$ EXIT
- Escreva as ações semânticas para geração de código para a máquina de pilha para ambos comandos. Preste especial atenção nas informações necessárias em cada produção, onde a informação é gerada e como a mesma é transferida entre as produções.

1) (3.0 pontos) Considerando as produções e ações semânticas a seguir:

$\text{com_cond} \rightarrow \text{if } \{E.\text{true}=\text{geralabel}; E.\text{false}=\text{geralabel}\}E \text{ then Com else Com}_2 \{ \text{Final} := \text{GeraLabel}; \text{ComCond.cod} := E.\text{cod} \parallel \text{gen}(E.\text{true}) \parallel \text{Com.cod} \parallel \text{gen}(\text{goto Final}) \parallel \text{gen}(\text{label}, E.\text{false}) \parallel \text{Com}_2.\text{cod} \parallel \text{gen}(\text{label}, \text{Final}) \}$

$E \rightarrow \{ E_1.\text{true} := E.\text{true}; E_1.\text{false} := \text{GeraLabel} \} E_1 \text{ or } \{ E_2.\text{true} := E.\text{true}; E_2.\text{false} := E.\text{false} \} E_2 \{ E.\text{cod} := E_1.\text{cod} \parallel \text{gen}(\text{label } E_1.\text{false}) \parallel E_2.\text{cod} \}$

$E \rightarrow \{ E_1.\text{true} := \text{geralabel}; E_1.\text{false} := E.\text{false} \} E_1 \text{ and } \{ E_2.\text{true} := E.\text{true}; E_2.\text{false} := E.\text{false} \} E_2 \{ E.\text{cod} := E_1.\text{cod} \parallel \text{gen}(\text{label } E_1.\text{true}) \parallel E_2.\text{cod} \}$

$E \rightarrow id_1 > id_2 \{ E.\text{cod} := \text{gen}(\text{if } id_1 > E_1 \text{ goto } E.\text{true}) \parallel \text{gen}(\text{goto } E.\text{false}) \}$

$E \rightarrow \{ E_1.\text{true} := E.\text{true}; E_1.\text{false} := E.\text{false} \} (E_1) \{ E.\text{cod} := E_1.\text{cod} \}$

$\text{Com} \rightarrow id := \text{cte} \{ \text{Com.cod} := \text{gen}(id := \text{cte}) \}$

mostre a árvore de derivação anotada para a sentença:

```
if (a>b and b>c ) or c>d
  then a:=0
  else b:=0;
```

2)(3.0 pontos) Mostre o C3E gerado para o trecho de código a seguir, considerando que V é um array[1..20] of integer e cada integer ocupa dois bytes:

```
for i:=1 to 10 do
  begin
    aux:=V[i] ;
    V[i]:=V[21-i] ;
    V[21-i]:= aux ;
  end ;
```

4)(2 pontos)Represente o bloco básico a seguir, gerado para a atribuição $a[i]=b[i]+c[i]$, através de um grafo e gere o código otimizado a partir do grafo.

$T1=i-2$

$T2=T1*2$

$T3=B[T2]$

$T4=i-2$

$T5=T4*2$

$T6=C[T5]$

$T7=T3+T6$

$T8=i-2$

$T9= T8*2$

$A[T9]=T7$

[Voltar para Tópicos](#)

3) Represente o bloco básico a seguir através de um grafo e gere o código otimizado a partir do grafo, considerando que as únicas variáveis do bloco básico que devem estar atualizadas ao final do bloco são as variáveis a e T1.

T1 := a + c

T2 := T1 + c

T3 := a + c

T3 := T3 + d

T4 := c + b

T5 := T4 + d

T2 := T2 * T4

T8 := T7 * T2

A := T8

3) (2.5 pontos) Mostre como o código abaixo seria representado em um grafo dirigido acíclico e mostre o C3E obtido a partir do grafo.

$t1 = i * 4$

$aux = v + t1$

$t2 = 9 - i$

$t3 = t2 * 4$

$t4 = v + t3$

$t5 = i * 4$

$v = t4 + t5$

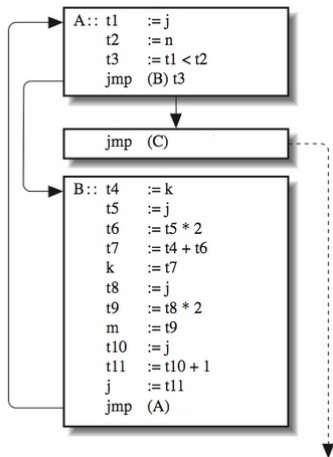
$t6 = 9 - i$

$t7 = t6 * 4$

$v = aux + t7$

$i = i + 1$

2) (2.5 pontos) Considere o grafo de fluxo de controle abaixo:



Mostre o grafo acíclico dirigido correspondente ao bloco B e mostre o código gerado a partir do grafo acíclico dirigido. Considere que ao final do bloco B apenas as variáveis k e j devem estar vivas.

1) (2.5 pontos) Considerando as produções e ações semânticas a seguir:

```

com_cond →      if                      {E.true=geralabel; E.false=geralabel}
                  E then Com else Com2  { Final := Geralabel;
                                           ComCond.cod := E.cod ||
                                           gen(label, E.true)||Com.cod||gen(goto Final)||
                                           gen(label, E.false)||Com2.cod||gen(label, Final)}

E →              { E1.true := E.true ;E1.false := Geralabel}
                  E1 or                    { E2.true := E.true;E2.false := E.false}
                  E2                      { E.cod := E1.cod || gen ( label E1.false ) || E2.cod }
E →              { E1.true := geralabel ;E1.false := E.false}
                  E1 and                   { E2.true := E.true;E2.false := E.false}
                  E2                      { E.cod := E1.cod || gen ( label E1.true ) || E2.cod }
E →              id1 > id2                 { E.cod := gen (if id1 > id2 goto E.true) || gen (goto E.false)}
E →              id1 = id2                { E.cod := gen (if id1 > id2 goto E.true) || gen (goto E.false)}
E →              {E1.true:=E.true; E1.false:=E.false}
                  (E1)                    { E.cod := E1.cod}
Com →            id := cte                 {Com.cod:=gen(id := cte)}
    
```

mostre o C3E gerado para a sentença:

```

if (d=31)or(d=30 and (m=4 or m=6 or m=9 or m=11))
then a:=0
else b:=0;
    
```

2)(2.5 pontos) Uma forma de implementar o case do pascal é colocar juntos, no C3E, o teste e o código de cada entrada. Assim, o trecho a seguir:

```
case dia of
    0: write("domingo");
    1: write("segunda");
    else write("sei lá");
end;
```

poderia gerar o código:

```
    if dia<>0 goto L1
    write("domingo")
    goto LFIM
L1:    if dia<>1 goto L2
    write("segunda")
    goto LFIM
L2:    write("sei lá")
LFIM
```

Faça um esquema de tradução para gerar o C3E para o case usando a abordagem descrita.

3)(2.5 pontos) Mostre o código gerado para a expressão abaixo:

$$C := A[B[i, j + 1], B[i, j - 1]]$$

considerando as declarações a seguir:

```
var C: integer;  
    A, B: array[11..20, 11..20] of integer;
```

4) (2.5 pontos) Considere o C3E gerado para o código a seguir:

```
var M,X:array[1..10] of integer;  
    N:array[1..20] of integer;  
for i:=1 to 10 do  
    begin  
        N[i]:=M[i];  
        N[i+10]:=X[i];  
    end;
```

[Voltar para Tópicos](#)

```
i=1
Laço: if i>10 goto Fim
      T1=i-1
      T2=T1*2
      T3=M[T2]
      T4=i-1
      T5=T4*2
      N[T5]:=T3
      T6=i-1
      T7=T6*2
      T8=X[T7]
      T9=i+10
      T10=T9-1
      T11=T10*2
      N[T11]=T8
      i=i+1
      goto Laço
```

Aplice as transformações para gerar o código o mais otimizado possível. Indique, a cada passo, qual a transformação utilizada.

i=1

Laço: if i>10 goto Fim

T1=i-1

T2=T1*2

T3=M[T2]

T4=i-1

T5=T4*2

N[T5]:=T3

T6=i-1

T7=T6*2

T8=X[T7]

T9=i+10

T10=T9-1

T11=T10*2

N[T11]=T8

i=i+1

goto Laço

Fim:

i=1

Laço: if i>10 goto Fim

T1=i-1

T2=T1*2

T3=M[T2]

T5=T1*2

N[T5]:=T3

T7=T1*2

T8=X[T7]

T9=i+10

T10=T9-1

T11=T10*2

N[T11]=T8

i=i+1

goto Laço

i=1

Laço: if i>10 goto Fim

T1=i-1

T2=T1*2

T3=M[T2]

N[T2]:=T3

T8=X[T2]

T9=i+10

T10=T9-1

T11=T10*2

N[T11]=T8

i=i+1

goto Laço

T1=0

T9=11

Laço: if T1>9 goto Fim

T2=T1*2

T3=M[T2]

N[T2]:=T3

T8=X[T2]

T10=T9-1

T11=T10*2

N[T11]=T8

T1=T1+1

T9=T9+1

goto Laço

T2=0
T10=10

Laço: if T2>18 goto Fim

T3=M[T2]
N[T2]:=T3
T8=X[T2]
T11=T10*2
N[T11]=T8
T2=T2+2
T10=T10+1
goto Laço

T2=0
T11=20

Laço: if T2>18 goto Fim

T3=M[T2]
N[T2]:=T3
T8=X[T2]
N[T11]=T8
T2=T2+2
T11=T11+2
goto Laço

1) (5.0 pontos) Considere o C3E gerado para o código a seguir, que escreve os primos de 2 a 200 pelo Crivo de Eratóstenes:

```
for (j=2;j <=200;j++)  
    if ( v [ j ]==0)  
    {  
        printf ( "\%d" , j );  
        for (k=2*j ; k<=200;k+=j )  
            v [ k ]=1;  
    }
```

```
j=2
L1:  if j>200 goto L2
      T1=4*j
      T2=V[T1]
      if T2<>0 goto L3
      param j
      call escreve
      k=2*j
L5:  if k>200 goto L3
      T3=k*4
      V[T3]=1
      k=k+j
      goto L5
L3:  j=j+1
      goto L1
```

L2:

Aplice as transformações vistas em aula para gerar o código o mais otimizado possível, começando pela montagem dos blocos básicos, etc.... Indique, a cada passo, qual a transformação utilizada. Considere que ao final desse código, nenhuma das variáveis precisa estar viva.

```
j=2
L1:  if j>200 goto L2
    T1=4*j
    T2=V[T1]
    if T2<>0 goto L3
    param j
    call escreve
    k=2*j
L5:  if k>200 goto L3
    T3=k*4
    V[T3]=1
    k=k+j
    goto L5
L3:  j=j+1
    goto L1
L2:
```

```
j=2
L1:  if j>200 goto L2
    T1=4*j
    T2=V[T1]
    if T2<>0 goto L3
    param j
    call escreve
    k=2*j
    T3=k*4
L5:  if T3>800 goto L3
    V[T3]=1
    T4=4*j
    T3=T3+T4
    goto L5
L3:  j=j+1
    goto L1
L2:
```

j=2

L1: if j>200 goto L2

T1=4*j

T2=V[T1]

if T2<>0 goto L3

param j

call escreve

k=2*j

T3=k*4

T4=4*j

L5: if T3>800 goto L3

V[T3]=1

T3=T3+T4

goto L5

L3: j=j+1

goto L1

L2:

j=2

L1: if j>200 goto L2

T1=4*j

T2=V[T1]

if T2<>0 goto L3

param j

call escreve

T3=8*j

T4=4*j

L5: if T3>800 goto L3

V[T3]=1

T3=T3+T4

goto L5

L3: j=j+1

goto L1

L2:

j=2

L1: if j>200 goto L2

T1=4*j

T2=V[T1]

if T2<>0 goto L3

param j

call escreve

T3=8*j

L5: if T3>800 goto L3

V[T3]=1

T3=T3+T1

goto L5

L3: j=j+1

goto L1

L2:

- j não pode ser removido porque é parâmetro da escreve
- T1 e T2 não podem ser removidos porque são índices do vetor

2) (2.5 pontos) O código a seguir inverte um vetor

```
int v[10];  
for (i=0; i<5; i++)  
{  
    aux=v[i];  
    v[i]=v[9-i];  
    v[9-i]=aux;  
}
```

e um possível C3E equivalente é:

$i = 0$

laco: if $i=5$ goto fim

$t1 = i * 4$

$aux = v[t1]$

$t2 = 9 - i$

$t3 = t2 * 4$

$t4 = v[t3]$

$t5 = i * 4$

$v[t5] = t4$

$t6 = 9 - i$

$t7 = t6 * 4$

$v[t7] = aux$

$i = i + 1$

goto laco

fim:

Aplice as otimizações vistas em aula para obter um código otimizado. Mostre a transformação feita a cada passo.

1) (4.0 pontos) Mostre o C3E gerado para o trecho de código a seguir (2.0 pontos) e otimize o código reduzindo ao máximo as variáveis de indução, removendo expressões laço-invariantes e removendo expressões repetidas (2.0 pontos)

```
var  a:array[2..10] of integer;  
aux,i:integer;
```

```
for i:=3 to 9 do  
begin  
  aux:=a[i];  
  a[i]:=a[i+1];  
  a[i+1]:=aux;  
end;
```


- 3) (2 pontos) Considere o C3E gerado para o código a seguir:

```
for (i=0;i<10;i++)  
    for (j=0;j<10;j++)  
        M[i,j]=0;
```

```
    i=0
```

```
Laço2:  if i>9 goto Fim
```

```
    j=0
```

```
Laço1:  if j>9 goto LB2
```

```
    T1=i*10
```

```
    T2=T1+j
```

```
    M[T2]=0
```

```
    j=j+1
```

```
    goto Laço1
```

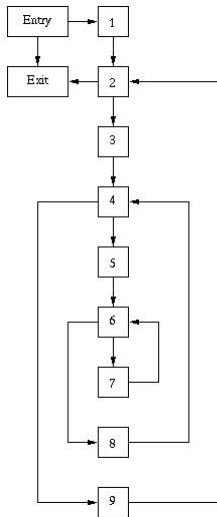
```
LB2:    i=i+1
```

```
    goto Laço2
```

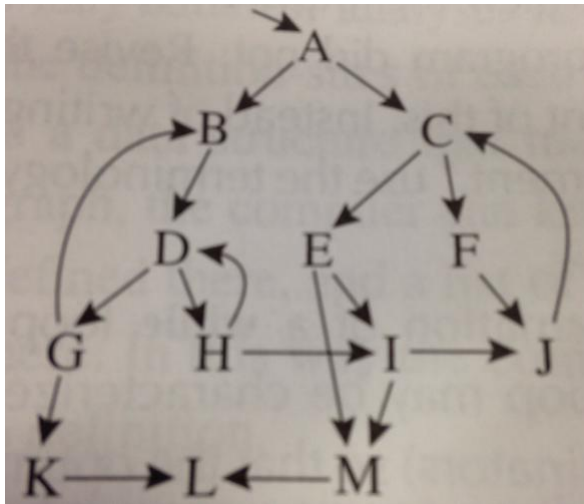
```
Fim:
```

Aplice as transformações vistas em aula para gerar o código o mais otimizado possível. Indique, a cada passo, qual a transformação utilizada.

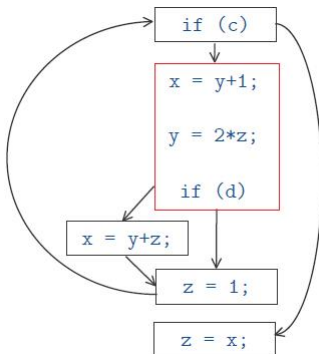
2) (2.0 pontos) Mostre a árvore de dominadores para o Grafo de Fluxo de Controle a seguir:



1) (2.0 pontos) Mostre a árvore de dominadores para o Grafo de Fluxo de Controle a seguir:



3) (1.5 pontos) Identifique quais variáveis estão vivas ao final de cada bloco básico do Grafo de Fluxo de Controle a seguir. Considere que, ao final do nodo de saída, apenas a variável z deve estar viva.



[Voltar para Tópicos](#)

4) (1.5 pontos) Considere a função recursiva a seguir:

```
void esc1N (int N)
{
    if (N<=1) return ;
    if (N%2==0)
    {
        printf ("%d",N);
        esc1N (N-2);
        return ;
    }
    esc1N (N-1);
}
```

Aplique a transformação vista em aula para remover a recursão de cauda, gerando uma função equivalente não recursiva.

2) (1.5 pontos) Considere o código a seguir, que chama N vezes a função esclaN:

```
for ( i=1; i<=N; i++)  
    esclaN(N);
```

que pode gerar o C3E a seguir:

```
    i=1  
l1:  if i>N goto l2  
    param N  
    call esclaN,1  
    i=i+1  
    goto l1
```

l2:

Aplique a transformação vista em aula para desdobramento de laços, desdobrando por um fator de desdobramento igual a 2.

3) (1.5 pontos) Considere a função recursiva a seguir para escrever todos os elementos de uma lista:

```
void lista (struct nodo * lista )
{
    if ( lista != NULL)
    {
        printf ( "%d" , lista -> valor )
        lista ( lista -> prox );
    }
}
```

Aplice a transformação vista em aula para remover a recursão de cauda, gerando uma função equivalente não recursiva.

1) (2.5 pontos) O código abaixo conta a quantidade de divisores de N . Mostre um possível C3E equivalente a ele, mostrando como é feita a divisão do mesmo em blocos básicos. Considere que o C3E possui uma instrução para cálculo de resto de divisão inteira.

```
For ( i = 1; i <= N; i = i + 1)
    if ( N % i == 0)
        d = d + 1;
```


1) (2.5 pontos) Considere o C3E gerado para o código a seguir, que calcula a série

$$\frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \frac{x^{1000}}{1000!}$$

```
soma=0;
for (i=1;i <=1000;i++){
fat=1;
for (j=1;j<=i ;j++) fat=fat*j ;
termo=x^i / fat ;
soma=soma+termo ;
}
```

```

soma=0
i=1
Laço: if i>1000 goto fim
      fat=1
      j=1
LF:   if j>i goto F2
      fat=fat*j
      j=j+1
      goto LF
F2    T1=X^i
      termo=T1/fat
      soma=soma+termo
      i=i+1
      goto Laço

```

fim:

Mostre o grafo de fluxo de controle correspondente e mostre, para cada bloco básico, quais variáveis estão vivas na entrada e na saída do mesmo. Considere que ao fim da execução apenas as variáveis X e $soma$ estão vivas.

4) (2.5 pontos) Utilizando o algoritmo para análise de uso subsequente ao final dessa prova, mostre a cada linha do código a informação de vida e próximo uso de cada uma das variáveis envolvidas ao início e ao fim de cada comando. Considere que ao final do bloco básico as únicas variáveis que devem estar vivas são as variáveis v e i .

$$1) t1 = i * 4$$

$$2) aux = v + t1$$

$$3) t2 = 9 - i$$

$$4) t3 = t2 * 4$$

$$5) t4 = v + t3$$

$$6) t5 = i * 4$$

$$7) v = t4 + t5$$

$$8) t6 = 9 - i$$

$$9) t7 = t6 * 4$$

$$10) v = aux + t7$$

$$11) i = i + 1$$

5) (2.0 pontos) Mostre o C3E gerado para o comando a seguir considerando, na numeração das variáveis temporárias, a ordem em que o código foi gerado. Coloque na resposta as produções da gramática que foram utilizadas na derivação com as respectivas ações semânticas (está no caderno).

$a := (a + b + c) * (a + b + d) * (c + b + d);$

7) (2.0 pontos) A reutilização de variáveis temporárias baseia-se no fato que, a não ser que ocorram otimizações, cada variável temporária é utilizada apenas uma vez, podendo ser reutilizada após o uso. O trecho de código do exercício anterior utiliza 8 variáveis temporárias. Considerando que a única variável que deve estar viva ao final do código é a variável A, reescreva o código (o código original, antes da otimização) utilizando o menor número de variáveis temporárias possível.

8) (2.0 pontos) Utilizando o algoritmo de geração de código visto em aula, gere o código objeto para o código do exercício 2 (o original, antes da otimização). Considere uma máquina com 3 registradores, R0, R1 e R2, inicialmente vazios. Mostre ao final de cada instrução do C3E o estado dos descritores de registradores e descritores de variáveis. Considere que ao final do bloco básico as únicas variáveis que devem estar vivas são as variáveis a, b, c e d.

1. Para o trecho de código a seguir:

```
Var A,B,C: array [5..10,5..10] of integer;  
    lin , col , ind : integer;  
begin  
    for lin:=5 to 10 do  
        for col:=5 to 10 do  
            begin  
                C[lin , col]:=0;  
                for ind:=5 to 10 do  
                    C[lin , col]:=C[lin , col]+A[lin , ind]*B[ind , col];  
                end;  
            end;  
        end;  
    end;
```

- a. Mostre o C3E gerado utilizando o padrão Tnn para variáveis temporárias e Lnn para labels. A numeração dos labels e variáveis temporárias deve estar de acordo com as ações semânticas da gramática (2 pontos)
- b. Identifique os líderes e monte o diagrama de fluxo (1 ponto)
- c. Otimize o código gerado (2 pontos)

2) Mostre o C3E gerado para o comando a seguir considerando, na numeração das variáveis temporárias, a ordem em que o código foi gerado.

$a := (a + b) * (a - b + d) / (c + b + d);$

4) Utilizando o algoritmo de geração de código visto em aula, gere o código objeto para o código do exercício 2 (o original, antes da otimização). Considere uma máquina com 3 registradores, R0, R1 e R2, inicialmente vazios. Mostre ao final de cada instrução do C3E o estado dos descritores de registradores e descritores de variáveis. Considere que ao final do bloco básico as únicas variáveis que devem estar vivas são as variáveis a, b, c e d.

2) (3.0 pontos) Considere a produção a seguir para geração de código de avaliação de expressões.

$$\begin{aligned} E &\rightarrow E_1 / T \{E.place := cria_temp; E.cod := E_1.cod \parallel T.cod \\ &\parallel gen(/, E.place, E_1.place, T.place)\} \\ E &\rightarrow T \{E.cod := T.cod \ E.place := T.place\} \end{aligned}$$

Escreva uma produção equivalente, não recursiva, com as devidas ações semânticas

3) (3.0 pontos) Escreva uma gramática não recursiva que reconheça expressões pós-fixadas com os operadores $+$, $-$, $*$ e $/$ e operandos de um dígito, e retorne no atributo E.inf a expressão infixada equivalente, podendo conter parênteses redundantes. Por exemplo, para a expressão $423+*$ as ações semânticas devem retornar $(4 * (2 + 3))$

1)(3 pontos) Considerando a seguinte declaração de array:
M:array[1..10,1..10,1..10] of integer; Mostre o código de três endereços gerado para o seguinte trecho de programa em pascal:

```
for i := 1 to 10 do
  for j := 1 to 10 do
    for k := 1 to 10 do
      M[i , j , k]:=M[i , j , k+1]+ 1;
```

2)(4 pontos)Escreva uma DDS para gerar código de três endereços para referências a elementos de arrays [1..10,1..10] of integer em pascal na seguinte produção:

$E \rightarrow \text{id} [E , E]$

3)(3 pontos)Monte o grafo de fluxo do código do exercício 1 e otimize-o removendo expressões laço invariantes e variáveis de indução.

1) (2.5 pontos) Considere uma sublinguagem de tipos que contem apenas inteiros e structures em que os campos são inteiros ou outras structures. Uma structure é descrita pela string (campo1 * campo2 * campo3 ...) e cada campo é descrito por (nome_campo * tipo). Inteiros são descritos como "int". Assim, a structure a seguir:

```
struct {int a;  
        int b;  
        struct {int c;  
                int d;} e;}
```

seria descrita como

```
struct((a*int)*(b*int)*(e*struct((c*int)*(d*int))))
```

Complete o esquema de tradução abaixo para que reconheça os dois tipos descritos acima (int e struct) e gere no atributo .tp o string com a descrição do tipo reconhecido.

tipo → **int** {tipo.tp="int"}

tipo → **struct** {LCampos} ...

Lcampos → ...

Para simplificar a análise considere que:

- a) Cada campo é descrito por um identificador e um tipo, não havendo descrições de campo em que o tipo é associado a mais de um identificador como int a,b,c;
- b) Não são associados nomes às structs, não devendo ser reconhecidas construções como struct nome {...};

tipo $\rightarrow$ int {tipo.tp="int"}

tipo $\rightarrow$ struct {LCampos}

{tipo.tp="struct("+LCampos.txt+")"}

Lcampos $\rightarrow$ Campo ; RLCampos

{LCampos.txt=Campo.txt+RLCampos.txt}

RLCampos $\rightarrow$ Campo; *RLCampos*₁

{RLCampos.txt="*" + Campo.txt + RLCampos1.txt}

RLCampos $\rightarrow \epsilon$ {RLCampos.txt=""}

Campo $\rightarrow$ id : tipo

{Campo.txt="(" + id.txt + "*" + tipo.tp + ")"}

2) (2.5 pontos) Considere a versão simplificada do bloco try...catch a seguir:

```
try {  
    comandos...  
    throw cte1  
    comandos...  
    throw cte2  
} catch (cte1, cte2) comando;
```

em que exceções são geradas pelo comando throw dentro do bloco e quando identificadas pelo catch o comando após o catch é executado. Faça um esquema de tradução para reconhecer um bloco try-catch conforme descrito e verificar se:

- a) Não há throws gerando exceções que não são tratadas pelo catch
- b) A lista de constantes no catch não contém constantes para os quais não há um throw no bloco.

Bloco $\rightarrow$ **try** {Lcomandos} **catch** (Lconst) comando; { if
Lcomandos.lista!= Lconst.lista printf("Erro!!!") }

Lcomandos $\rightarrow$ comando LComandos1
{Lcomandos.lista=comando.e + LComandos1.lista}

Lcomandos $\rightarrow$ ϵ {Lcomandos.lista=" "}

comando $\rightarrow$ **throw** cte {comandos.e=cte}

comando $\rightarrow$ outro {comandos.e=" "}

4) (2.5 pontos) Faça um esquema de tradução (produções e ações semânticas) para reconhecer declarações de vetores com ou sem inicialização em C, como as declarações a seguir:

int a[4]={1,2,3,4}; int b[10];

O esquema de tradução deve retornar em um atributo .cod o código correspondente às inicializações e verificar se a quantidade de inicializadores corresponde à dimensão do vetor. Caso o número de inicializadores não seja igual ao número de elementos do vetor deve ser emitida uma mensagem de erro. Para simplificar, considere que uma declaração contem a declaração de apenas um vetor (não deve reconhecer, por exemplo, `int a[10],b[10];`); O código de inicialização correspondente à primeira declaração acima deve ser:

`a[0] = 1`

`a[1] = 2`

`a[2] = 3`

`a[3] = 4`

- 1) (2.5 pontos) Considere a gramática a seguir que gera expressões envolvendo produtos de variáveis e constantes:

$$E \rightarrow E * T \mid T$$

$$T \rightarrow \text{cte} \mid \text{id}$$

- Especifique ações semânticas na gramática para gerar o string equivalente à expressão reconhecida, mas efetuando, na medida do possível, a multiplicação dos termos da expressão que forem constantes:
- Ex:
 - A expressão "3 * 2 * a" gera o string "6 * a"
 - A expressão "3 * 0 * x" gera o string "0 * x"
 - A expressão "3 * 1 * y" gera o string "3 * y"

- Assuma a existência de funções para converter de strings para valores inteiros e vice-versa. Não é necessário remover a recursão da gramática. Considere que o operador de multiplicação é comutativo e associativo, de modo que a expressão " $3 * x * 2$ " é equivalente a " $3 * 2 * x$ " de modo que o produto " $3 * 2$ " pode ser efetuado durante a análise e a expressão resultante deve ser " $6 * x$ " ou " $x * 6$ ".
 - Dica: Monte a expressão apenas com as variáveis, mantendo o produto das constantes em separado, acrescentando-o à expressão apenas ao final.

- Resposta:

```
E → E1 * T {E.cte=E1.cte*T.cte; if (E1.id=="") E.id=T.id;  
else if (T.id=="") E.id=E1.id; else E.id=E1.id+"*" +T.id; if  
(E.id=="") E.str=itoa(E.cte); else if (E.cte==0) E.str="0";  
else if (E.cte==1) E.str=E.id; else  
E.str=E.id+"*" +itoa(E.cte)}  
E → T {E.cte=T.cte;E.id=T.id;E.str=T.str; }  
T → cte {T.cte=cte.val;T.str=cte.lex;T.id="";}  
T → id {T.cte=1;T.str=id.lex;T.id=id.lex; }
```

- Obs:

- A função itoa converte um valor inteiro para um string
- O atributo .cte mantém o produto (valor numérico) de todas as constantes que aparecem na expressão
- O atributo .id mantém um string com o produto de todos os identificadores que aparecem na expressão
- O atributo .str mantém o resultado final, o string com os identificadores e no final do string, a constante resultado do produto de todas as constantes

- 2) (2.5 pontos) Considere uma sublinguagem de tipos que contem como tipo básico apenas inteiros, e como construtores apenas pointers. Assim,
 - `int *v;` seria descrita como `"pointer(int)"`
 - `int **v;` seria descrito como `"pointer(pointer(int))"`
- Escreva um esquema de tradução (gramática e ações semânticas) que reconheça os tipos descritos acima (`int`, `pointers para ints`, `pointers para pointers int`, etc...) gerando no atributo `.tp` o string com a descrição do tipo reconhecido (apenas a especificação do tipo, sem o identificador da variável).

1) (2.5 pontos) Considere a gramática a seguir que gera atribuições de expressões envolvendo pointers e arrays unidimensionais (ex: $x = (*a)[3]$ ou $y = (*a[3])[3]$)

$\text{atrib} \rightarrow \text{id} = E$

$E \rightarrow * E_1$

$E \rightarrow T [E_1]$

$E \rightarrow T$

$T \rightarrow \text{id}$

$T \rightarrow \text{cte}$

$T \rightarrow (E)$

Acrescente ações semânticas para gerar C3E.

2) (2.5 pontos) Usando o esquema de tradução do exercício anterior, mostre a árvore de derivação **anotada** para a expressão $x = (*a)[3]$

- 1) (2.5 pontos) Considere a gramática a seguir que reconhece atribuições de expressões envolvendo produtos de variáveis e constantes:

$$\text{atrib} \rightarrow \text{id} = E$$
$$E \rightarrow E * T \mid T$$
$$T \rightarrow \text{cte} \mid \text{id}$$

- Especifique ações semânticas na gramática para gerar C3E para a atribuição mas efetuando, na medida do possível, a multiplicação dos termos da expressão que forem constantes:
- Ex:
 - A expressão "x = 3 * 2 * a" gera o C3E equivalente 6 * a (t1=6*a x=t1)
 - A expressão "x = 3 * 1 * y" gera o C3E equivalente a 3 * y

- Assuma a existência de funções para converter de strings para valores inteiros e vice-versa. Não é necessário remover a recursão da gramática. Considere que o operador de multiplicação é comutativo e associativo, de modo que a expressão " $3 * x * 2$ " é equivalente a " $3 * 2 * x$ " de modo que o produto " $3 * 2$ " pode ser efetuado durante a análise e a expressão resultante deve ser " $6 * x$ " ou " $x * 6$ ".
 - Dica: Monte a expressão apenas com as variáveis, mantendo o produto das constantes em separado, acrescentando-o à expressão apenas ao final.

- 2) (2.5 pontos) O operador de referência de pointer em C3E pode ser representado pela instrução
 - $a = *b$
- Complete o esquema de tradução abaixo com ações semânticas para reconhecer o uso de referências de pointers em expressões do tipo $*x[3]$, $**x[3]$, etc., e gere o C3E equivalente. Lembre que o operador $[]$ tem prioridade maior que o operador $*$. O início da gramática seria

$$E \rightarrow *E \mid F$$

$$F \rightarrow id[cte]$$
- Ex: a expressão $**X[3]$ geraria o C3E
 - $t1 = X[3]$
 - $t2 = *t1$
 - $t3 = *t2$
- Considere que no C3E a constante do índice do vetor pode ser aplicada diretamente no endereço-base do vetor sem precisar somar ou multiplicar por valor nenhum.

1) (2.5 pontos) Considere o C3E gerado para o código a seguir, que escreve os fatores primos que compõem um número:

```
d=2;
leia(n);
while (n>1)
    if (n%2==0){
        n=n/d;
        escreva(d);}
    else d=d+1;
```

```
d=2
leia(n)
```

```
L1:  if n<=1 goto fim
```

```
t1=n %d
if t1>0 goto else
```

```
n=n/d
escreva(d)
goto F1
```

```
else:  d=d+1
```

```
F1:  goto L1
```

```
fim:
```

Converta esse código para a forma SSA renumerando as variáveis e adicionando as funções ϕ que forem necessárias.

1) (2.0 pontos) O operador de pós-incremento em C (++) aplicado a uma variável, incrementa o valor da variável e resulta, como expressão, no valor original da variável, antes do incremento.

Assim, o trecho de código:

```
a=3;
```

```
printf("%d",a++);
```

escreverá o valor de a antes do incremento (3), mas após o printf a variável a terá o valor 4. O operador de pós-incremento só pode ser aplicado a variáveis, de modo que a expressão `3++` é inválida.

Faça um esquema de tradução (gramática e ações semânticas) que reconheça e gere código de 3 endereços para expressões envolvendo somas de constantes e variáveis, com ou sem a aplicação do operador de pós-incremento. Ex: `a + b++ + 3`. O esquema de tradução deve verificar que o operador de pós incremento seja aplicado somente a identificadores.

- Solução:
- $E \rightarrow E_1 + T \{E.place=criatemp;$
 $\{E.cod=E_1.cod+T.cod+gen(+,E.place,E_1.place,T.place)\}$
- $E \rightarrow T \{E.place=T.place;E.cod=T.cod\}$
- $T \rightarrow id \{T.place=id.lex;T.cod=" \ "}\}$
- $T \rightarrow cte \{T.place=criatemp;T.cod=gen(=,T.place,cte.lex)\}$
- $T \rightarrow id++ \{T.place=criatemp;$
 $T.cod=gen(=,T.place,id.lex)+gen(+,id.lex,id.lex,1)\}$

2) (1.5 pontos) O operador vírgula aplicado a duas expressões faz com que as duas sejam avaliadas e retorna como resultado o valor da segunda expressão. Assim, no comando $a=(2*b,3+c)$, a variável a recebe o valor da expressão $3+c$. Faça um esquema de tradução para gerar código de 3 endereços para expressões com constantes e variáveis, e os operadores de soma e vírgula. Considere que o operador de soma tem maior prioridade que o operador vírgula e que ambos operadores são associativos à esquerda.

• Solução:

- $E \rightarrow E_1, T \{E.place = T.place; E.cod = E_1.cod + T.cod\}$
- $E \rightarrow T \{E.place = T.place; E.cod = T.cod\}$
- $T \rightarrow T_1 + F \{T.place = criatemp; T.cod = T_1.cod + F.cod + gen(+, T.place, T_1.place, F.place)\}$
- $T \rightarrow F \{T.place = F.place; T.cod = F.cod\}$
- $F \rightarrow id \{F.place = id.lex; F.cod = " "\}$
- $F \rightarrow cte \{F.place = criatemp; F.cod = gen(=, F.place, cte.lex)\}$

- 3) (1.5 pontos) Na avaliação reduzida (short-circuit) de expressões lógicas, se o resultado da expressão for atribuída a uma variável, deve ser gerado código para atribuir 0 ou 1 à variável, dependendo do resultado da expressão lógica. Assim, o comando "teste = a>b;" geraria o código:

```
if a>b goto L1 // Código da expressão "a>b"  
goto L0
```

```
L0: teste=0      // Código gerado na produção da atribuição  
    goto L2  
L1: teste=1  
L2: ...
```

- Enquanto que o comando "teste = a + b" geraria a atribuição numérica
 - t0=a+b
 - teste = t0

- Considere a produção para comando de atribuição:

Atrib $\rightarrow$ id = E

- Adicione ações semânticas para gerar C3E para a atribuição, considerando que se a expressão for do tipo lógico, deve ser gerado o código mostrado acima, mas se for do tipo numérico, deve ser gerado o código da atribuição numérica. Considere a existência de um atributo E.tipo, que retorna se a expressão é uma expressão lógica ou aritmética. Considere também a existência dos atributos E.true e E.false que, passados a E, contém o label para onde a expressão deve desviar, caso seja uma expressão lógica (se a expressão for aritmética, os labels não serão utilizados pela expressão).

```
Atrib → id = {E.true=geralabel; E.false=geralabel}E {if  
(E.tipo==numerico) Atrib.cod=E.cod+gen(=,id.lex,E.place) else  
{lbfim=geralabel; Atrib.cod =  
gen(:,E.false)+gen(=,id,0)+gen(goto,lbfim)+gen(:,E.true)+  
gen(=,id.lex,1)+gen(:,lbfim)}}}
```

1) (2.5 pontos) Considere a gramática a seguir para gerar expressões aritméticas com constantes, o operador de subtração e parênteses. Insira ações semânticas para gerar, a partir da análise de uma expressão contendo parênteses redundantes, uma expressão equivalente contendo apenas os parênteses obrigatórios. Por exemplo, na expressão $3-((2-1)-1)$, os parênteses internos são desnecessários, não afetando a ordem de avaliação dos operadores, enquanto que os parênteses externos alteram a ordem de avaliação. Não é necessário remover a recursão da gramática.

$$E \rightarrow E-T$$

$$E \rightarrow T$$

$$T \rightarrow \text{dig}$$

$$T \rightarrow (E)$$

2) (2.0 pontos) Represente o bloco básico a seguir através de um grafo e gere o código otimizado a partir do grafo, considerando que as únicas variáveis do bloco básico que devem estar atualizadas ao final do bloco são as variáveis a e $T1$.

$T1 := a + b$

$T2 := T1 + c$

$T3 := a + b$

$T4 := T3 + d$

$T5 := c + b$

$T6 := T5 + d$

$T7 := T2 * T4$

$T8 := T7 * T6$

$A := T8$

5) (2.0 pontos) Para o trecho de código a seguir:

```
Var A,B:array [1..6] of integer;  
C:array [5..11] of integer;  
    ind : integer;  
begin  
    for ind := 1 to 6 do  
        C[ind]:=A[ind]+B[ind];  
    end;
```

- Mostre o C3E gerado utilizando o padrão Tnn para variáveis temporárias e Lnn para labels.
- A numeração dos labels e variáveis temporárias deve estar de acordo com as ações semânticas da gramática.
- Identifique os líderes, monte o diagrama de fluxo e otimize o código gerado.

2)(4.0 pontos) Mostre um possível código gerado para a procedure A1 no trecho abaixo. O código deve conter o código de entrada na procedure, a chamada à procedure A com a preparação dos parâmetros e o código de saída da procedure.

```
Procedure A(B,C:integer );  
Begin  
End;
```

```
Procedure A1(B1,C1:integer );  
Var i,j:integer;  
Begin  
    For i:=1 to 10 do  
        For j:=1 to 10 do  
            A(i+1,j+1);  
        End;  
    End;
```


9) (1 ponto) Descreva como seria o código gerado para que o procedimento A a seguir pudesse acessar a variável V1 no trecho a seguir:

```
Procedure B;  
  var V1:integer;  
  Procedure A;  
    var V2:integer;  
    begin  
      v2:=v1;  
    end;  
  begin  
  end;
```

1) (1.5 pontos) O C3E abaixo calcula a soma dos números primos entre 2 e 50. Monte o grafo de fluxo de controle correspondente (identificação de líderes, blocos básicos, etc.).

S=0

n=2

L0: if n>50 goto Lfim

d=2

L1: T0=n%d

if T0==0 goto L2

d=d+1

goto L1

L2: if n<>d goto L3

S=S+n

L3: n=n+1

goto L0

Lfim:

2) (1.5 pontos) O C3E a seguir efetua o produto escalar da linha 2 de dois vetores:

$s = 0$

$i = 1$

laco: if $i > 10$ goto fim

$t0 = i - 1$

$t1 = 40 + t0$

$t2 = t1 * 4$

$t3 = v[t2]$

$t4 = w[t2]$

$t5 = t3 * t4$

$s = s + t5$

$i = i + 1$

goto laco

fim:

Aplique as otimizações vistas em aula para obter um código otimizado. Mostre a transformação feita a cada passo. Considere que somente s deve estar vivo ao final do laço.

3) (2. pontos) Mostre como o código abaixo seria representado em um grafo dirigido acíclico e mostre o C3E obtido a partir do grafo.

$t_0 = -b$

$t_1 = t_0 + d$

$t_2 = 2 * a$

$t_3 = t_1 / t_2$

$r_1 = t_3$

$t_4 = t_0 - d$

$t_5 = 2 * a$

$t_6 = t_4 / t_5$

$r_2 = t_6$

$a = a + 1$

- 2) (2.5 pontos) Considere uma sublinguagem de tipos que contem como tipo básico apenas inteiros, e como construtores pointers e arrays unidimensionais. Assim,

int *v[10]; seria descrita como $v = \text{array}(0..9, \text{pointer}(\text{int}))$

int (*v)[10]; seria descrito como $v = \text{pointer}(\text{array}(0..9, \text{int}))$

int **v; seria descrito como $v = \text{pointer}(\text{pointer}(\text{int}))$

- Complete o esquema de tradução abaixo para que reconheça os dois tipos descritos acima (int e struct) e gere no atributo .tp o string com a descrição do tipo reconhecido.

tipo -> int

{tipo.tp="int"}

tipo -> struct {LCampos} ...

Lcampos -> ...

- Para simplificar a análise considere que:
 - a) Cada campo é descrito por um identificador e um tipo, não havendo descrições de campo em que o tipo é associado a mais de um identificador como int a,b,c;
 - b) Não são associados nomes às structs, não devendo ser reconhecidas construções como struct nome ...;

- 1) (2.5 pontos) Escreva um esquema de tradução (gramática e ações semânticas) que reconheça números entre 1 e 99 e retorne como atributo do símbolo inicial um string com o número por extenso. Exemplo: se o número reconhecido for 23, o esquema deve retornar o string "vinte e três". Considere que você tem disponíveis as seguintes listas de strings:
 - `char dezenas[10]={"dez", "vinte", "trinta" ..., "noventa"}`
 - `char unidades[10]={"um", "dois", "três" ... "nove"}`
 - `char unidez[10]={"onze", "doze", ..., "dezenove"}`

```
N → D U    {if (D.v==0) N.t=unidades[U.val]
              else if (U.v==0)N.t=dezenas[D.v]
              else if (D.v==1) N.t=unidez[U.v]
              else N.t=dezenas[D.v]+" e "+unidades[U.v]}

D → dig     {D.v=dig.val}
D → ε       {D.v=0}
U → dig     {U.v=dig.val}
```

- 2) (2.5 pontos) Considere a gramática a seguir que gera árvores binárias de inteiros positivos (em forma linearizada):
 - $\text{barvore} \rightarrow (\text{número b\bar{a}rvore b\bar{a}rvore}) \mid \text{nil}$
- Escreva ações semânticas para verificar se uma árvore binária é uma árvore de busca, ou seja, se todos os valores da primeira subárvore são menores ou iguais ao valor do número corrente e todos os valores da segunda subárvore são maiores ou iguais ao valor do número corrente. Por exemplo, $(2 (1 \text{ nil nil}) (3 \text{ nil nil}))$ está ordenada, mas $(1 (2 \text{ nil nil}) (3 \text{ nil nil}))$ não está ordenada. Sugestão: associe a cada árvore um atributo com o valor máximo na árvore, um atributo com o valor mínimo na árvore e um atributo com a informação se ela é ordenada ou não


```
b -> (número  $b_1$   $b_2$ ) { b.nil=false;  
    b.maior=numero;  
    if ( $b_1$ .nil==false &&  $b_1$ .maior>b.maior) b.maior= $b_1$ .maior}  
    if ( $b_2$ .nil==false &&  $b_2$ .maior>b.maior) b.maior= $b_2$ .maior}  
    b.menor=numero;  
    if ( $b_1$ .nil==false &&  $b_1$ .menor<b.menor) b.menor= $b_1$ .menor}  
    if ( $b_2$ .nil==false &&  $b_2$ .menor<b.menor) b.menor= $b_2$ .menor}  
    if ( $b_1$ .bal==true &&  $b_2$ .bal==true &&  
         $b_1$ .maior<numero &&  $b_2$ .menor>numero) b.bal=true}  
b -> nil {b.nil=true b.bal=true}
```

- 1) (2.5 pontos) A função DECODE do SQL compara uma expressão com uma lista de valores fornecidos retornando a constante associada ao valor igual ao valor da expressão. Se o valor da expressão não é igual a nenhum valor da lista, o decode retorna o valor default, último valor da lista, conforme o exemplo abaixo:
 - DECODE (warehouse_id, 1, 2322, // se warehouse_id é 1, a expressão retorna 2322
3, 4327, // se warehouse_id é 3, a expressão retorna 4327
7, 1245, // etc.etc.etc.
9, 2359,
9999} // se não for nenhum dos anteriores, retorna 9999
- Faça um esquema de tradução em que uma expressão possa derivar uma chamada a Decode ($E \rightarrow \text{Decode}(\dots)$). O esquema deve reconhecer chamadas da função decode e gerar C3E "inline" para a mesma que retorne no atributo .place o valor selecionado.

$E \rightarrow \text{Decode } \{ E_1 \{ E.p = \text{criatemp}; \text{Lbfim} = \text{geralabel};$
 $\text{L.e1p} = E_1.p \text{ L.p} = E.p \text{ L.f} = \text{Lbfim} \}$
 $\text{L} \{ E.\text{cod} = E_1.\text{cod} + \text{L.cod} + \text{gen}(\text{Lbfim}.) \}$
 $L \rightarrow , v_1 v_2 \{ L_1.p = L.p; L_1.e1p = L.e1p; L_1.f = L.f \} L_1$
 $\{ \text{Lb} = \text{geralabel}; \text{L.cod} = \text{gen}(\text{if } L.e1p \neq v_1 \text{ goto Lb}) +$
 $\text{gen}(\text{Lp} = v_2) + \text{gen}(\text{goto Lbf}) + \text{gen}(\text{Lb}.) + L_1.\text{cod} \}$
 $L \rightarrow , v \} \{ \text{L.cod} = \text{gen}(\text{L.p} = v) \}$

- 2) (2.5 pontos) Na aritmética de ponteiros, em operações que envolvam incremento de ponteiros a unidade de incremento é o tamanho do tipo base do ponteiro. Assim, se p é um pointer para inteiro de 4 bytes ($\text{int } *p$) o incremento de p (p.ex. $p++$) é na verdade um incremento de 4 bytes. Faça um esquema de tradução que reconheça o operador de pós incremento aplicado a ponteiros e variáveis int ($E \rightarrow id++$) e gere o C3E correspondente, retornando os atributos `.place` e `.cod`. Considere que está disponível uma função `busca_tipo(char id[])` que devolve "int" ou "ptr" e uma função `busca_tam_base(char id[])` que devolve o tamanho do tipo base apontado pelo identificador `id`. Lembre que o valor retornado pela expressão de pós-incremento é o valor original da variável, antes de ser incrementada, mas que o valor da variável ao final do pós-incremento é o valor incrementado.

```
E → id ++ {E.p = criatemp();  
            if busca_tipo(id) == "int" tam = 1;  
            else tam = busca_tam_base(id);  
            E.cod = gen(E.p = id) + gen(id = id + tam);}
```

- 2) (2.5 pontos) Considere o C3E gerado para o código a seguir, que copia uma matriz $M[10][10]$ para um vetor $V[100]$.

```
for ( i=0, iv=0; i < 10; i++)  
    for ( j=0; j < 10; j++)  
        v [ iv++]=M[ i ][ j ];
```

```
iv=0  
i=0  
L1:  if i>=10 goto fim1  
    j=0  
L2:  if j>=10 goto fim2  
    t0=i*10  
    t1=t0+j  
    t2=t1*4  
    t3=M[t2]  
    V[iv]=t3  
    iv=iv+1  
    j=j+1  
    goto L2  
fim2: i=i+1  
    goto L1  
fim1:
```

- Remoção do $t0 = i*10$ do laço interno por ser laço-invariante no laço interno

```
        iv=0
        i=0
L1:      if i>=10 goto fim1
        j=0
L2:      if j>=10 goto fim2
        t0=i*10
        t1=t0+j
        t2=t1*4
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        j=j+1
        goto L2
fim2:    i=i+1
        goto L1
fim1:
```

```
        iv=0
        i=0
L1:      if i>=10 goto fim1
        j=0
        t0=i*10
L2:      if j>=10 goto fim2
        t1=t0+j
        t2=t1*4
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        j=j+1
        goto L2
fim2:    i=i+1
        goto L1
fim1:
```

- Substituição de j por t1 ($j = t1 - t0$)

```
iv=0
i=0
L1:  if i>=10 goto fim1
    j=0
    t0=i*10
L2:  if j>=10 goto fim2
    t1=t0+j
    t2=t1*4
    t3=M[t2]
    V[iv]=t3
    iv=iv+1
    j=j+1
    goto L2
fim2: i=i+1
    goto L1
fim1:
```

```
iv=0
i=0
L1:  if i>=10 goto fim1
    t0=i*10
    t1=t0
L2:  if t1>=t0+10 goto fim2
    t2=t1*4
    t3=M[t2]
    V[iv]=t3
    iv=iv+1
    t1=t1+1
    goto L2
fim2: i=i+1
    goto L1
fim1:
```



```

        iv=0
        i=0
L1:      if i>=10 goto fim1
        t0=i*10
        t1=t0
L2:      if t1>=t0+10 goto fim2
        t2=t1*4
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t1=t1+1
        goto L2
fim2:    i=i+1
        goto L1
fim1:

```

```

        iv=0
        i=0
L1:      if i>=10 goto fim1
        t0=i*10
        t1=t0
        t4=t0+10
L2:      if t1>=t4 goto fim2
        t2=t1*4
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t1=t1+1
        goto L2
fim2:    i=i+1
        goto L1
fim1:

```

- Eliminando t1 e substituindo por t2 ($t1=t2/4$):

```

iv=0
i=0
L1:  if i>=10 goto fim1
    t0=i*10
    t1=t0
    t4=t0+10
L2:  if t1>=t4 goto fim2
    t2=t1*4
    t3=M[t2]
    V[iv]=t3
    iv=iv+1
    t1=t1+1
    goto L2
fim2: i=i+1
    goto L1
fim1:

```

```

iv=0
i=0
L1:  if i>=10 goto fim1
    t0=i*10
    t1=t0
    t2=t1*4
    t4=t0+10
L2:  if t2>=4*t4 goto fim2
    t3=M[t2]
    V[iv]=t3
    iv=iv+1
    t2=t2+4
    goto L2
fim2: i=i+1
    goto L1
fim1:

```

e

- Eliminando t1 e substituindo por t2 ($t1=t2/4$):

```
        iv=0
        i=0
L1:      if i>=10 goto fim1
        t0=i*10
        t1=t0
        t2=t1*4
        t4=t0+10
        t5=4*t4
L2:      if t2>=t5 goto fim2
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t2=t2+4
        goto L2
fim2:    i=i+1
        goto L1
fim1:
```

```
        iv=0
        i=0
L1:      if i>=10 goto fim1
        t0=i*10
        t1=t0
        t2=t1*4
        t4=t0+10
        t5=4*t4
L2:      if t2>=t5 goto fim2
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t2=t2+4
        goto L2
fim2:    i=i+1
        goto L1
fim1:
```

- Substituindo t1 por t0 (propagação de cópia):

```
      iv=0
      i=0
L1:   if i>=10 goto fim1
      t0=i*10
      t1=t0
      t2=t1*4
      t4=t0+10
      t5=4*t4
L2:   if t2>=t5 goto fim2
      t3=M[t2]
      V[iv]=t3
      iv=iv+1
      t2=t2+4
      goto L2
fim2: i=i+1
      goto L1
fim1:
```

```
      iv=0
      i=0
L1:   if i>=10 goto fim1
      t0=i*10
      t2=t0*4
      t4=t0+10
      t5=4*t4
L2:   if t2>=t5 goto fim2
      t3=M[t2]
      V[iv]=t3
      iv=iv+1
      t2=t2+4
      goto L2
fim2: i=i+1
      goto L1
fim1:
```

- Laço externo: Substituindo i por $t0$ ($i=t0/10$):

```

        iv=0
        i=0
L1:     if i>=10 goto fim1
        t0=i*10
        t2=t0*4
        t4=t0+10
        t5=4*t4
L2:     if t2>=t5 goto fim2
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t2=t2+4
        goto L2
fim2:   i=i+1
        goto L1
fim1:

```

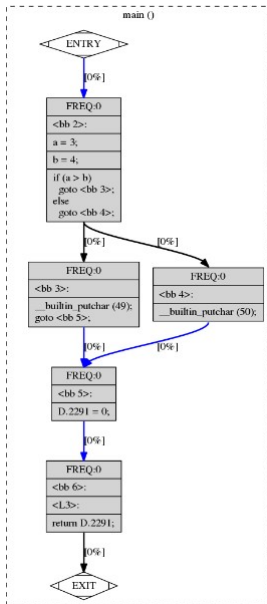
```

        iv=0
        t0=0
L1:     if t0>=100 goto fim1
        t2=t0*4
        t4=t0+10
        t5=4*t4
L2:     if t2>=t5 goto fim2
        t3=M[t2]
        V[iv]=t3
        iv=iv+1
        t2=t2+4
        goto L2
fim2:   t0=t0+10
        goto L1
fim1:

```

- Diretivas do gcc para gerar arquivos de I.R.:
 - gcc -fdump-tree-all-graph-lineno <target.c>
 - gera arquivos.dot, com o grafo de fluxo de controle.
 - P.ex. para o programa abaixo:

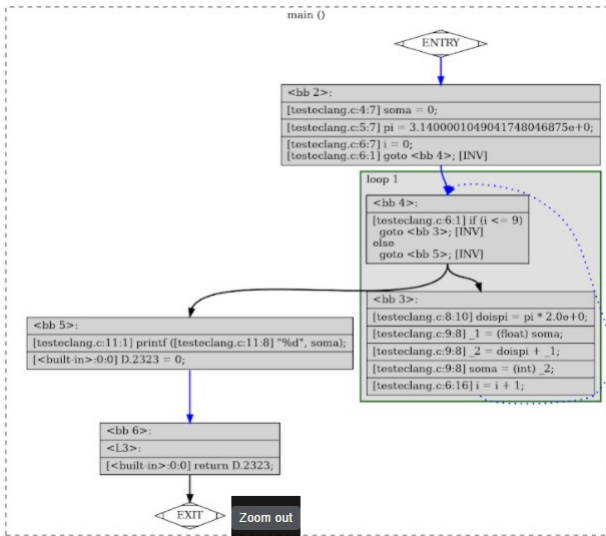
```
int main{
int a=3,b=4;
if (a>b) printf("1");
else printf("2");}
```
 - gera o gráfico a seguir após converter o .dot para .png
 - dot -Tpng teste.c.011t.cfg.dot -o cfg.png



- E alguns dos arquivos intermediários para o programa abaixo são:

```
#include <stdio.h>
int main()
{
    int i ,soma=0;
    float pi=3.14;
    for ( i=0;i <10;i++)
    {
        float doispi=2*pi;
        soma+=doispi;
    }
    printf( "%d" ,soma );
}
```


- testclang.c.012t.cfg.dot

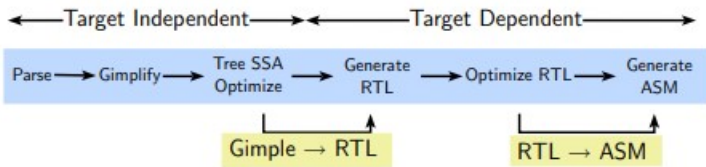


Estudo de Caso: As representações internas do GCC

- Durante o processo de compilação o GCC utiliza 3 diferentes formatos internos de representação, convenientes para cada etapa da compilação.
 - GENERIC é uma representação na forma de uma árvore sintática abstrata (AST), dependente de cada linguagem fonte (lembrando que o gcc, gnu compiler collection, suporta várias linguagens)
 - GIMPLE é a representação intermediário, na forma de um código de 3 endereços, sobre a qual são feitas otimizações independentes da linguagem e da plataforma-alvo.
 - RTL (Register Transfer Language) é uma representação em nível mais baixo, que leva em conta características da máquina-alvo, como arquitetura de registradores e modos de endereçamento.

- O processo completo de compilação pode chegar a 196 passos (excluído o pré-processamento e a montagem do assembly gerado pelo compilador), sendo que alguns passos são executados diversas vezes.
- A lista de passos pode ser encontrada no arquivo `passes.c`, no diretório de fontes do gcc (Google `passes.c` também funciona).
- Os passos podem ser divididos, de modo geral, em passos sobre o GIMPLE e passos sobre o RTL.

- A figura abaixo mostra uma primeira divisão dos passos do gcc.
- No Parsing o arquivo-fonte é transformado em uma representação de árvore sintática abstrata, em GENERIC
- No Gimplify é transformado no código de 3 endereços GIMPLE
- Após, é transformado em uma representação SSA onde são feitas mais algumas otimizações
- Por fim ele é convertido para RTL, passa por otimizações e é convertido para assembly



- A tabela abaixo agrupa alguns passos sobre o GIMPLE, mostrando em que passos são feitas algumas das otimizações.
- No total são feitos 136 passos

Pass Group	Examples	Number of passes
Lowering Gimple	IR, CFG Construction	12
Interprocedural Optimizations	Conditional Constant Propagation, Inlining, SSA Construction	36
Intraprocedural Optimizations	Constant Propagation, Dead Code Elimination, PRE	40
Loop Optimizations	Vectorization, Parallelization	24
Remaining Intraprocedural Opts.	Value Range Propagation, Rename SSA	23
Generating RTL		01
Total number of passes on Gimple		136

- A tabela abaixo agrupa alguns passos sobre o RTL, mostrando em que passos são feitas algumas das otimizações.
- No total são feitos 84 passos

Pass Group	Examples	Number of passes
Intraprocedural Optimizations	CSE, Jump Optimization	15
Loop Optimizations	Loop Invariant Movement, Peeling, Unswitching	7
Machine Dependent Optimizations	Register Allocation, Instruction Scheduling, Peephole Optimizations	59
Assembly Emission and Finishing		03
Total number of passes on RTL		84

- A lista de otimizações do GCC com uma pequena descrição de cada uma, bem como os flags para ativação pode ser encontrada nas opções de linha de comando:

```
gcc -c - -help=optimizers
```

```
C:\teste\gimple>gcc -c --help=optimizers
The following options control optimizations:
  -O<number>          Set optimization level to <number>
  -Ofast              Optimize for speed disregarding exact standards
                      compliance
  -Og                 Optimize for debugging experience rather than
                      speed or size
  -Os                 Optimize for space rather than speed
  -faggressive-loop-optimizations Aggressively optimize loops using language
                      constraints
  -falign-functions   Align the start of functions
  -falign-jumps       Align labels which are only reached by jumping
  -falign-labels      Align all labels
  -falign-loops       Align the start of loops
```

- Também é possível gerar um dump da saída de cada passo, permitindo acompanhar a representação e as ações executadas em cada passo.
- Para ver a saída de cada passo usa-se a opção:
-fdump-< ir >-<pass>
- onde <ir> é
 - tree - passos antes da geração do rtl
- <pass> pode ser : gimple , cfg etc.
- Use -all para ver todos os dumps
 - rtl - passos sobre o rtl
- <pass> pode ser : expand, greg, vreg etc.
- Use -all para ver todos os dumps
- Pode-se usar também a opção -da para gerar todos os dumps do rtl
- Exemplo:
gcc -fdump-tree-all -fdump-rtl-all test.c

- O trecho de código abaixo (teste.c) foi compilado gerando os dumps do gimple.

```
int i,a=1,b=1;
for (i=0;i<10;i++)
    a=a*i+2*++b;
```

- No passo 3 o **for** foi "desmanchado" em desvios condicionais e incondicionais, são introduzidos os labels necessários e o dump teste.c.003t.original resultou

```
int i;
int a = 1;
int b = 1;
i = 0;
goto <D.1840>;
<D.1839>;
a = a * i + ++b * 2;
i++;
<D.1840>;
if (i <= 9) goto <D.1839>; else goto <D.1841>;
<D.1841>;
```

- Observa-se no código anterior que no código gerado a condição do for fica no final do laço, e antes do laço é feito um desvio incondicional para a condição do for.
- No passo 4 as expressões maiores são "quebradas" com o uso de temporários e o código é convertido para o formato de 3 endereços, resultando no dump teste.c.004t.gimple abaixo:

```
int i;  
int a = 1;  
int b = 1;  
i = 0;  
goto <D.1840>;  
<D.1839>;  
D.1843 = a * i;  
b = b + 1;  
D.1844 = b * 2;  
a = D.1843 + D.1844;  
i = i + 1;  
<D.1840>;  
if (i <= 9) goto <D.1839>; else goto <D.1841>;  
<D.1841>;
```

- No passo 11 (teste.c.011t.cfg) é gerado o grafo de fluxo de controle e identificados os laços. Os labels do passo anterior são substituídos pelos labels de entrada de cada bloco básico. No início do dump são apresentadas as informações sobre os laços e a estrutura dos blocos básicos, identificando os sucessores de cada bloco.

```
:: 2 loops found
::
;; Loop 0
;; header 0, latch 1
;; depth 0, outer -1
;; nodes: 0 1 2 3 4 5
;;
;; Loop 1
;; header 4, latch 3
;; depth 1, outer 0
;; nodes: 4 3
;; 2 succs { 4 }
;; 3 succs { 4 }
;; 4 succs { 3 5 }
;; 5 succs { 1 }
```

```
<bb 2>:  
a = 1;  
b = 1;  
i = 0;  
goto <bb 4>;  
<bb 3>:  
D.1843 = a * i;  
b = b + 1;  
D.1844 = b * 2;  
a = D.1843 + D.1844;  
i = i + 1;  
<bb 4>:  
if (i <= 9)  
goto <bb 3>;  
else  
goto <bb 5>;  
<bb 5>:  
return;
```

- No passo 15 (teste.c.015t.ssa) o grafo de fluxo de controle é transformado para a forma SSA e são inseridas as funções phi nas entradas dos blocos básicos que tem mais de um predecessor.

```
<bb 2>:
a_4 = 1;
b_5 = 1;
i_6 = 0;
goto <bb 4>;
<bb 3>:
_7 = a_2 * i_1;
b_8 = b_3 + 1;
_9 = b_8 * 2;
a_10 = _7 + _9;
i_11 = i_1 + 1;
<bb 4>:
# i_1 = PHI <i_6(2), i_11(3)>
# a_2 = PHI <a_4(2), a_10(3)>
# b_3 = PHI <b_5(2), b_8(3)>
if (i_1 <= 9)
goto <bb 3>;
else
goto <bb 5>;
<bb 5>:
return;
```